



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

Department of Biosciences

Master's Degree in Quantitative Biology

MYCELIUM

A Unified Pipeline for Causal Gene Regulatory Network Inference from Single-Cell Perturbation Data

Supervisor (Human Technopole)

Dr. Andrea Sottoriva

Internal Advisor (University of Milan)

Dr. Beatrice Bodega

Candidate

Patrick Sheehan

Matriculation Number 50458A

Academic Year 2025 / 2026

Signed by Andrea Sottoriva

13.07.2026

Dedicated to Angeline

*Who in less than two months time I will have the incredible joy and privilege of calling
my wife*

*This dream of moving to Italy, the thesis, and the life that we have together is only possible
because of her unyielding love and support. The content and exhaustive nature of this
thesis is therefore dedicated to her.*

Acknowledgements

My thanks to my supervisor, Dr. Andrea Sottoriva, for hosting this thesis at Human Technopole and supporting the work.

Thank you to my internal advisor, Dr. Beatrice Bodega, whose Advanced Molecular Biology course was a highlight of my degree and the reason I asked her to advise this thesis.

To Dr. Chinmaya Joisa, who gave me access to his group's compute at the University of North Carolina at Chapel Hill after I had lost my own, thank you. That access is a large part of why this thesis came together in time.

Michele Calabrò has been a mentor and a friend, and he helped me shape and work through the ideas behind FUNGI. A good deal of how the method took form came out of those conversations, and I am grateful for them.

My thanks to Dr. Anna Moroni, head of the Quantitative Biology programme at the University of Milan, for helping me into the programme and for her guidance since.

Thank you to Madeline Dale, Luca Frattegiani, and Clara Canavese, Andrey Tyshevich, David Mas Ponte, and Alexandra Livanova from my lab, for the company and the support along the way.

My thanks to Anastasiia Romanova, our lab's administrator, who kept everything coordinated and handled more logistics on my behalf than I can count.

Thank you to Konstantin Winter, our team's project manager, for his support.

To the Booties, Lillian, Erfan, Angelina, Alex, and Samuel, my fellow students in Milan, thank you for being in the trenches with me through every exam and every setback.

To Kat Vanhüßen, thank you for the endless voice notes between Milan and New York, for listening through all of it, and for the shared love of plants.

Thank you to Noah Clark, a fellow scientist who understood the work from the inside and never needed me to explain why it was hard.

To Noah Keefer and Ian Cook, thank you for keeping me sane across the time zones, one game at a time.

My thanks to Nicole Mooseburger, whose own move abroad to Oxford helped give me the nerve to make mine.

To Nell Crosby, an old friend and roommate, who walked her road into medical school while I walked mine into graduate school, thank you.

Thank you to Salvador Carriaga-Lambert, a fellow American abroad, for the good company.

To Luna, a fellow intern who became a good friend, thank you.

My thanks to Larisa Burka, for her generosity with study materials.

To Antonio, Alessio, and Thashini, from my programme, thank you for making the hardest stretches better.

Thank you to Shadi, Alessia, Maks, Ayan, Leon, Matteo, Yaroslav, Ryan, Sofya, Joaco, Christo, and Stef, for the friendship and the support.

To Simone Trini and Alberto Russo, whom I met during my internship in Anna Moroni's lab, thank you.

And to everyone in Milan who helped make this thesis possible, in ways large and small, thank you.

Author's Note

This thesis was written under conditions that its results do not advertise. The MYCELIUM pipeline is the product of roughly nine months of work, and for almost all of that time it was developed on the high-performance computing cluster of Human Technopole, where the data this project depends on was also stored. On 19 June 2026 a total blackout struck the medium-voltage network supplying the campus; the main feeder cell of the power line was damaged, the data centre was brought down under a controlled shutdown, and the notice that reached us that afternoon asked that we all limit activity to the absolute minimum and switch off equipment wherever possible. What was meant to be a brief interruption became a loss of access, to both the compute and the data, that lasted until four days before this thesis was due.

The months in between were a forced adaptation. Recovering the code from version control, I converted several enterprise-scale pipelines to run on a five-year-old laptop with six CPU cores and a single RTX 2070 Max-Q, a small fraction of the thousand-plus cores and generous GPU nodes the cluster had offered. Some of the heaviest work, the HYPHAE co-expression inference and any large-scale model training, simply could not be done there, and it was precisely that constraint that produced two of the instruments this thesis is built on: MOREL and RHIZO, deliberately lightweight predictive models designed to run locally while preserving the dataset-agnostic character of the whole project. The path back to scale came through the generosity of others. Dr. Chinmaya Joisa, of the University of North Carolina at Chapel Hill, gave me access to his university's compute cluster, which returned large CPU workloads to reach and, more than anything, bought the time needed to finalize and test the super-graph idea; without it the RPE1 HYPHAE co-expression graph would have been substantially weaker. To develop and test RHIZO on a modern GPU I paid out of pocket for cloud compute on an RTX 5090, and on 10 July I regained access to the Human Technopole cluster and the data held on it. The honest consequence is that most of the load-bearing, large-scale results in this thesis were generated in the four days before the deadline. The setback was overcome, but not without cost: a number of investigations it made impossible are still outstanding, and they are set out in the conclusion. MYCELIUM is a methodology, and a methodology outlives the loss of any one cluster; the core findings and claims stand as they are, and the work they open is straightforward to carry forward, which is exactly what will happen.

Table of Contents

Acknowledgements.....	3
Author's Note.....	4
Table of Contents	5
1.1 — The Gene Regulatory Network.....	17
1.1.1 — What a gene regulatory network is.....	17
1.1.2 — Regulators, hubs, and terminal effectors.....	17
1.1.3 — Why the GRN is one of the great unsolved problems	18
1.1.4 — Applications of a faithful GRN	18
1.1.5 — Scale and complexity.....	19
1.2 — Gene Regulatory Networks: Topology and Macro-Properties	19
1.2.1 — GRNs as directed, weighted graphs	19
1.2.2 — Topological signatures of biological GRNs.....	19
1.3 — The Central Problem: Predicting Perturbation Responses	20
1.3.1 — From regulatory wiring to perturbation prediction	20
1.3.2 — The space of possible perturbations	21
1.3.3 — What a perturbation response means computationally	22
1.3.4 — Target identification as a prediction problem.....	22
1.4 — Measuring Perturbations: CRISPR Functional Genomics.....	23
1.4.1 — CRISPR perturbation modalities.....	23
1.4.2 — From classical genetics to pooled screens	23
1.4.3 — Perturb-seq: pairing genotype to transcriptome at single-cell resolution.....	24
1.4.4 — Genome-scale Perturb-seq: the Replogle 2022 datasets	24
1.5 — Single-Cell RNA-seq: Signal and Noise	25
1.5.1 — Technology overview	25
1.5.2 — Technical noise and systematic variation	26
1.5.2.1 — Ambient RNA and doublets	26
1.5.2.2 — Library size and sequencing depth.....	27
1.5.2.3 — Batch effects.....	27
1.5.2.4 — Systematic variation	27
1.5.3 — Knockdown efficiency and the escaper problem	28
1.6 — Inferring and Pruning Gene Regulatory Networks	29
1.6.1 — Inferring a candidate GRN from data	29
1.6.1.1 — Established inference methods.....	29
1.6.1.2 — Co-expression versus causal inference.....	29
1.6.2 — The dense-to-sparse problem	29
1.6.2.1 — Why prune: dense inference and over-smoothing.....	29
1.6.2.2 — Top-weight pruning.....	30
1.6.2.3 — K-nearest-neighbor pruning	30

1.6.2.4 — Toward a biologically grounded pruning	30
1.7 — From Graph to Prediction: Graph Neural Networks	32
1.7.1 — Message passing and why topology matters.....	32
1.7.2 — Models for perturbation-response prediction	32
1.7.3 — SPECTRA and why graph quality drives performance	32
1.8 — The MYCELIUM Pipeline and Thesis Roadmap	33
1.8.1 — The pipeline at a glance	33
1.8.2 — What each module contributes.....	33
1.8.2.1 — SPORE	33
1.8.2.2 — SHROOM	33
1.8.2.3 — FUNGI.....	34
1.8.3 — Scope and contributions of this thesis.....	34
1.8.4 — Roadmap of the thesis	34
2.1 — Why Data Preparation Decides Everything Downstream.....	36
2.1.1 — Data quality as the ceiling on inference	36
2.1.2 — Data leakage: the subtler, more dangerous failure	36
2.1.3 — The case for a generalizable, reproducible preparation framework	37
2.2 — SPORE: A Modality-Aware Preparation Pipeline.....	37
2.2.1 — Design principles and pipeline shape	37
2.2.2 — Ingestion and detection (Phases 0–1)	37
2.2.3 — Cell-level quality control (Phases 2–4)	38
2.2.4 — Perturbation-aware filtering: escapers and knockdown efficiency (Phase 5)	39
2.2.5 — Gene-level triage (Phase 6)	40
2.3 — Splitting Without Leakage (Phase 7)	41
2.3.1 — The zero-shot split: hold out whole perturbations, not random cells.....	41
2.3.2 — Stratification by transcriptional severity	41
2.3.3 — Three files, three roles, and how leakage is prevented end-to-end	41
2.4 — Preparing the Training Substrate (Phases 8–12)	43
2.4.1 — Feature selection on the training split (Phase 8)	43
2.4.2 — Normalization (Phase 9)	44
2.4.3 — Confounder mitigation and batch correction (Phase 10)	44
2.4.4 — Cell-line separation (Phase 11)	45
2.4.5 — Metacell aggregation (Phase 12)	45
2.5 — CHITIN: Causal Heuristics Isolating Transcriptomic Intervention Noise.....	46
2.5.1 — The correction target: a convergent, shared axis.....	47
2.5.2 — Why it is hard: a confound made of the same biology	47
2.5.3 — CHITIN as a family of corrections	47
2.5.4 — Judging a correction without being deceived by it.....	48
2.6 — From Clean Data to a Candidate Network	48

3.1 — What a dense GRN is, and where it sits in MYCELIUM	50
3.1.1 — The dense parent as the pipeline's quality ceiling.....	50
3.1.2 — Nodes, directed weighted edges, and the full candidate edge space	50
3.1.3 — Why the generator stays dense	50
3.1.4 — The worked substrates: RPE1 and the VCC H1-hESC data.....	50
3.2 — Two schools of thought: association and causation	51
3.2.1 — Co-expression (association) versus causal (interventional) inference	51
3.2.2 — Why GRN inference is benchmarked, not solved	52
3.3 — HYPHAE: Heuristic Yield of Parallel Hub Assembled Edges	52
3.3.1 — What HYPHAE is	52
3.3.2 — Method: tree-ensemble feature importance	53
3.3.3 — The HYPHAE realization: bootstrapping, early stopping, and the tuned learner.....	53
3.3.4 — Engineering: parallel inference and consolidation	54
3.3.5 — What it captures, and HYPHAE as a valid standalone GRN.....	54
3.3.6 — Cost, hardware, and the bridge to a causal generator.....	54
3.4 — SHROOM: Self-training Heuristic for Regulatory Orientation Of Mechanisms	54
3.4.1 — What SHROOM is	54
3.4.2 — Method: PSGRN self-training.....	55
3.4.3 — The interventional signal: why SHROOM is causal.....	55
3.4.4 — Engineering: GPU pseudolabels and single-pass scoring	55
3.4.5 — What it captures, and SHROOM as a valid standalone GRN.....	56
3.4.6 — Perturbation-bounded coverage, and the bridge to fusion.....	56
3.5 — The super-graph: merging association and causation	57
3.5.1 — Motivation: complementary generators and the wisdom of crowds.....	57
3.5.2 — The MYCELIUM super-graph: best of both, compensating weaknesses.....	57
3.5.3 — Ensemble technique: rank normalization and consensus operators.....	58
3.5.4 — Merge order: prune-then-fuse versus fuse-then-prune	59
3.6 — Evaluating a dense generator, and why the real test is downstream	59
3.6.1 — Two evaluation axes: biological and causal precision	59
3.6.2 — The real test is the downstream model.....	60
4.1 — Motivation.....	62
4.2 — Design Philosophy: Dataset Agnosticism and Configurable Bounds.....	62
4.2.1 — Data-derived diagnostic bounds (the default)	62
4.2.2 — Literature-sourced fallback bounds (FUNGI-supplied).....	63
4.2.3 — User bounds-override mode (fully custom).....	63
4.2.4 — Mix-and-match, with RPE1 as the worked example	63
4.3 — Biological Topology Targets.....	63
4.3.1 — α — Power-law degree exponent	63
4.3.1.1 — Biological Context.....	63

4.3.1.2 — Optimization Rationale.....	64
4.3.1.3 — Quantitative Identity	64
4.3.2 — <i>Gini</i> — Regulatory Inequality	66
4.3.2.1 — Biological Context.....	66
4.3.2.2 — Optimization Rationale.....	66
4.3.2.3 — Quantitative Identity	66
4.3.3 — $Gini_{in}$ — Regulatory-input inequality	68
4.3.3.1 — Biological Context.....	68
4.3.3.2 — Optimization Rationale.....	68
4.3.3.3 — Quantitative Identity	68
4.3.4 — S_{max} — Hub Fraction	69
4.3.4.1 — Biological Context.....	69
4.3.4.2 — Optimization Rationale.....	69
4.3.4.3 — Quantitative Identity	69
4.3.5 — Q — Modularity.....	70
4.3.5.1 — Biological Context.....	70
4.3.5.2 — Optimization Rationale.....	70
4.3.5.3 — Quantitative Identity	71
4.3.6 — C — Clustering Coefficient.....	71
4.3.6.1 — Biological Context.....	72
4.3.6.2 — Optimization Rationale.....	72
4.3.6.3 — Quantitative identity	72
4.3.7 — ρ — Degree assortativity.....	73
4.3.7.1 — Biological Context.....	73
4.3.7.2 — Optimization Rationale.....	74
4.3.7.3 — Quantitative Identity	74
4.3.8 — Reciprocity — Directed link reciprocity	74
4.3.8.1 — Biological Context.....	74
4.3.8.2 — Optimization Rationale.....	75
4.3.8.3 — Quantitative Identity	75
4.4 — The Utopian Loss and Bound Calibration	76
4.4.1 — The utopian ideal and what limits it: pruning-only reachability.....	76
4.4.2 — The λ_{eff} estimator	77
4.4.3 — The diagnostic probe suite (default, data-derived bounds)	77
4.4.3.1 — The power-law exponent (α)	78
4.4.3.2 — The degree-inequality probes ($Gini_{out}$ and $Gini_{in}$).....	78
4.4.3.3 — The hub-fraction bound (S_{max})	78
4.4.3.4 — The clustering probe (C)	78
4.4.3.5 — The assortativity probe (ρ).....	79

4.4.3.6 — The literature-prior targets (reciprocity, and the substrate-limited Q)	79
4.4.4 — λ — user-controlled target density	79
4.4.5 — The utopian loss function.....	79
4.5 — The DASH Scoring Kernel.....	80
4.5.1 — The full DASH kernel.....	81
4.5.2 — Priors and hyperparameters: why the kernel mixes fixed and swept terms	82
4.5.3 — Source-quantile edge weighting $W_q(\beta)$	82
4.5.4 — Feed-forward-loop topology term (δ).....	82
4.5.5 — Biological priors (fixed).....	83
4.5.5.1 — Perturbation-impact prior (ψ)	83
4.5.5.2 — Pleiotropy priors: source χ_s and target χ_t	83
4.5.5.3 — Causal-output prior (ρ_c)	83
4.5.5.4 — Reachability-diffusion prior (v / RDF)	84
4.5.6 — m_{intra} — intra-community edge boost.....	84
4.5.7 — Source-Conditioned Bridge Effective Resistance (SCBER)	84
4.5.7.1 — Motivation: inter-module bridge preservation	84
4.5.7.2 — Effective resistance via randomized Laplacian sketch.....	85
4.5.7.3 — Source-conditioned formulation	85
4.5.8 — Motif repair	86
4.5.9 — Hyperparameter search space (the swept set).....	86
4.6 — Integration of GRN Signal	87
4.6.1 — Stability gate.....	88
4.6.2 — Causal-edge refinement via W_q substitution.....	88
4.7 — The Optimization Pipeline	88
4.7.1 — Data ingestion.....	88
4.7.2 — Diagnostic calibration.....	89
4.7.3 — Candidate-pool construction and dense-parent metrics	89
4.7.4 — Expansive search (Sobol)	89
4.7.5 — Refinement.....	91
4.7.6 — Champion selection	92
4.7.7 — Graph output	93
4.8 — Beyond Biological Topology: Application-Specific Optimization	93
4.8.1 — The dual objective: biological shape first, then a secondary target	93
4.8.2 — Use case: a downstream perturbation-prediction model (SPECTRA).....	94
4.8.3 — Use case: the topology-aware arbiter (RHIZO).....	94
4.8.4 — Use case: an industry-standard model (GEARS)	95
4.8.5 — Why the DASH kernel enables this.....	95
4.9 — Baseline Pruning Methods	95
4.9.1 — Top-K by weight.....	95

4.9.2 — K-nearest-neighbors graph	96
4.10 — Datasets and Experimental Setup	96
4.11 — From Dense Parent to Curated Graph	97
5.1 — Why the pipeline ends in a predictive model.....	99
5.1.1 — A graph's worth is a model's held-out prediction.....	99
5.1.2 — Two instruments, one claim: MOREL and RHIZO	99
5.2 — MOREL: Mechanistic Ontology-free Regulatory Estimation by Linear-propagation.....	100
5.2.1 — What MOREL is: a directed linear-propagation instrument.....	100
5.2.2 — The load-bearing design: the only path runs through the edges.....	101
5.3 — MOREL: how it forms a prediction	101
5.3.1 — Perturbation injection and directed propagation.....	101
5.3.2 — Per-gene ridge readout on the propagated features	102
5.3.3 — Training on the specific residual.....	103
5.4 — RHIZO: Regulatory Hop Inference, Zero Ontology	103
5.4.1 — The arbiter: from a curated graph to a perturbation prediction	103
5.5 — RHIZO: Zero Ontology	104
5.5.1 — What "zero ontology" means, and what it excludes.....	104
5.5.2 — Why graph-agnosticism makes RHIZO the clean instrument.....	104
5.5.3 — Alignment with the MYCELIUM philosophy.....	104
5.6 — RHIZO: Architecture, a lightweight message-passing model.....	105
5.6.1 — Message passing over the regulatory graph (the FAGCN basis)	105
5.6.2 — Propagating a perturbation from its source, and the $\psi(0)=0$ invariant.....	105
5.6.3 — Graph-derived edge features.....	106
5.6.4 — Training objective and the lightweight design.....	107
5.7 — RHIZO: Capabilities, the switchable components.....	107
5.7.1 — Topology-conditioned readout (FiLM).....	107
5.7.2 — Over-squash-aware readout.....	108
5.7.3 — Directed regulatory edge features (what a co-expression graph lacks).....	108
5.8 — Evaluation regimes: in-distribution and zero-shot	109
5.8.1 — rung1: in-distribution graph quality	109
5.8.2 — zeroshot: the held-out coverage boundary	109
5.8.3 — What the models are scored on.....	110
5.9 — From predictive models to verdict	110
6.0 — Overview: From a Constructed Substrate to a Downstream Verdict	113
6.1 — The Two Testbeds: Dense versus Sparse Perturbation Coverage.....	113
6.2 — The Evaluation Lens: Why Regulator-Specificity Is the Decision Axis	114
6.2.1 — The metrics, defined.....	114
6.2.2 — Why de-confounding is not optional: the CHITIN evidence	116
6.3 — Building the Substrate: From Single Cells to the Super Graph.....	116

6.3.1 — The generators, and the fusion	116
6.3.2 — The graphs are good intrinsically, on both datasets	117
6.4 — The Crown Jewel: Substrate Biology Decides Perturbation-Specificity	119
6.5 — Why the Super Win Is Real: The Mechanism Controls	122
6.6 — FUNGI's Distinct Contribution: A Legible Graph at Predictive Parity	124
6.6.1 — The pruning wash, stated plainly	124
6.6.2 — Legibility at parity, and a longer hub hierarchy	124
6.6.3 — FUNGI is literature-faithful on both datasets, and beats every structure-blind graph	126
6.7 — The Result Generalizes: Density, Cell Line, and an Independent Instrument.....	127
6.8 — Attribution: The Graph Carries the Signal, and RHIZO Reads It.....	128
6.9 — Honest Boundaries: Where the Advantage Ends.....	129
6.10 — RHIZO Capacity and Architecture: The Pruner Question	131
6.11 — Discussion Synthesis: The Two-Pillar Case, Consolidated	132
7.1 — What MYCELIUM Delivers	134
7.2 — The Recipe for the Best GRN	134
7.3 — Boundaries and the Next Step	134
References.....	135

Thesis Summary

Mapping Your Cellular Expression: Learning Interventional Unified-regulatory Maps

Introduction

A gene regulatory network (GRN) is the map of which genes control which others, and it is the object a biologist reaches for to reason about how a cell will respond when a gene is switched off. Single-cell CRISPR screens, known as Perturb-seq, now measure the transcriptome-wide response to knocking down thousands of individual genes, one gene at a time, across hundreds of thousands of cells. That capability turns a long-standing question into a concrete prediction problem. Given a regulatory map of a cell, can we predict how its transcriptome will respond to a perturbation we have not yet run. A method that does this well lets a biologist prioritise experiments in silico before spending time and reagents at the bench, and it is the target the new class of "virtual cell" models is built to hit.

Three hard facts make the problem difficult. There is no ground-truth GRN to train against, so every inferred network is an estimate that can only be judged indirectly. The inference methods that produce those estimates return dense, noisy graphs that a downstream model cannot use as they stand. And the perturbation data is dominated by a large shared response, driven in these data by cell-cycle arrest, that imitates regulatory biology and inflates any naive accuracy score.

The problem

This thesis takes the three problems together and asks one question. Can interventional single-cell data be turned into a single curated regulatory map whose biological composition and topology measurably change what a downstream model predicts, judged by an evaluation that the shared-response confound cannot fool. The practical goal follows directly. Build the best GRN the data allows, and then prove that the graph, rather than the model reading it, is what carries the result.

The MYCELIUM pipeline

The contribution is MYCELIUM (Mapping Your Cellular Expression: Learning Interventional Unified-regulatory Maps), a complete and reproducible route from raw single-cell perturbation counts to a biologically calibrated GRN and a downstream test of that GRN. It runs end to end, and it is deliberately modular. The stages are designed to work together, and each one is also a standalone tool that solves its own problem, so a user can take only the piece they need. Someone who wants a clean dense co-expression graph can run HYPHAE on its own. Someone who has any dense graph and wants a usable, biologically faithful topology can point FUNGI at it. The pipeline is the recommended path, and the parts are independently useful.

SPORE prepares the substrate, and it is the quiet workhorse the rest of the pipeline stands on. It cleans the raw single-cell data, recovers the perturbation coverage that naive filtering throws away, and builds gene-disjoint train and test splits with a verified leakage firewall, so that no gene seen in training appears in evaluation. That firewall is what makes every downstream number trustworthy rather than a memorization artifact.

SHROOM infers the causal graph. It self-trains a classifier on the measured perturbation responses, so its edges point from a regulator to the genes that actually shift when that regulator is knocked down. Because a gene earns out-edges only by being perturbed, this graph is highly regulator-specific and, by construction, covers only the perturbed genes. HYPHAE infers the complementary co-expression graph across the whole panel using tree-based regression. It contributes broad coverage of every gene, and on its own it carries no directed regulatory specificity, which is exactly why fusing it with a causal core, rather than using it alone, is the winning move.

FUNGI is the first of the two central characters. It prunes a dense graph down to a working topology, and it does so against explicit, measurable targets rather than a heuristic threshold. It optimizes the pruned graph to sit inside the literature-reported bands for hub concentration, edge reciprocity, and degree assortativity, which turns "network quality" from a vague

aspiration into an objective a method can be held to. FUNGI is substrate-agnostic, so it prunes a causal graph, a co-expression graph, or a fused graph with the same machinery.

The Super graph is the second central character and the headline artifact. It fuses the SHROOM causal core with the HYPHAE co-expression scaffold, so a single graph carries the causal core's regulator-specificity and the scaffold's whole-panel coverage at the same time. This composition is the thesis's answer to how to build the best GRN available from perturbation data.

Two evaluators close the loop. RHIZO is a directed graph neural network that reads a graph and predicts a perturbation response, and it is the model held fixed while the graph is swapped, so any change in prediction is attributable to the graph. MOREL is a deliberately simple, node-feature-free propagator that carries no learned embeddings, so the graph is the only channel it can use. MOREL is cheap to run and it isolates one question cleanly, namely whether a graph's topology carries usable signal at all. The whole pipeline was developed and run on a single consumer workstation, whereas the perturbation-aware inference it builds on was demonstrated on large compute clusters, which widens who can actually use it.

Results

The best GRN is the Super graph, and the evidence is consistent across two very different datasets and two independent evaluators. At matched edge count and in-distribution, every causal-containing graph, and the fused Super graph above all, encodes regulator-specific signal that pure co-expression categorically lacks. On the de-confounded axis built to strip the shared response, the causal and Super graphs are strongly positive while every co-expression graph sits at or below zero. The result holds on a dense-coverage RPE1 screen and on a stem-cell screen with roughly ten times sparser coverage, so it is a property of the substrate rather than of one dataset.

Control experiments show the advantage is genuine regulatory structure. Scrambling the targets while preserving degree, and reversing edge directions, both collapse the advantage to the co-expression floor, while binarising the edge weights leaves it intact, so the signal lives in the specific directed wiring and its topology rather than in degree or fine weights. Read by RHIZO, the Super graph clears, in-distribution, the simple linear baseline that recent deep perturbation models have repeatedly failed to beat.

FUNGI earns its place on interpretability. On raw accuracy the choice of pruning method is a wash, and that parity survives a doubled-capacity model and a full architecture sweep, so it is a real property rather than an under-powered comparison. FUNGI is the only pruner whose graph lands inside the literature topology bands on both cell lines, which delivers a biologically legible network that predicts as well as an opaque one. MOREL confirms the same story from the other direction, since FUNGI's real wiring beats a degree-matched shuffle of itself and every structure-blind graph when topology is the only signal available.

The boundaries are stated plainly. At held-out perturbations a causal graph has no out-edges to offer and every method converges to the linear-baseline ceiling, which is the current field-wide result rather than a shortcoming of this pipeline. Additional coverage buys breadth of raw prediction but not sharper regulator-specificity, a divergence that itself confirms why the de-confounded evaluation was necessary.

Conclusion

MYCELIUM delivers a complete, reproducible, and modular pipeline that converts interventional single-cell data into one curated, topologically faithful regulatory map, together with a downstream demonstration that the map's composition improves perturbation-specific prediction on two cell lines and two evaluators. The recipe for the best GRN is concrete. Fuse a specific causal core with a broad co-expression scaffold into a Super graph, and prune it to literature-faithful topology with FUNGI. The transferable contributions are the framing of GRN quality as measurable topological faithfulness, the fused Super composition principle, the

leakage-safe substrate construction, and a de-confounded evaluation that catches its own confounds, all achieved on a single workstation. The natural next step, held back only by available compute, is a causal graft that gives held-out genes out-edges and carries the causal advantage from the in-distribution regime into true zero-shot generalization.

CHAPTER 1

Introduction & Background

How a cell's genes control one another, why that regulatory wiring is so hard to reconstruct, and why the shape of the network is the property that matters.

1.1 — The Gene Regulatory Network

1.1.1 — What a gene regulatory network is

A gene regulatory network (GRN) is the directed graph that describes how the genes of a cell control one another's expression. Its nodes are genes, and a directed edge from gene *A* to gene *B* asserts that the product of *A* changes how much of *B* the cell produces. The gene that exerts the effect is the regulator, and the gene that receives it is the target ¹. What an edge acts on is the target's expression level, the amount of messenger RNA the cell transcribes from that gene, and the combined expression levels of every gene are what make a liver cell a liver cell and a stem cell a stem cell ². The GRN is the control layer that sets those levels.

An edge stands for a physical regulatory mechanism rather than a mathematical abstraction. In the canonical case the regulator is a transcription factor, a protein encoded by the regulator gene that binds specific cis-regulatory DNA sequences near the target and either recruits or blocks the machinery that transcribes it ³. Binding that promotes transcription makes the edge activating, and binding that represses transcription makes it repressing ³. Regulation also runs through indirect and non-transcriptional routes, including signaling intermediates and control that acts after transcription, and a single GRN edge summarizes the net regulatory effect of one gene on another whatever the precise molecular path between them ¹.

Because an edge stands for a real mechanism, the properties of that mechanism carry into the graph, and a model that discards them loses information the cell actually uses. Edges are signed, because an activating and a repressing interaction of equal strength drive the target in opposite directions, so the sign is part of the data rather than an annotation added afterward ¹. Edges are weighted, because regulation is graded rather than all-or-nothing, and a dominant regulator can set most of a target's expression while a weak one only nudges it ¹. Edges are directed, because the fact that *A* regulates *B* says nothing about whether *B* regulates *A*, and most regulatory relationships act in a single direction ¹.

A gene is rarely only a regulator or only a target. The same gene usually receives edges from its own upstream regulators while sending edges to the genes downstream of it, so regulator and target are roles a gene plays on particular edges rather than fixed identities ¹. Two genes can regulate each other directly, longer cycles of mutual regulation are common, and these feedback loops are part of what lets a fixed wiring diagram produce the dynamic, self-stabilizing behavior of a living cell ¹. The behavior of a cell emerges from this wiring taken as a whole, and reconstructing it at the level of the entire network, rather than one interaction at a time, is the central aim of this thesis.

1.1.2 — Regulators, hubs, and terminal effectors

Regulatory influence is distributed very unevenly across a GRN. A small minority of genes, the transcription factors, act as hubs that reach many targets at once ⁴. A single transcription factor can regulate from tens to several hundred targets, and the most connected of them sit at the head of an entire cell-type or developmental program ⁴. The core regulators of embryonic stem cells make the point concrete, where a handful of factors such as OCT4, SOX2, and NANOG together occupy and control a large share of the genes that maintain the stem-cell state ⁵. Changing the activity of a hub shifts a whole coordinated program of downstream genes rather than one gene's output, which is why hubs are the genes whose control matters most for moving a cell from one state to another ².

At the other end of the distribution sit the terminal effectors. These genes, including many that encode structural, metabolic, and housekeeping proteins, receive regulatory input but send out little or no regulation of their own ³. They are the genes the network acts on rather than the genes it acts through. Transcription factors make up only a small share of the genome, on the order of 1,600 of the roughly 20,000 protein-coding genes, so most genes sit toward the effector

end of the spectrum ³. The result is a sharp asymmetry, a few genes of very high out-degree and a long tail of genes with almost none.

This hierarchy is real but not rigid, and the difference matters for how the network must be inferred. A gene's role depends on cellular context, so a factor that dominates as a hub in one cell type or condition can be largely inactive in another, because the set of edges that are actually live changes with the state of the cell ². Roles can also be conditional within a single network, where a gene serves as an effector of one program while acting as a local regulator inside another ². What carries forward into the rest of this thesis is the shape of the distribution rather than the label on any one gene. That hub-dominated, heavy-tailed structure is the clearest signature separating a genuine GRN from a random graph, and recovering it from noisy inferred data is what the FUNGI stage of this work is built to do.

1.1.3 — Why the GRN is one of the great unsolved problems

For a given cell type there is, in principle, one true GRN, the complete and correct set of regulatory edges, with their signs and strengths, that governs that cell. That network has yet to be fully mapped or measured for any human cell type ¹. The field can resolve fragments of it, individual regulatory circuits and condition-specific modules worked out in detail, and it can build genome-wide estimates of it, but a faithful and complete regulatory map for even a single human cell type does not yet exist ⁶.

Two obstacles keep the true network out of reach. The first is the sheer size of the search. A human cell expresses on the order of twenty thousand genes ⁷, so the number of candidate directed edges runs to hundreds of millions, and the number of ways those edges can combine into the multi-gene regulatory logic a cell actually uses is far larger still, well beyond what any direct experiment can test ⁷. The second obstacle is that regulation cannot be read off a static genome. It has to be inferred from how expression changes when the system is disturbed ⁸, and that signal is partial, noisy, and tangled with the technical and biological structure of the data.

Because the true network is unknown, GRN reconstruction is an inference problem rather than a lookup problem, and that shapes how the field measures progress. Every method returns an estimate, and that estimate can be judged only indirectly, against the partial set of experimentally validated interactions already known, against measurements held out from the data, and against how well the network predicts expression changes it was not trained on ⁶. Benchmarks built on real data show that this evaluation is genuinely hard ⁶, and that methods which look strong on simulated data do not always carry over to real cells ⁹. A network this difficult to reconstruct is only worth the effort because a faithful one would be so useful, and the uses are the subject of the next section.

1.1.4 — Applications of a faithful GRN

A true GRN is valuable because it turns a list of genes into a model that can be acted on. The clearest case is target selection in drug discovery. Without a map of the regulatory wiring, choosing which gene to switch off or on to push a diseased cell back toward a healthy state is largely a matter of trial and error, whereas a GRN points to the regulators whose manipulation would propagate through the right downstream program ¹⁰. The same map narrows a genome-scale question to a short list of mechanistically motivated candidates.

A true GRN also removes much of the experiment. Given the wiring, a computational model can predict how a cell's expression shifts when a chosen gene is silenced, so a researcher can rank thousands of candidate genes in software and take only the most promising few to the bench instead of testing each one experimentally ¹¹. This is the use the downstream model of this thesis is ultimately built to serve.

In disease genetics, a GRN supplies the mechanism that statistical association leaves out. Genome-wide association studies link genetic variants to traits and diseases, but the large majority of disease-associated variants fall outside protein-coding regions and act by changing

regulation rather than protein sequence ¹². A GRN connects such a variant, sitting in or near a regulator, to the downstream genes and programs its mis-regulation would disturb, which turns a statistical association into a testable mechanistic hypothesis ¹⁰.

The hub structure that makes a GRN hard to reconstruct is also what makes it powerful for cell engineering. Converting one cell type into another can often be driven by forcing the activity of a small number of master regulators, and a GRN is what identifies which regulators sit at the top of the target cell's program ^{5,13}. The common thread across all of these uses is that a regulatory map converts description into prediction. A reconstruction accurate enough to support them, for even one cell type, would be a substantial advance for both basic biology and its translation into therapies.

1.1.5 — Scale and complexity

The difficulty of reconstruction scales sharply with the size of the network. A well-characterized signaling pathway might involve on the order of fifty genes, small enough that the wiring can be drawn by hand and reasoned about one edge at a time. The regulatory graph of a whole human cell, over roughly twenty thousand expressed genes ⁷, is a different kind of object. The number of possible directed edges grows with the square of the number of genes, which takes the space of candidate edges from a few thousand for a pathway to hundreds of millions for a genome.

At genome scale, direct measurement cannot keep pace with that growth. Mapping the network edge by edge would mean resolving how each gene responds to a change in every other gene, a number of experiments that is infeasible even with high-throughput methods, and the readout would still be confounded by technical and biological noise ⁹. A full-genome GRN therefore has to be inferred from the expression data the field can realistically collect ¹⁴. That inferred object is not a random tangle, though. Even at genome scale a real regulatory network has a definite and recognizable architecture, and that architecture, the shape a correct GRN should have, is what the next section lays out.

1.2 — Gene Regulatory Networks: Topology and Macro-Properties

1.2.1 — GRNs as directed, weighted graphs

Treating the GRN as a graph gives a small set of numbers that describe its shape and let us reason about it quantitatively. The graph is a set of genes together with the directed, signed, weighted edges already described, and it is summarized by its adjacency, the table that assigns every ordered pair of genes a weight that is zero when no edge exists and otherwise records the strength of regulation from the first gene to the second. Two quantities read straight off that table carry most of what follows. A gene's out-degree is the number of genes it regulates, and its in-degree is the number of regulators acting on it.

The out-degree is the quantity that matters most for the rest of this thesis. Collected across all genes, the list of out-degrees, the out-degree sequence, is what distinguishes a biological network from a random one, because it records how regulatory output is shared among genes. A random graph spreads out-degree evenly, giving most genes a similar number of outgoing edges, whereas a biological network concentrates it on a few hub regulators and leaves the rest with almost none ⁴. The properties in the next section are, for the most part, statements about the shape of that out-degree sequence and the wiring patterns that accompany it.

1.2.2 — Topological signatures of biological GRNs

A real GRN differs from a random graph in a handful of measurable ways, and each of these topological signatures reflects something about how regulation is organized. They are stated here in general terms; each is formalized and turned into an explicit optimization target in Chapter 4, where FUNGI is built.

The first signature is a scale-free degree distribution. Out-degrees follow a heavy-tailed, roughly power-law shape, with most genes regulating few or no targets and a small number of hub regulators controlling very many⁴. This is the distributional form of the hub-and-effector asymmetry already described, and it is why a correct GRN cannot look like a network in which every gene has a similar number of outgoing edges. A second signature, disassortativity, concerns which genes the hubs attach to. Hubs tend to connect to low-degree targets rather than to one another, so the network avoids a tightly interconnected core of super-regulators¹⁵. A third signature is modularity, the organization of genes into communities that are densely connected within a functional program and sparsely connected between programs^{16,17}, which mirrors the way a cell runs distinct regulatory programs that are largely separable from one another. A fourth signature is the enrichment of small recurring circuits, above all the feed-forward loop, in which a regulator acts on a target both directly and through an intermediate¹⁸. The feed-forward loop occurs far more often than chance and shapes the timing and noise-tolerance of a response¹⁹, which is why its presence is a mark of genuine regulatory logic rather than random wiring. Taken together, these signatures are what a reconstructed GRN should exhibit to resemble the real thing, and matching them is the concrete objective Chapter 4 pursues.

Figure 1.1 contrasts this heavy-tailed scale-free shape with the Poisson distribution of a random graph on the same axes.

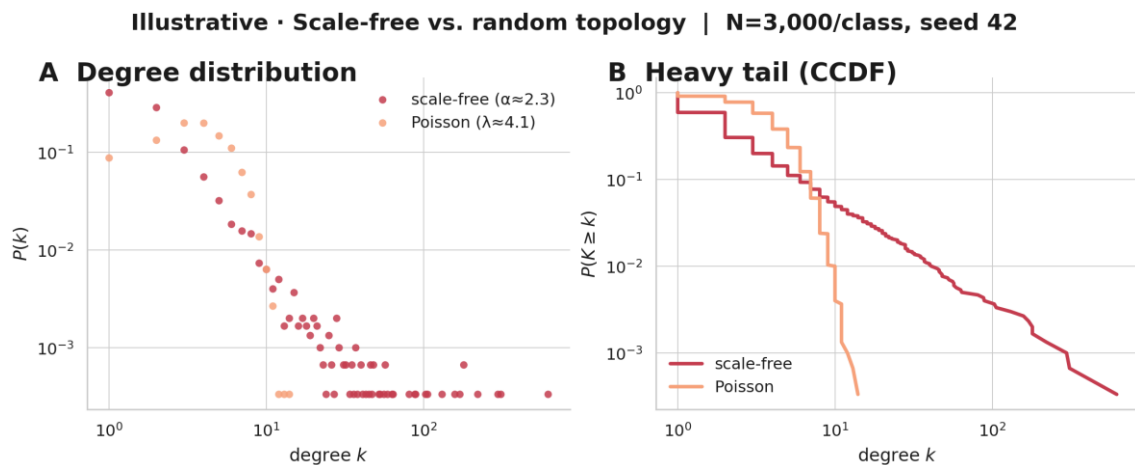


Figure 1.1 | Scale-free versus random network topology. Two simulated out-degree distributions on shared log-log axes, one scale-free (power-law, alpha near 2.3) and one random (Poisson) with a matched mean. The scale-free distribution has a heavy tail, where most genes regulate a few targets and a small number of hub genes regulate many. That heavy tail is the signature separating a biological gene regulatory network from a random graph. The data are illustrative.

1.3 — The Central Problem: Predicting Perturbation Responses

1.3.1 — From regulatory wiring to perturbation prediction

A cell's behavior is set by its regulatory wiring, so a faithful map of that wiring should let us do more than describe the cell. It should let us predict how the cell reacts when the wiring is disturbed. To study that reaction we first need a way to disturb the wiring on purpose. The basic move is to change the activity of a chosen gene, most often by silencing it so the cell makes little or none of its product, and then read out what happens to every other gene. This deliberate change to a gene's activity is a perturbation⁸. How a perturbation is applied at the bench is a separate question, taken up when we turn to the data; here the concept is what matters.

A GRN turns a perturbation into a prediction. Given the wiring and the identity of a perturbed gene, the effect of silencing that gene can be propagated along the edges leaving it, through its targets, and onward through the cascade, to estimate how the expression of the rest of the

genome shifts in response ¹¹. This is the central computational problem of the thesis. The input is a GRN together with a perturbation identity, and the output is the predicted change in expression across all genes, the perturbation response. A model that solves it can be asked what happens to a cell when any chosen gene is silenced, including genes that were never perturbed in any experiment ¹¹.

1.3.2 — The space of possible perturbations

The reach of such a model is what makes it worth building. The genome already presents one combinatorial explosion, the hundreds of millions of candidate regulatory edges among its genes. A second explosion sits on top of that one, the space of the perturbations themselves. A human cell has on the order of twenty thousand genes that could be silenced one at a time ⁷, roughly two hundred million distinct pairs that could be silenced together, and a far larger number of higher-order combinations. These combinations are not redundant, because genes interact, and the response to silencing two genes together is often not the sum of the two single responses, so a combination can expose regulatory relationships that neither single perturbation reveals ²⁰.

Only a thin slice of this space has ever been measured. Even the largest single-cell screens perturb on the order of one to a few thousand genes, and almost always one gene at a time ²¹. Measuring every single-gene perturbation, let alone every pair, is not feasible, and the informative combinations sit almost entirely in the unmeasured remainder. Closing the gap between what has been measured and what one would want to know is what a predictive model is for, since it generalizes from the measured slice to the rest of the space ¹¹.

Figure 1.2 sets the size of this space against how little of it any single screen can measure.

Illustrative · The combinatorial explosion | n=5,000-gene panel

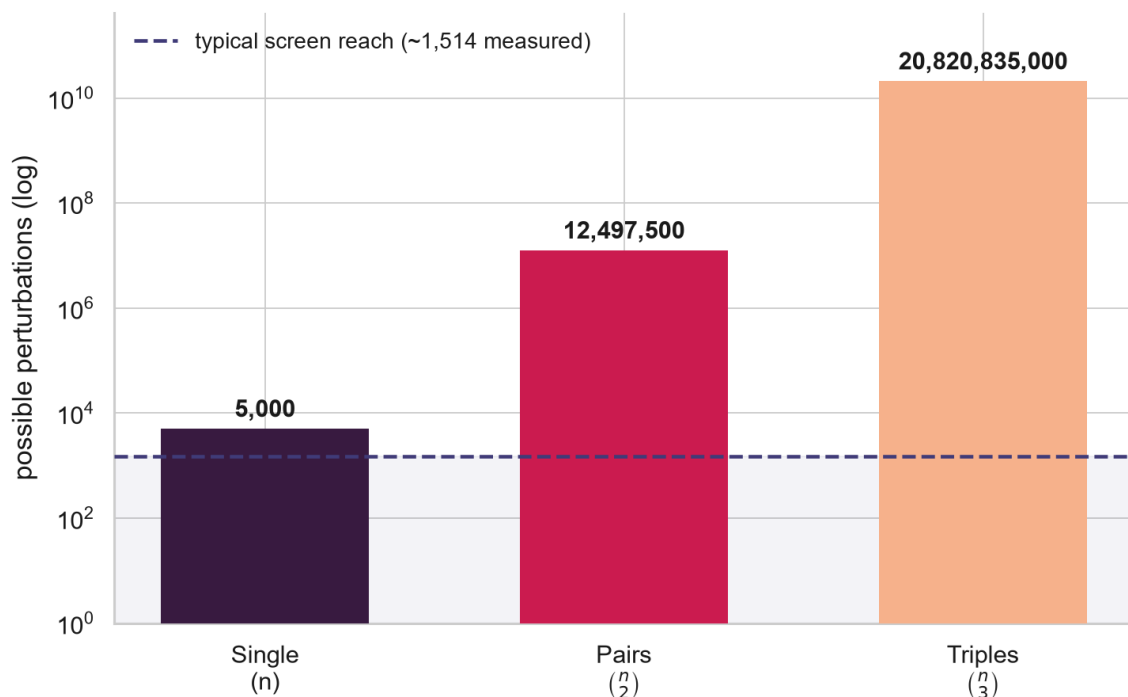


Figure 1.2 | The combinatorial explosion of possible perturbations. The number of candidate perturbations grows from single genes to pairs to higher-order combinations, and it quickly outruns what any single screen can measure (shaded band). This gap is why a predictive model is needed rather than exhaustive measurement. Counts are illustrative for a panel of size n .

1.3.3 — What a perturbation response means computationally

To make this a problem a model can be scored on, the perturbation response has to be written as something measurable. The quantity of interest is the shift in expression that the perturbation causes. For each gene, that shift is the difference between its expression in perturbed cells and its expression in matched control cells that were left alone, a quantity called differential expression ²². Taken across the genome, the response becomes a vector with one differential-expression value per gene, a full account of how the transcriptome moves when the target is silenced ²².

With that readout the problem is stated precisely. A model here means a computational function, fit to data, that maps an input to a predicted output. It receives a GRN and the identity of a perturbed gene and returns a predicted differential-expression vector, which is scored against the measured vector on perturbations held out from training ¹¹. Two families of model are in use. One learns the mapping directly from expression data, with no explicit network, inferring the response from the patterns seen across many measured perturbations ²³. The other propagates the perturbation over an explicit gene graph using a graph neural network, a model that passes information along a graph's edges so that each gene's predicted state is updated from its neighbors ^{11,24}. The two families differ in almost everything else but share one dependence, since both are only as good as the structure they are given, which returns the burden once more to the quality of the GRN.

Figure 1.3 states this computation in a few lines of code.

```
build_rpel_lfc.py
sc.pp.normalize_total(ad, target_sum=1e4)
sc.pp.log1p(ad)

def grp_mean(mask):
    return np.asarray(X[mask].mean(0)).ravel()

mu_ctrl = grp_mean(split == "control")
...
m = (split == sp) & (gene == gname)
S = grp_mean(m) - mu_ctrl    # response vector
```

Figure 1.3 | A perturbation response, defined computationally. The response of a perturbation is the mean log-normalised expression shift of its perturbed cells relative to matched control cells, taken across the gene panel. This differential-expression contract is the quantity every downstream stage predicts and is scored against.

1.3.4 — Target identification as a prediction problem

The decision that a faithful GRN ultimately supports, from choosing which gene to drug to reading the consequences of a disease-associated variant, comes down to which gene or genes to perturb in order to move a cell toward a wanted state. Set against the problem just defined, that decision is perturbation prediction run in reverse. Forward prediction asks what a chosen perturbation does. Target identification asks which perturbation would produce a chosen response ¹⁰.

This reversal is only useful when the forward prediction can be trusted over perturbations that no one has run. A model confined to the measured slice can rank only the genes that were already tested, a small and biased corner of the space. A model that generalizes can score any candidate gene, or any combination of genes, in software, and return the few whose predicted response comes closest to the goal ¹¹. Target identification then becomes a search across the full

space of possible perturbations rather than a lookup over the handful that happen to have been measured, which is the difference between re-confirming known targets and proposing new ones. The quality of that search inherits the quality of the prediction, and the quality of the prediction inherits the quality of the GRN, so producing a network good enough to search over is the prerequisite for all of it.

1.4 — Measuring Perturbations: CRISPR Functional Genomics

1.4.1 — CRISPR perturbation modalities

A perturbation stays a concept until there is a way to apply one, and the way used throughout this thesis is CRISPR interference. A guide RNA, a short programmable RNA sequence, directs a catalytically dead Cas9 protein to a chosen gene, where the bound complex physically obstructs the transcriptional machinery and represses the gene without cutting the DNA ²⁵. The result is a targeted, reversible reduction in the gene's expression, a knockdown, applied to whichever gene the guide is designed against ²⁵. Because the guide is only a sequence, the same machinery can be aimed at any gene by swapping the guide, which is what makes CRISPR interference programmable across the whole genome ²⁶.

CRISPR interference is one of three related ways to perturb a gene, and the other two are worth naming because the literature this thesis draws on uses all of them. In CRISPR knockout, an active Cas9 cuts the DNA at the target, and the cell's error-prone repair leaves small insertions or deletions that shift the reading frame and abolish the protein ²⁷. The choice between interference and knockout is a real tradeoff rather than a matter of taste ²⁸. Interference never touches the DNA sequence, so it leaves the gene, its folding, and the surrounding genomic and epigenetic context intact, which makes it the gentler and reversible option, but the knockdown it produces is partial and varies from strong to weak across genes and cells ^{26,28}. Knockout removes the gene product far more completely because the coding sequence itself is broken, but the double-strand break and its repair are a blunter intervention that can disturb the local genomic and epigenetic landscape, so its thoroughness comes with a risk of side effects that interference avoids ²⁸. Both are used, and both produce the loss-of-function perturbation this thesis models. The third modality, CRISPR activation, carries a transcriptional activator on the same dead Cas9 and so raises a target's expression instead of lowering it ²⁶. Activation is the tool for gain-of-function questions. This thesis does not use it, and it is named only to complete the picture, since interference and knockout lower a gene's activity, activation raises it, and all three are programmed by the same guide RNA.

1.4.2 — From classical genetics to pooled screens

Applying one perturbation at a time is the classical approach to genetics, where a single gene is disrupted and its consequences are studied in isolation. It is thorough and slow, and it does not scale to the twenty thousand genes of a human cell. Pooled screening removes the one-at-a-time bottleneck. A library of guide RNAs, one or more per target gene, is delivered into a population of cells, and the whole pool is grown and assayed together, so one experiment carries thousands of distinct perturbations at once ²⁹.

For a pooled screen to be interpretable, each cell has to carry a known single perturbation, and the design that guarantees this is delivery at low multiplicity of infection. The guide-carrying virus is added sparingly enough that, by the statistics of random uptake, most cells that take up any guide take up exactly one, and the small fraction that would receive two or more is kept low by construction ²⁸. The guide then integrates into that cell's genome and acts as a stable barcode that records which gene was hit, so sequencing the guide reads out the perturbation identity of every cell ²⁹.

What a screen can report depends on what is measured at the end. The earliest pooled screens used a fitness readout, sequencing the guides before and after a period of growth to find

perturbations that made cells proliferate or die, which identifies essential genes but yields only a single number per perturbation ²⁹. A transcriptomic readout is far richer, since it records the full expression state of each perturbed cell rather than its survival, and that richer readout is what turns a screen into data a regulatory-network method can use. A pooled screen can also place more than one guide per cell on purpose. Most screens, including the data used here, are single-perturbation, one guide and one silenced gene per cell. A combinatorial screen instead delivers two or more guides per cell to measure how genes act together and to expose the non-additive interactions that single perturbations cannot ²⁰. Combinatorial designs are the right instrument for mapping interaction landscapes, but this thesis works with single-perturbation data throughout, both because that is what the genome-scale resource provides and because the regulatory wiring it sets out to reconstruct is already identifiable from single-gene effects.

1.4.3 — Perturb-seq: pairing genotype to transcriptome at single-cell resolution

Perturb-seq is the method that supplies the transcriptomic readout at scale. It joins a pooled CRISPR screen to single-cell RNA sequencing, so that for every cell in the pool it records both the identity of the perturbation and the resulting genome-wide expression profile ^{8,30}. Each cell becomes one paired observation, a perturbation on the input side and a full transcriptome on the output side, which is exactly the paired data a perturbation-response model consumes.

Linking the two halves reliably is the technical crux. Early implementations read the perturbation off the expressed guide barcode, and later ones capture the guide directly alongside the messenger RNA so the assignment stays unambiguous at high throughput ³¹. Each captured transcript is also tagged with a unique molecular identifier before amplification, so the expression profile reflects true molecule counts rather than amplification artifacts ³². A Perturb-seq experiment therefore returns a large table of cells, each labeled with its perturbation and described by a quantitative expression vector.

1.4.4 — Genome-scale Perturb-seq: the Replogle 2022 datasets

Perturb-seq reached genome scale in the datasets this thesis uses. Replogle and colleagues applied CRISPR-interference Perturb-seq across more than two and a half million human cells, perturbing essentially every expressed gene and recording the transcriptome of each cell ²¹. Two cell lines anchor the resource and this work, RPE1 and K562, each screened over its panel of essential genes ²¹.

The two lines are chosen for complementary reasons. RPE1 is a near-diploid retinal pigment epithelial line, valued because its close-to-normal chromosome complement makes it a clean background for reading regulatory effects ²¹. K562 is a chronic myeloid leukemia line and one of the most heavily used human cell lines in all of functional genomics. It is a primary reference line for the ENCODE project, which mapped its transcription-factor binding, chromatin state, and accessibility in exhaustive detail ³³, and that depth of prior characterization is why K562 recurs as a reference background across the perturbation-screening and regulatory-genomics literature. Screening both a near-diploid and a deeply-characterized cancer line is part of what makes the Replogle resource the field's reference dataset for genome-scale Perturb-seq. Every downstream stage of this thesis is developed and evaluated on these panels, on the order of a thousand perturbed target genes read out against several thousand expressed genes per cell ²¹. The exact panels, gene counts, and preprocessing are set out where the data enters the pipeline; what matters here is that the input to everything that follows is genome-scale, single-cell, single-perturbation CRISPR-interference data.

Figure 1.4 places the RPE1 substrate used here against the smaller screens that came before it.

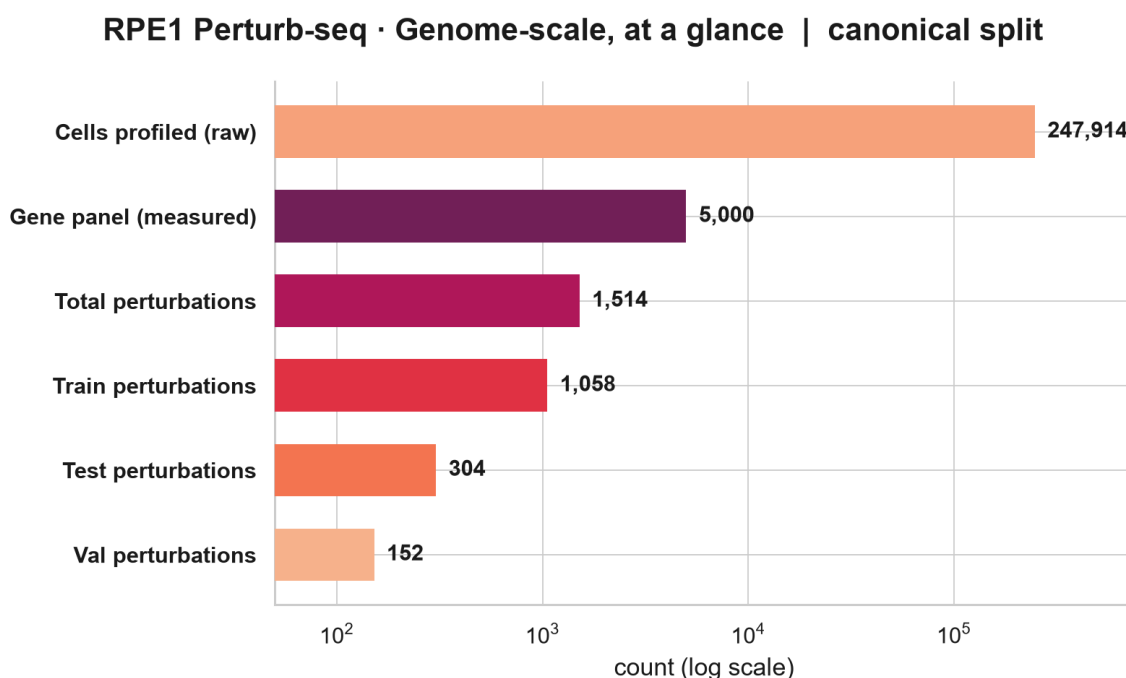


Figure 1.4 | The scale of genome-scale Perturb-seq. The RPE1 essential-gene substrate used throughout this thesis carries on the order of 1,500 perturbations read out against a panel of roughly 5,000 genes over millions of cells, set against the far smaller pooled screens that preceded it. Data from Replogle and colleagues (2022).

1.5 — Single-Cell RNA-seq: Signal and Noise

The value of Perturb-seq comes from its single-cell readout, and so do most of its difficulties. Reading each perturbed cell by single-cell RNA sequencing is what pairs a perturbation to a full transcriptome, but the measurement is shallow, noisy, and structured in ways that can masquerade as biology. This section covers what the measurement is and, more importantly, what makes its signal hard to trust, because those difficulties are what the SPORE stage of the pipeline is built to handle.

1.5.1 — Technology overview

In the droplet-based protocol the datasets use, individual cells are isolated into nanoliter droplets, each carrying a bead that tags every transcript it captures with a barcode identifying the cell of origin ³⁴. Reverse transcription and sequencing then produce, for each cell, a count of how many transcript molecules were captured for each gene ³⁴.

Two features of that readout carry downstream, and they pull in opposite directions. The first is precision in counting. Each captured molecule carries a unique molecular identifier, so transcripts are counted as distinct molecules rather than as amplified copies, which removes most amplification bias ³². The second is the scale of the output. The experiment produces a matrix of cells by genes, where each entry is the integer number of messenger-RNA molecules captured for that gene in that cell, and this count matrix is the raw material every later stage consumes.

The count matrix is severely sparse. A single cell yields only a limited number of captured molecules, so most genes register zero counts in most cells even when the gene is expressed, an effect known as dropout ³⁵. Sparsity is not a minor nuisance. It means any single cell is a noisy, incomplete snapshot of its own transcriptome, which is the root cause of several problems below and much of the reason cells are later aggregated before a network is inferred.

Figure 1.5 shows this dropout directly, plotting the fraction of zero counts against mean expression for every gene.

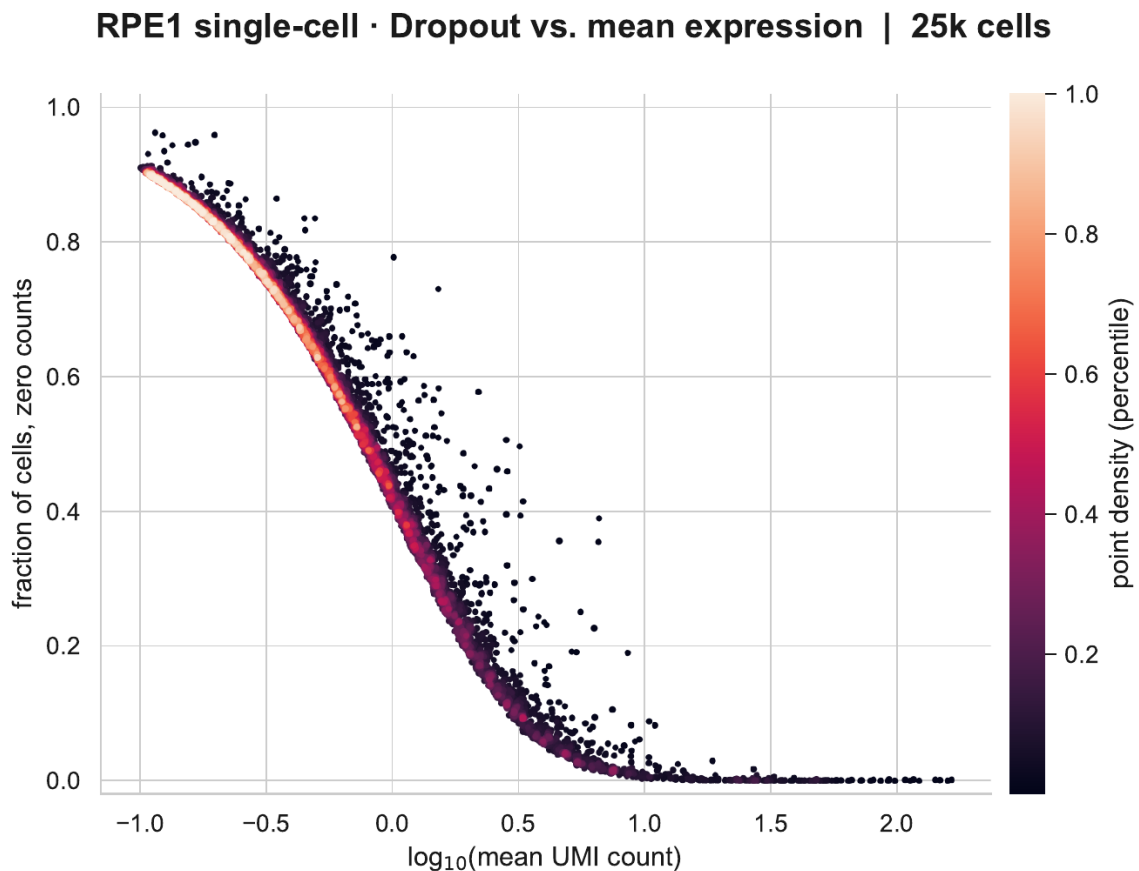


Figure 1.5 | Dropout and sparsity in single-cell RNA-seq. Each point is a gene, plotting its fraction of zero counts against its mean expression, with point colour showing local (kNN) density. The heavy dropout at low expression is the sparsity that motivates the denoising and aggregation steps of preprocessing.

1.5.2 — Technical noise and systematic variation

The count matrix carries two kinds of unwanted variation, and telling them apart organizes the rest of this section. The first is random technical noise, which corrupts individual measurements independently and can be modeled and averaged down ³⁵. The second is systematic variation, coherent structure shared across many cells that is not the signal of interest and, because it is correlated rather than random, is not removed by averaging ³⁶. The next three subsections cover the main sources of random technical noise, and the fourth treats systematic variation, which is the harder and more consequential of the two.

1.5.2.1 — Ambient RNA and doublets

The first kind of random technical noise comes from the droplet chemistry itself, in two forms. Ambient RNA is cell-free transcript, released by lysed or dying cells, that floats in the input suspension and is captured alongside a droplet's real cell, so every profile is contaminated by a background of transcripts the cell never expressed ³⁷. Doublets are droplets that captured two cells rather than one, producing a single chimeric profile that mixes two transcriptomes and can masquerade as a spurious intermediate cell state ³⁵. Both are estimated and removed during quality control, and both matter here because a perturbation-response method reads the contaminated or chimeric profile as if it were one cell's genuine response.

1.5.2.2 — Library size and sequencing depth

Cells also differ in how deeply they were sequenced. Sequencing depth is the total number of transcript molecules recovered from a cell, its library size, and it varies widely from cell to cell for reasons unrelated to biology, including differences in how efficiently each cell's messenger RNA was captured and how many reads it received ³⁵. A gene can therefore show more counts in one cell than in another simply because that cell yielded more molecules overall, not because the gene is more highly expressed. Raw counts are consequently not comparable across cells and are normalized, typically by scaling each cell to a common total, before any expression is compared ³⁵. Normalization is routine but not free, since the scaling itself can distort the very differences a downstream method is trying to read.

1.5.2.3 — Batch effects

Beyond individual cells, whole groups of cells can differ for technical reasons. When samples are processed in separate sequencing lanes, on different days, with different reagent lots, or from different culture passages, the cells in each group carry a shared technical signature, a batch effect, that shifts their expression coherently relative to other groups ³⁵. Batch effects are dangerous precisely because they are shared, so they look like real, coordinated biological differences, and if a perturbation happens to be measured mostly in one batch, the batch signature and the perturbation signal cannot be separated after the fact. Careful experimental design and batch-correction methods reduce them, and they are the first concrete instance of the broader problem the next subsection defines.

1.5.2.4 — Systematic variation

Systematic variation is any coherent, non-random difference in expression between the cells being compared that does not come from the biological signal of interest, and a batch effect is only its most familiar special case ³⁶. In a perturbation experiment the comparison that matters is between perturbed cells and control cells, and systematic variation is the part of that difference produced by something other than the perturbation itself. It is signal in the statistical sense, coherent and reproducible, but it is the wrong signal.

The sources are several, and one of them is easy to overlook yet arguably dominant. Selection biases in which cells survive and are captured, differences in cell-cycle stage or growth arrest, viability differences, and any passage or batch structure that aligns with the perturbation all contribute ³⁶. On top of these sits the generalized stress state a cell enters simply by being perturbed. Silencing a gene, together with the selection and handling a screen imposes, pushes many cells into a shared stress-and-arrest response that appears as a large, coherent transcriptional shift common to many different perturbations rather than specific to any one of them ^{21,36}. The unfolded-protein response is a well-characterized example of such a program, coherent enough across perturbed cells to be dissected in its own right ³⁰. Because this stress signature is shared across perturbations, it is exactly the kind of coherent, off-target structure that a regulatory method can mistake for regulatory biology.

This is a field-level problem, not a quirk of one dataset. A systematic study across ten datasets, three technologies, and five cell lines found that the consistent transcriptional differences between perturbed and control cells are large enough that methods claiming to predict perturbation responses can score well simply by reproducing this systematic component, without capturing any regulator-specific effect ³⁶. A model can look accurate while learning the confound rather than the biology. For GRN inference the danger takes a different form with the same root, since edges inferred from a perturbed-versus-control contrast dominated by systematic variation reflect whatever coherent structure separates the two populations, which may have nothing to do with the regulatory wiring the method is meant to recover.

What makes systematic variation hard is that it is entangled with the real signal rather than separable from it. Random noise can be averaged away because it is uncorrelated with the

perturbation, but systematic variation is correlated with it, sometimes because the perturbation genuinely changes the confounder. Silencing a gene can slow the cell cycle, so a cell-cycle shift is at once a confound and a real downstream consequence of the perturbation ³⁶. A correction aggressive enough to remove the confounding part of such a shift will also remove the genuine regulatory part, and a correction gentle enough to preserve the biology will leave the confound in place. Telling the two apart requires assumptions about which axes of variation are biological, and those assumptions are exactly what is uncertain. Correcting systematic variation without destroying causal signal therefore remains an open problem on this kind of data, and the approach taken throughout the pipeline built here is to measure and account for it rather than to aggressively subtract it, with an explicit correction developed and evaluated as a separate component and revisited in Chapter 6. What carries into the rest of this thesis is that systematic variation is real, large, and not the target, so every downstream stage is designed with the perturbed-versus-control confound in mind.

Figure 1.6 shows perturbed cells collapsing onto this shared axis, with the regulator-specific signal as the smaller off-axis residual.

Illustrative, synthetic · Perturbed cells share one confound axis | N=1,200 cells

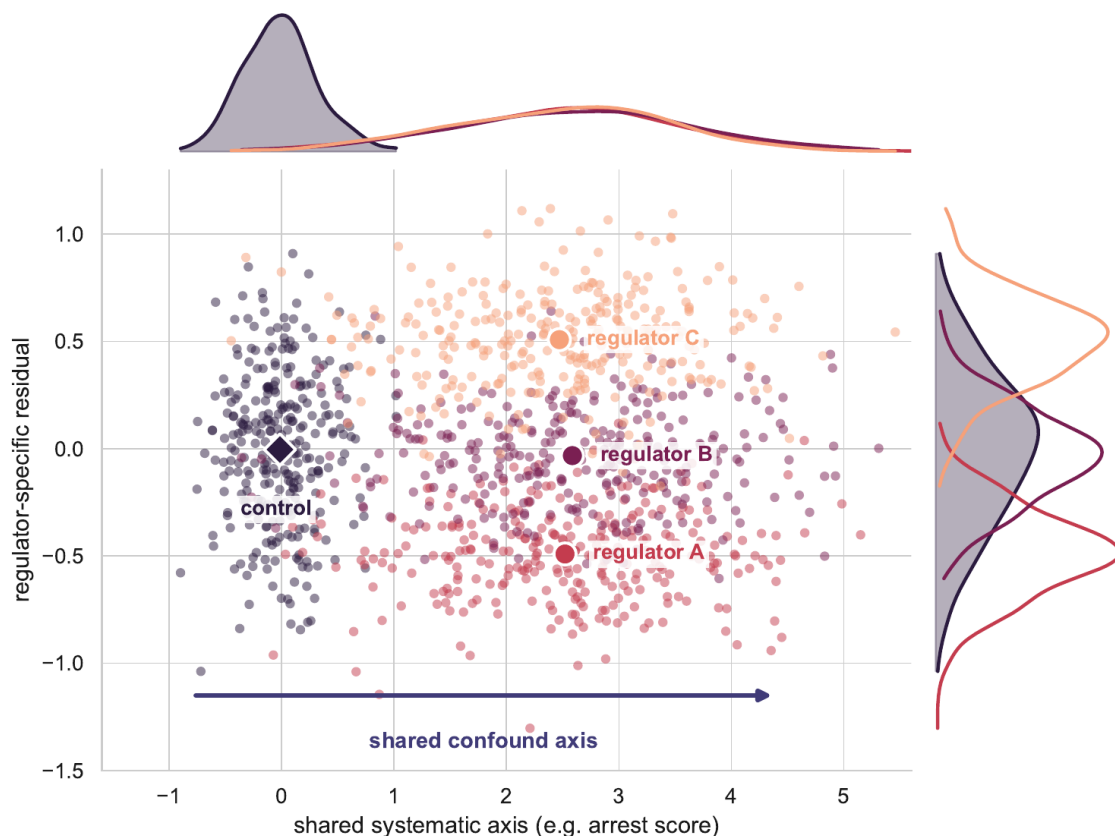


Figure 1.6 | Regulator-specific signal against the shared systematic axis. Perturbed cells collapse onto a single shared response direction (cell-cycle arrest) regardless of which essential gene was targeted, and the regulator-specific signal sits as a smaller residual off that axis. This shared axis is the systematic variation the pipeline has to separate from real regulatory signal.

1.5.3 — Knockdown efficiency and the escaper problem

One assumption sits under all of the above, that a cell carrying a guide against a gene has actually had that gene silenced, and it does not always hold. CRISPR interference reduces expression by a variable amount rather than eliminating it, so knockdown efficiency ranges from strong to negligible across genes and across cells ²⁶. Some cells take up a guide yet show little or no reduction in the target, and these escapers carry the label of a perturbation without its effect

²¹. Left in the data they blur the signal a regulatory method is trying to read, since a cell counted as perturbed but effectively unperturbed contributes a contradictory example. The perturbation label and the perturbation effect are therefore not the same thing, and separating them by each cell's measured knockdown is a data-cleaning problem handled in the SPORE stage of the pipeline. Detecting escapers, removing the droplet artifacts and batch structure above, and deciding how to cope with sparsity are all the same kind of problem, turning a raw, noisy count matrix into data an inference method can trust, which is where the pipeline begins once the inference problem itself has been set out.

1.6 — Inferring and Pruning Gene Regulatory Networks

1.6.1 — Inferring a candidate GRN from data

With clean expression data in hand, the next step is to turn it into a candidate network. GRN inference is the family of methods that score, for every pair of genes, how strongly one gene's expression predicts another's, and read a regulatory edge off a high score ⁶. Every such method returns a weighted graph over all genes, and, as the pruning problem below makes clear, that graph comes out dense. The methods differ in how they compute the scores and, more fundamentally, in whether those scores reflect mere association or genuine regulatory direction.

1.6.1.1 — Established inference methods

The established workhorses of GRN inference are tree-based. GENIE3 scores a candidate edge by how much one gene improves a random-forest prediction of another's expression ¹⁴, and GRNBoost2 computes the same kind of importance with gradient-boosted trees for speed at single-cell scale ³⁸. These two are worth singling out because they are the field's standard reference methods, the baselines against which new inference approaches are routinely measured, and inference quality across such methods is assessed through curated benchmarking frameworks such as BEELINE ⁶. Their design also carries their main limitation. They were built for observational expression data and modest gene panels, and benchmarks on real interventional data show that methods strong on small or synthetic problems do not always transfer to the genome scale ⁹.

1.6.1.2 — Co-expression versus causal inference

Underneath the choice of scoring machinery is a deeper split in what an inferred edge is claimed to mean. Co-expression methods read edges from statistical co-variation, taking genes that move together across cells as evidence of a regulatory link. SCENIC is a representative example, building modules of co-expressed genes around candidate regulators and refining them with motif evidence ³⁹. The weakness of the whole co-expression family is intrinsic to it, since correlation alone cannot say which of two co-varying genes regulates the other, or whether both are driven by a third, so a co-expression edge carries no reliable direction. Causal methods attack that weakness by using perturbations. When the data records what happens after a specific gene is silenced, the direction of influence can be read from the intervention itself, because the silenced gene is the cause and the genes that shift are the effects ⁸. This is what interventional benchmarks were created to reward, and what separates methods that recover regulatory direction from those that recover co-expression ⁹. The dense-graph generator this thesis builds on belongs to the causal, perturbation-aware family, and the method itself is introduced with SHROOM in Chapter 3.

1.6.2 — The dense-to-sparse problem

1.6.2.1 — Why prune: dense inference and over-smoothing

An inference method assigns a score to every possible edge, so its raw output is effectively a complete graph, with a nonzero weight between almost every pair of genes. This dense graph is

unusable as it stands. Most of its edges are weak, spurious correlations rather than regulatory relationships, and a graph in which everything connects to everything reproduces none of the structure of a real GRN, which is scale-free, disassortative, and modular rather than uniform ⁴. The dense graph is also actively harmful to a downstream graph model, which works by repeatedly updating each gene from its neighbors. On a near-complete graph every gene's signal blends into every other gene's within a few steps, and the distinct regulatory hierarchy collapses into a uniform blur, an effect known as over-smoothing ⁴⁰. Extracting a usable network therefore means pruning the dense graph down to the edges that matter.

The difficulty is that pruning is underdetermined. Many different sparse graphs can be cut from the same dense one, and which edges are kept is what decides whether the result reproduces the hub hierarchy and modular structure of a real GRN or is merely smaller. The target is clear even if reaching it is not. A good pruned graph should reproduce the topological signatures of a real GRN, the scale-free hub hierarchy, the disassortative wiring, the modular community structure, and the feed-forward-loop enrichment, rather than optimizing any single one of them. The two baseline strategies below each prune by optimizing one simple criterion, and each falls short of that target in an instructive way. They are the comparators the later evaluation measures against.

1.6.2.2 — Top-weight pruning

The simplest strategy keeps the globally highest-weighted edges and discards the rest. Top-weight pruning trusts the inference signal completely, so an edge that scored high stays. It is a natural baseline and a fair upper bound on what raw signal alone can achieve, since any method that cannot beat it is adding nothing beyond the inference score. Its weakness is that it is blind to topology. Edge scores are not calibrated per gene, so a few hub genes with many high-scoring outgoing edges absorb most of the edge budget, and the pruned graph ends up dominated by a handful of super-hubs with almost everything else left bare. The result is a concentration of regulatory output far more extreme than a real GRN allows, where even the largest master regulators reach only tens to hundreds of targets, a bounded fraction of the genome rather than a dominant share of it ^{3,4}.

1.6.2.3 — K-nearest-neighbor pruning

The opposite strategy fixes runaway hubs by force. K-nearest-neighbor pruning keeps, for each gene, only its top-K outgoing edges, so every regulator ends with the same out-degree. It guarantees that no single gene can dominate and that every gene retains its strongest targets, producing the flattest possible network. That flatness is exactly the weakness. Real regulatory output is deeply unequal, a few hubs controlling hundreds of genes while most control almost none, and forcing every gene to an identical out-degree erases that hierarchy entirely ⁴. A K-nearest-neighbor graph has no hubs by construction, which discards the single most characteristic feature of a biological GRN and leaves a downstream model no master-regulator structure to exploit.

1.6.2.4 — Toward a biologically grounded pruning

The two baselines fail in mirror-image ways. Top-weight maximizes hub dominance and overshoots inequality, K-nearest-neighbor eliminates it and undershoots, and each hits its one target while missing the biology as a whole. The lesson is that no single simple criterion yields a faithful graph, because the signatures of a real GRN have to hold together, and pushing hard on one of them in isolation works against the others. This suggests what a better method must do. It should treat the biological signatures themselves as the objective and search for the sparse graph that matches all of them at once, rather than any one alone ⁴¹.

How much this matters depends on scale. On a network of tens or hundreds of genes there is not enough room for a strong hub hierarchy or an over-smoothing problem to develop, so a top-weight or K-nearest-neighbor graph is often adequate and the choice of pruning barely bites. At

the scale MYCELIUM targets, thousands to tens of thousands of genes, the shape of the graph starts to carry real weight. A network that matches the true architecture is more useful as a map of the cell's regulation, and it is simultaneously more useful to a downstream graph model, which reads the graph as the medium of its computation rather than as a passive input, so a biologically correct shape serves the model at the same time as it serves the map ^{11,40,41}. A pruning method that turns the topology of a real GRN into a target and cuts a dense inferred graph down to meet it is the problem the later chapters take up, once the pipeline that produces the dense graph has been described.

Figure 1.7 sketches this transformation, from a dense inferred parent to the pruned scale-free graph.

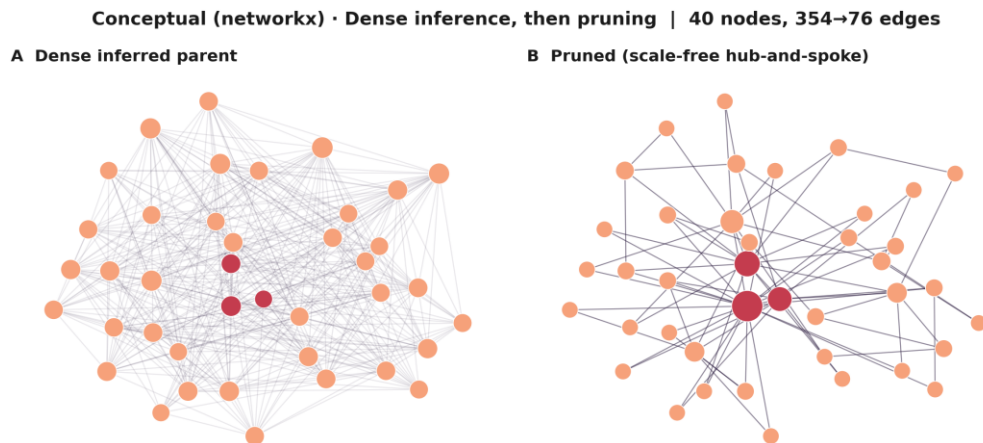


Figure 1.7 | From a dense inferred parent to a pruned scale-free graph. A near-complete dense inference graph (left) and the sparse hub-and-spoke network recovered from it after topology-aware pruning (right), drawn over the same nodes. Recovering the right graph is the job of the pruning stage.

Figure 1.8 shows these two baseline failure modes next to a scale-free target.

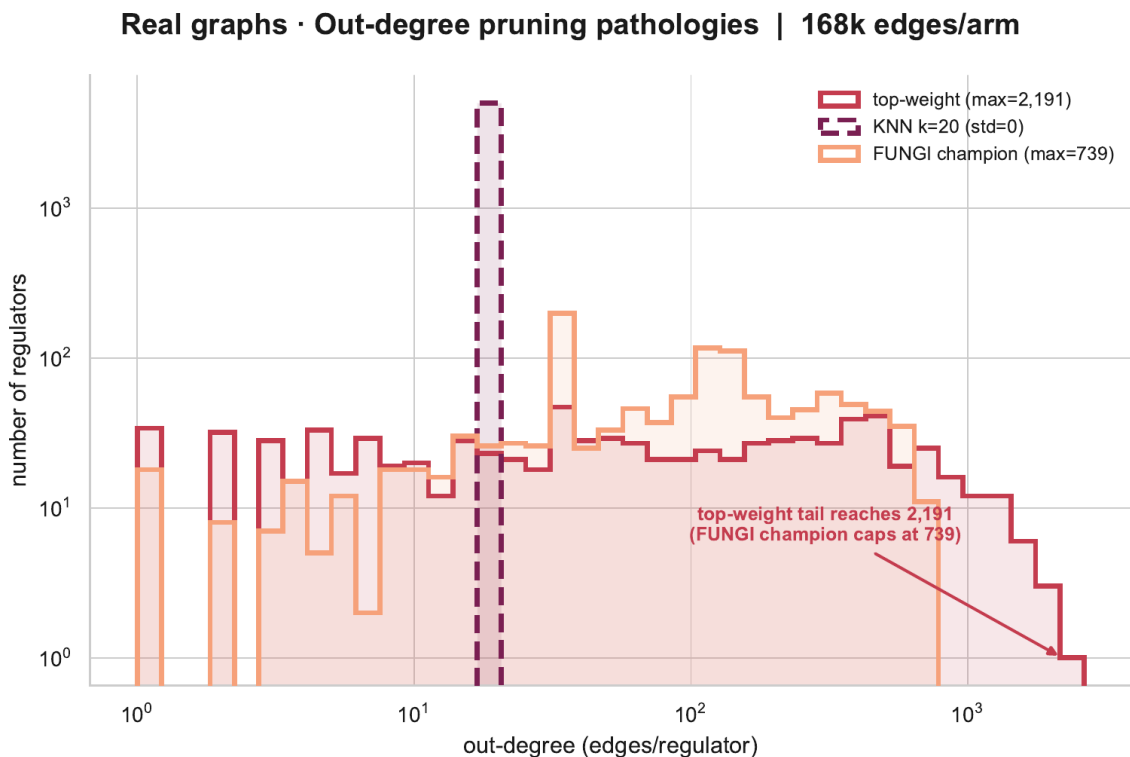


Figure 1.8 | Pruning failure modes in out-degree. Out-degree distributions for an over-concentrated top-weight prune, a flat degree-capped nearest-neighbour prune, and a scale-free target. The two baselines show the

pathologies a topology-aware objective has to avoid, one concentrating all output on a few nodes and one forcing every node to the same degree.

1.7 — From Graph to Prediction: Graph Neural Networks

1.7.1 — Message passing and why topology matters

A pruned GRN is only useful if a model can turn it into predictions, and the model class built for graph-structured data is the graph neural network. Its mechanism is message passing. Each gene starts with a feature vector, and in each layer every gene updates its vector by aggregating the vectors of its graph neighbors, so information flows one hop along the edges per layer and, over several layers, spreads across the network ²⁴. Predicting the response to a perturbed gene amounts to injecting a signal at that gene and letting message passing carry its effect outward along the regulatory edges, so the graph a graph neural network runs on is the medium of the computation rather than a passive input ²⁴.

This is what makes topology load-bearing. Message passing can only move information along edges that exist, so a missing regulatory edge is a channel the model cannot use, and a spurious edge is a channel that leaks signal where none should flow. On a dense graph the failure is the over-smoothing described for the pruning problem, where a few rounds of aggregation blend every gene's vector into every other's and the model can no longer tell regulators from targets ⁴⁰. A graph neural network's predictions are therefore bounded by the quality of its graph, which is why producing a biologically faithful sparse graph sets a ceiling on how well any downstream model can perform.

1.7.2 — Models for perturbation-response prediction

Several models already predict perturbation responses, and the three this thesis benchmarks against are chosen because they span the design space, one using no graph at all, one using a fixed external graph, and one using a language-model prior. Naming them together first makes the contrast the pipeline turns on visible before the details.

The no-graph approach is scGen, which learns the response in a latent space built from expression data alone and shifts a cell from an unperturbed to a perturbed state by vector arithmetic in that space, with no explicit network anywhere in the model ²³. The fixed-graph approach is GEARS, a graph neural network that runs over a knowledge graph of gene-gene relationships drawn from gene-ontology annotations, which lets it predict the effect of perturbing genes, and combinations of genes, that it never saw in training ¹¹. The language-model approach is scLAMBDA, which pairs large-language-model embeddings of genes with a generative model to predict single and combinatorial responses ⁴². Across these three the structure each leans on is different, no graph, a fixed external graph, and a learned prior, and that difference is what the next subsection builds on.

1.7.3 — SPECTRA and why graph quality drives performance

SPECTRA is the downstream model of this pipeline, a graph-based perturbation-response predictor built to run on the dataset-specific GRN the earlier stages produce ⁴³. It keeps the graph-based prediction idea that GEARS demonstrates but changes what the graph is. GEARS runs its predictions over a gene-ontology graph that is fixed in advance and identical for every dataset, so the graph encodes generic prior knowledge rather than the regulatory wiring of the particular cells being modeled ¹¹. SPECTRA replaces that fixed, generic graph with the GRN inferred and calibrated from the actual perturbation data of the cell line in question.

The bet this sets up is direct. A graph built from the real regulatory data of a cell line should carry more of the right regulatory channels than a one-size-fits-all ontology, and a message-passing model given that graph should predict perturbation responses more accurately. The pipeline is built and validated up to the point where SPECTRA consumes its graph, and SPECTRA

is the stage at which a more faithful GRN would show up as a measurable gain in perturbation-response prediction. That is the payoff the whole pipeline is aimed at.

1.8 — The MYCELIUM Pipeline and Thesis Roadmap

1.8.1 — The pipeline at a glance

The preceding sections describe a field boxed in by three hard facts, and MYCELIUM is the response to them taken together. There is no ground-truth GRN to learn from, so every network is an estimate that can be judged only indirectly^{1,6}. The methods that produce those estimates return dense, noisy graphs that a downstream model cannot use as they stand. And the perturbation data those methods run on is shot through with systematic variation that can masquerade as regulatory biology³⁶. A fourth difficulty compounds the first three, since most inference methods are demonstrated on small networks of tens to hundreds of genes, and much of the noise and difficulty is what appears only when the same task is scaled to the thousands or tens of thousands of genes a genome-scale map requires^{6,9}.

MYCELIUM, for Mapping Your Cellular Expression: Learning Interventional Unified-regulatory Maps, takes interventional single-cell data and turns it into a single, curated regulatory map that a downstream model can reason over. It is not one algorithm but a pipeline, the aggregate of three stages that each answer one of these problems. SPORE prepares the data, SHROOM infers a network from it, and FUNGI prunes that network into a biologically faithful graph. Run in sequence, SPORE to SHROOM to FUNGI carry a raw perturbation dataset through to the calibrated sparse GRN that a model such as SPECTRA consumes. The rest of this section introduces each stage in turn and maps the road through the chapters that follow.

1.8.2 — What each module contributes

The three stages are designed to run together as one pipeline, and each also stands on its own as a usable tool. Each is introduced here only far enough to place it in the whole, with the mechanisms and results left to the chapters that follow.

1.8.2.1 — SPORE

SPORE is the data-preparation stage, and it is the most self-contained of the three, usable on its own as a general toolkit for cleaning and structuring single-cell perturbation data. It applies modality-aware quality control, removes the ambient RNA and doublets that corrupt droplet data, and filters the escaper cells whose guide fired without silencing the target, so that a cell labeled as perturbed has actually been perturbed^{21,35}. It splits the data into training and held-out sets so that later evaluation stays honest. Where single-cell sparsity would otherwise bury the signal, SPORE can also aggregate transcriptionally similar cells under the same perturbation into metacells, whose summed counts give a more stable estimate of expression⁴⁴. Metacell aggregation is one option among the ways SPORE can treat the data rather than a required step, and the two problems deferred earlier, escapers and sparsity, are both resolved at this stage.

1.8.2.2 — SHROOM

SHROOM is the inference stage, and it produces the first network. It runs a perturbation-aware inference method, PSGRN, over SPORE's cleaned data, using the record of what happens when specific genes are silenced to score candidate regulatory edges in a way that reflects direction rather than mere co-expression⁴⁵. Its output is a dense parent graph, a weighted, near-complete network in which almost every gene pair carries some score. That graph is not usable on its own, for the reasons already set out, but it is the raw regulatory signal from which a good graph can be cut. SHROOM's role is to convert prepared data into that candidate network as faithfully as the interventional signal allows.

1.8.2.3 — FUNGI

FUNGI is the pruning stage, and it is where the dense graph is sculpted into a biologically realistic shape. It takes SHROOM's parent graph and cuts it down to a sparse network chosen to match the topological signatures of a real GRN, the scale-free hub hierarchy, the disassortative wiring, the modular structure, and the feed-forward-loop enrichment, rather than any single one of them. It does this by treating those signatures as an explicit objective and searching, with a scoring kernel built on a domain-knowledge pruning approach, for the sparse graph that satisfies them together ⁴¹. Where each baseline heuristic optimized one criterion and missed the biology as a whole, FUNGI optimizes toward the biology directly, and its output is the calibrated sparse GRN the pipeline delivers.

1.8.3 — Scope and contributions of this thesis

The contribution of this thesis is the MYCELIUM pipeline itself, a complete and reproducible route from single-cell perturbation data to a biologically calibrated GRN, built from the three stages above and evaluated at each step. Two things set it apart. The first is the framing that a GRN's value lies in its topological faithfulness, which turns network quality from a vague aspiration into a set of measurable targets a method can be held to. The second is reach, since the whole pipeline runs end to end on a single workstation, which puts genome-scale GRN inference within range of a standard lab setup rather than a dedicated compute cluster. The pipeline delivers the calibrated GRN and validates it at each stage up to the point where a downstream model consumes it, which is where the payoff in perturbation-response prediction is measured.

1.8.4 — Roadmap of the thesis

The chapters ahead follow the pipeline in order. Chapter 2 takes up SPORE and the problem of turning raw perturbation data into a trustworthy dataset, including the escaper filtering and metacell aggregation named here. Chapter 3 develops SHROOM and the inference of a dense parent graph from that data. Chapter 4 is FUNGI, where the topological signatures become formal optimization targets and the pruning method is built in full. Chapter 5 reports what the pipeline produces, and Chapter 6 steps back to what those results mean, what remains open, including the correction of systematic variation and the downstream evaluation, and where the work goes from here. The natural place to begin is where the pipeline begins, with the data, and turning that raw, noisy data into something an inference method can trust is the subject of the next chapter.

CHAPTER 2

SPORE

Systematic Preprocessing and Optimization for Robust Evaluation

Turning millions of raw, noisy perturbed cells into a clean, leakage-safe substrate that every later stage of the pipeline can trust.

A spore is where a mycelium begins its life. Here too the pipeline takes root, germinating raw measurements into the ground everything downstream grows from.

2.1 — Why Data Preparation Decides Everything Downstream

A gene regulatory network is only as trustworthy as the data it is inferred from, so before any inference method runs, the data itself has to be made trustworthy. That preparation does two distinct jobs, and the rest of this chapter is organized around them. The first job is cleaning, removing the technical artifacts and mislabeled cells that an inference method would otherwise read as biology. The second is splitting, partitioning the data so that the network is built and evaluated without any information leaking from the parts held back for testing. Both are easy to underestimate, and getting either one wrong quietly caps how good every downstream result can be.

2.1.1 — Data quality as the ceiling on inference

Data quality sets a hard ceiling on the quality of any inferred network, because an inference method has no way to distinguish a real regulatory signal from a convincing artifact. Whatever structure survives into its input, the method treats as evidence and writes into its edges. The single-cell artifacts introduced in the previous chapter are exactly the kind of structure that survives and misleads. Ambient RNA adds a background of transcripts a cell never expressed, so two genes can appear co-expressed only because they share the same contamination³⁷. Doublets fuse two cells into one profile and manufacture co-expression patterns that belong to no real cell³⁵. Differences in sequencing depth and the pervasive dropout of single-cell data make the same gene read as present in one cell and absent in another for purely technical reasons³⁵. Every one of these is a correlation that an inference method can mistake for regulation.

The perturbation label carries its own failure mode, and it is the more dangerous one because it corrupts the causal signal directly. A cell tagged with a guide against a gene is assumed to have had that gene silenced, but CRISPR interference produces a variable knockdown, and some cells escape it almost entirely²⁶. Such an escaper carries the label of a perturbation without its effect, entering the analysis as a cell that should show a regulatory response and does not²¹. Left in the data, escapers blunt and contradict the very signal a causal method depends on. Because inference cannot tell a clean cell from a contaminated one, or a real knockdown from an escaper, the quality of the prepared data is the ceiling on the quality of the network, and no amount of downstream sophistication can lift a result above it.

2.1.2 — Data leakage: the subtler, more dangerous failure

Cleaning protects the signal, and splitting protects the honesty of the evaluation, guarding against a failure called data leakage. Leakage is the contamination of the training data with information that should have been held back for testing, so a model ends up credited for predicting something it was in effect allowed to see⁴⁶. What it produces is measured performance that looks strong and then collapses on genuinely new data. This is not a fringe concern. A survey across seventeen scientific fields found leakage driving overoptimistic, irreproducible results in hundreds of published studies, in several cases making complex models appear to beat simple baselines that, once the leak was closed, they did not actually beat⁴⁷.

Perturbation modeling is especially exposed to leakage, because of the shared structure described in the previous chapter. Perturbed cells carry a large, coherent systematic-variation signature that recurs across many different perturbations rather than being specific to any one of them³⁶. If cells from the same perturbation are allowed into both the data used to build a model and the data used to test it, the model can win by recognizing that perturbation's signature instead of predicting its regulatory effect, and because the systematic-variation component makes even unrelated perturbations look partly alike, the channel through which such recognition leaks is wider than it first appears³⁶. A model can then post an excellent test score by matching signatures it has effectively already seen. Preventing this is not a matter of being careful during analysis but of how the data is partitioned in the first place, which is the design the splitting stage of SPORE is built around and the subject of a dedicated section below.

2.1.3 — The case for a generalizable, reproducible preparation framework

Cleaning and leakage-safe splitting could in principle be handled by a pile of dataset-specific scripts, but doing them well at genome scale calls for a single, general framework, under three pressures that a script handles badly. Scale is the first. The genome-scale Perturb-seq data this thesis works from runs to millions of cells ²¹, so every step has to hold to a fixed memory budget rather than assume the whole matrix fits in memory at once. Reproducibility is the second, and it is a precondition for the result being credible at all, since a preparation pipeline is only reproducible if it is deterministic, logged, and captured in configuration rather than reconstructed from memory of what was run ⁴⁸. Generalization is the third, and it is what a framework earns over a script. A pipeline that detects a dataset's modality, gene-identifier format, and perturbation structure can adapt to a new dataset without being rewritten, where a bespoke script would have to be rebuilt each time ⁴⁹.

This thesis meets those three demands with SPORE, short for Systematic Preprocessing and Optimization for Robust Evaluation, the data-preparation stage of the MYCELIUM pipeline. SPORE is built as a general, reproducible framework rather than a one-off cleaning script, and its task is to turn a raw, million-cell perturbation dataset into the clean, leakage-safely-split substrate that the inference stage consumes. The sections that follow open it up, first its design and the journey a cell takes through its cleaning phases, then the splitting stage that stands between the data and any downstream leakage, and finally the shaping that readies the training data for network inference.

2.2 — SPORE: A Modality-Aware Preparation Pipeline

SPORE is organized as an ordered sequence of numbered phases, each responsible for one transformation of the data and run back to back, so that a raw dataset enters at one end and a clean, split substrate leaves at the other. This section describes the design that holds across all of the phases and then follows the data through the cleaning phases in order, from ingestion up to the point just before the split. The split itself, and the shaping that comes after it, are taken up in the sections that follow.

2.2.1 — Design principles and pipeline shape

Four design choices shape every phase of SPORE. The most basic is the phased architecture itself. The pipeline is a fixed sequence of numbered stages, each reading the previous stage's output and writing its own, which keeps every transformation auditable and lets a run resume from an intermediate stage rather than restart from scratch. SPORE is also self-configuring, through a dedicated detection stage that inspects each dataset and infers the properties the later phases need, so the same pipeline adapts to a new dataset without hand-editing. Memory-safety is the third concern, and the one that dictates much of the engineering, since SPORE keeps the expression matrix sparse throughout and offloads its heaviest phases to isolated worker subprocesses so that a dataset of millions of cells stays within a fixed memory budget instead of being assumed to fit in RAM ²¹. The last choice is to default to non-destructive edits. Cleaning phases flag cells rather than remove them, and a flagged cell is deleted only when the configuration explicitly calls for it, so every removal decision can be inspected and reversed before any data is lost. Across all of this SPORE builds on the AnnData data structure and the scanpy toolkit, the standard substrate for single-cell analysis in Python ⁵⁰.

2.2.2 — Ingestion and detection (Phases 0–1)

The data's journey begins with ingestion. SPORE reads the raw dataset and converts its expression matrix to a sparse representation in chunks, caching the result so the expensive conversion runs only once ⁵⁰. Nothing biological changes at this stage, since the conversion only alters how the same counts are stored. Detection comes next, and it is deliberately read-only. Before any cleaning touches the data, SPORE inspects the raw matrix and infers the dataset's

modality, the format of its gene identifiers while harmonizing them to a common naming, its perturbation structure (whether each cell carries a single guide or a combination), and any cell-line labels already present. These detected properties propagate to every later phase, which is what lets one pipeline configure itself to a new dataset instead of requiring a human to specify each of these by hand.

2.2.3 — Cell-level quality control (Phases 2–4)

With the data ingested and characterized, the cleaning begins, and the first target is the cells themselves. Cell-level triage computes per-cell quality metrics, the total molecule count and the fraction of counts coming from mitochondrial genes, and filters out the cells whose metrics mark them as low-quality or dying, with a guard that relaxes the mitochondrial filter for single-nucleus data where it does not apply³⁵. Two droplet artifacts are handled after that. Ambient RNA, the background of cell-free transcripts that contaminates every droplet, is estimated either from empty droplets in the raw data or, when only filtered data is available, as a global expression signature, and a cell that combines a high ambient score with a low molecule count is flagged as a likely empty or dying droplet³⁷. Doublets, the droplets that captured two cells and produced one chimeric profile, are scored with Scrublet, which simulates artificial doublets from the data and trains a classifier to recognize them; for datasets above roughly a hundred and fifty thousand cells, SPORE fits Scrublet on a subsample and projects the remaining cells through a learned low-dimensional space, keeping the computation linear in memory^{35,51}. Each of these artifacts was introduced as a concept in the previous chapter, and here each one is measured and removed in practice.

Figure 2.1 shows the pre-filter distributions and the thresholds applied to them.

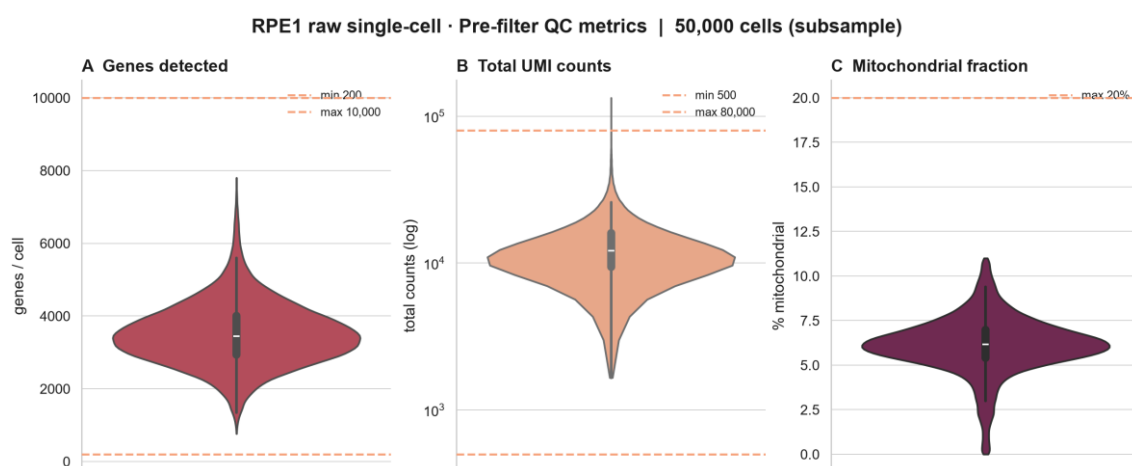


Figure 2.1 | Pre-filter cell quality control. Distributions of genes per cell, total UMI count (log scale), and mitochondrial content across cells before filtering, with the minimum and maximum acceptance thresholds marked. Cells outside these gates, empty droplets, low-complexity cells, and dying cells, are removed.

Figure 2.2 shows the ambient-RNA and doublet gates as a layered-density view.

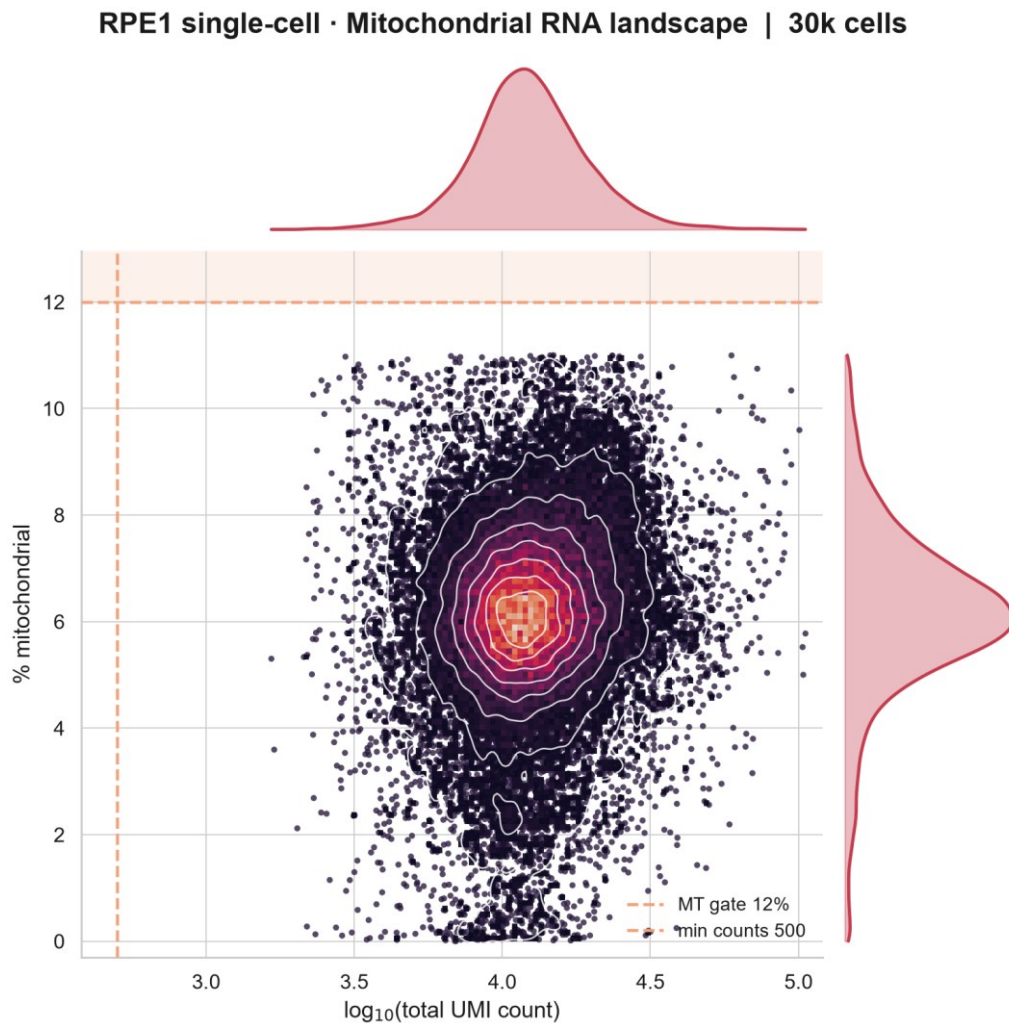


Figure 2.2 | Ambient-RNA and doublet gating. Joint density of the two quality-control axes, with the retained population and the flagged danger zone shaded and white contours marking where genuine single cells concentrate. This layered-joint view is the form SPORE uses for its cleaning decisions.

2.2.4 — Perturbation-aware filtering: escapers and knockdown efficiency (Phase 5)

Cleaning the cells is not enough, because a cell can be technically pristine and still carry the wrong label. The escaper-filtering phase is the perturbation-aware cleaning step, and it is where the perturbation label is reconciled with the perturbation effect. For each guide-bearing cell, the expression of its target gene is compared against the distribution of that gene across the non-targeting control cells, and cells that fail to show the expected shift are removed as escapers, with the direction of the test set by the modality so that interference and knockout retain the strongly-silenced cells while activation retains the strongly-induced ones^{21,26}. The same comparison yields a knockdown-efficiency score for each perturbation, a record of how well each guide actually worked. Combinatorial labels are supported natively, with each gene in a compound perturbation tested against its own control distribution⁸. This phase delivers on the escaper problem raised in the previous chapter, so that a cell counted as perturbed has actually been perturbed.

Figure 2.3 shows knockdown efficiency per target and the escapers that fall below threshold.

RPE1 train split, Phase 5 · Guide performance landscape | 888 targets

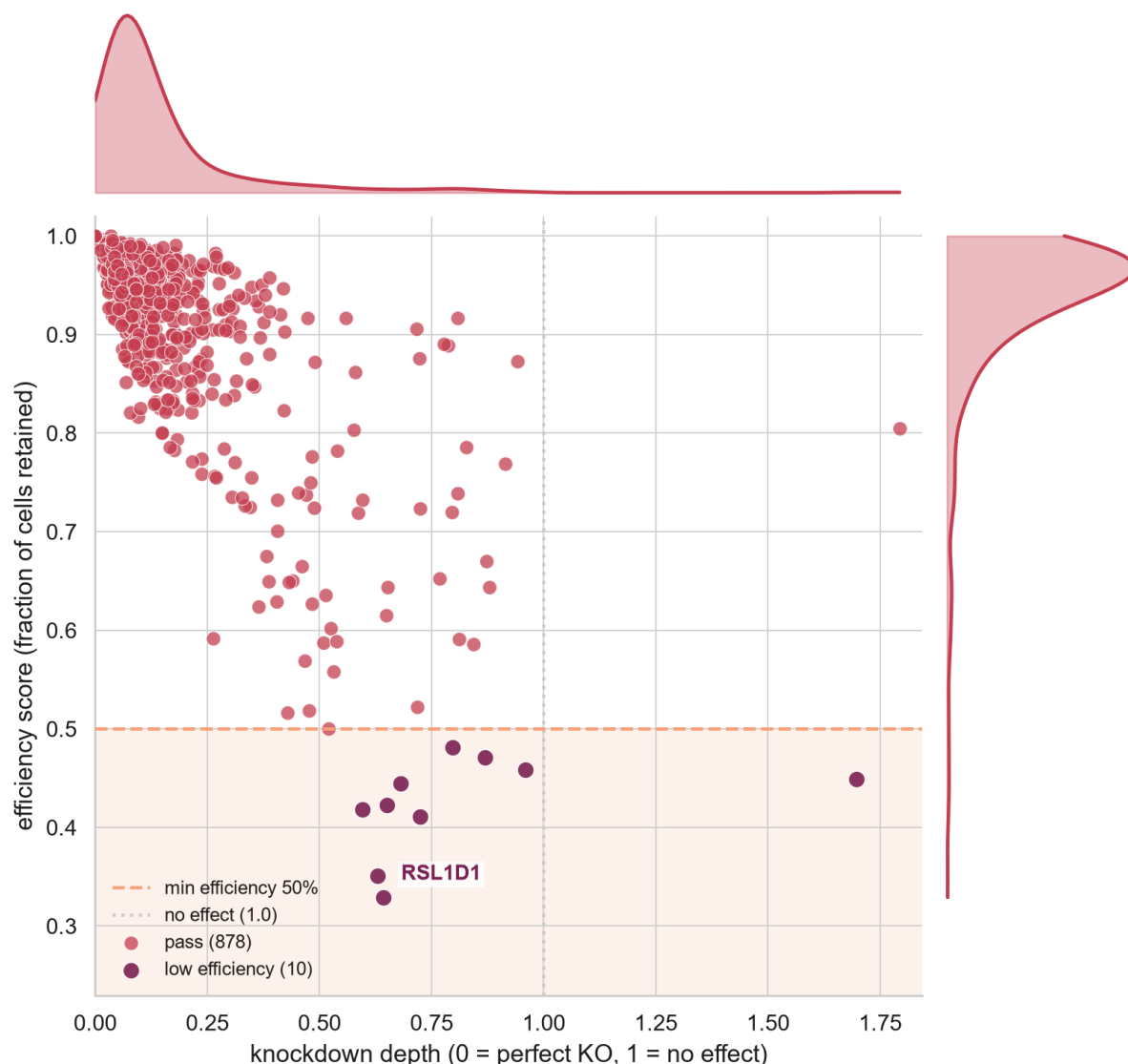


Figure 2.3 | Guide performance and escapers. Per-target knockdown efficiency plotted against knockdown depth. Targets below the efficiency threshold (shaded) are escapers, cells whose perturbation label does not correspond to a real change in expression, and SPORE handles them separately.

2.2.5 — Gene-level triage (Phase 6)

The last cleaning phase turns from cells to genes. Gene-level triage removes genes too sparsely or lowly expressed to carry usable signal, shrinking the feature space before the costly later stages. The detail that matters is what it protects. Any gene that is a perturbation target is rescued from removal, including the individual genes parsed out of a combinatorial label, so that the regulators the inference stage most needs to reason about are never dropped for being lowly expressed^{21,35}. With the cells cleaned and the gene set triaged, the data is ready to be divided, which is the most consequential decision SPORE makes.

Figure 2.4 traces the gene counts through triage and the penetrance-based rescue.

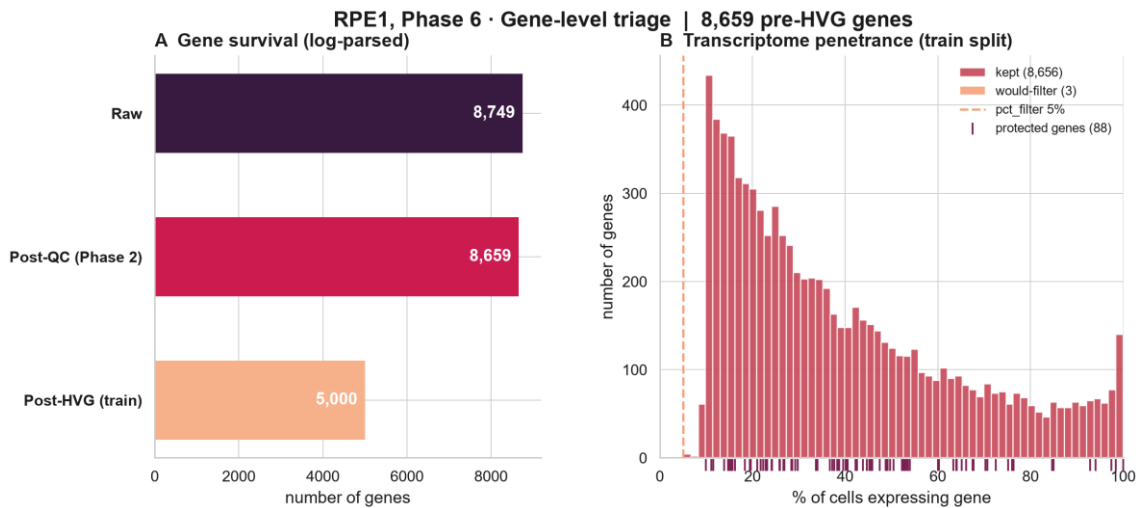


Figure 2.4 | Gene-level triage. Staged gene counts through the filtering phases (left) and the penetrance distribution split into kept and filtered genes (right). Perturbation targets are rescued and kept even where the generic filter would drop them.

2.3 — Splitting Without Leakage (Phase 7)

Every step so far has improved the data in place. Splitting is different, because it does not change the cells at all; it decides which cells the downstream model is allowed to learn from and which are held back to judge it. Get that division wrong and the leakage described earlier seeps in no matter how clean the data is. SPORE's splitting stage is built to make that leakage impossible rather than merely unlikely, and this section explains how.

2.3.1 — The zero-shot split: hold out whole perturbations, not random cells

The core decision is to split on whole perturbations rather than on individual cells. A naive split that shuffled cells at random would place cells from the same perturbation on both sides of the divide, letting a model see a perturbation's response during training and then be tested on more cells of that same perturbation, which rewards memorizing a signature rather than predicting it. SPORE instead performs a zero-shot split, in which an entire perturbation target is assigned wholly to one partition, so no cell of a held-out perturbation ever appears in training ¹¹. The downstream model is therefore forced to generalize to genetic states it has never seen, which is both the ability that matters for the target-identification use the whole pipeline is built toward and the only honest thing to measure ^{11,20}.

2.3.2 — Stratification by transcriptional severity

A zero-shot split carries a risk of its own, that the held-out perturbations happen to be systematically easier or harder than the ones kept for training. SPORE controls for this by stratifying on transcriptional severity. Each perturbation is scored by the absolute size of its mean expression shift away from the non-targeting controls, a measure of how strongly it moves the transcriptome ²². Perturbations are then binned by that severity, and the training, validation, and test partitions are sampled proportionally from every bin, so all three carry the same spread of weak and strong effects rather than concentrating the strong ones in a single partition. The split can also be specified as absolute counts instead of fractions, a safeguard that keeps a large perturbation panel from producing a validation or test set too small to be meaningful.

2.3.3 — Three files, three roles, and how leakage is prevented end-to-end

The split produces three files, and their separation is the guarantee the rest of the pipeline rests on. SPORE writes a training set, a validation set, and a test set, one AnnData file each, along with

the exact record of which perturbations went to which partition. One rule among these is both simple to state and load-bearing to enforce. Only the training file is passed to SHROOM to infer the dense graph, so the network is built exclusively from training-split perturbations and never encodes information from the held-out ones. The validation file is reserved for choosing and tuning the downstream model, and the test file is left untouched until the final evaluation of the perturbation-response model, so that the number it produces reflects performance on perturbations nothing upstream has seen. Because inference sees only the training split, and every later computation is bounded by what that split contains, a held-out perturbation's response cannot flow into either the graph or the score that grades it. The danger this forecloses is concrete. If even a few cells of a test perturbation reached SHROOM, the inferred graph could carry that perturbation's fingerprint, and the eventual evaluation would be measuring recognition rather than prediction, which is exactly the inflated and irreproducible result that leakage produces⁴⁷. Keeping the three files strictly apart, with only the training set feeding the network, is how SPORE turns leakage-safety from a matter of analyst discipline into a property of the pipeline's structure³⁶.

Figure 2.5 shows the split allocation and confirms the three splits carry the same difficulty.

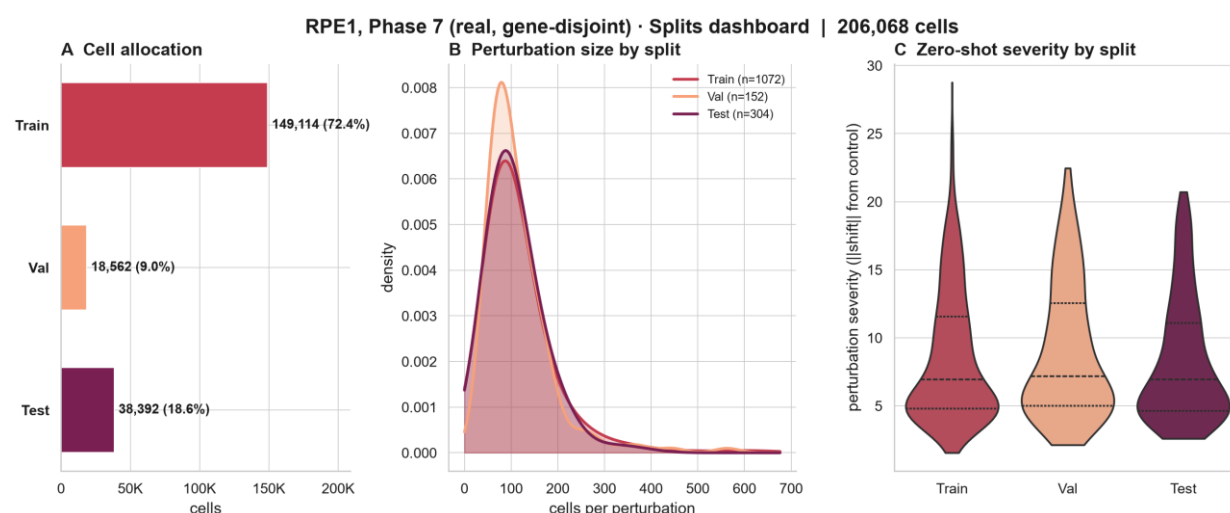


Figure 2.5 | Leakage-safe splits. Cell allocation across train, validation, and test (left), the perturbation-severity distribution per split (centre), and per-split zero-shot difficulty (right). The three splits carry the same difficulty distribution while holding out whole perturbations rather than random cells.

Figure 2.6 gives that firewall as a single assertion.

```
diagnostics.py::report_phase7_diagnostics
train_perts = set(res["train_labels"])
val_perts   = set(res["val_labels"])
test_perts  = set(res["test_labels"])

leak_tv = len(train_perts.intersection(val_perts))
leak_tt = len(train_perts.intersection(test_perts))
leak_vt = len(val_perts.intersection(test_perts))

if leak_tv == 0 and leak_tt == 0 and leak_vt == 0:
    print("[PASS] Target sets are strictly mutually exclusive.")
else:
    print("[FAIL] DATA LEAKAGE DETECTED!")
```

Figure 2.6 | The leakage firewall. The gene-disjoint guarantee written as a single assertion, confirming that training perturbations share no gene with validation or test, with only the training split flowing onward to network inference.

2.4 — Preparing the Training Substrate (Phases 8–12)

Everything after the split is the final shaping of the data into the form the inference stage expects, and it runs under one discipline. Where a step has to learn something from the data, it learns it from the training split alone and applies the result to the validation and test sets, so the leakage-safety established by the split is not quietly undone by the steps that follow. The phases below carry the data from a clean, split matrix to the feature-selected, per-cell-line substrate SHROOM consumes.

2.4.1 — Feature selection on the training split (Phase 8)

The first shaping step selects the genes worth keeping. Single-cell data measures far more genes than any model needs, and most of them vary too little across cells to inform a regulatory network, so SPORE reduces the panel to a few thousand highly variable genes chosen by the Seurat v3 procedure^{35,52}. The selection is computed on the training split alone and then applied to all three, so the feature set carries no information from the held-out perturbations.

Variance-based selection has a blind spot that matters more here than in most analyses. It keeps the genes that vary the most, but some of the most important regulators are among those that vary the least. Transcription factors are the hub regulators that sit at the top of a cell's programs⁴, and many of the canonical ones are highly conserved and held at stable, modest expression levels, so their variance across cells is low and a top-n highly-variable-gene cut can drop them entirely³. Losing them is close to the worst outcome for a regulatory network, because these are exactly the genes whose edges the network exists to recover. SPORE guards against this with a protected-gene list, a curated set of genes force-included in the feature panel regardless of their variance, so that a conserved master regulator survives selection on the strength of its biological importance rather than its statistical variability.

The protected list is curated per cell line and organized by priority. Its top tier holds the core master regulators whose absence would distort the network's topology outright, such as the identity transcription factors that define a cell type, and lower tiers cover critical pathway and chromatin regulators and a broader regulatory context that is rescued only when the feature budget allows. The mechanism is also open-ended. Every perturbation target is protected automatically, since a gene that was silenced has to remain in the panel, and a user can extend the list with any custom set of genes their study depends on. The targets of a drug screen are a

natural example. When SPORE was applied to the Tahoe drug-screen data, its 227 screen targets were force-included so that the downstream analysis kept every gene the screen was designed around, whatever their variance. This is what lets a variance-driven feature selection serve a regulatory goal rather than work against it.

Figure 2.7 shows the selection on the mean-variance plane, fit on the training split alone.

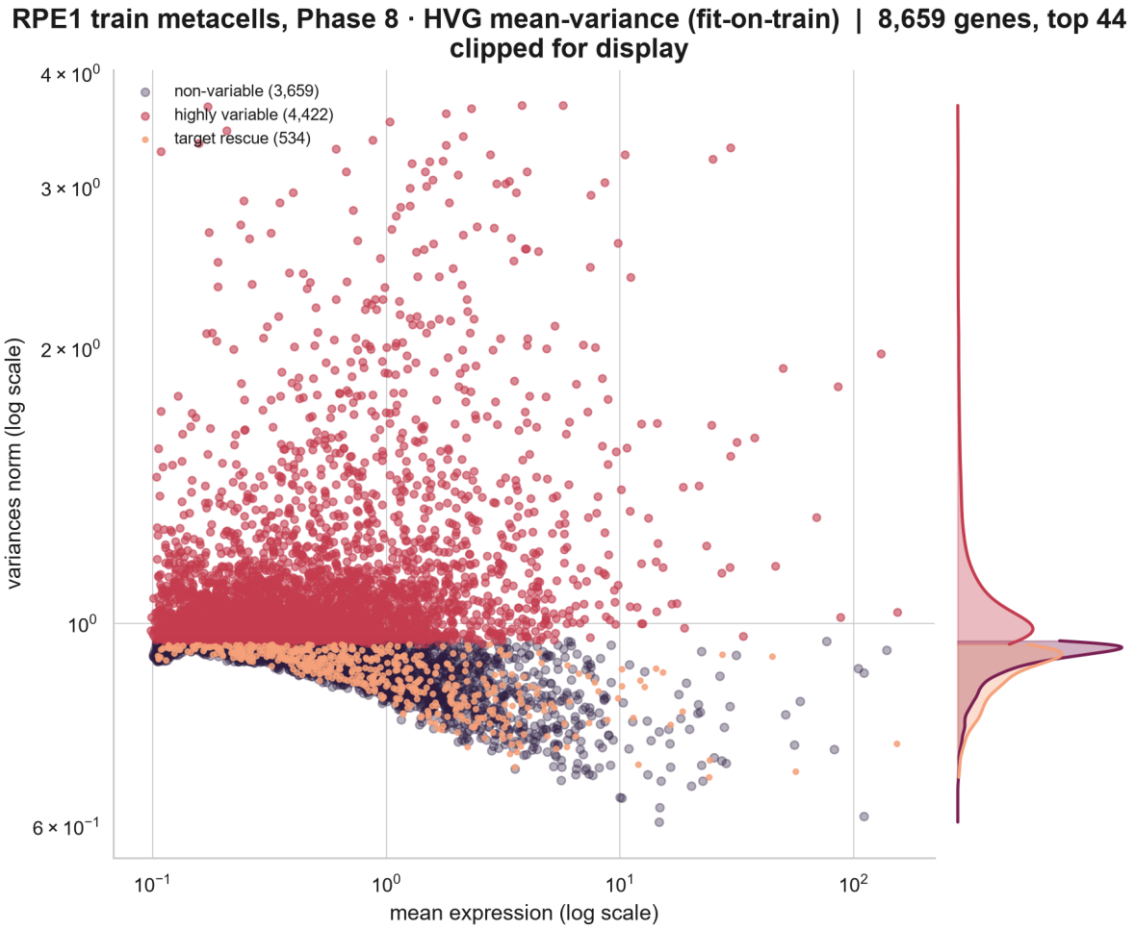


Figure 2.7 | Highly-variable-gene selection on the training split. The mean-variance relationship with the selected highly-variable core and the rescued perturbation-target and cell-cycle genes marked. Selection is fit on the training split alone, which keeps validation and test genes out of the feature choice.

2.4.2 — Normalization (Phase 9)

With the gene panel fixed, the counts are normalized. Raw counts are not comparable across cells because library size varies for technical reasons, so SPORE scales each cell to a common total and applies a log transform before any expression is compared³⁵. This step learns nothing across cells, since each cell is rescaled by its own total, so it can be applied to the three splits independently with no risk of leakage. SPORE performs the log transform in place, which avoids the large memory spike a naive copy would cause on a matrix of this size.

2.4.3 — Confounder mitigation and batch correction (Phase 10)

Normalized expression still carries batch structure, the coherent technical differences between cells processed in different lanes, on different days, or from different cell lines that have nothing to do with the perturbation. SPORE mitigates it by working in a reduced space. It fits a principal-component projection on a representative subsample and projects the full matrix into that space, then applies Harmony to the resulting embeddings, an integration method that iteratively pulls together cells differing only by batch while preserving genuine biological differences⁵³.

The temporary cell-cycle marker genes used to score each cell's phase are removed once the embeddings are built, so the feature set handed downstream stays at its intended size ³⁵.

Figure 2.8 shows the RPE1 training cells in a principal-component embedding once they have passed through the full SPORE pipeline.

RPE1 train, Phase 11 · Global bivariate topology (PCA) | 50,000 cells

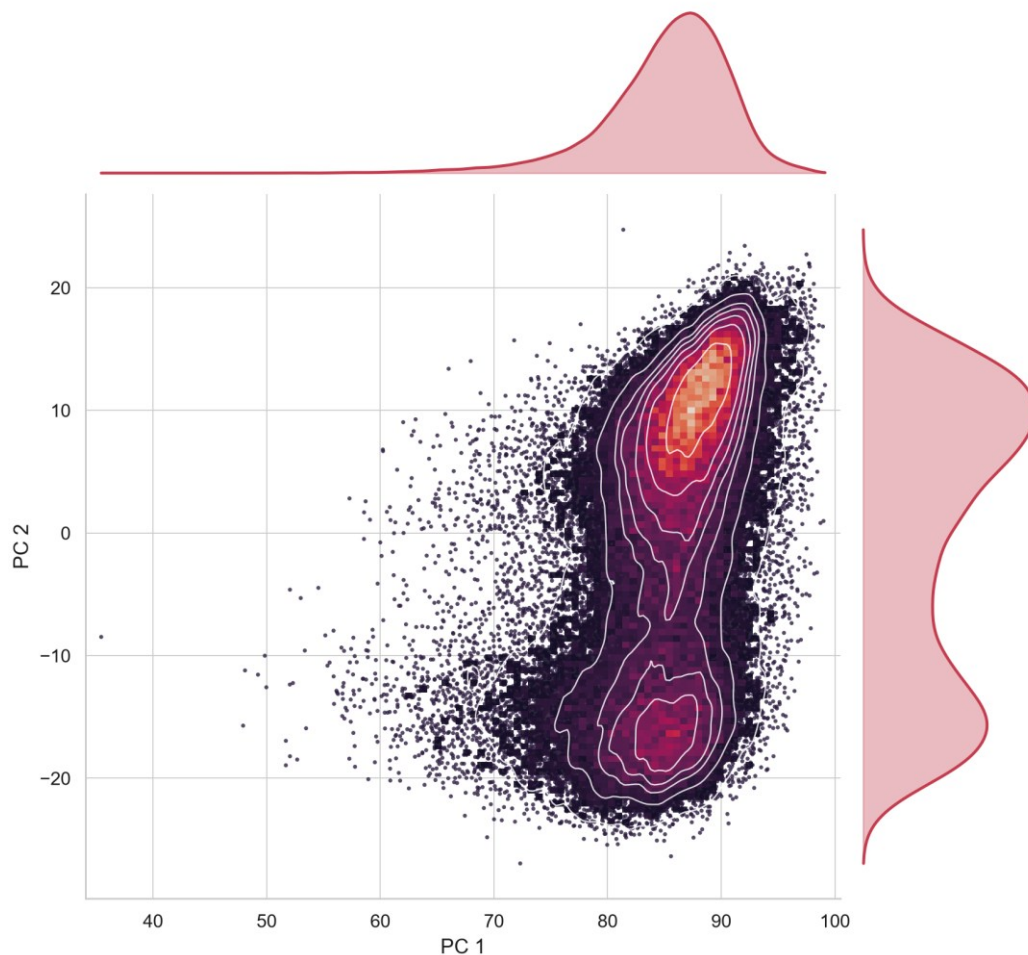


Figure 2.8 | The processed RPE1 expression landscape. A principal-component embedding (PC1 versus PC2) of the RPE1 training cells after the SPORE pipeline, shown as a joint density with its marginals. This cleaned, integrated single-cell structure is what the dense-graph generators of the next chapter read.

2.4.4 — Cell-line separation (Phase 11)

The Replogle resource contains more than one cell line, and the lines behave differently enough that they should not be modeled as a single population ²¹. SPORE detects and separates them using whatever evidence is available, reading existing labels when the dataset provides them, clustering on the batch-corrected embedding when the number of lines is known, and falling back to fully automatic clustering with a stability check when it is not. Before committing to a separation, it validates that the detected populations differ in their regulatory context rather than merely in intensity, so that two states of one program are not mistaken for two lines. Each cell line then proceeds through the remaining steps on its own, which lets the inference stage build a separate network for each line.

2.4.5 — Metacell aggregation (Phase 12)

Single-cell sparsity, the same dropout that makes any one cell an unreliable snapshot of its transcriptome ³⁵, can leave a regulatory method with too little signal per cell to work from. The

standard response is to aggregate transcriptionally similar cells under the same perturbation into metacells, whose pooled counts are far less sparse and give a more stable estimate of expression ⁴⁴. Aggregation trades single-cell resolution for stability, so how it is done, and which cells it is applied to, decides whether the trade is worth making.

Applying the same aggregation uniformly to every cell turns out to be the wrong move, and the reason is what the control cells are for. In a perturbation screen the non-targeting control cells are the shared reference against which everything else is measured. The differential expression that defines each perturbation's effect ²², the escaper test ²¹, the estimate of systematic variation ³⁶, and the co-expression structure that seeds network inference ¹⁴ are all read relative to the controls, so the fine-grained, cell-to-cell variation among the controls is what anchors the baseline the whole pipeline is calibrated against. Aggregating the controls collapses that variation and blurs the reference, which weakens the downstream network even as it reduces sparsity. The far more numerous perturbed cells carry no such role, so aggregating them mostly averages away noise the pipeline does not need at single-cell resolution.

SPORE's metacell stage is therefore hybrid. The perturbed cells are aggregated with a mild, fixed grouping, a MiniBatchKMeans clustering at $k=2$ ⁵⁴ that pools each perturbation's cells into metacells, while the control cells are kept as individual single cells. Because the controls are only a small fraction of the dataset, preserving them costs almost none of the speedup, and the perturbed side still shrinks enough to make the downstream stages substantially cheaper. The scheme keeps the reference sharp where the pipeline needs precision and coarsens the data where it can afford to, a balance that a single uniform aggregation cannot strike. Like the rest of this stage, the hybrid metacell substrate is an option rather than a requirement, and single-cell input remains the conservative default.

Figure 2.9 shows that branch in code, where perturbed cells aggregate and controls stay single.

```
agg_variants.py (control-preserving metacell builder)
for pert in unique:
    mask = pert_values == pert
    X_sub = X[mask]
    if pert == control_label:
        # controls stay SINGLE-CELL -- no aggregation
        for row in X_sub:
            all_exprs.append(row); all_ncells.append(1)
    else:
        # perturbed cells -> MiniBatchKMeans metacells (k=2)
        exprs, ncells = aggregate_group_mbk(X_sub, cluster_emb, k=2, seed=seed)
        all_exprs.extend(exprs); all_ncells.extend(ncells)
```

Figure 2.9 | The control-preserving metacell rule. Perturbed cells are aggregated into MiniBatchKMeans ($k=2$) metacells while control cells stay at single-cell resolution. Keeping the controls single-cell preserves the co-expression backbone the later stages depend on.

2.5 — CHITIN: Causal Heuristics Isolating Transcriptomic Intervention Noise

The one stage of SPORE whose success is not guaranteed. CHITIN estimates the shared cell-cycle-arrest program that every essential-gene perturbation drives the cell into, and subtracts it, so that the regulator-specific signal is what remains.

Every phase to this point is settled engineering, a known artifact met with an established fix. The last thing SPORE confronts breaks that pattern. Systematic variation is an open research problem, and the stage SPORE devotes to it, CHITIN, short for Causal Heuristics Isolating Transcriptomic Intervention Noise, is the one part of the pipeline whose success is not guaranteed. It earns its own treatment because the problem it takes on is as much conceptual as computational.

2.5.1 — The correction target: a convergent, shared axis

To correct systematic variation, one first has to say precisely what is being removed. In the essential-gene Perturb-seq data this pipeline works from, the dominant unwanted signal is a single convergent axis. Silencing an essential gene stresses the cell, and in a p53-competent line like RPE1 that stress resolves into a shared cell-cycle-arrest program that looks much the same whichever essential gene was hit ^{21,36}. Because nearly every perturbation drives cells along this same axis, it dominates the difference between perturbed and control cells and swamps the smaller, regulator-specific shifts that actually encode the regulatory wiring ³⁶. This is the signal CHITIN sets out to estimate and subtract, so that what remains is the perturbation-specific response rather than the shared arrest response.

Figure 2.10 shows this convergence onto the shared arrest axis.

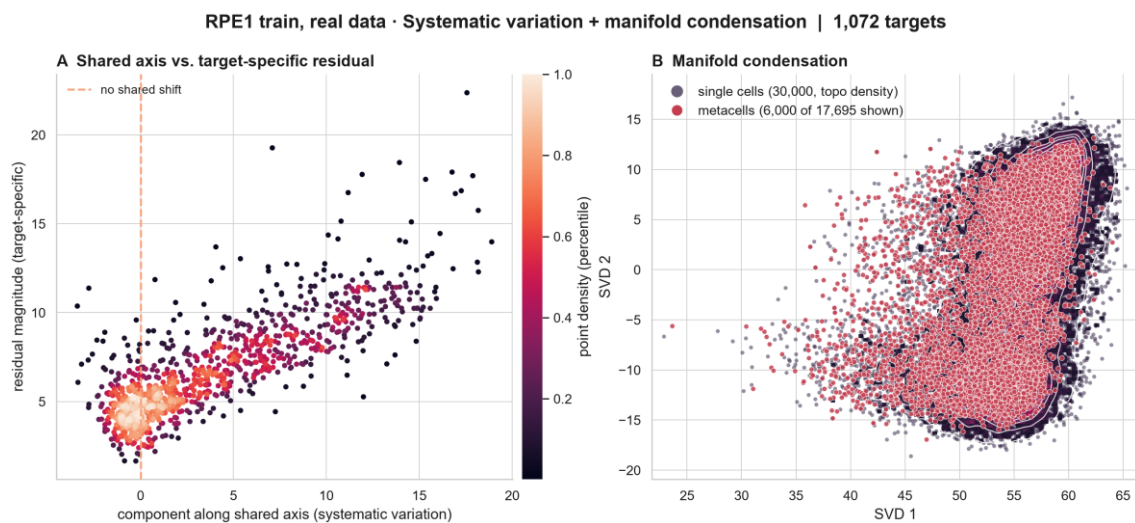


Figure 2.10 | The shared systematic-variation axis. Perturbed cells converge onto one shared response direction (cell-cycle arrest) regardless of which essential gene was targeted, and the regulator-specific signal is the smaller residual. This shared axis is what the CHITIN corrections attempt to remove.

2.5.2 — Why it is hard: a confound made of the same biology

The obstacle is that this axis is not cleanly separable from the signal of interest, because it is partly made of that signal. The shared arrest program is at once the dominant confound, since it separates perturbed cells from controls no matter which gene was targeted, and a genuine downstream consequence of the perturbations themselves, since knocking out an essential gene really does drive the cell toward arrest ³⁶. The confounding signal and the causal signal are therefore built from the same molecules. A correction strong enough to remove the shared axis will strip away some of the real regulatory biology that travels with it, and a correction gentle enough to preserve that biology will leave much of the confound behind. CHITIN, in short, attempts to subtract a confound made largely of the same biology it is trying to preserve, and that tension is a property of the data rather than of any one method. It is what makes systematic-variation removal a hard, open problem rather than a routine cleaning step, and why the outcome cannot be known before the correction is run.

2.5.3 — CHITIN as a family of corrections

Because no single operation is obviously right, CHITIN is designed as a family of corrections rather than one fixed method, spanning a spectrum of aggressiveness. At the blind end sit methods that estimate the shared axis ³⁶ directly from the control cells and subtract it from every cell, whether by removing the dominant direction of variation, regressing out the leading components of the systematic manifold, or subtracting a nearest-neighbor estimate of the background state. At the targeted end sits the removal of one specific, named program, most

naturally the cell-cycle signature the shared axis is largely composed of, taken out by regressing against curated cell-cycle phase markers ³⁵ rather than against a blindly-estimated direction. Latent-variable models that try to learn a perturbation-specific representation, and cell-filtering approaches that discard the most confounded cells outright, are further members of the same family at different points on the same trade-off. Every variant is fit on each cell line's training-split control cells and applied to that line's own splits, so the correction, like every other data-derived quantity in SPORE, learns nothing from the held-out perturbations. What separates the members is how surgically they act, and that is the axis along which the later chapters weigh them.

2.5.4 — Judging a correction without being deceived by it

A final difficulty has less to do with correcting systematic variation than with knowing whether a correction helped, and it shapes how CHITIN has to be evaluated. The trap is that the obvious measures of a good perturbation model reward the confound. A corrected dataset can score well simply by reproducing the shared arrest axis, because that axis is large, reproducible, and common to every perturbation, so a metric that credits it will call a correction successful even when nothing has been done for the regulator-specific signal ³⁶. Judging CHITIN honestly therefore requires a measure that is independent of the shared response, one that asks whether the regulator-specific, causal signal survived rather than whether the overall perturbed-versus-control difference still looks large. Building that evaluation is part of the problem CHITIN poses, and the verdict it eventually returns belongs to the later chapters. For the pipeline itself, the standing decision is to keep CHITIN as an optional, under-investigation stage rather than a default, and to revisit it as future work. Its place in the pipeline is fixed; its verdict is not.

2.6 — From Clean Data to a Candidate Network

A raw Perturb-seq dataset enters SPORE as a matrix of millions of cells, contaminated by ambient RNA and doublets, carrying mislabeled escapers, and shot through with technical and systematic variation. What leaves is a different object. The cells have been cleaned and the escapers removed, the gene panel has been reduced to a feature-selected core, and the whole has been partitioned into training, validation, and test sets, one set of files per cell line, each ready for inference. Depending on how the pipeline is configured, the cells may also have been aggregated into metacells or corrected for systematic variation.

One guarantee outranks the rest, and the chapters that follow depend on it. Only the training split is handed onward to build the network, so the graph SHROOM infers, and every evaluation that eventually grades it, are sealed off from the held-out perturbations by the structure of the pipeline rather than by the care of whoever runs it. With a clean, leakage-safe training substrate in hand, the task turns from preparing the data to inferring a network from it, which is where the next chapter begins.

CHAPTER 3

Dense Graph Generation

HYPHAE & SHROOM

Turning the clean expression matrix into a network. Two complementary dense-graph generators, one built on co-expression and one on perturbation, and the super-graph that fuses their strengths.

3.1 — What a dense GRN is, and where it sits in MYCELIUM

Chapter 2 ends with a clean, leakage-safe training substrate and one job left undone. That expression matrix still has to become a network. This is the task of dense graph generation, and two complementary generators together with a fusion of the two carry it out. We first set out what a dense GRN is at this stage, why the pipeline holds the graph at full density rather than thresholding it early, and the properties of the worked substrates the generators run on.

3.1.1 — The dense parent as the pipeline's quality ceiling

FUNGI, the curation stage that follows, only ever removes edges. It never adds one and never re-weights the graph toward a relationship the generator did not already score. The sparse graph that reaches the downstream model is therefore always a subset of the dense graph produced here, which makes this stage the quality ceiling for everything downstream. A regulatory relationship that the generator failed to score cannot be recovered later, and a topology the dense parent cannot support is one no amount of pruning will reach. When a downstream target is missed, the first question is whether the parent graph could have supported it at all, because a target FUNGI cannot reach on a poor parent is a problem with the parent rather than the pruning. Chapter 4 returns to this when it formalizes pruning-only curation. The consequence for this chapter is the ordering it follows. Build the richest, most faithful dense graph first, and defer every thresholding decision to the curation stage that can make it with full knowledge of the graph.

3.1.2 — Nodes, directed weighted edges, and the full candidate edge space

A dense GRN at this stage is a directed, weighted graph over the gene panel. The nodes are the genes carried forward from SPORE, and the generator assigns a score to every candidate directed pair ($i \rightarrow j$), one number for the hypothesis that gene i regulates gene j . With V genes the candidate edge space is the full $V \times V$ set of ordered pairs, so for the 5,000-gene panel used throughout this thesis the generator scores on the order of twenty-five million directed candidates. No edge is discarded and no score is thresholded. The output is effectively a complete graph in which almost every gene pair carries some weight, and the weight, not the presence of an edge, is what distinguishes a likely regulatory link from a spurious one. This is the object FUNGI prunes into a sparse graph, where most candidate pairs are dropped and only the edges that carry regulatory signal survive.

3.1.3 — Why the generator stays dense

Holding the graph at full density out of the generator is a deliberate division of labor. A generator could threshold its own output, keeping only its top-scoring edges per gene or above some global cutoff, but that decision would be made greedily, by one method, with no view of the network the edges form. Thresholding is done once instead, globally, and with topology awareness, downstream in FUNGI, where the cut can be driven by what the whole graph should look like rather than by each edge's score in isolation. Keeping the generators dense also keeps them honest as inputs to fusion, because two graphs can only be merged edge for edge if neither has already thrown its low-ranked edges away. The generator's task is therefore to score every candidate as well as it can. Deciding which of those scores become edges belongs to the curation stage.

3.1.4 — The worked substrates: RPE1 and the VCC H1-hESC data

The generators in this chapter run on two Perturb-seq substrates, and the same three properties matter for each. The primary substrate is the RPE1 genome-scale Perturb-seq data, used as the worked example throughout the thesis²¹. Its gene panel is the feature-selected 5,000-gene core, which sets the node set and the candidate edge space both generators score over. Its expression input is the hybrid metacell substrate Chapter 2 constructs, in which each perturbation's cells

are pooled into metacells while the control cells are kept single, so both generators read the same de-noised yet control-sharp matrix. Its perturbations are split into disjoint training, validation, and test sets, and only the training split is used to build the graph, the firewall that keeps the downstream evaluation honest ^{8,21}.

The second substrate is the H1 human embryonic stem cell (H1-hESC) dataset from the Virtual Cell Challenge, an open perturbation-prediction benchmark whose held-out task is to predict the expression response to silencing a gene the model was never trained on ⁵⁵. It carries roughly three hundred CRISPRi perturbations of H1-hESC, split into 150 training, 50 validation, and 100 held-out test perturbations, and it enters this thesis as a second cell line on which to test whether the generators' behavior replicates beyond RPE1 ⁵⁵. Its far smaller perturbation set matters later in this chapter, because how many genes were perturbed sets a hard limit on what a causal generator can cover, a limit that is starker on H1-hESC than on RPE1.

The provenance of both datasets and the construction of their splits belong to Chapter 2. What matters for the generators is the panel they score over, the matrix they read, and the guarantee that the graph they produce has never seen a held-out perturbation.

3.2 — Two schools of thought: association and causation

The two generators this chapter builds come from two different answers to one question, which is what an inferred edge actually means. That question splits GRN inference into an association school and a causal school, and the split is the reason the chapter builds a generator of each kind rather than settling on one. This section frames the distinction and the benchmarks the field uses to adjudicate it, and leaves the method detail of each school to the pillar that develops it.

3.2.1 — Co-expression (association) versus causal (interventional) inference

An association method reads an edge from statistical co-variation, treating two genes that move together across cells as evidence of a regulatory link, while a causal method reads an edge from perturbation response, taking the genes that shift after a specific gene is silenced as its downstream targets. The difference decides what an edge can claim. Co-expression cannot say which of two co-varying genes regulates the other, or whether a third gene drives both, so its edges carry weak direction ¹⁴. A perturbation fixes that direction from the intervention itself, because the silenced gene is the cause and the genes that respond are the effects ^{8,9}.

Neither school is a strict superset of the other, and the reason is coverage. An association method can score an edge out of any gene, because every gene has expression that co-varies with the rest, so it covers the whole panel. A causal method can only score an edge out of a gene that was actually perturbed, because its signal is the response to silencing that gene, so its coverage is bounded by how many genes the screen knocked down. The two schools therefore trade off along opposite axes. Association buys full coverage at the cost of direction, and causation buys direction at the cost of coverage. This complementary profile is what the chapter exploits. It builds HYPHAE, a co-expression generator with full coverage, and SHROOM, a causal generator with sharp direction, then fuses the two into a super-graph that aims to inherit both strengths. The methods behind each generator are developed in the two pillars that follow.

Figure 3.1 shows the same neighbourhood read both ways, as a symmetric association and as a directed causal effect.

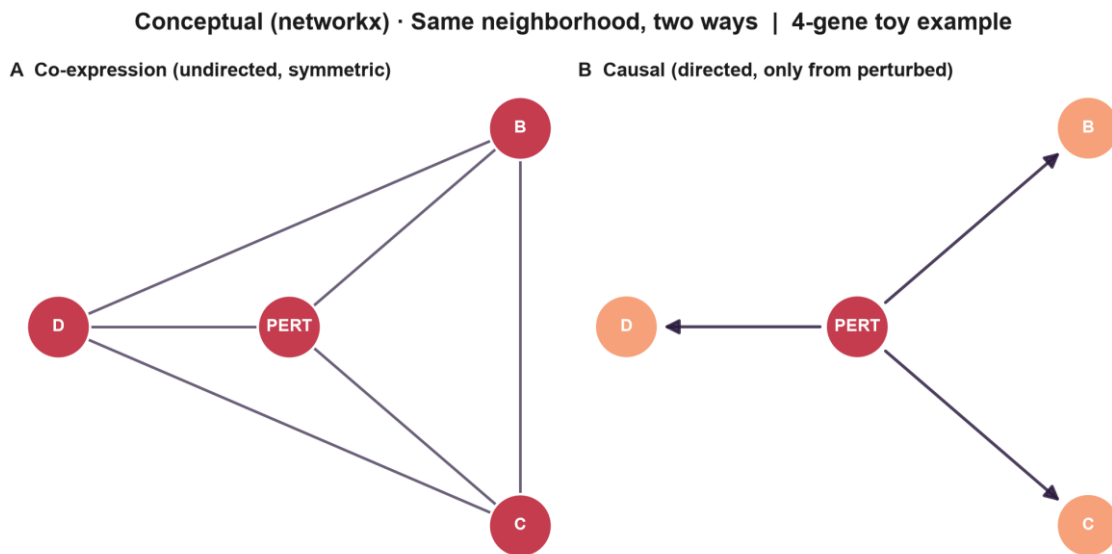


Figure 3.1 | The same gene neighbourhood, read two ways. A co-expression method treats genes that move together as an undirected association, while a causal method reads direction from perturbation and points an edge from the silenced gene to the genes its knockdown moves. These are the association and causal schools the chapter builds a generator for.

3.2.2 — Why GRN inference is benchmarked, not solved

No ground-truth GRN exists to fit against, and many different networks reproduce the same expression data equally well, so GRN inference is underdetermined and the field measures progress against community benchmarks rather than a single correct answer ⁶. Three of these benchmarks set the reference points this chapter relies on. DREAM5 assessed more than thirty inference methods across bacterial, yeast, and in-silico networks and is the standard for community inference ⁵⁶. CausalBench evaluates network inference on single-cell perturbation data, the interventional setting the causal school targets ⁹. BEELINE benchmarks single-cell GRN methods specifically and is the reference for that regime ⁶. One DREAM5 result carries directly into this chapter. No single method was best across all networks, and aggregating the predictions of complementary methods gave more robust performance than any method alone ⁵⁶. That finding is the empirical seed of the super-graph, and the fusion section returns to it. How a dense generator is scored in this pipeline, against biological and causal precision, is taken up at the end of the chapter.

3.3 — HYPHAE: Heuristic Yield of Parallel Hub Assembled Edges

The association generator. HYPHAE scores an edge from how genes co-vary across the expression matrix, which lets it place an out-edge from every gene in the panel and gives it the full coverage a perturbation-derived method cannot reach.

“By ceaselessly remodelling themselves they can navigate mazes, solve complex routing problems and expertly explore their surroundings.”

Merlin Sheldrake, Entangled Life

3.3.1 — What HYPHAE is

HYPHAE is the association generator of the two the chapter builds. It produces a dense graph in which every gene carries outgoing edges, inferred purely from how genes co-vary across the expression matrix, with no perturbation information used to place an edge. That full coverage is its defining property and the reason it earns a place alongside a causal generator, because it can

supply out-edges for the genes a perturbation-derived method leaves blind. HYPHAE sits in the tree-ensemble co-expression lineage that GENIE3 established and GRNBoost2 scaled ^{14,38}, and the next subsections give its method, its concrete realization in code, and how it is run at scale.

3.3.2 — Method: tree-ensemble feature importance

HYPHAE infers edges the way GENIE3 does. For each target gene it fits a regression that predicts that gene's expression from the expression of all other genes, and it reads the fitted model's feature importances as directed edges, one from each predictor gene to the target ¹⁴. A gene whose expression sharply improves the prediction of the target is a candidate regulator of it, so the importance of a predictor becomes the weight of a `Regulator → Target` edge. GENIE3 introduced this per-target regression decomposition with random forests and won the DREAM5 assessment with it ^{14,56}, and GRNBoost2 kept the same importance-as-edge principle while swapping in gradient-boosted trees for the speed needed at single-cell scale ³⁸. HYPHAE is a realization of that gradient-boosted form, fitting one LightGBM regression per gene and taking each model's gain importance, the total reduction in prediction error a feature contributes across all of its tree splits, as the edge weight.

Figure 3.2 shows the step in code, where a fitted model's feature importances are read out as directed edges.

```
HYPHAE/2_run_HYPHAE_worker.py::_train_gene_worker
# one LightGBM model PER TARGET GENE -- runs for all 5,000 genes
X = X_shared[:, feature_indices] # every OTHER gene's expression
y = X_shared[:, target_idx]      # this gene's expression

model = lgb.train(params, dtrain, num_boost_round=n_estimators,
                  valid_sets=[dval],
                  callbacks=[lgb.early_stopping(50)])

# gain importance -> directed Regulator -> Target edges
total_importance += model.feature_importance(importance_type="gain")
...
records = [{"Target": target_gene, "Regulator": name, "Importance": score}
           for name, score in zip(features_used, avg_importance) if score > 0]
```

Figure 3.2 | Importance becomes an edge. For each target gene, HYPHAE fits a gradient-boosted regression on the expression of every other gene and reads each predictor's gain importance as the weight of a directed edge into that target. A predictor that sharply improves the target's prediction becomes a candidate regulator of it.

3.3.3 — The HYPHAE realization: bootstrapping, early stopping, and the tuned learner

A single tree-ensemble fit gives a noisy importance ranking, so HYPHAE stabilizes each gene's edges by bootstrapping. It trains twenty LightGBM models per target gene, each with a different random seed and a fresh 80/20 split of the metacells into a training and a validation set, and averages the gain importances across the twenty replicates into the final edge weights. Every replicate is trained with early stopping against its own held-out validation loss, halting once that loss has not improved for fifty rounds, so a gene with weak regulatory signal spends few rounds and a gene with clear signal is fit fully, and the stopping criterion is measured on metacells the model did not train on rather than on its training error.

The LightGBM settings are tuned for metacell-aggregated Perturb-seq input rather than left at their defaults. The per-split feature fraction is lowered from `0.9` to `0.7`, so each tree split sees a smaller random subset of the roughly five thousand candidate predictors, which sharpens the randomization that separates true regulators from merely co-expressed genes across the bootstrap replicates. The maximum tree depth is capped at `8` rather than left unlimited, because metacell expression is already smoothed by aggregation and deep trees on smooth data chase noise. The boosting-round ceiling is raised to `1000`, an upper bound that early stopping rarely reaches, with clear-signal hub genes typically halting in the low hundreds of rounds. Only

edges with a strictly positive averaged importance are written, so a predictor that never contributed to any split for a target produces no edge.

3.3.4 — Engineering: parallel inference and consolidation

The five thousand per-gene regressions are independent of one another, which makes HYPHAE straightforward to parallelize. Inference is split across a ten-task compute array, each task taking a contiguous slice of the gene list, and within a task the per-gene fits are distributed across thirty-two worker cores. The expression matrix is written once to a shared memory map that all thirty-two workers read, so the matrix is not copied thirty-two times into memory. Each gene's result is checkpointed to its own file as it completes, and a task that restarts skips every gene already on disk, so the job survives a wall-time limit or a failed task by resuming where it stopped rather than recomputing. Once all ten tasks finish, a single consolidation step stitches their outputs into one edge list and runs a pre-flight diagnostic before the graph is handed to FUNGI, checking that no edge is duplicated, that no importance is missing, and reporting the node and edge counts, the degree distribution, and the weight statistics, along with how many genes were dropped for never receiving a positive importance. The result is one dense parent graph, a CPU-cluster job that runs in roughly eighteen hours on this substrate.

3.3.5 — What it captures, and HYPHAE as a valid standalone GRN

HYPHAE captures statistical co-regulation across the entire panel, and its signature strength is coverage. Because every gene is used as a predictor for every other gene, every gene receives outgoing edges, including the held-out perturbation targets that a causal generator cannot reach. Its weaknesses follow from the same association basis. Co-expression is close to symmetric, so the direction of a HYPHAE edge is weakly identified, and two genes that co-vary because a third drives both, or because they belong to the same complex, score as strongly as a true regulator and its target¹⁴. These limits do not disqualify it as a network in its own right. The GENIE3 and GRNBoost2 methods it realizes are established, benchmark-topping GRN inference approaches^{14,38,56}, a co-expression network is a legitimate and widely used regulatory prior, and its full coverage makes it usable wherever perturbation data is unavailable. HYPHAE is a complete regulatory graph on its own, not merely an ingredient for the fusion that comes later.

3.3.6 — Cost, hardware, and the bridge to a causal generator

HYPHAE is expensive to produce. Five thousand per-gene tree-ensemble fits, each bootstrapped twenty times with early stopping, add up to a large CPU workload that runs for roughly eighteen hours across a ten-task, several-hundred-core array. It is a CPU-cluster job by design, and rebuilding it for a new dataset means paying that cost again. This compute cost, together with the conceptual limit that association cannot supply the direction of regulation, is what motivates the second generator. A method that reads direction from perturbation and is fast enough to rebuild per dataset on a single accelerator would answer both, and that is what SHROOM is built to be.

3.4 — SHROOM: Self-training Heuristic for Regulatory Orientation Of Mechanisms

The causal generator. SHROOM reads the direction of regulation from perturbation, so its edges point from a silenced gene to the genes its knockdown moves, and it rebuilds on a single GPU rather than a CPU cluster.

3.4.1 — What SHROOM is

SHROOM is the causal generator of the pair. It reads the direction of regulation from perturbation rather than from co-variation, so its edges point from a regulator to the genes its knockdown moves, and it is built to run per dataset on a single GPU rather than a CPU cluster. It

is a faithful implementation of PSGRN, the perturbation self-training method of Song et al. ⁴⁵, and it is the fast, directed counterpart to HYPHAE. The subsections that follow give the self-training method, the interventional feature construction that makes its edges causal, how it runs on a GPU, and the coverage limit that motivates fusing it with HYPHAE.

3.4.2 — Method: PSGRN self-training

SHROOM scores edges with a single self-training step. It trains one global binary classifier over every directed gene pair, using pseudolabels derived from correlation as the training target, and then re-scores every pair with that same classifier to produce the edge weights ⁴⁵. The self-training name comes from this structure. The pseudolabels are not a curated ground truth but a coarse signal computed from the data itself, a correlation strong enough to be called a candidate edge, and the classifier learns to separate likely edges from unlikely ones on features that carry more than correlation, then applies that learned rule to the whole graph. A gene pair scores high when its features look like those of the correlation-seeded positives, and the output importance is the classifier's predicted probability for that pair. PSGRN's published implementation is CPU-based and, in this project, only ever ran on the now-offline compute cluster. SHROOM reimplements the same formulation as a GPU-native Python process, chosen so the causal generator is fast enough to rebuild for each dataset.

3.4.3 — The interventional signal: why SHROOM is causal

What makes a SHROOM edge causal rather than co-expression is the feature set the classifier reads. Each directed pair from a source gene to a target gene is described by four numbers. Two are baselines, the mean expression of the source and of the target across the control cells. The other two are interventional, the mean expression of the source in the cells where the source itself was knocked down, and the mean expression of the target specifically in those same source-knockdown cells. That fourth feature is the one that reads the effect of intervening on the source, because it measures where the target sits when the source is silenced, and it is undefined when the source was never perturbed. The construction is asymmetric by design. The features for the pair from gene A to gene B are computed from A's knockdown, while the features for B to A are computed from B's knockdown, so the score of an edge depends on which gene was the intervention target and the edge therefore carries a direction. This is the signal association methods lack, and reading it from the intervention is what licenses a causal claim about the edge ^{8,9}.

Figure 3.3 shows the four-feature construction that gives a SHROOM edge its direction.

```
SHROOM/shroom.py::build_training_arrays
# defaults to NaN; only rows for PERTURBED source genes get filled
int_mean_other = np.full((n_genes, n_genes), np.nan, dtype=np.float32)
for gene, int_idx in pert_map.items(): # one row per PERTURBED gene
    src_i = gene_lookup[gene]
    int_mean_other[src_i] = interv_only.mean(dim=0) # tgt mean, src knocked down
...
# the four causal features, per directed (src, tgt) pair -- verbatim from build_training_arrays
feature_matrix[:, 0] = np.broadcast_to(obs_mean.reshape(-1, 1), (n_genes, n_genes))[off_diag]
feature_matrix[:, 1] = np.broadcast_to(int_mean.reshape(1, -1), (n_genes, n_genes))[off_diag]
feature_matrix[:, 2] = np.broadcast_to(int_mean_self.reshape(-1, 1), (n_genes, n_genes))[off_diag]
feature_matrix[:, 3] = pseudo["int_mean_other"][off_diag] # NaN if src never perturbed
```

Figure 3.3 | The four features that make a SHROOM edge causal. Each directed pair is described by two control baselines and two interventional means, the source and target expression in the cells where the source was knocked down. The interventional pair reads the effect of silencing the source, and it is undefined when the source was never perturbed, which is what ties SHROOM's coverage to the perturbation set.

3.4.4 — Engineering: GPU pseudolabels and single-pass scoring

The pseudolabels and features are computed on the GPU in two passes over the expression matrix. The first pass handles every gene at once, z-scoring the control cells and taking a single matrix multiplication to get the absolute correlation of every gene with every other, which supplies the pseudolabel row for each non-perturbed source. The second pass visits each

perturbed source gene in turn and overwrites that source's correlation row using a matched sample of control and knockdown cells, and at the same time reads off the interventional means from the knockdown cells. Thresholding the resulting correlations gives the binary pseudolabels, the classifier is fit once on the four-feature matrix over all off-diagonal pairs, and it scores every pair in a single pass to yield the dense graph. There is one round of training and one of scoring, with no iterative relabeling, and because the heavy correlation work is a couple of GPU matrix operations rather than five thousand separate regressions, SHROOM rebuilds a whole dense graph on one accelerator in a fraction of HYPHAE's cluster time. That speed is the point of a GPU-native causal generator.

3.4.5 — What it captures, and SHROOM as a valid standalone GRN

SHROOM captures a directed causal backbone. The interventional features give its edges a regulatory orientation and a precision-per-edge that co-expression cannot reach, because an edge scored from what actually changed when the source was silenced is closer to a regulatory statement than one scored from what merely co-varies with it. It is a complete GRN in its own right, a faithful realization of a published and benchmarked causal method ⁴⁵, producing directed, interventionally grounded edges that association methods cannot, and it scores well on causal precision against held-out perturbation response, the evaluation the chapter takes up at its close. Where HYPHAE offers coverage, SHROOM offers direction and precision.

3.4.6 — Perturbation-bounded coverage, and the bridge to fusion

SHROOM's limitation is the mirror image of its strength. The interventional features exist only for genes that were actually perturbed, so only a perturbed gene carries a genuine causal out-signal, and every gene that was never knocked down is effectively a sink with incoming edges but no outgoing ones. Source coverage is therefore capped at the panel's perturbation coverage, and that cap is severe at genome scale. On the RPE1 panel roughly a thousand of the five thousand genes were perturbed, so at most about a fifth of the panel can be a regulator, and on the H1-hESC panel only 108 of the 5,024 genes were perturbed, close to two percent. The root cause is a mismatch of regime. PSGRN was designed and validated on data where every gene was perturbed, its native full-coverage setting, whereas it is run here at five-thousand-gene scale against a few hundred to a thousand perturbations, far outside that design point. SHROOM is sparser, directed, and higher-precision-per-edge than HYPHAE, and lower-coverage. That complementary profile, full coverage without direction on one side and direction without coverage on the other, is exactly what the super-graph is built to exploit.

Figure 3.4 makes the coverage ceiling concrete, showing what fraction of the panel each generator can even emit an edge from.

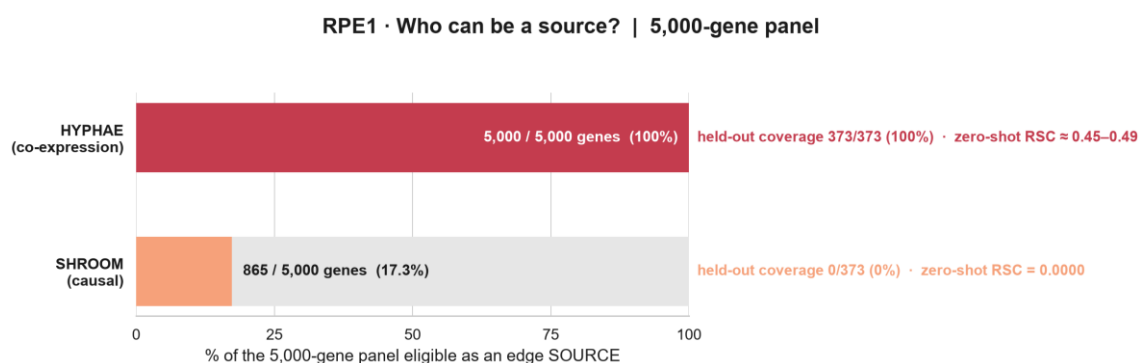


Figure 3.4 | Who is allowed to be a source. The fraction of the 5,000-gene panel that can emit an out-edge. HYPHAE scores an out-edge from every gene and covers the whole panel, while SHROOM can only score one from a perturbed gene and reaches roughly one gene in six. The rank-score correlation printed at each bar is the downstream consequence. A co-expression graph still predicts held-out responses at about 0.47, while the causal graph scores 0.0000 on the genes it never perturbed.

Figure 3.5 shows the mechanism behind that ceiling in the out-degree distributions of the three arms.

RPE1 · Out-degree: a few causal hubs vs broad co-expression | 5,000-gene panel

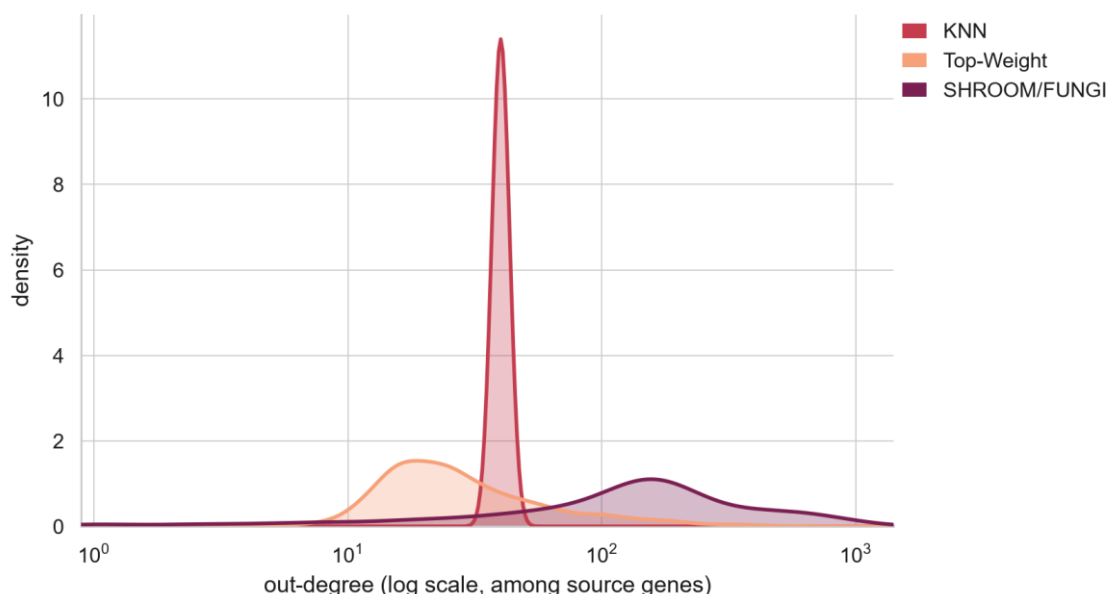


Figure 3.5 | Why the ceiling exists, in the out-degree distributions. The co-expression baselines, KNN and Top-Weight, spread their out-edges across all 5,000 genes at modest degree, while SHROOM concentrates every causal out-edge on a few hundred perturbed hubs and leaves roughly four thousand genes at out-degree zero. A handful of perturbed regulators carry the entire causal signal.

3.5 — The super-graph: merging association and causation

3.5.1 — Motivation: complementary generators and the wisdom of crowds

The reason to build two generators and combine them, rather than pick the better one, comes straight from the benchmark that opened this chapter. DREAM5's central finding was that no single inference method is best across networks, and that aggregating the predictions of complementary methods is more robust than any method alone ⁵⁶. NIMEFI carried that result into practice for this exact setting, aggregating several ensemble feature-importance GRN methods by rank and showing the aggregate outperforms its members ⁵⁷. HYPHAE and SHROOM are complementary in precisely the way that makes aggregation pay. One captures association across the whole panel, the other captures causal direction on the perturbed genes, and their edge signals are partly orthogonal. Fusing them is the wisdom-of-crowds argument applied to a co-expression graph and a causal graph.

3.5.2 — The MYCELIUM super-graph: best of both, compensating weaknesses

The super-graph is built to inherit each generator's strength where the other is weak. HYPHAE's full coverage supplies out-edges for the held-out genes that SHROOM's perturbation-bounded inference leaves as sinks, and SHROOM's interventional direction sharpens the near-symmetric edges that HYPHAE cannot orient. Each generator's blind spot is the other's strong suit, which is the condition under which a fusion can be better than either input. The compensation is limited to the two conceptual weaknesses. Fusion addresses coverage and direction, and it does nothing for compute cost, because producing the super-graph still means producing both generators, so HYPHAE's CPU-cluster expense is paid in full.

Whether that combination pays off is the reader's natural question, and it does, in a way that splits along the same coverage-versus-direction line. For genes the graph was built on, the prune-then-fuse super-graph is the strongest of every graph tested at reproducing the measured perturbation responses, and fusing the causal and co-expression signals beats the pure co-expression baseline outright, which on the de-confounded measure is the worst of the graph-based arms. For genes that were never perturbed, where a causal graph is structurally silent, the super-graph predicts only as well as the co-expression coverage it inherits from HYPHAE, and the causal core adds little beyond that reach. Chapter 6 develops this comparison in full, with its controls, baselines, and significance; the two-sided result is what the rest of the pipeline is designed around.

3.5.3 — Ensemble technique: rank normalization and consensus operators

Fusing the two graphs takes two steps, and the first is rank normalization. HYPHAE's edge weights are gradient-boosted gain importances that can run to very large values, while SHROOM's are classifier probabilities bounded in the unit interval, so the raw weights are not comparable and averaging them directly would let one scale dominate. HYPHAE and SHROOM are therefore each converted to ranks before fusion. Every edge weight in a graph becomes a normalized rank in the unit interval, its average rank divided by the number of edges, and an edge absent from a graph is assigned rank zero so it sits below every edge that graph actually scored. On this common footing the two graphs are combined with a consensus operator from the rank-aggregation family that NIMEFI brought to GRN inference ⁵⁷. The default operator is a Borda count. The Borda count is a voting method, in which each voter ranks the candidates and each candidate is scored by its rank position summed or averaged across all voters, so the winner is whoever ranks consistently high with every voter rather than whoever any single voter puts first ⁵⁸. Applied here, the two generators are the voters and the candidate regulatory edges are the candidates. Each generator has already ranked every edge by weight, and the Borda score of an edge is the mean of its two ranks, the average of where HYPHAE places it and where SHROOM places it, with an adjustable weight on the two that defaults to an even split. An edge both generators rank near the top earns a high mean rank and rises in the fused graph, while an edge only one generator favors is pulled down by the other's low rank. The alternative operator is a min-rank, the smaller of an edge's two ranks, which is stricter still and lets an edge score high only when both generators rank it high. Both operators encode the same claim, that an edge an association method and a causal method independently rank highly is more trustworthy than an edge only one of them favors, and the agreement between SHROOM and HYPHAE is the reliable backbone the fusion is after.

The production super-graph is built with the Borda operator, and applied in the prune-then-fuse order taken up next, which is why the arm that carries it through the downstream evaluation is labelled `'ptf_borda'`, for prune-then-fuse Borda. It is the recipe the rest of the thesis means by the super-graph.

Figure 3.6 shows the fusion in a single expression, the Borda mean of the two rank vectors.

```
fuse_graphs.py
def rank01(w):
    return (rankdata(w, method="average") / len(w)).astype(np.float32)

def scatter_rank(si, ti, w, N):
    # dense N*N rank array; missing edges stay 0 (tie_0)
    arr = np.zeros(N * N, dtype=np.float32)
    arr[si * N + ti] = rank01(w)
    return arr

rA = scatter_rank(aS, aT, aW, N) # A = causal-maximalist (SHROOM/PSGRN)
rB = scatter_rank(bS, bT, bW, N) # B = co-expression-maximalist (HYPHAE)
fused = (w * rA + (1.0 - w) * rB).astype(np.float32) # default w=0.5
```

Figure 3.6 | Rank fusion in one expression. Each generator's edge weights are converted to normalized ranks, and the Borda score of an edge is the mean of its two ranks, so an edge both HYPHAE and SHROOM rank highly rises in the fused graph while an edge only one favours is pulled down. An edge absent from a graph takes rank zero.

3.5.4 — Merge order: prune-then-fuse versus fuse-then-prune

The order in which fusion and pruning are applied is a real engineering decision, and the wrong order breaks the graph. Fusing the two dense parents first and pruning the consensus afterward can shatter. A rank consensus of a directed causal graph and a near-symmetric co-expression graph inherits the co-expression graph's symmetry, so nearly every edge appears alongside its reverse and the consensus has a reciprocity close to one. FUNGI prunes toward a directed backbone with a reciprocity in a low target band, far from one, and it cannot resolve a graph in which almost every edge is reciprocated into a directed structure, so a large fraction of nodes lose all their edges and the pruned graph collapses. A prototype run on the H1-hESC substrate makes the failure concrete. FUNGI pruning a co-expression graph on its own is healthy, driving that graph's reciprocity from 0.9999 down to 0.145 and bringing its topology into the target ranges, but pruning the fused consensus shattered it, orphaning twenty to thirty percent of the nodes with reciprocity stuck near one, and not one of the 256 pruning settings searched produced a usable graph. This failure is structural rather than a matter of tuning, because FUNGI's collapse check is applied to the pruned edge set independently of the reciprocity objective, so neither widening nor disabling the reciprocity target rescues a fused-then-pruned consensus. The order that works is prune-then-fuse. FUNGI prunes SHROOM and HYPHAE separately, where each is a single directed graph it can resolve, and only then are the two pruned graphs fused. Each pillar reaches a directed backbone before it meets the other, so the fusion no longer has a near-reciprocal parent to choke on. Prune-then-fuse is the adopted order for the production super-graph. The FUNGI pruning mechanism this rests on is developed in Chapter 4, and the head-to-head showing the prune-then-fuse super-graph beats structure-blind consensus prunes is in Chapter 6.

3.6 — Evaluating a dense generator, and why the real test is downstream

3.6.1 — Two evaluation axes: biological and causal precision

A dense generator is judged on two axes, and the two disagree in a way that shaped the whole ensemble. The first is biological precision, whether the graph's top edges recover known interactions. This is scored against curated interaction databases, most directly STRING, whose edges are weighted heavily by co-expression evidence⁵⁹, so an edge that reproduces a known association counts as a hit. The second is causal precision, whether the graph's top edges reflect genuine regulatory control. This is scored against held-out perturbation response, in the style of the interventional benchmarks, by asking whether the predicted regulators of a gene actually

move it when they are silenced in cells the graph was not built from ⁹. A third check sits alongside them, testing whether an edge's apparent effect is specific to its regulator rather than an echo of the systematic, shared perturbation response that runs through the whole dataset. The two main axes rank the generators in opposite orders. HYPHAE wins on biological precision, because co-expression is exactly what STRING rewards, while SHROOM wins on causal precision, because its interventional features are built to recover regulatory direction. Neither generator is simply better, and that split is the concrete evidence behind the wisdom-of-crowds argument for fusing them.

Table 3.1 shows the split directly, with each generator winning the axis its own design targets.

Table 3.1 | The evaluation asymmetry. Top-1,000 edge precision on the RPE1 5,000-gene panel, measured on two axes. Biological precision scores the top edges against a STRING-based gold standard, while causal precision asks whether the predicted regulators actually move their targets when silenced in held-out cells. The co-expression school wins the first axis and the causal school wins the second.

Precision axis	Co-expression	Causal (SHROOM)
Biological precision (top-1,000, STRING-based gold standard)	0.567	0.250
Causal precision (top-1,000, held-out perturbation)	0.636	0.992

Co-expression figures use Pearson correlation as the representative of the association school; the HYPHAE tree-ensemble was not separately scored on these axes. Values from the evaluation-framework audit on the RPE1 5,000-gene panel.

3.6.2 — The real test is the downstream model

Precision on either axis is necessary, and neither is sufficient. A dense graph is a means, not an end, and its real worth is whether it helps the downstream model predict how cells respond to perturbations it has not seen. A graph can score well against a curated database and still be the wrong input for a model, and a graph can lose a little edge-level precision and still be the better substrate once a model propagates signal across it. This is why the evaluation that decides the pipeline is not a precision score on the graph itself but the performance of the model that consumes it. That reframing sets up the rest of the thesis. Chapter 4 takes the dense super-graph built here and curates it with FUNGI, pruning it toward the topology of a real regulatory network. Chapter 5 introduces RHIZO, the downstream model that reasons over the curated graph to predict perturbation responses. Chapter 6 puts the two together and asks the question this chapter has been building toward, whether a curated, topology-aware graph lets that model predict held-out responses better than the structure-blind baselines. The dense graph is where the pipeline's quality ceiling is set. Whether the pipeline reaches it is decided downstream.

CHAPTER 4

FUNGI

Functional Unraveling of Network Geometry for Inference

Pruning reframed as optimization. FUNGI treats the shape of a real regulatory network as an explicit objective and searches for the sparse graph that matches it, curating the dense parent into a usable regulatory map.

The fungus is the mature body the whole mycelium grows toward, the form that finally takes shape above ground. FUNGI plays that part here, carving the dense, formless parent graph down to the structured network the rest of the pipeline was reaching for.

4.1 — Motivation

Every earlier stage of this pipeline exists to hand FUNGI a dense, noisy graph, and FUNGI's job is to turn that graph into a usable one. A perturbation-aware inference method assigns a score to every candidate regulatory edge, so its raw output is effectively a complete graph in which almost every gene pair carries some weight ⁴⁵. That dense parent is unusable as a regulatory map and actively harmful as a model input. Most of its edges are weak correlations rather than regulatory relationships, none of the hub hierarchy or modular structure of a real GRN survives at that density ⁴, and a downstream graph model run over it over-smooths, blending every gene's signal into every other's until regulators and targets can no longer be told apart ⁴⁰. The graph has to be pruned down to the edges that matter.

Pruning is easy to do and hard to do well. The same dense graph can be cut to a sparse one in an enormous number of ways, and the simplest strategies each fix on a single quantity. Top-weight pruning keeps the highest-weighted edges, a k-nearest-neighbor rule caps every gene's out-degree, and a connectivity backbone holds the network together at minimum cost. Each hits its one target and misses the biology as a whole, and these are the baseline methods this chapter later measures against. The question FUNGI answers is which pruning yields the graph that is at once the most biologically faithful and the most useful to the downstream model that will consume it.

FUNGI, for Functional Unraveling of Network Geometry for Inference, reframes that question as an optimization problem. Instead of applying a fixed rule, it treats the shape of a real GRN as an explicit objective and searches for the sparse graph that matches it. The defining topological signatures of a biological network, its scale-free hub hierarchy, its regulatory inequality, its modular and motif structure, become target bounds, and a specialized scoring function, the DASH kernel, ranks candidate edges so that the surviving graph is driven toward those targets ⁴¹. Pruning becomes a hyperparameter search over a biologically-grounded objective. What sets FUNGI apart is that the objective is not hard-coded. The target bounds are read from the data itself, so the method adapts to whatever dataset it is given rather than assuming in advance what a regulatory network should look like. That data-driven, configurable objective is the design philosophy the next section sets out.

4.2 — Design Philosophy: Dataset Agnosticism and Configurable Bounds

FUNGI does not hard-code what a gene regulatory network looks like. The topology targets it optimizes toward are bounds, one interval per parameter, and where those bounds come from is a deliberate design choice rather than a constant baked into the method. This is what makes FUNGI dataset-agnostic. The same engine can tune a network for a retinal epithelial line, a leukemia line, or a dataset it has never seen, because the definition of a good graph travels with the data rather than with the code. Three sources can supply each bound, and they can be mixed freely, one parameter at a time.

4.2.1 — Data-derived diagnostic bounds (the default)

By default, FUNGI derives its target bounds from the dense parent graph itself. Before any pruning, a suite of diagnostic probes runs over the parent, and for each topology parameter these probes estimate the interval that a biologically faithful sparse graph should fall in for this particular dataset ⁴¹. The bounds are therefore calibrated to the data at hand rather than imported from another cell type or organism, which is precisely what lets one pipeline adapt to a new dataset without hand-tuning. The mechanics of these probes are taken up in the calibration section; here what matters is that the default answer to what this graph should look like is computed from the data, not assumed.

4.2.2 — Literature-sourced fallback bounds (FUNGI-supplied)

Diagnostics are not always the right call. A probe can be unreliable when the dense parent is too sparse or too structureless to estimate a bound from, and a user may simply not trust a data-derived interval for a parameter whose biological range is already well established. For these cases FUNGI ships a set of curated, literature-sourced fallback bounds for every parameter, ready to use in place of the diagnostic estimate. The reciprocity target is a clean example. Its band is taken directly from the network-science literature on directed link reciprocity⁶⁰, and FUNGI defaults to that literature band rather than probing for it. A user who prefers established biology to a per-run estimate can fall back to these curated bounds for any parameter, with no change to the engine.

4.2.3 — User bounds-override mode (fully custom)

The final source is the user. FUNGI exposes every target bound in its configuration, so a user can override any parameter with an interval of their own, instructing the search to hold the power-law exponent between 1.5 and 1.75, say, or to widen the hub-fraction band, and the optimization adopts it directly. This is what turns FUNGI from a fixed procedure into a tool. A user with a specific hypothesis about the topology they want, or a downstream application with its own structural requirements, can encode that intent in the objective rather than accept the defaults.

4.2.4 — Mix-and-match, with RPE1 as the worked example

These three sources are not mutually exclusive; they are chosen one parameter at a time. A single run can take its power-law and clustering bounds from the diagnostics, its reciprocity band from the supplied literature fallback, and a hand-set hub-fraction interval from the user, all in the same objective. The RPE1 configuration used throughout this thesis illustrates the mix. Its diagnostic probes place the power-law exponent in roughly [2.2,2.5] and the in-degree concentration near [0.55,0.69], the hub fraction is bounded near [0.09,0.18] and the feed-forward-loop clustering near [0.10,0.16], while the reciprocity band [0.02,0.12] is supplied from the literature rather than probed. The edge density is set separately by the user as a target the search prunes toward. The specific values matter less here than the fact that a single working objective is assembled from all three sources at once, which is what dataset agnosticism looks like in practice.

4.3 — Biological Topology Targets

FUNGI prunes toward the shape of a real gene regulatory network, and this section defines the parameters that make that shape measurable. Each is a graph-theoretic quantity on which a biological GRN and a random graph demonstrably differ, and each is developed the same way, through its biological context, its role in the optimization, and its quantitative definition. FUNGI can optimize toward all of them simultaneously, and which are engaged on any given run depends on what the dense parent graph can support, a point taken up with the utopian loss in the next section.

4.3.1 — α — Power-law degree exponent

The power-law degree exponent, denoted as α , serves as the foundational macro-topological constraint within the FUNGI, within the FUNGI framework. While the DASH equation relies on a multivariate balance of local network motifs and functional signals, α acts as the global boundary condition. This ensures that the algorithm prunes away dense, spurious connections while preserving the scale-free hierarchy characteristic of true cellular networks.

4.3.1.1 — Biological Context

Gene Regulatory Networks (GRNs) are not random graphs where all nodes (genes) have the same approximate number of edges. In graph theory, the degree distribution $P(k)$ is the

probability that a randomly chosen node has a degree k (i.e., they have k -number of edges). In a random graph this number of connections would follow a Poisson distribution, in which most nodes would have approximately k edges. This would make finding outlier nodes with very high number of connections or nodes that have almost no connections statistically unlikely.⁶¹ However, biology does not follow this type of distribution. In biology transcription factors dominate edge counts. Furthermore, most genes in fact are terminal effectors and may only receive one edge from a local hub or transcription factor and regulate nothing downstream of them. This kind of topology defined by a small cohort of transcription factors that regulate hundreds of downstream genes while the vast majority of genes only regulate a handful of targets if any at all is known as a scale free topology.^{4,62.}

Scale free networks follow a power-law distribution where

$$P(k) \sim k^{-\alpha}$$

In these networks we see what is known as a "long tail" distribution as a number of genes act as hubs controlling hundreds of downstream targets. The parameter α dictates how steep this distribution is. With a high α (e.g. > 3) the transcription factors in the network lose the ability to properly regulate the network and the GRN behaves more like a random graph. With a low α (e.g., < 2) the network is controlled by a few super-hubs to an extreme degree. True Biological GRNs exist at a state where α is between 2.0 and 3.0.^{63,64.}

4.3.1.2 — Optimization Rationale

When FUNGI first receives the raw parent graph from the GRN inference engine it is not yet scale free. The raw data is extremely noisy and often times exhibits nearly 100% density (i.e., nearly all nodes connect to all other nodes in the graph). This dense and noisy graph has a degree distribution that resembles a Gaussian distribution and would confuse a downstream Graph Neural Network (GNN) as the identity of transcription factors is lost in the dense noisy graph. This leads to "over smoothing" where the GNN sees messages from all nodes to all other nodes and true causal biological signals are lost.^{40.}

To regain the hierarchy of signal distribution that resembles biology we use α as a prior for our DASH equation. By doing so, FUNGI optimizes the network until we regain the signal distribution pattern that resembles the biological reality FUNGI is designed to optimize for. We balance α between under-pruning which would result in far too noisy of a signal for the GNN to use correctly and over-pruning in which we would have an extreme degree of distribution disparity. This balance between over and under pruning allows for the extraction of the true causal backbone of the GRN from the original raw data.

4.3.1.3 — Quantitative Identity

To calculate α we use the the Maximum Likelihood Estimate (MLE), where, given the out-degree sequence $\{k_i\}$ of the optimized graph, $k_{\min}$ must be established. $k_{\min}$ is the threshold degree above which the power-law behavior truly begins. At this point we observe the lowest degree threshold above which the topological distribution reliably follows a scale-free power-law, filtering out baseline noise. To establish the value for $k_{\min}$ we employ a brute force the Kolmogorov-Smirnov (KS) test.^{65.} This allows us to run this loop

- **Iterate:** Look at every unique degree d present in our graph
- **Assume:** For each value of d we test we work the assumption that $k_{\min} = d$. We then remove all nodes with a degree less than d
- **Calculate:** Using the MLE equation what α would be for the remaining nodes
- **Compare:** Looking at our empirical distribution (our actual data) and a theoretical power-law distribution (data generated using the assumed $k_{\min}$ and calculated α). We compare our empirical distribution (the actual data) against a theoretical power-law distribution. This KS Statistic allows us to see the maximum vertical distance between the empirical distribution and the currently observed power-law distribution.

- **Select:** We then repeat this for every possible value of d the resulting "true" $k_{\min}$ is the value that results in the smallest KS distance. We then use this $k_{\min}$ value for our MLE equation

The maximum Likelihood Estimate equation we use to calculate our value of α is

$$\hat{\alpha} = 1 + n \left[\sum_{i=1}^n \ln \frac{k_i}{k_{\min} - \frac{1}{2}} \right]^{-1}$$

where

- $\hat{\alpha}$ is the estimated power-law degree exponent of the candidate network
- n is the total number of nodes (genes) in the network that possess a degree greater than or equal to the threshold $k_{\min}$
- i is the index variable used to iterate through the n valid nodes
- k_i is the specific out-degree (number of outgoing regulatory edges) of i -th node
- $k_{\min}$ is the minimum degree threshold above which the topological distribution reliably follows a scale-free power-law.
- $-\frac{1}{2}$ is the mathematical correction factor used to adapt the continuous MLE formula for discrete, integer-based data (edge counts)

Ultimately, this calculated $\hat{\alpha}$ is fed directly into FUNGI's DASH equation during the Sparsification pipeline. By continuously comparing this empirical estimate against the biologically targeted bounds of [2.0,3.0], the engine precisely calibrates its penalty and rewards signals, dynamically steering the network out of a saturated state and into a biologically valid, scale-free topology.

Figure 4.1 contrasts the scale-free out-degree distribution FUNGI targets with the Poisson distribution of a random graph.

RPE1 · Scale-free causal wiring vs a random null | same mean out-degree (142), same n=1188 sources

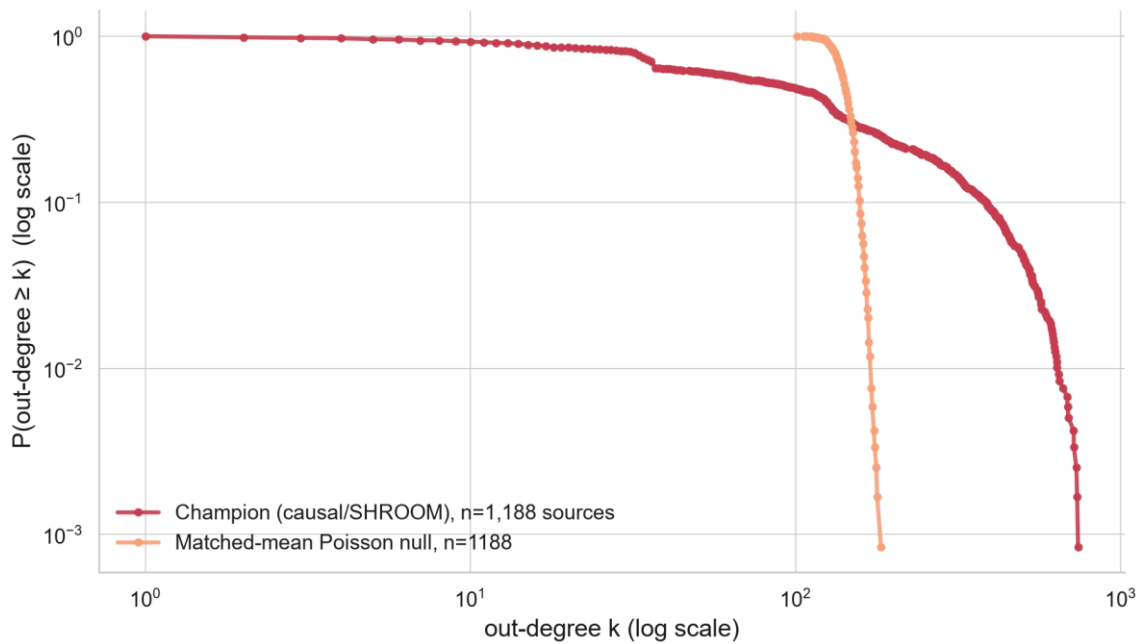


Figure 4.1 | Scale-free versus random out-degree. A power-law out-degree distribution, most genes regulating few targets and a few hubs regulating many, against a Poisson distribution of matched mean on shared log-log axes. FUNGI prunes toward the scale-free shape, since the dense parent's near-uniform degrees resemble the random case.

4.3.2 — Gini — Regulatory Inequality

The Gini coefficient, denoted as G , serves as the measure of regulatory inequality within the FUNGI framework. While the power-law exponent α defines the mathematical decay of connectivity, the Gini Coefficient quantifies the concentration of regulatory authority. By monitoring the disparity between master regulators and terminal effectors, FUNGI ensures that the resulting network maintains a biologically authentic hierarchy of control rather than collapsing into a uniform distribution of noise.

4.3.2.1 — Biological Context

As established in section 4.3.1, Gene Regulatory Networks follow a scale-free topology where a small cohort of transcription factors regulate hundreds of downstream targets while the vast majority of genes act as terminal effectors with few or no outgoing regulatory edges ^{4,62}. The power-law exponent α captures the rate at which this regulatory influence decays as we descend from master regulators to terminal effectors. However, α alone does not tell us how much the network's total regulatory output is concentrated in the hub tier versus distributed across the effector tier. This is the question that the Gini coefficient is designed to answer.

The Gini coefficient, expressed as G , originally developed as a measure of economic income inequality; maps directly onto the biology of gene regulation. ⁶⁶. In an economic context, a high Gini coefficient describes a network where a small number of transcription factors claim a disproportionate share of the total regulatory edge budget. Master regulators such as OCT4 and SOX2 in hESCs are analogous to the high-wealth tail of an income distribution. ⁵. They regulate hundreds of downstream targets while the vast majority of genes regulate few if any targets at all.

Critically, a biologically realistic GRN is neither perfectly equal nor infinitely unequal. A network where every gene regulates exactly the same number of targets would have a Gini of 0 and would be biologically meaningless. It would erase the functional distinction between transcription factors and effector genes entirely. Conversely, a network where a single transcription factor claims essentially all regulatory edges would have a Gini approaching 1 and would be equally unrealistic, as a real cellular regulatory programs are distributed across a coordinated hierarchy of master regulators, local hubs, and terminal effectors ⁶². True biological GRNs exist in an intermediate regime that reflects this hierarchy.

4.3.2.2 — Optimization Rationale

When FUNGI receives the raw dense parent graph from the GRN inference engine, the degree distribution is approximately uniform. Since the parent graph is a very dense and noisy system nearly all genes connect to nearly all other genes with similar edge counts, producing a Gini coefficient close to zero. This egalitarian structure is biologically wrong. It erases the master regulator identity of transcription factors, gives the downstream GNN no meaningful structural signal to exploit, and produces the same over-smoothing problem described in section 4.3.1 ⁴⁰. FUNGI uses the Gini coefficient as a complementary constraint to α within the DASH equation and utopia loss function. While α enforces that the degree distribution follows the correct power-law shape, Gini enforces that the resulting distribution has the correct degree of inequality. Making sure that the hub tier collectively claims the right portion of the total edge budget relative to the effector tier.

4.3.2.3 — Quantitative Identity

The Gini coefficient is computed from the out-degree sequence of candidate graph. That is to say the number of outgoing regulatory edges for each gene. Given the out-degree sequence $\{k_i\}$ for $i = 1, \dots, n$ genes sorted in non-decreasing order such that $k_1 \leq k_2 \leq \dots \leq k_n$, the Gini coefficient (G) is defined as

$$G = \frac{2 \sum_{i=1}^n i \cdot k_i}{n \sum_{i=1}^n k_i} - \frac{n+1}{n}$$

where

- n is the total number of genes (nodes) in the network
- k_i is the out-degree of the i -th gene when the sequence is sorted ascending
- i is the rank of gene i in the sorted sequence (1 = lowest degree)
- The subtracted term $\frac{n+1}{n}$ is a normalization correction that ensures a perfectly equal distribution yields $G = 0$

This formula is the discrete analogue of the Lorenz curve area calculation.⁶⁷ The Lorenz curve is constructed by plotting, for each cumulative of genes ranked from lowest to highest out-degree, the cumulative fraction of total edges they collectively hold. In a perfectly equal network this curve is the diagonal line of perfect equality. In a real GRN it bows below the diagonal. Where the bottom fraction of genes by out-degree collectively holds a much smaller fraction of total edges than their numerical share would imply. The Gini coefficient is the normalized area between the line of perfect equality and the empirical Lorenz curve, with $G = 0$ indicating the curve lies exactly on the diagonal and $G = 1$ indicating the curve collapses to the axes.

Figure 4.2 shows the same inequality as a Lorenz curve, the area between the curve and the diagonal being the Gini coefficient.

RPE1 · Out-degree inequality: the Lorenz curve | 5,000-gene panel, 1,188 active sources

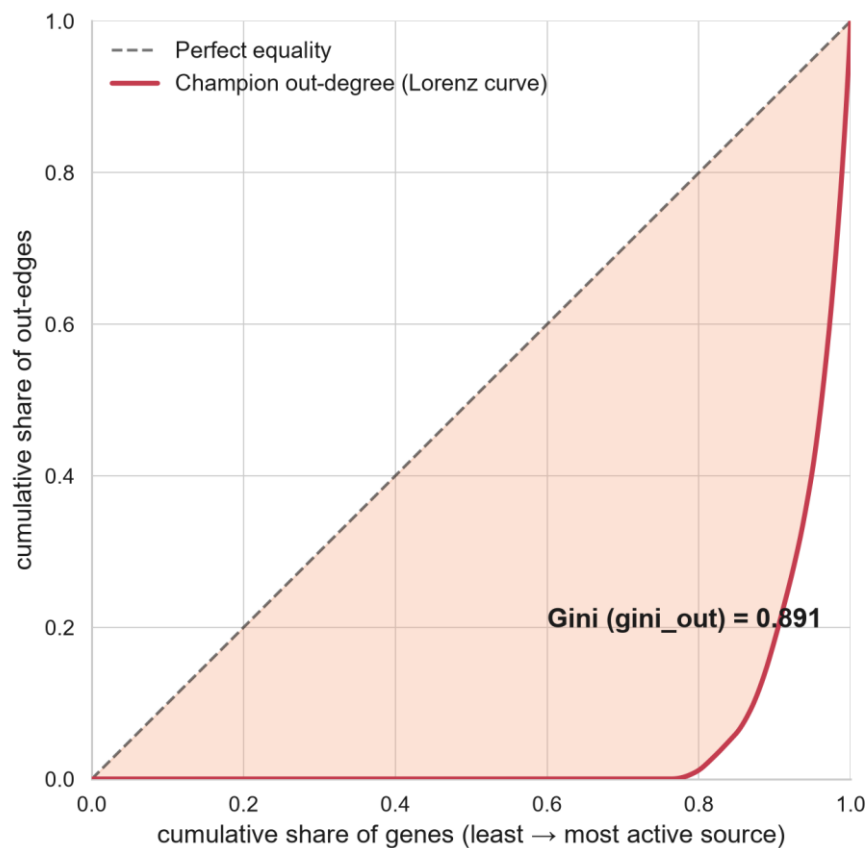


Figure 4.2 | The Lorenz curve of regulatory output. Genes are ranked by out-degree along the horizontal axis and the cumulative share of edges they hold is plotted on the vertical. A perfectly equal network follows the diagonal, a real regulatory network bows below it, and the shaded area between the two is the out-degree Gini coefficient.

4.3.3 — Gini_{in} — Regulatory-input inequality

The in-degree Gini coefficient extends the measure of regulatory inequality to the receiving side of the network. Where the out-degree Gini of the previous subsection quantifies how unequally genes distribute regulatory control, the in-degree Gini quantifies how unequally they receive it. These are two distinct properties of the same directed graph, and a biologically faithful GRN has structure on both, so FUNGI constrains each in its own right.

4.3.3.1 — Biological Context

The out-degree side of a gene regulatory network is famously unequal, with a small cohort of transcription-factor hubs and a long tail of terminal effectors. The in-degree side has its own structure, less often discussed but no less real. Most genes are governed by a modest committee of regulators, a handful of transcription factors whose combined activity sets the gene's expression level, while a smaller set of genes sit at the convergence points of many regulatory programs and integrate input from a large number of upstream factors ^{4,62}.

The asymmetry between the two sides is biologically grounded rather than incidental. Broadcasting is cheap and concentrated, since a single transcription factor can bind cis-regulatory sequence at hundreds of promoters ³, so the out-degree distribution is heavy-tailed and its inequality is extreme. Integration is more constrained, since a promoter has finite regulatory real estate and a gene can only be co-regulated by so many factors at once, so the in-degree distribution, while still unequal, is typically milder and less heavy-tailed than the out-degree distribution. A realistic GRN therefore shows meaningful but bounded input inequality, neither a flat distribution in which every gene is controlled by the same number of regulators nor an extreme one in which a few genes absorb nearly all regulatory input.

Because the two inequalities are distinct, a candidate graph can look correct on one and wrong on the other. A pruning that faithfully reproduces the out-degree hierarchy can still leave the in-degree distribution washed flat, producing a network whose broadcasting structure survives but whose integration structure has been erased. Measuring input inequality on its own axis is what prevents that failure from passing unnoticed.

4.3.3.2 — Optimization Rationale

The in-degree Gini constrains a degree of freedom that the out-degree parameters leave untouched. The exponent α , the out-degree Gini, and the hub fraction all describe the broadcasting side of the network and say nothing about how regulatory input is distributed, so FUNGI can satisfy every one of them while still producing a graph whose in-degree distribution is uniform or spiked. Binding the in-degree concentration into its own band closes that gap and shapes the integration structure of the network as deliberately as its broadcast structure. Its target band is read from the dense parent by the in-degree diagnostic probe, so like the other diagnostic targets it is calibrated to the dataset rather than imported as a fixed constant.

4.3.3.3 — Quantitative Identity

The in-degree Gini is the Gini coefficient of the previous subsection applied to the in-degree sequence rather than the out-degree sequence. Given the in-degrees $\{k_i^{\text{in}}\}$ for $i = 1, \dots, n$ genes sorted in non-decreasing order, it is

$$G_{\text{in}} = \frac{2 \sum_{i=1}^n i k_i^{\text{in}}}{n \sum_{i=1}^n k_i^{\text{in}}} - \frac{n+1}{n},$$

where k_i^{in} is the number of incoming regulatory edges received by gene i , i is its rank in the sorted sequence, and the $\frac{n+1}{n}$ term normalizes a perfectly equal distribution to $G_{\text{in}} = 0$. The interpretation carries over directly from the out-degree case: G_{in} is the normalized area between

the line of perfect equality and the Lorenz curve of the in-degree distribution, now measuring how concentrated the reception of regulation is rather than its dispersal.

4.3.4 — $S_{\max}$ — Hub Fraction

The Hub Fraction, denoted as $S_{\max}$, provides a targeted measure of extreme regulatory concentration within the FUNGI framework. While parameters like α and Gini evaluate the network's macroscopic distribution, $S_{\max}$ isolates the single most influential node to ensure its connectivity remains biologically plausible. By anchoring the absolute top end of the degree distribution, this metric prevents the sparsification engine from collapsing into artificial hyper-hubs while ensuring a genuine master regulator successfully emerges

4.3.4.1 — Biological Context

While the Gini Coefficient characterizes the overall inequality of regulatory output across all genes, it says nothing specific about the single most dominant regulator in the network. In real biological GRNs, the most connected transcription factor occupies a qualitatively distinct role from even the second or third most connected gene. Master regulators such as OCT4 in pluripotent stem cells do not simply sit at the top of a continuous degree distribution. Instead they act as a singular organizational anchor of the regulatory program, with direct reach over a substantial but bounded fraction of the transcriptome.⁵

$S_{\max}$ captures this directly by measuring what fraction of all genes in the network are directly regulated by the single most connected hub. It is a focused, interpretable quantity. If $S_{\max} = 0.05$, the most connected transcription factor in the network sends outgoing regulatory edges to 5% of all genes. This is biologically meaningful in a way that neither α nor Gini can capture alone, since both of those metrics describe properties of the whole distribution rather than the behavior of its single most extreme member.

Too large of an $S_{\max}$ implies a biologically unrealistic hyper-hub that contains a single transcription factor that directly regulates a dominant fraction of the transcriptome, which is not supported by experimental evidence even for the most promiscuous master regulators.⁶⁸ Too small an $S_{\max}$ implies that no gene has emerged as a genuine master regulator, which is equally inconsistent with the known architecture of gene regulatory programs in any well-characterized cell type.⁶⁹

4.3.4.2 — Optimization Rationale

$S_{\max}$ plays a particularly important role in FUNGI because it is directly coupled to the κ hyperparameter, which sets the per-source hub cap. κ controls the maximum number of outgoing edges any single gene is permitted to claim during edge selection. $S_{\max}$ is therefore both a diagnostic topology metric and a soft constraint on κ itself. A configuration that produces an $S_{\max}$ outside the target interval is signaling not only that the network topology is wrong, but that the hub cap is either too permissive or too restrictive for the dataset at hand.

As with the rest of the parameters, the target bounds for $S_{\max}$ are estimated empirically from the perturbation data by the phase 0 diagnostic probes rather than being imported as fixed biological constants. The appropriate largest hub fraction depends on the specific regulatory program being modeled and the biological context being analyzed.

4.3.4.3 — Quantitative Identity

$S_{\max}$ is defined simply as the fraction of all genes in the network that receive a direct outgoing edge from the single highest out-degree node. It is expressed as

$$S_{\max} = \frac{\max_i(k_i^{\text{out}})}{n}$$

where:

- $S_{\max}$ is the largest hub fraction of the candidate network.

- k_i^{out} is the out-degree of gene i (the number of outgoing regulatory edges it sends to other genes).
- $\max_i(k_i^{\text{out}})$ is the out-degree of the single most connected gene in the network
- n is the total number of genes in the network.

The simplicity of this formula is deliberate. Unlike α and Gini, which require fitting procedures or sorted summations over the full degree sequence, S_{max} is a direct single-observation statistic. It makes no assumptions about the shape of the underlying degree distribution and is computed in constant time once the out-degree array is known. This also makes it the most interpretable of the six topology parameters. Its value has a direct biological reading as the direct regulatory reach of the dominant master regulator of the transcriptome.

4.3.5 — Q — Modularity

The modularity metric, denoted as Q , evaluates the community structure of the inferred network within the FUNGI framework. While parameters like α or Gini govern the overall distribution of regulatory power, Q ensures that this power is correctly compartmentalized into functionally coherent neighborhoods. By actively penalizing architectures that lack biological compartmentalization, this metric guarantees that the downstream Graph Neural Network learns localized propagation patterns reflective of true cellular processes.

4.3.5.1 — Biological Context

Genes do not regulate one another at random across the transcriptome. Real biological GRNs are organized into functional modules made up of coherent clusters of genes that are densely interconnected with one another and relatively sparsely connected to genes outside their cluster. These modules correspond to meaningful units, for example the cell cycle module, pluripotency maintenance module, metabolic response module, and so on.^{16,17} The existence of this modular organization is not incidental to gene regulation. It is the mechanism by which cells execute distinct, separable regulatory programs simultaneously without catastrophic cross-talk between them.

Modularity is the standard graph-theoretic measure of how clearly a network is partitioned into such communities.⁷⁰ A high Q indicates that the network has a strong internal community structure where edges are concentrated within modules and relatively rare between them. A low Q indicates that edges are distributed more uniformly across the network without meaningful cluster separation, resembling a random graph.

For a GNN tasked with predicting the downstream effects of a CRISPRi perturbation, the modularity allows the GNN to learn that a perturbation to a gene in the pluripotency module propagates strongly within the module and weakly into, say, the cell cycle module. Without this community structure the GNN's message passing operates over a flat, undifferentiated topology and loses the capacity to learn module-specific propagation patterns. Too little modularity and the network is topologically indistinct from noise. Too much and the modules become so isolated from one another that the GNN cannot learn cross-module regulatory relationships that genuinely exist in the biology.

4.3.5.2 — Optimization Rationale

The raw dense parent graph produced by LightGBM inference has low modularity by construction⁴⁵. When nearly every gene connects to nearly every other gene with similar edge weights, there is no meaningful community structure for a partitioning algorithm to detect. FUNGI must recover this structure through sparsification, and Q acts as one of the six simultaneous constraints in the utopia loss function that guides the search toward a biologically valid topology.

Q is particularly important as a complement to α and Gini. It is entirely possible to construct a network that has the correct power-law degree distribution and correct degree of hub inequality while still having no meaningful modular organization, which occurs if hub genes send their

edges uniformly across the transcriptome rather than concentrating them within functional neighborhoods. Q closes this gap by directly penalizing topologies that lack community structure regardless of whether their degree distribution looks correct.

As with the other parameters, the target interval for Q is estimated from the perturbation expression data by the Phase 0 diagnostic probes rather than being fixed in advance.

4.3.5.3 — Quantitative Identity

FUNGI computes Q using the standard Newman-Girvan modularity formula applied to the undirected projection of the candidate graph ⁷⁰. Because modularity is a property of an undirected community partition, the directed candidate graph is first symmetrized by collapsing bidirectional edges and summing their weights, producing an undirected weighted graph on which community detection is run.

A partition $\mathcal{C}$ is simply a complete assignment of every gene in the network to exactly one community. Every gene belongs to one community and one community only, there is no overlap. The partition itself is the label a gene in question belongs to. Gene i is assigned to community label c_i while gene j is assigned label c_j and so on for all n genes. The modularity score Q is then a single number that evaluates the quality of that particular labeling is. Specifically, whether genes within the same community are more connected to each other than you would expect if edges were placed randomly.

The modularity of a partition $\mathcal{C}$ is given by

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where:

- Q is the modularity of the partition
- $m = \frac{1}{2} \sum_{ij} A_{ij}$ is the total edge weight in the undirected graph
- A_{ij} is the weight of the edge between genes i and j in the undirected graph
- $k_i = \sum_{ij} A_{ij}$ is the weighted degree of the gene i
- $\frac{k_i k_j}{2m}$ is the expected edge weight between genes i and j under a null model that preserves degree sequence but places edges at random
- $\delta(c_i, c_j)$ is the Kronecker delta function, which equals 1 if genes i and j are assigned to the same community and 0 otherwise

The term inside the sum, $A_{ij} - \frac{k_i k_j}{2m}$, is the deviation of the observed edge weight from what would be expected by chance given the degrees of the two endpoints. Summing this deviation only over pairs within the same community (enforced by the δ term) and normalizing by $2m$ yields a value in $(-1,1)$. A partition where 2 communities have far more internal edges than chance expectation produces a high Q , while a partition that does no better than random produces $Q \approx 0$ ⁷¹.

Community assignments are determined in FUNGI by igraph's implementation of Louvain algorithm (`community_multilevel()`), which optimizes Q directly through a greedy multi-level merging procedure. ⁷². The algorithm does not require the number of communities to be specified in advance. It discovers the partition that maximizes Q organically from the graph structure. The resulting Q is then evaluated against the data-derived target interval $[Q_{lo}, Q_{hi}]$ within the utopia loss function.

4.3.6 — C — Clustering Coefficient

The Clustering Coefficient, denoted as C , measures the local density of regulatory neighborhoods within the FUNGI framework. While modularity (Q) evaluates large-scale compartmentalization, C quantifies the prevalence of localized triangular motifs, primarily Feed-Forward Loops (FFLs).

By optimizing for an intermediate clustering density, FUNGI ensures the resulting characteristic of true biological circuits, preventing it from deteriorating into disjointed linear chains of collapsing into overly saturated cliques.

4.3.6.1 — Biological Context

Modularity, as covered in section 4.3.5, tells us that a GRN is organized into distinct functional communities. The clustering coefficient C asks a related but meaningfully different question. Within those communities, how densely are local neighborhoods of individual genes interconnected with one another? Specifically, C measures the prevalence of triangles in the network. These are configurations where gene A regulates gene B , gene A also regulates gene C , and gene B in turn regulates gene C , forming a closed three-node loop.

In the context of gene regulation this triangular motif has a specific and well-studied biological identity, the Feed-Forward loop, or FFL. The FFL is one of the most enriched network motifs in experimentally characterized GRNs across organisms, from bacterial transcriptional networks to mammalian developmental programs.¹⁸ Its prevalence is not accidental. An FFL provides a form of regulatory redundancy and signal coherence. If a master regulator activates an intermediate regulator and a shared target gene, the target receives two independent regulatory inputs that must be integrated. This architecture confers properties such as noise filtering, pulse generation, and response acceleration depending on the signs of the regulatory interactions involved.^{19,73} The clustering coefficient is therefore not only a graph-theoretic abstraction but a direct proxy for how FFL-rich the inferred network is.

A GRN with a low value of C is one where regulatory relationships are sparse and linear, chains of regulation without the triangular reinforcement that characterizes real biological circuits. A GRN with too high a clustering coefficient risks becoming so locally cliquish that information propagates in tight isolated loops rather than flowing coherently across the network. True biological GRNs occupy an intermediate regime where FFLs are meaningfully enriched relative to a random graph but do not dominate the topology to the exclusion of longer-range regulatory paths.

4.3.6.2 — Optimization Rationale

The clustering coefficient occupies a conceptually distinct role among FUNGI's six topology parameters because it is the parameter most directly tied to the DASH kernel's FFL scoring term⁴¹. The DASH equation includes the term $\exp(\delta \cdot \tilde{T}_{st})$, where $\tilde{T}_{st}$ is the degree-normalized triangle count for each candidate edge. It is a measure of how many FFL triangles a given edge participates in. The hyperparameter δ controls how aggressively this FFL participation boosts an edge's selection probability. C in the utopia loss function is therefore both an outcome being measured and an indirect signal about whether δ and k_{core} are calibrated correctly for the dataset. A champion graph whose C falls below the target interval is telling us that the FFL boost was insufficient to recover the triangle density expected from biology. A C above the target interval suggests the FFL term was too aggressive and the network has collapsed into an overly triangulated local structure at the expense of longer-range connectivity.

4.3.6.3 — Quantitative identity

FUNGI computes C as the global clustering coefficient of the undirected projection of the candidate graph using igraph's `transitivity_undirected()` function⁷⁴. As with the modularity calculation, the directed candidate graph is first symmetrized by collapsing bidirectional edges and summing their weights before clustering coefficient is evaluated.

The global clustering coefficient is defined as the ratio of closed triangles to the total number of connected triplets present in the network⁷⁵

$$C = \frac{3 \cdot (\text{number of closed triangles in the graph})}{\text{number of connected triplets of nodes}}$$

which can be expressed more formally as

$$C = \frac{\sum_i t_i}{\sum_i \tau_i}$$

where each component is defined as follows. the index i runs over all genes in the network. t_i is the number triangles that pass through gene i . This gives us the number of pairs of neighbors of gene i that are also connected to each other, forming a closed three-gene loop involving gene i . This number of pairs of genes that are both neighbors of gene i where each component is defined as follows. The index i runs over all genes in the network t_i is the number of triangles that pass through gene i (representing the number of pairs of neighbors of gene i that are also connected to each other, forming a closed three-gene loop). τ_i is the total number of connected triplets centered on gene i regardless of whether those two are connected to each other. A connected triplet centered on gene i . A connected triplet simply any pair of genes (j, k) such that both j and k have edges to i , regardless whether j and k are connected to each other. It becomes a closed triangle if and only if j and k are also connected to each other. The factor of 3 in the numerator of the first expression arises because each triangle in the graph is counted once for each of its three member nodes when summing t_i over all i , while each triangle contributes to the triplet count of only two of its three nodes at a time, so the factor corrects for this asymmetry and ensures $C \in [0,1]$.

An equivalent and perhaps more intuitive way to read C is as a global probability. If you pick a gene at random, then pick two of its regulatory neighbors at random, C is the probability that those two neighbors are themselves connected. When this probability is high, regulatory neighborhoods are tightly interconnected and FFL motifs are abundant. When it is low, a gene's neighbors are largely unaware of one another and regulation proceeds through independent parallel paths rather than reinforcing triangular circuits.

This computed value is then evaluated against the data derived target interval $[C_{lo}, C_{hi}]$ within the utopia loss function, with its contribution weighted by the confidence assigned by Phase 0 clustering probe.

4.3.7 — ρ — Degree assortativity

Degree assortativity ρ measures the company that a network's hubs keep, capturing whether high-degree genes attach preferentially to other high-degree genes or to the low-degree periphery. Where $S_{\max}$ bounds the size of the single largest hub, ρ governs how the hubs wire into the rest of the network, and biological gene regulatory networks are distinctive in being disassortative.

4.3.7.1 — Biological Context

Assortativity is a mixing property. In an assortative network, high-degree nodes preferentially connect to one another, folding the hubs into a dense, mutually-connected core; in a disassortative network, high-degree nodes connect outward to low-degree ones and avoid each other. Social networks tend to be assortative, since well-connected individuals associate with other well-connected individuals, whereas biological, technological, and regulatory networks are consistently disassortative ¹⁵.

In a gene regulatory network this disassortativity has a direct regulatory reading. A master-regulator transcription factor sends the overwhelming majority of its outgoing edges to terminal effectors, the low-degree genes it actually controls, rather than to other master regulators. The regulatory hierarchy fans out from the top rather than folding in on itself. A small number of high-level regulators may cross-regulate one another, but the dominant wiring pattern is hub-to-periphery, and it is this pattern that gives the network its layered, top-down organization ⁶².

A GRN whose hubs clustered together would concentrate the network's control, and its points of failure, into a single dense core, and it would collapse the layered regulatory hierarchy into a tangled centre. Neither matches how cellular regulation is organized. Preserving disassortativity

keeps the reconstructed network's control structure spread across the hierarchy the biology actually uses rather than piled into a hub clique.

4.3.7.2 — Optimization Rationale

Assortativity is easy to get wrong by accident, which is why it earns an explicit target. A pruning that rewards high-weight edges tends to drift assortative, because the strongest-scoring edges frequently run between well-connected genes, and selecting them pulls the hubs toward one another and builds exactly the dense core biology avoids. Targeting ρ on the negative side of zero holds the pruned network in the disassortative regime, forcing hub edges outward to low-degree targets and preserving the hub-to-periphery wiring that separates a real GRN from a hub-clustered graph. The target band is placed on the negative side of zero, consistent with the measured disassortativity of transcriptional networks ¹⁵.

4.3.7.3 — Quantitative Identity

ρ is Newman's assortativity coefficient, defined as the Pearson correlation of the degrees at the two ends of an edge ¹⁵. Taking j_i and k_i as the degrees of the nodes at the ends of edge i , summed over all M edges,

$$\rho = \frac{M^{-1} \sum_i j_i k_i - [M^{-1} \sum_i 1/2 (j_i + k_i)]^2}{M^{-1} \sum_i 1/2 (j_i^2 + k_i^2) - [M^{-1} \sum_i 1/2 (j_i + k_i)]^2},$$

which takes values in $[-1,1]$. A positive ρ marks an assortative network in which similar-degree nodes connect, a negative ρ a disassortative one in which hubs attach to low-degree nodes, and a ρ near zero a network with no degree correlation at all. Biological GRNs sit at negative values, and the utopian loss scores the measured ρ against a target band on that negative side.

4.3.8 — Reciprocity — Directed link reciprocity

Reciprocity is the one topology parameter here that speaks to the directed, two-way structure of regulation, measuring how often an edge from gene A to gene B is matched by a return edge from B to A . It is the target FUNGI adopted to fill a reachable, biologically-grounded slot in its objective.

4.3.8.1 — Biological Context

Regulation is overwhelmingly one-directional. A transcription factor regulates its target, and the target only rarely regulates the factor in return. A minority of gene pairs, however, do regulate each other mutually, and these two-node feedback loops are not noise in the wiring. They underlie bistable switches, mutual-repression toggles, and the self-stabilizing circuits that lock a cell into a committed state ¹, so the presence of a controlled amount of reciprocity is a genuine feature of biological regulation rather than an artifact ⁶⁰.

Real gene regulatory networks therefore carry a small, non-random amount of reciprocity, high enough to encode genuine feedback yet far below the saturating reciprocity of an undirected graph, in which every edge is trivially mutual by construction. The reciprocated fraction is a compact summary of how much of this two-way structure a candidate graph has retained. A graph with essentially zero reciprocity has discarded the feedback motifs that give regulation its dynamics, while a graph whose reciprocity approaches one has collapsed toward an undirected object and thrown away the causal asymmetry that makes a GRN a directed regulatory map rather than a symmetric correlation network.

Reciprocity earns its place in the objective because it clears every bar a target must clear at once. It is reachable on the inference substrate, its band is grounded in the network-science literature, it is biologically interpretable as the amount of feedback wiring, and it can be probed on the dense parent. That combination is rarer than it sounds, since several otherwise-appealing

topological quantities fail at least one of these tests, and reciprocity passing all four is what qualified it as a target.

4.3.8.2 — Optimization Rationale

Reciprocity guards the directed character of the pruned graph. Because the inference substrate is directed and causal, an uncontrolled pruning can drift toward either extreme, stripping out the reciprocated pairs entirely and losing the feedback structure, or over-preserving mutual edges until the graph reads as undirected. Its band comes from the literature on directed link reciprocity⁶⁰ and is supplied as a fallback that the diagnostics also default to. Its adoption was checked against a degree-preserving direction shuffle, in which reshuffling the edge directions while holding the degree sequence fixed collapses the measured reciprocity. This confirms that the reciprocity FUNGI reads is genuine directionality rather than an artifact of the degree sequence. Targeting it holds the feedback structure of the network in a biological range without letting the graph slide toward an undirected object.

4.3.8.3 — Quantitative Identity

The simplest measure of reciprocity is the fraction of directed edges that are mutual,

$$r = \frac{\sum_{i \neq j} a_{ij} a_{ji}}{\sum_{i \neq j} a_{ij}},$$

where $a_{ij} = 1$ when a regulatory edge runs from gene i to gene j and 0 otherwise. This raw fraction conflates genuine reciprocity with the reciprocity a graph would show by chance at its edge density, so FUNGI uses the density-corrected measure of Garlaschelli and Loffredo, which rescales r against its random-graph expectation so that the reported value isolates reciprocity beyond what density alone produces and remains comparable across graphs of different sparsity⁶⁰. The corrected value is scored against the reciprocity band inside the utopian loss.

Figure 4.3 shows where the champion graph lands on each engaged target.

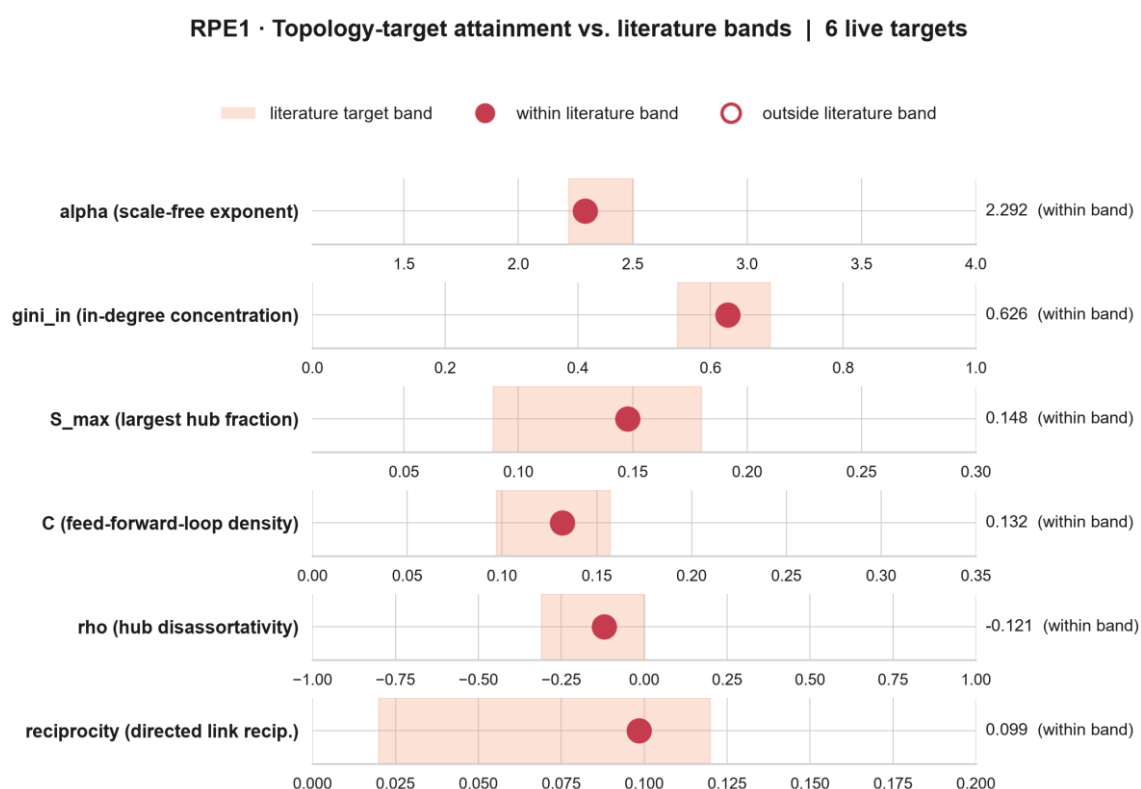


Figure 4.3 | Topology-target attainment. The champion graph's value on each of the six engaged targets against its literature band. All six sit inside band, from the scale-free exponent (2.29) and in-degree concentration

(0.63) to the largest-hub fraction (0.15), feed-forward-loop density (0.13), hub disassortativity (-0.12), and directed reciprocity (0.10). Out-degree Gini and modularity are unreachable on this parent and are disabled.

4.4 — The Utopian Loss and Bound Calibration

Section 4.3 defines the shapes FUNGI prunes toward and the band each one should fall in. This section is how FUNGI turns that catalogue of target bands into a single number it can minimize, and how the bands themselves are set from the data. The object that does the scoring is the utopian loss. It measures how far a candidate graph sits from a graph that would satisfy every engaged target at once, and the whole hyperparameter search is the search for the graph that drives this loss as close to zero as the substrate allows.

4.4.1 — The utopian ideal and what limits it: pruning-only reachability

The utopian graph is the single graph that would land every engaged topology target inside its band at once, the graph at which the loss is exactly zero. FUNGI aims for it on every run, but reaching it is not guaranteed, and the reason is a hard constraint on what FUNGI is allowed to do. FUNGI makes only destructive edits. It removes edges from the dense parent graph and never adds an edge the parent did not already contain. The set of graphs FUNGI can reach is therefore bounded entirely by the content of the dense parent. A target the parent has no structure to support cannot be pruned into existence, since pruning can only ever take structure away.

This makes the loss substrate-aware in a way that carries real information. When every engaged target is reachable from the parent, a zero-loss graph exists and the search can find it. When a target is not reachable on a given parent, a nonzero loss floor remains no matter how the graph is pruned, and that floor is a diagnostic rather than a dead end. It says one of two things. Either the target is wrong for this biology, or the parent is too sparse or too structureless to express it, which is the quantitative case for building the graph from a denser upstream generator. FUNGI reports not only the best available pruning of a given parent but whether that parent can carry the biology at all.

This is why the set of engaged targets depends on the substrate. On the current dense parent, six targets are reachable and drive the loss, the power-law exponent, in-degree Gini, the hub fraction, the clustering coefficient, assortativity, and reciprocity. Out-degree Gini and modularity are not reachable on this parent, so they are disabled and contribute nothing to the loss rather than holding every graph above a floor it can never clear. Disabling is not deletion. Both targets stay defined in the objective and re-engage on a denser parent that can support them, so a future generator with richer structure reopens them with no change to the loss itself.

Figure 4.4 sketches why some targets are out of reach, since FUNGI only removes edges.

FUNGI · Pruning-only reachability: the shrinking feasible region | conceptual, no data

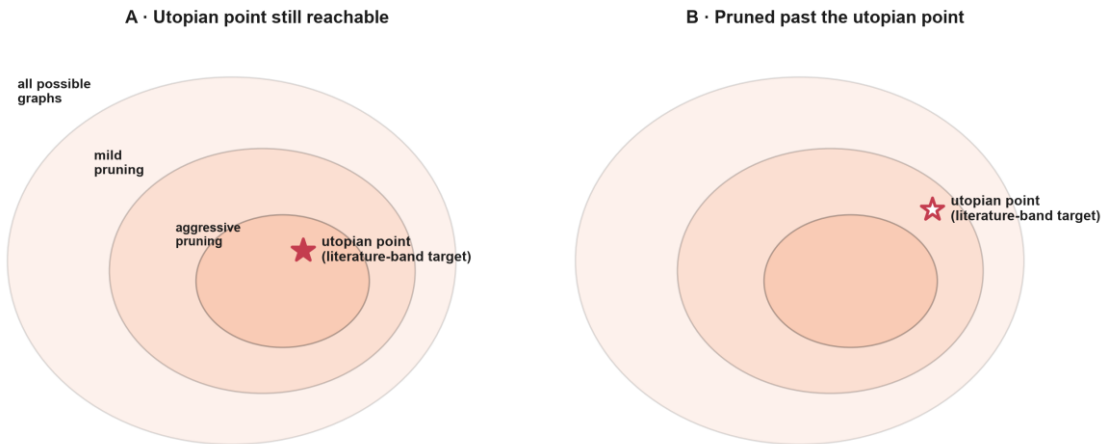


Figure 4.4 | Pruning-only reachability. FUNGI makes only destructive edits, so every graph it can produce is a subgraph of the dense parent, and the set of reachable topologies is bounded by what the parent contains. A target the parent cannot support leaves a nonzero loss floor, a diagnostic that the parent is too sparse rather than a tuning failure.

4.4.2 — The λ_{eff} estimator

Before any target band is set, FUNGI estimates how many regulatory edges per gene the perturbation data can actually support, a quantity it calls λ_{eff} . The estimate is a cascade-corrected count of how many genes each perturbation directly regulates. For every active perturbation, FUNGI sums the genes that respond to it but discounts genes that respond to many different perturbations, since a gene that lights up under almost every knockdown is more likely an indirect cascade effector than a direct target. The discount is an inverse-square-root weighting by how many perturbations each responding gene appears under, and FUNGI takes the weighted median of this direct-reach count across all perturbations.

A single estimator misjudges the density on some panel types, so FUNGI pairs the reach count with a second estimate and keeps the smaller of the two. The reach count overshoots on a targeted panel, where the perturbed genes are deliberately chosen high-impact regulators whose reach far exceeds an average gene's. A singular-value effective rank of the perturbation-response matrix, which counts how many independent regulatory programs the data contains by thresholding its singular values at the optimal hard cutoff, overshoots instead on a genome-scale panel⁷⁶. Taking the minimum selects the conservative, data-appropriate estimate for each kind of panel automatically. λ_{eff} then anchors the probe bounds, so that target bands defined relative to density are calibrated to the density the data can support rather than to an absolute number. It is a property of the data, and it is distinct from λ , the target density the user sets in section 4.4.4.

4.4.3 — The diagnostic probe suite (default, data-derived bounds)

Each target band comes from one of three sources, developed in section 4.2, a diagnostic probe that reads the band off the data, a literature fallback FUNGI supplies when the data cannot support a probe, or a value the user fixes directly. The default is the probe, and this section describes it. For each engaged target FUNGI runs a Phase-0 probe that measures the target on the dense parent and the perturbation data and returns two quantities, a band and a confidence. The band becomes the interval the loss scores against, and the confidence becomes the target's weight in the loss, so a target the data pins down sharply penalizes departures harder than one the data constrains only loosely.

Two safeguards keep the probes honest. Each probe is literature-bounded, so a band read from the data is clipped into the range published gene-regulatory-network measurements consider plausible, and a noisy probe cannot wander to a biologically absurd interval. Each probe is also validated against a shuffle. A probe run on degree-shuffled data that returns the same band it returned on the real data is not reading genuine structure, and FUNGI rejects it rather than trusting it. The probes derive their bands without hand-set scaling constants, so the calibration is a property of the data and the literature rather than of a tuned constant. The subsections below take each probe in turn, giving the statistic it computes, the substrate it reads it from, and what its band constrains.

4.4.3.1 — The power-law exponent (α)

FUNGI does not fit the scale-free exponent of section 4.3.1 directly. It derives α from the parent's out-degree inequality through the Lorenz-curve mapping $\alpha = 1/2G + 3/2$, where G is the out-degree Gini the next probe measures. A direct maximum-likelihood fit, the standard Clauset estimator, drifts upward as the candidate pool's density changes, so the same parent reports a different exponent at every edge budget ⁶⁵. The Gini-derived exponent does not drift, because it reads a global property of the degree distribution rather than fitting only the tail above a cutoff, and in a bootstrap comparison of several estimators it was the one that held inside the reported gene-regulatory range across densities. The exponent band is the interval this mapping carries over from the out-degree Gini band, and its confidence follows the Gini probe's.

4.4.3.2 — The degree-inequality probes ($Gini_{out}$ and $Gini_{in}$)

Two probes measure how unequally edges are distributed across genes, one on each side of the edge. The out-degree probe takes the Gini coefficient of the out-degree distribution over the top-weight slice of the parent, capturing how concentrated regulatory output is in a few source hubs. The in-degree probe does the same on the in-degree distribution, capturing how concentrated regulatory input is on a few heavily-targeted genes. Both read the slice of the parent whose size matches the intended edge budget, so the inequality they measure is the inequality the pruned graph will inherit rather than the inequality of the full dense parent. The Gini coefficient is the same statistic in both cases, the area between the Lorenz curve of the degree distribution and the line of perfect equality ⁶⁶, estimated on the observed degrees ⁶⁷. The two bands enter the loss separately, because a regulatory network is far more unequal in its output than in its input, and the biology of that asymmetry is developed in sections 4.3.2 and 4.3.3 ⁶².

4.4.3.3 — The hub-fraction bound (S_{max})

The hub-fraction target of section 4.3.4 is the one band FUNGI sets from theory rather than from a data probe. The largest hub's reach cannot be read reliably off the dense parent, because the parent's most-connected gene reflects inference density more than biology, so a probe that took the observed maximum out-degree would track how dense the parent happened to be. Instead FUNGI floors the band at the natural cutoff a scale-free network of the graph's size and density implies, which scales as $\sqrt{\lambda N}/N$ in the gene count N and the target density λ , and caps it at the fraction of the genome the largest known master regulators reach ^{5,68}. The band is therefore a statement about how large a single hub can plausibly be, not a measurement of how large the parent's happens to be.

4.4.3.4 — The clustering probe (C)

The clustering probe measures the global transitivity of the parent's top-weight slice, the fraction of connected gene triples that close into triangles ⁷⁵. FUNGI builds the undirected graph of the top-weight candidate edges, removes the duplicate edges a directed-to-undirected collapse creates, and computes the transitivity with the standard igraph routine ⁷⁴. Because a feed-forward loop is a closed directed triple, this transitivity is the graph-level signature of the feed-forward enrichment the kernel's motif term targets, and its band tells the loss how much

triangle structure the pruned graph should carry. The biological role of that structure is developed in section 4.3.6.

4.4.3.5 — The assortativity probe (ρ)

The assortativity probe measures whether high-output genes tend to regulate high-input genes or low-input ones, the degree correlation across edges that section 4.3.7 defines. FUNGI reads it from the perturbation data rather than from the graph. For each perturbed source it takes the L2 norm of that source's differential-expression vector, a measure of how strongly the source drives its targets, and for each gene it takes the L2 norm of that gene's column in the same matrix, a measure of how strongly the gene is driven. The Spearman correlation between the two over all matched genes is the assortativity estimate, negative when strong drivers act on weakly-driven effectors, which is the disassortative wiring characteristic of regulatory networks¹⁵. Its half-width widens when the correlation is statistically weak, so a poorly-determined sign enters the loss as a soft prior rather than a sharp target.

4.4.3.6 — The literature-prior targets (reciprocity, and the substrate-limited Q)

Not every target has a data probe. Reciprocity, the fraction of edges that are mutual, is set entirely from the literature. Directed regulatory networks are strongly non-reciprocal, so FUNGI fixes the band at the low reciprocated fraction reported for real directed networks rather than measuring it, because the reciprocity of the dense parent reflects the inference method's symmetry more than the biology's⁶⁰. This is the one target FUNGI calibrates by literature reference alone and never by a data probe, and the choice is deliberate rather than a missing probe. Deriving the band from the parent would make the target circular, since it would ask the pruned graph to be as reciprocal as an input whose reciprocity is a property of the inference method rather than of the biology. Modularity, Q , is the mirror case. A probe for it exists, but on a sparse causal parent the achievable modularity is capped well below the biological range by the parent's own structure, so FUNGI carries Q as a soft prior that a denser generator can re-engage rather than as an active target the current substrate can reach. Which of these targets is engaged, and where its band sits, is a property of the substrate, fixed once for a given parent at calibration time.

4.4.4 — λ — user-controlled target density

λ is the number of edges per gene the pruned graph is built to, and it sets the edge budget $\lfloor n\lambda \rfloor$ the selection step spends (section 4.5.9). Unlike the target bands, λ is not probed from the data. The user sets it, centered on a chosen density such as forty edges per gene and explored across a search window around that center, for instance thirty to fifty. It is fixed only in the sense that the diagnostic suite does not probe for it, and it is otherwise fully configurable. The right density is a downstream-consumer choice rather than a property the data dictates, since how dense a graph the downstream model trains best on is a modeling decision, not a biological measurement. λ_{eff} informs a sensible default, but the user owns the final value.

4.4.5 — The utopian loss function

The utopian loss sums a smooth boundary penalty over every engaged target and takes the square root of the total,

$$\mathcal{L} = \sqrt{\sum_{\tau \in \text{engaged}} w_{\tau} P(x_{\tau}; \ell_{\tau}, h_{\tau})},$$

where x_{τ} is the measured value of target τ on the candidate graph, $[\ell_{\tau}, h_{\tau}]$ is its band, and w_{τ} is its confidence weight. The per-target penalty P is zero whenever the measured value falls inside the band. Outside the band it is the squared relative distance past the nearer edge, capped so that no single runaway target dominates the sum, and ramped in by a sigmoid onset that begins

after a small buffer inside the boundary. A candidate graph reaches zero loss exactly when every engaged target sits inside its band at once, which is the utopian graph of section 4.4.1.

The smooth penalty is a deliberate choice over a hard one. A hard in-or-out penalty gives the optimizer no information about a graph that misses a band, since every out-of-band graph would score equally badly. The buffered squared-distance penalty instead rises with distance from the band, so a graph that nearly satisfies a target scores better than one that is far from it, and the search has a gradient to follow from outside the band toward it. Disabled targets carry a weight of zero and drop out of the sum, so the objective can reach zero on the reachable target set without the disabled targets holding it above the floor. The hub-fraction term additionally couples to the selection geometry, since part of its penalty measures how far the largest hub's out-degree exceeds the per-gene cap κ . This ties the topology target directly to the hub cap that produces it (section 4.5.9), so a graph is penalized both for an out-of-band hub fraction and for a hub that breaches the cap meant to bound it.

Figure 4.5 gives the loss in code, a smooth per-target band penalty combined under a square root.

```
engine.py :: calculate_utopia_loss._p_smooth
def _p_smooth(par, obs, ub, lw, buffer_frac=0.10, sharpness=5.0):
    b = ub[par]
    w = _safe(lw[par], 1.)
    o = _safe(obs, 0.)
    bound_width = max(abs(b[1] - b[0]), 1e-6)
    buffer = bound_width * buffer_frac
    if b[0] <= o <= b[1]:
        return 0.
    if o < b[0]:
        raw_dist = (b[0] - o) / max(abs(b[0]), 1e-6)
        dist_beyond = max(0., (b[0] - o) - buffer)
    else:
        raw_dist = (o - b[1]) / max(abs(b[1]), 1e-6)
        dist_beyond = max(0., (o - b[1]) - buffer)
    base_penalty = raw_dist ** 2
    onset = 1.0 / (1.0 + np.exp(-sharpness * (dist_beyond / bound_width)))
    return w * min(base_penalty, 4.0) * onset
```

Figure 4.5 | The utopian loss. Each engaged target contributes a smooth penalty that is zero inside its band and grows with the squared distance outside it, and the penalties are confidence-weighted and combined under a square root. The loss reaches zero exactly when every engaged target sits inside its band at once.

4.5 — The DASH Scoring Kernel

Sections 4.3 and 4.4 describe what FUNGI is trying to build and how it recognizes success. The topology targets set the shape the pruned graph should take, and the utopian loss collapses that shape into a single number that scores a whole graph. Neither one acts on an individual edge. The loss can only compare finished graphs against one another, and it says nothing about which edges any particular graph ought to have kept. The DASH kernel fills that gap. It assigns every candidate edge in the dense parent a single scalar score, and FUNGI builds each candidate graph by keeping the highest-scoring edges under a per-gene budget. The kernel is the only place in the pipeline where a decision is made about one edge at a time, so it is where every source of evidence FUNGI holds about an edge is combined into one number: the inferred regulatory weight, the edge's role in local feed-forward motifs, the perturbation biology of its source gene, and the edge's position relative to the network's community structure. FUNGI's kernel adapts the domain-knowledge pruning approach of DASH, which prunes a dense regulatory model by scoring its edges against data-derived and biological priors rather than by magnitude alone ⁴¹.

4.5.1 — The full DASH kernel

For a candidate edge $e = (s \rightarrow t)$ running from source gene s to target gene t , the DASH kernel assigns the score

$$\omega(e) = W_q(e)^\beta \cdot e^{\delta \tilde{T}_{st}} \cdot \pi(s)^\psi \cdot R(s)^\nu \cdot m_{\text{intra}}^{\mathbb{1}[\text{intra}(e)]} \cdot \text{ER}(e)^\eta \cdot \chi_s(s) \chi_t(t) \cdot \rho_c(s),$$

a product of eight factors, each carrying its own piece of evidence about the edge. $W_q(e)^\beta$ is the inferred regulatory signal, reweighted so that magnitudes are comparable across source genes (section 4.5.3). $e^{\delta \tilde{T}_{st}}$ rewards edges that close feed-forward-loop triangles (section 4.5.4). $\pi(s)^\psi$, $R(s)^\nu$, $\chi_s(s)\chi_t(t)$, and $\rho_c(s)$ are the biological priors that inject perturbation and expression evidence the co-expression weight alone cannot see (section 4.5.5). $m_{\text{intra}}^{\mathbb{1}[\text{intra}(e)]}$ strengthens within-community edges (section 4.5.6), and $\text{ER}(e)^\eta$ is the source-conditioned bridge term that protects inter-module connectivity, developed in full in section 4.5.7.

FUNGI evaluates this score in log-space, which turns the product into a sum and lets the whole candidate pool be scored in a single batched operation on the GPU,

$$\log \omega(e) = \beta \log W_q(e) + \delta \tilde{T}_{st} + \psi \log \pi(s) + \nu \log R(s) + \mathbb{1}[\text{intra}(e)] \log m_{\text{intra}} + \eta \log \text{ER}(e) + \log \chi_s(s) + \log \chi_t(t) + \log \rho_c(s).$$

The multiplicative form is deliberate. Each factor acts as an independent gain on the edge's odds of selection, so no single term can veto an edge that every other line of evidence supports, and a factor set to its neutral value drops out cleanly. A swept exponent of zero collapses its factor to one, and a prior array of all ones contributes nothing to the sum. This is also how FUNGI ablates the kernel. Every factor carries an on/off flag, and disabling one zeroes its log-contribution once at setup rather than deleting it, so any term can be switched out to isolate the effect of the others without touching the rest of the kernel ⁴¹.

The kernel only ranks edges. It does not by itself decide how many survive. FUNGI keeps the top $[n\lambda]$ edges by ω , where n is the number of genes and λ is the target edges-per-gene density (section 4.4.4), subject to a per-gene cap that stops any one gene from consuming the budget (section 4.5.9). The score sets the order; the density and the cap set the size and the ceiling.

Figure 4.6 shows the DASH kernel as the engine assembles it, a sum of one term per line of evidence in log-space.

```
engine.py :: compute_omega_batch_gpu
def compute_omega_batch_gpu(ctx, beta, delta, psi, nu, m_intra, k_core_eff):
    """Batched DASH omega, log-space. (B,) CUDA tensors -> (B, Ne) omega."""
    if ctx.T_bucket_stack is not None:
        T_batch = lookup_T_bucket_rows(ctx, k_core_eff)
    else:
        T_batch = torch.zeros((len(beta), ctx.Ne), device=ctx.log_wq.device,
                               dtype=torch.float32)

    b, d, p, n, m = (beta.view(-1, 1), delta.view(-1, 1), psi.view(-1, 1),
                     nu.view(-1, 1), m_intra.view(-1, 1))
    log_num = (b * ctx.log_wq.unsqueeze(0)
               + d * T_batch
               + p * ctx.log_pi_base.unsqueeze(0)
               + n * ctx.log_rdf_base.unsqueeze(0)
               + torch.log(m) * ctx.intra_mask_f.unsqueeze(0)
               + ctx.log_static.unsqueeze(0))
    return torch.exp(log_num)
```

Figure 4.6 | The DASH kernel in log-space. Each candidate edge's score is a product of eight factors, the inferred weight, the feed-forward reward, the biological priors, the community boost, and the bridge term, evaluated as a sum of logs so the whole candidate pool is scored in one batched GPU operation. A factor set to its neutral value drops out cleanly, which is also how the kernel is ablated.

4.5.2 — Priors and hyperparameters: why the kernel mixes fixed and swept terms

The kernel contains two kinds of quantity, and the distinction between them is the design decision at the center of FUNGI. A prior is a per-edge or per-gene value computed once from the data or the biology before the search begins and held fixed throughout it. A hyperparameter is a scalar the search varies from one candidate graph to the next. $W_q, \tilde{T}, \pi, R, \chi_s, \chi_t, \rho_c$, and ER are priors, each a fixed array indexed by edge or gene. $\beta, \delta, \psi, \nu, m_{\text{intra}}$, together with the selection scalars κ and k_{core} , are hyperparameters that the Sobol search explores (section 4.5.9).

Several priors are paired with a swept exponent that controls how hard that prior pushes. β tunes the sharpness of the weight prior, δ the strength of the feed-forward reward, ψ the influence of the perturbation-impact prior, and ν the influence of the diversity prior. The remaining priors carry a fixed internal damping and enter the kernel with a unit exponent. The two pleiotropy priors are raised to a fixed power $\zeta = 0.5$, the causal-output prior to a fixed power $\phi = 0.5$, and the bridge term to a fixed η , all set once and never searched.

Not every quantity is a hyperparameter, and three considerations decide which become swept and which stay fixed. The first is search cost. Every swept dimension multiplies the number of Sobol samples needed to cover the space at a given resolution, so each added hyperparameter is paid for in compute. The second is identifiability. Two exponents that push a graph in the same direction cannot be separately pinned by a loss that only sees the finished graph, so making both of them free would let the search wander a ridge of equivalent solutions without improving the fit. The third is anchoring. The pleiotropy and causal-output priors are conservative, bounded damping factors whose role is to nudge the ranking with perturbation evidence, and their scale is fixed better by the biology that defines them than by an optimizer chasing a topology target. Fixing them keeps the search focused on the levers that genuinely reshape the graph and treats the domain-knowledge priors as evidence rather than as free parameters ⁴¹.

4.5.3 — Source-quantile edge weighting $W_q(\beta)$

The raw material of the kernel is the regulatory weight FUNGI inherits from SHROOM, the LightGBM importance score that the inference stage assigns to each candidate edge ⁴⁵. These raw magnitudes are not comparable across source genes. A regulatory hub with thousands of outgoing edges and a sparse regulator with a handful sit on entirely different weight scales, so ranking edges by raw importance would let the busiest genes dominate the graph through the sheer magnitude of their scores rather than through the quality of any single edge.

W_q removes this bias by replacing each edge's raw weight with its rank inside its own source gene's outgoing-weight distribution, normalized to $[0,1]$. The strongest outgoing edge of every gene lands near 1 regardless of that gene's absolute weight scale, and the weakest near 0. The exponent β then sets how sharply the kernel prefers each gene's top edges over its weaker ones. A β near zero flattens W_q toward uniform and effectively ignores the inferred signal, while a large β sharpens the preference until only each gene's strongest edges carry appreciable weight in the score. Casting the weight as a within-source quantile makes β a gradient across the real signal range of every gene rather than an amplifier of the arbitrary magnitude gaps between one gene and another. On the RPE1 configuration β is searched over $[0.5, 4.0]$.

4.5.4 — Feed-forward-loop topology term (δ)

The term $e^{\delta \tilde{T}_{st}}$ is the kernel's one direct handle on local triangle structure, and it is the mechanism behind the clustering-coefficient target of section 4.3.6. A feed-forward loop is the three-gene motif in which a regulator acts on a target both directly and through an intermediate, the enriched circuit whose biological role section 4.3.6 develops. The kernel rewards an edge in proportion to how much of that structure it completes.

$\tilde{T}_{st}$ is the degree-normalized count of feed-forward triangles that the edge $s \rightarrow t$ participates in. FUNGI counts these triangles over the top $t_e = n \cdot k_{\text{core}}$ highest-weight edges of the candidate

pool rather than the entire dense parent, so k_{core} sets the radius of the neighborhood in which motif structure is assessed. A larger k_{core} counts triangles across a broader core subgraph; a smaller one restricts the count to the strongest edges. The raw triangle count is normalized by $\sqrt{d_{\text{out}}(s) \cdot d_{\text{in}}(t)}$ and clipped to $[0,1]$, so a hub does not accumulate triangle credit simply by having many edges through which triangles can pass. The exponent δ scales the resulting reward. When δ is large, edges embedded in dense feed-forward structure are pushed sharply up the ranking, which raises the clustering coefficient of the pruned graph; when δ is near zero, the kernel is indifferent to triangle participation. On the RPE1 configuration δ is searched over $[0,3.0]$ and k_{core} over $[8,28]$ ⁴¹. Feed-forward loops are among the most enriched motifs in characterized regulatory networks, which is why recovering them is a target rather than an accident of pruning ¹⁸.

4.5.5 — Biological priors (fixed)

The four biological priors inject evidence that the co-expression weight cannot carry. W_q knows only how strongly the inference engine scored an edge, and two edges with identical weights can have very different regulatory standing depending on the perturbation biology of the genes at their ends. Each prior is a per-gene multiplier computed once, before the search, from the Phase-0 differential-expression results and the gene-feature space, and each targets a specific way in which the raw weight can mislead.

4.5.5.1 — Perturbation-impact prior (ψ)

The perturbation-impact prior $\pi(s)$ rewards source genes that are causally productive. It is the number of differentially expressed genes a source's knockdown produces, normalized by that source's out-degree in the parent graph, so it measures differentially expressed genes caused per available outgoing edge rather than raw footprint. The normalization is what makes it an efficiency rather than a volume score. A hub that produces many downstream changes simply because it has thousands of edges to fire through is not rewarded, whereas a gene that produces a large response from a small set of edges is. The exponent ψ , searched over $[0,3.0]$ on the RPE1 configuration, sets how strongly this efficiency boosts a source's edges, and the prior is clipped to a bounded range so no single gene's perturbation response can overwhelm the rest of the kernel ⁴¹.

4.5.5.2 — Pleiotropy priors: source χ_s and target χ_t

The pleiotropy prior measures how often a gene appears as a differentially expressed gene across all training perturbations, read directly from the column sums of the Phase-0 differential-expression matrix. A gene that responds to many independent knockdowns is a convergence point where many upstream pathways meet, and such genes tend to carry substantial regulatory output of their own ⁶². FUNGI applies this signal on both ends of an edge. $\chi_s(s)$ boosts edges leaving a convergence gene, and $\chi_t(t)$ boosts edges arriving at one, so an edge between two frequently-responding genes is favored from both directions. The signal is defined for every gene in the panel rather than only the perturbed ones, so a validation or test gene that happens to be a frequent responder in the training perturbations receives the boost organically, which keeps the prior generalizable across the leakage-safe split. The value is log-damped and raised to the fixed power $\zeta = 0.5$, which holds it in a conservative bounded range so it nudges the ranking without dominating the weight signal.

4.5.5.3 — Causal-output prior (ρ_c)

The causal-output prior $\rho_c(s)$ is distinct from the degree-assortativity ρ of section 4.3.7 despite sharing the Greek letter; one is a per-gene kernel prior, the other a whole-graph topology target. $\rho_c(s)$ measures the differentially expressed genes a source produces per outgoing edge, the same efficiency quantity as π but applied as a boost that never drops below one. It exists to repair a failure mode of the perturbation-impact prior. At a high ψ , a gene with a below-average

differential-expression count receives $\pi(s) < 1$, which actively penalizes its outgoing edges even though the gene carries genuine causal evidence. ρ_c counteracts this by guaranteeing that any gene with non-zero causal output scores at least neutral, so real regulators are never suppressed for being modest rather than prolific. It is log-damped, floored at one, and raised to the fixed power $\phi = 0.5$, on the same conservative scale as the pleiotropy prior ⁴¹.

4.5.5.4 — Reachability-diffusion prior (ν / RDF)

The reachability-diffusion prior $R(s)$ measures the regulatory diversity of a source gene, defined as the mean pairwise cosine distance among its top-weighted targets in the expression-feature space. A high value marks a genuine multi-program regulator whose targets span several dissimilar expression programs, the profile of a transcription factor coordinating distinct metabolic, structural, and signaling responses. A low value marks a gene whose targets all sit in one co-expression bloc, the profile of a chromatin or cell-cycle hub whose high importance score reflects indirect effects propagating through a single shared program rather than genuine regulatory breadth. The weight, pleiotropy, and causal-output priors cannot separate these two cases, because a bloc hub scores high on outgoing weight, carries a wide perturbation footprint, and participates in feed-forward triangles just as a true multi-program regulator does. R discounts the bloc hubs and favors regulators whose targets are diverse among themselves. The exponent ν , searched over $[0, 2.0]$ on the RPE1 configuration, sets the strength of this discount. Favoring diverse regulators raises the fraction of edges that cross expression-space boundaries, which is the structure a downstream heterophilic graph neural network exploits ⁷⁷.

4.5.6 — m_{intra} — intra-community edge boost

m_{intra} is a single multiplicative boost applied to edges that fall inside a community, under the Leiden partition that SCBER already computes for its bridge detection (section 4.5.7). It enters the kernel as $m_{\text{intra}}^{\mathbb{1}[\text{intra}(e)]}$, so an intra-community edge is scaled by m_{intra} and a cross-community edge is left unchanged. The value is searched over $[1.0, 4.0]$ on the RPE1 configuration, where 1.0 leaves the kernel unchanged.

The boost exists to give the optimizer a positive force toward modularity that counterbalances SCBER's inter-module bridge term. The bridge term deliberately preserves edges that connect communities (section 4.5.7), which protects global connectivity but pulls against the compartmentalized structure the modularity target of section 4.3.5 rewards. Without a counterweight, promoting bridges depresses Q . m_{intra} supplies that counterweight by strengthening within-community wiring, so FUNGI can satisfy the modularity target by building genuine community structure rather than by relaxing the target itself. This is the lever that lifts the achieved modularity into its biological range while the bridge term keeps the communities connected.

4.5.7 — Source-Conditioned Bridge Effective Resistance (SCBER)

The kernel terms to this point score an edge on local evidence, its own weight, its role in nearby feed-forward motifs, and the perturbation biology of its source gene. None of them sees whether an edge is the only thing holding two parts of the network together. A pruned regulatory graph is the medium a downstream model passes signal over, so if pruning severs the few edges that bridge otherwise-separate modules, a perturbed gene's effect can no longer reach the parts of the network it regulates. SCBER is the kernel term that protects those bridges. It is the $ER(e)^n$ factor of the assembled kernel, and it raises an edge's score in proportion to how structurally irreplaceable it is as a connector.

4.5.7.1 — Motivation: inter-module bridge preservation

A message-passing graph neural network moves information along the edges of the graph it is given, which makes the graph the medium of the computation. When two communities of genes

are joined by only a few edges, every signal that must pass between them is forced through that narrow channel, and a network reading the pruned graph cannot carry a perturbation's effect across the bottleneck. This breakdown of long-range information flow is over-squashing, and the effective resistance between two nodes is its precise measure, with message-passing networks propagating information further when the resistance between nodes is low ⁷⁸. A greedy prune that keeps only the highest-weight edges is blind to this structure. It will drop a low-weight edge that is the sole link between two modules while keeping many redundant high-weight edges inside a single well-connected community, because weight alone says nothing about a connection's structural role.

The naive correction, boosting every edge by its effective resistance, is too indiscriminate. It rewards all bridges equally, including redundant cross-module links that only duplicate connectivity another edge already provides, and by promoting cross-community edges across the board it inflates the number of bridges until the compartmental structure a real regulatory network shows is washed out. SCBER conditions the boost on community structure so it acts only where it is needed. It applies the effective-resistance reward to inter-module edges alone, the edges that genuinely connect one community to another, and leaves within-module edges to be scored by the rest of the kernel. A bridge with no structural alternative is preferred over a redundant cross-module connection, while the dense wiring inside a community competes on its own weight and biology.

4.5.7.2 — Effective resistance via randomized Laplacian sketch

The effective resistance of an edge is the resistance measured across its two endpoints when the graph is treated as an electrical network with each edge a unit resistor. It is small when many parallel paths connect the endpoints and large when few do, which is the irreplaceable-connector property SCBER rewards. Effective resistance is also the basis of spectral graph sparsification, where a dense graph is reduced to a sparse one that preserves its spectral structure by sampling edges in proportion to their effective resistance ⁷⁹. Computing it exactly requires the pseudoinverse of the graph Laplacian, an operation cubic in the number of genes that is infeasible on a graph of thousands of nodes scored thousands of times across the search.

FUNGI estimates the resistances with a random projection instead. Spielman and Srivastava showed that all pairwise effective resistances can be recovered from a low-dimensional random projection of the Laplacian, which replaces the full inverse with a small number of linear solves ⁷⁹. FUNGI builds a grounded Laplacian from the symmetrized parent graph, draws a sketch of k random ± 1 vectors, and solves the Laplacian system against each one, so the squared distance between two nodes' sketch coordinates estimates the effective resistance between them. The ± 1 construction is the database-friendly form of the Johnson–Lindenstrauss projection, which preserves pairwise distances using only random signs and never materializes a dense projection matrix ⁸⁰. The sketch dimension follows the Johnson–Lindenstrauss bound of order $\ln n / \epsilon^2$, where ϵ controls the approximation quality, so the estimate is as tight as required while the cost stays near-linear in the number of edges. The raw resistances are clipped at a high percentile and rank-normalized to a bounded range before entering the kernel, so a single extreme value cannot dominate the term.

4.5.7.3 — Source-conditioned formulation

The conditioning that makes SCBER selective comes from a community partition of the parent graph. FUNGI detects communities with the Leiden algorithm, which improves on Louvain modularity optimization by guaranteeing that every community it returns is internally well-connected rather than accidentally split into disconnected pieces ⁸¹. Instead of fixing one resolution, FUNGI sweeps the Leiden resolution from coarse to fine and takes the first partition that is neither trivial nor degenerate. That partition must contain at least a minimum number of communities, and no single community may hold more than a set fraction of all genes, a rule that adapts the granularity to the graph rather than imposing a preset community count. The

partition is computed once and reused for both this term and the m_{intra} boost of section 4.5.6, through the same igraph Leiden implementation ⁷⁴.

With the partition fixed, FUNGI labels every candidate edge as intra-module or inter-module by whether its endpoints share a community. The effective-resistance reward is applied only to the inter-module edges, raised to the fixed exponent η , while intra-module edges take a neutral value and are left to the rest of the kernel. The exponent is held fixed rather than searched, for the same reason as the other biological priors, because its job is to preserve connectivity by a bounded, principled amount rather than to chase a topology target. The term therefore rewards exactly the edges whose removal would isolate a community and ignores the edges whose community is already densely wired, so FUNGI can compress the graph without cutting the thin channels a downstream model needs to move a perturbation's signal across the network.

Figure 4.7 shows the structure SCBER protects, a thin bridge that a weight-only prune would cut.

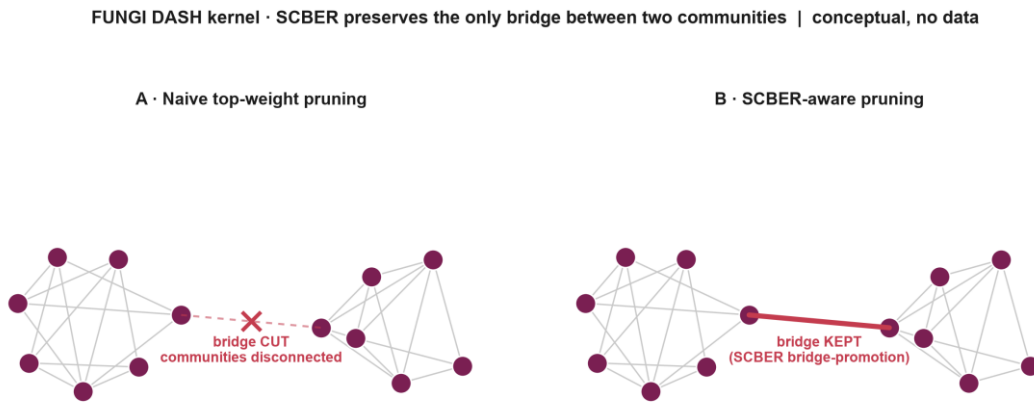


Figure 4.7 | Bridge preservation by SCBER. Two communities joined by a single low-weight bridge. A prune that keeps only the highest-weight edges cuts that bridge and severs the two communities, while SCBER rewards the edge in proportion to its effective resistance and keeps the channel a downstream model needs to carry a perturbation's signal across the network.

4.5.8 — Motif repair

Selecting the top-scoring edges under a per-gene budget can break feed-forward loops at the margin. When the third edge of a triangle scores just below the budget cutoff, ranking alone drops it and leaves an otherwise-recovered motif open. Motif repair is a budget-preserving swap pass that closes these gaps. It replaces up to three percent of the selected edges with unselected edges that complete open feed-forward triangles, and it makes a swap only when the closing candidate already outscores the weakest selected edge in ω . Each swap removes one edge and adds one, so the total edge count is preserved. The constraint that a swapped-in edge must outrank the edge it displaces keeps the repair from overriding the kernel's own ranking, so motif repair recovers triangle structure the top- ω cut narrowly missed without letting motif-completion override the evidence the kernel has already weighed.

4.5.9 — Hyperparameter search space (the swept set)

FUNGI's search is eight-dimensional. Seven of the dimensions are the kernel and selection hyperparameters, and the eighth is the target density. The kernel exponents are β (weight sharpness), δ (feed-forward reward), ψ (perturbation-impact strength), and ν (diversity strength). The selection geometry adds m_{intra} (the community boost of section 4.5.6), κ (the per-gene hub cap), and k_{core} (the feed-forward topology radius of section 4.5.4). On the RPE1 configuration these are searched over $\beta \in [0.5, 4.0]$, $\delta \in [0, 3.0]$, $\psi \in [0, 3.0]$, $\nu \in [0, 2.0]$, $m_{\text{intra}} \in$

$[1.0, 4.0]$, $\kappa \in [0.02, 0.35]$, and $k_{\text{core}} \in [8, 28]$. The eighth dimension, λ , is the edges-per-gene density the pruned graph is built to. Unlike the other seven, its band is set by the user rather than by a data probe, centered on a chosen density such as forty edges per gene and explored across a window around it (section 4.4.4).

The two selection scalars turn the kernel's ranking into a graph of the right size and shape. λ fixes the budget at $\lceil n\lambda \rceil$ edges. κ sets a per-gene cap on how many of those edges any one gene may contribute, which stops a handful of hubs from absorbing the entire budget. The cap is not uniform across genes. A per-gene multiplier derived from PageRank raises the cap for genuine hubs, so that the top one percent of genes by PageRank are allowed a larger share of edges while the long tail stays capped⁸². This soft cap lets the scale-free hub hierarchy of section 4.3.1 form, since real hubs are permitted the out-degree the biology gives them, without letting an unbounded hub flatten the rest of the graph. A small protected reserve, held to at most five percent of the budget, additionally guarantees that each perturbed source keeps its few strongest edges, so the genes for which FUNGI holds the most direct causal evidence are never pruned out entirely.

FUNGI explores this eight-dimensional space with a Sobol low-discrepancy sequence of 8192 samples, which spreads the samples far more evenly across the space than uniform random draws of the same size (section 4.7.4)⁸³. Every sample is one full setting of the eight quantities, scored by running the DASH kernel over the candidate pool, selecting the graph, and evaluating it against the utopian loss of section 4.4.

Table 4.1 lists the eight dimensions the search explores and the range each is searched over on the RPE1 configuration.

Table 4.1 | The FUNGI search space. The eight quantities the Sobol search varies, what each controls, and its RPE1 search range. Seven are kernel and selection hyperparameters; the eighth, the target density, is set by the user rather than probed from the data.

Symbol	Controls	RPE1 range
β	Signal trust (weight sharpness)	0.5 – 4.0
δ	Motif reward (feed-forward boost)	0 – 3.0
ψ	Perturbation-impact strength	0 – 3.0
ν	Directionality / diversity prior	0 – 2.0
m_{intra}	Community boost	1.0 – 4.0
κ	Per-gene hub cap (saturation)	0.02 – 0.35
k_{core}	Feed-forward core radius	8 – 28
λ	Target density (edges per gene, user-set)	30 – 50

4.6 — Integration of GRN Signal

The DASH kernel scores every edge from the co-expression structure of the dense parent. When an independent interventional signal is available for the same genes, FUNGI can fold it into that same scoring pass rather than carrying it as a separate graph. The engine exposes hooks for this that modify quantities the kernel already computes, and each one activates only when the dense parent supplies the matching evidence, so the integration is switchable rather than always applied. Two hooks illustrate the mechanism: a stability gate that rewards edges surviving resampling, and a substitution that lets an interventional causal-confidence score stand in for the co-expression weight W_q .

4.6.1 — Stability gate

An edge recovered every time the data are resampled is stronger evidence than an edge inferred once. The stability gate turns that reliability into score. When the dense parent reports how often each edge is selected across bootstrap replicates, FUNGI reads that frequency as S selections out of K resamples and converts it to a Beta-Bernoulli posterior log-odds, so an edge chosen in nearly every replicate reads as strong positive evidence and an edge chosen rarely as negative evidence⁸⁴. That log-odds is rescaled to a bounded multiplier centered on one, so the most reproducible edges are boosted and the least reproducible are damped, each by at most a fixed fraction. Edges with no stability record pass through unchanged. This applies stability selection at the scoring stage instead of as a hard inclusion cutoff, so a reproducible edge gains ground in the budget competition without a flaky edge being removed outright.

4.6.2 — Causal-edge refinement via W_q substitution

Causal-edge refinement changes which quantity the kernel weights in the first place. The source-quantile weight W_q ranks each edge by its co-expression strength within its source gene. Where an interventional causal-confidence score exists for an edge, FUNGI replaces that edge's W_q with the causal score, rank-normalized within the same source so the two quantities share a scale and compete fairly for the per-gene budget⁹. Edges without a causal score keep their co-expression W_q . The replacement is done per source rather than globally because the raw causal-confidence values compress near zero under a global normalization, which would zero out most of the edges the substitution is meant to strengthen. Ranking within each source preserves the causal order of that gene's outgoing edges while leaving the budget competition between genes intact.

These hooks give FUNGI a general route to combine a co-expression dense parent with interventional evidence inside a single scoring pass. The production RPE1 substrate is the pure PSGRN self-train graph, which carries no interventional columns, so the hooks stay inactive on that run and the kernel scores from the causal graph alone.

4.7 — The Optimization Pipeline

The previous sections describe FUNGI's pieces in isolation. The topology targets of section 4.3 set the shape the pruned graph should take, the utopian loss of section 4.4 collapses that shape into one number, and the DASH kernel of section 4.5 scores a single edge. A run has to bring these together in order, and this section is that walkthrough. FUNGI executes as a sequence of phases, each consuming the output of the last. An ingestion step loads the dense parent graph and aligns it to the expression data. A diagnostic calibration then reads the perturbation biology and fixes the targets and their weights. A preparation step filters the parent to a candidate pool and computes every fixed prior once. An expansive search scores thousands of candidate graphs against the loss, a refinement search sharpens the most promising region, and a champion-selection step returns the single graph the run commits to. The search machinery FUNGI reaches for spans three eras of applied mathematics, a low-discrepancy sampling sequence from the 1960s, a multi-objective desirability method from the 1980s, and a modern random-forest surrogate, each introduced alongside the phase that puts it to work.

4.7.1 — Data ingestion

The first step loads two inputs, the dense parent graph and the expression matrix, and places them on a common gene index. The dense parent is the weighted directed edge list SHROOM produces, read from disk as a table of source gene, target gene, and inferred regulatory weight⁴⁵. The expression matrix is the single-cell count data the diagnostic calibration reads to measure perturbation responses. FUNGI maps the parent graph's genes onto the expression matrix's gene order, so that every downstream array, the priors, the probe substrates, and the candidate-edge indices, refers to the same gene by the same integer. Treating an edge as a pair of integers rather than a pair of names is what lets the search run thousands of times at the speed it needs.

4.7.2 — Diagnostic calibration

The calibration stage turns the raw data into the targets the rest of the run optimizes against. It builds the perturbation-impact array and the differential-expression matrix from the loaded expression data, runs the diagnostic probe suite of section 4.4.3 on the dense parent, and returns the utopian bounds and confidence weights for every engaged topology target. The output is the fully specified loss of section 4.4, a fixed set of target bands and weights that the search scores every candidate graph against. The probe methods themselves are given in section 4.4.3; the point here is only that they run once, at the start of a run, and their result is cached, so the thousands of graph evaluations that follow reuse one calibration rather than recomputing it.

4.7.3 — Candidate-pool construction and dense-parent metrics

This stage prepares everything the kernel needs so that scoring a candidate graph during the search costs almost nothing. It begins by filtering the dense parent down to a candidate pool, keeping the highest-weight edges up to a configurable target density, since the search never selects from the full parent and the low-weight tail cannot survive pruning anyway. On that pool it then computes every fixed prior once, the source-quantile weights, the perturbation and pleiotropy and diversity priors of section 4.5.5, and the source-conditioned bridge resistances of section 4.5.7, and bakes them into a persistent GPU context the search reads without recomputation. Finally it measures the topology of the candidate pool itself. Because FUNGI only prunes, the pruned graph's topology is bounded by what the pool already contains, so these dense-parent metrics are what tell a run whether a target is reachable or sits beyond the structure the substrate can supply (section 4.4.1). With the pool fixed, the priors baked, and the reachability ceiling known, the run is ready for the expansive search that follows.

4.7.4 — Expansive search (Sobol)

The expansive search is FUNGI's coarse pass over the whole hyperparameter space. It samples the eight-dimensional space of section 4.5.9 with a Sobol sequence, a low-discrepancy sequence that fills the space far more evenly than independent random draws of the same size, so a fixed budget of samples covers the space without the clumps and gaps random sampling leaves⁸³. FUNGI draws a scrambled Sobol sequence of 8192 points, one point per candidate configuration, a power of two because the Sobol construction reaches its most uniform coverage at powers of two.

Every sample is one full setting of the eight quantities, and turning it into a score is the whole pipeline in miniature. FUNGI scores the candidate pool's edges with the DASH kernel at that setting, selects the highest-scoring edges under the per-gene budget, runs the motif-repair swap, and then puts the resulting graph through a viability gate before the loss ever sees it. The gate rejects graphs that are structurally broken regardless of their topology numbers, a graph that has fragmented into disconnected components, left too many genes with no edges at all, collapsed its budget onto a few saturated hubs, or fallen below a minimum clustering. A graph that fails the gate is assigned a sentinel loss and dropped from contention, and only the graphs that survive it are scored against the utopian loss of section 4.4. The order matters, because the loss compares topology bands and a broken graph can still post plausible-looking bands while being useless as a network, so FUNGI checks that a graph is a viable network first and how good a network it is second.

Each evaluation is made deterministic by seeding its motif-repair swap from a hash of its own parameters, so the same configuration always produces the same graph and the archive is reproducible. The 8192 evaluations are spread across parallel workers, or batched on the GPU, and written out incrementally as shards, so a run interrupted partway resumes from the shards already on disk rather than starting over. The output is an archive of every scored configuration, and the lowest-loss viable graph in it is the best the coarse search found and the seed for the refinement that follows.

Figure 4.8 shows the search draw in code, a scrambled Sobol sequence with a viability gate applied before the loss.

```
search.py :: generate_sobol_samples + _fungi_process_row
def generate_sobol_samples(n_genes, n_samples, hp_cfg, seed=42, ...):
    lower, upper, names = get_static_bounds(n_genes, hp_cfg, ...)
    m = int(np.ceil(np.log2(max(n_samples, 2))))
    ns = 2 ** m
    n_dim = len(lower)
    sampler = Sobol(d=n_dim, scramble=True, seed=seed)
    scaled = lower + sampler.random(n=ns) * (upper - lower)
    scaled = scaled[:n_samples]
    return scaled, lower, upper

# ... (elsewhere in search.py) _fungi_process_row's viability gate:
od = np.bincount(surv_s, minlength=ng)
idd = np.bincount(surv_t, minlength=ng)
active = int(np.count_nonzero(od + idd > 0))

sh, reason, gwcc_fraction = check_shatter(
    surv_s, surv_t, surv_W, od, ng, active, shatter_cfg)
if sh:
    return _shattered(reason, od, active, len(surv_s))
```

Figure 4.8 | Deterministic Sobol search. FUNGI draws a scrambled eight-dimensional Sobol sequence of 8,192 points, one per candidate configuration, and scores each by running the kernel, selecting the graph, and passing it through a viability gate before the loss. A structurally broken graph is rejected before it is scored, and each evaluation is seeded from its own parameters so the archive is reproducible.

Figure 4.9 shows the loss across that archive, a smooth basin with the champion sitting in an isolated low-loss needle.

FUNGI Phase 3 · Sorted Sobol loss curve | n=8,192, min = 0.000

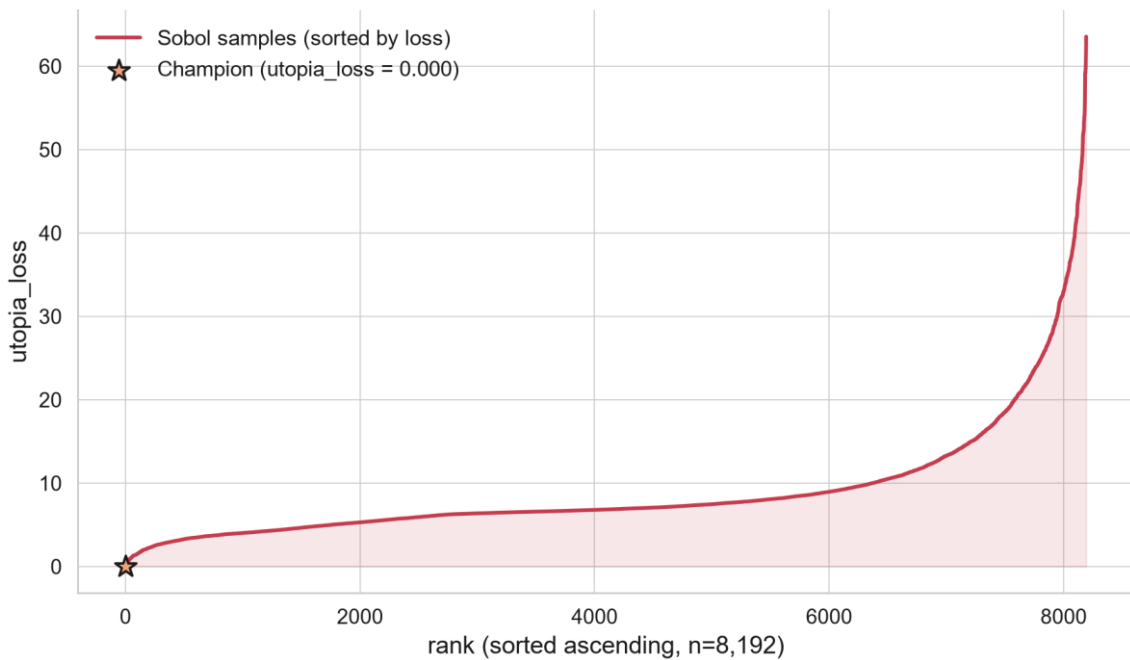


Figure 4.9 | The Sobol loss landscape. The utopian loss across the 8,192-sample archive, sorted from best to worst. The bulk of viable samples sit in a broad basin, and the champion occupies an isolated low-loss needle, which is why uniform Sobol sampling recovers it where the smoother global optimizers tested against it do not.

Figure 4.10 maps the viable region across pairs of the searched hyperparameters.

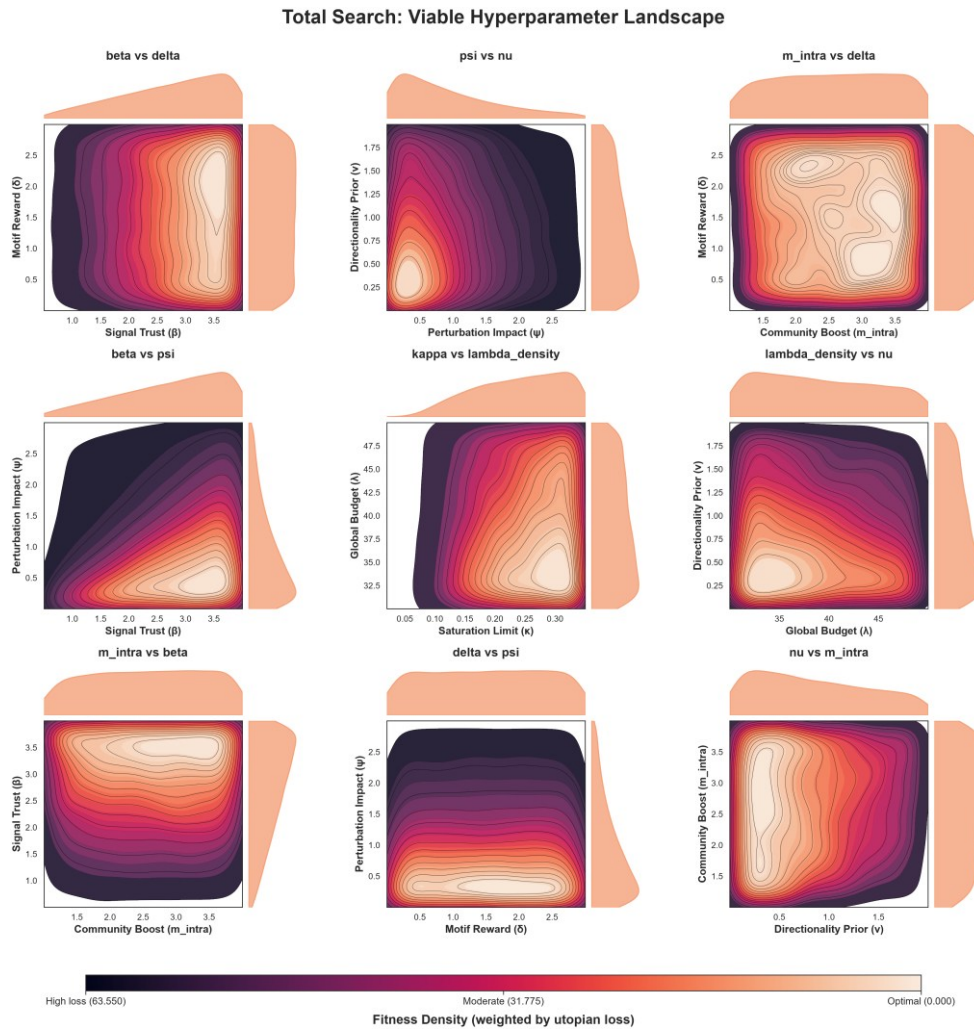


Figure 4.10 | The viable hyperparameter landscape. Fitness density across pairs of the searched hyperparameters, with marginal ridges. The viable region is broad and smooth across most pairs, so many different settings reach a biologically valid graph, and that slack is what the secondary objective later exploits.

4.7.5 — Refinement

The Sobol sweep is deliberately uniform, so it locates the promising regions of the space but does not settle into them. Refinement is the focused search that does. It takes the best-loss configurations from the archive, groups them into a small number of regions with density-based clustering, and searches each region locally instead of sampling the whole cube again⁸⁵. A hard cap on the number of regions, kept by quality, stops the search from fragmenting into dozens of shallow basins.

Each region is searched inside a trust region, a box drawn around the region's best configuration. A Gaussian-mixture, density-ratio sampler proposes new settings inside the box by modeling which parameter values separate good scores from bad ones and drawing toward the good, the tree-structured Parzen estimator strategy⁸⁶. The box adapts as the search proceeds, its radius growing on rounds that improve the best score and shrinking on rounds that do not, and a region is retired once its radius falls below a floor, which keeps effort on the regions that are still paying off.

A random-forest surrogate makes the local search cheap⁸⁷. It learns the map from a parameter setting to its score as the archive grows, from the losing configurations as well as the winning ones, and pre-screens the sampler's proposals so the expensive graph evaluations are spent on

the settings it predicts will score well. Learning from the losers is what lets it recognize which corners of a region to avoid, which a model fit to winners alone cannot do.

Refinement scores in two modes. While a region has not yet found a graph that reaches zero loss it minimizes the utopian loss, driving toward a biologically valid graph. Once a region reaches zero loss, every graph in it already satisfies the biology, so the search switches to a second objective that chooses among those valid graphs, a Derringer–Suich desirability that combines several secondary targets into one score through a weighted geometric mean⁸⁸. This second objective is what lets FUNGI prefer, among graphs that are equally biologically valid, the one best suited to a downstream consumer, and it is developed in full in section 4.8. Throughout, the global best is never displaced by a region-local update, so refinement can only improve on what the coarse search found.

Figure 4.11 compares the full Sobol draw with the low-loss refined subset for each searched hyperparameter.

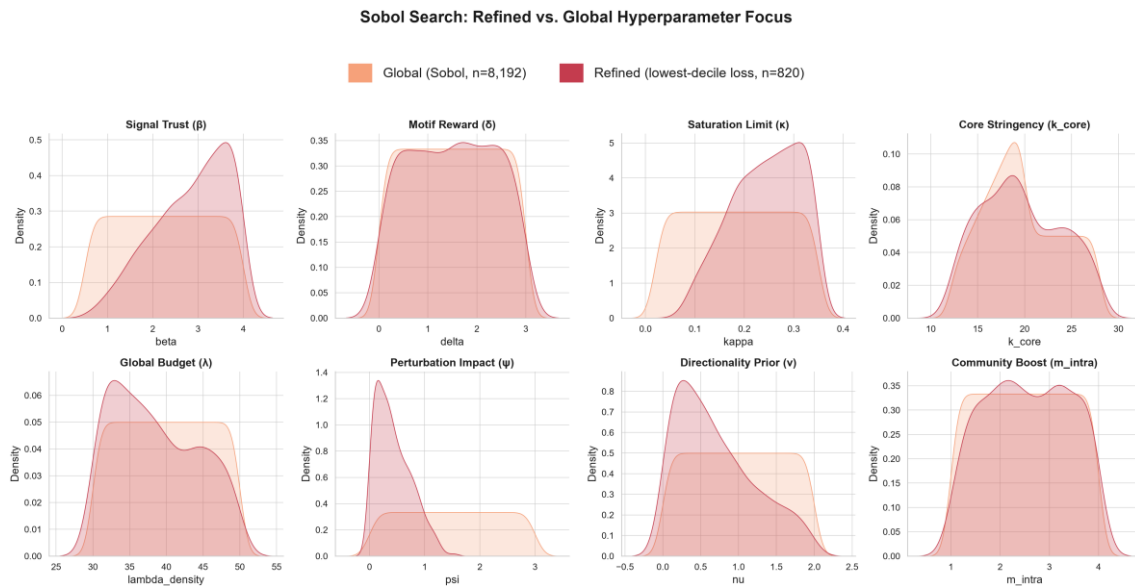


Figure 4.11 | Refined versus global hyperparameter focus. For each searched hyperparameter, the distribution over the full 8,192-sample Sobol draw against the lowest-loss decile. Most parameters sample as a broad plateau, and their low-loss subset simply concentrates within it. k_{core} is the exception. It sets the size of the core subgraph over which feed-forward structure is counted and edges are selected, so its extreme values push graphs to fail the viability gate and drop out of the scored archive, which is why its distribution peaks toward the middle of its range rather than holding the flat plateau the continuous kernel gains show.

4.7.6 — Champion selection

Refinement leaves the run with an archive of scored configurations, and champion selection picks the one FUNGI commits to and rebuilds it exactly. The winner is the configuration with the lowest utopian loss, or, once refinement has found graphs that reach zero loss, the one the desirability secondary ranks highest among them. Selecting the recipe is not the end of the step, because the search scored thousands of candidates with a fast, approximate topology computation to keep each evaluation cheap, and those approximate numbers are good enough to rank candidates but not to report. FUNGI reconstructs the champion graph from its recipe and re-measures it exactly. The reconstruction is deterministic, seeded from the same hash of the recipe's parameters that the search used and run with the same kernel settings, so the rebuilt graph is identical to the one the search scored rather than a near-copy of it. The champion's topology is then recomputed with the exact metrics in place of the fast approximations, so the numbers FUNGI reports for the graph it ships are measured rather than estimated.

4.7.7 — Graph output

The champion is written to disk in the same form as the dense parent it was pruned from, a weighted directed edge list of source gene, target gene, and score. Keeping the output format identical to the input is deliberate, because it lets the pruned graph drop into the downstream pipeline exactly where the dense parent would have gone, with no adapter in between. This edge list is the artifact the rest of the pipeline consumes, the graph a downstream perturbation-prediction model reads as the wiring it passes signal over, and producing it is the purpose of the FUNGI stage.

4.8 — Beyond Biological Topology: Application-Specific Optimization

4.8.1 — The dual objective: biological shape first, then a secondary target

A biologically valid graph is not a single graph. The topology targets of section 4.3 are bands rather than points, so many different prunings of the same dense parent all land inside them, and the utopian loss scores every one of them as equally valid. That slack is an opportunity. If a whole family of prunings is biologically indistinguishable, FUNGI is free to choose among them by some other criterion without giving up any biological fidelity, and the criterion it chooses is fitness for the model that will consume the graph. This is FUNGI's dual objective. The biological shape comes first, and a second objective then selects, among the graphs that already have that shape, the one best suited to a downstream consumer.

The two objectives are kept in a strict order, and the order is what makes the idea safe. The refinement stage minimizes the utopian loss until a region reaches zero loss, and only then switches to the secondary objective described in section 4.7.5, the Derringer–Suich desirability that ranks graphs on downstream-relevant structure⁸⁸. Because the switch happens only inside a region that has already reached zero loss, every graph the secondary ever compares is biologically valid, so the secondary can reorder those graphs but can never trade biological validity away for downstream fitness. The primary objective gates the secondary. FUNGI optimizes for the application only within the envelope of graphs that are already biologically correct.

Whether the secondary engages at all on a given run depends on the substrate, for the reason set out with reachability in section 4.4.1. FUNGI only prunes, so a zero-loss graph exists only when the dense parent can support one, and on a substrate where a target is unreachable the primary objective keeps a nonzero floor and FUNGI stays in biological-optimization mode with the secondary held in reserve. Where a zero-loss biological graph is reachable, the gating engages the secondary strictly among those graphs and reshapes which champion is selected, exactly as the dual-mode design intends. The capability is a property of the kernel and the search, and it activates wherever the biology can first be satisfied.

Figure 4.12 shows the payoff of optimizing for the application, a graph tuned for signal flow reaching more of each perturbation's targets.

RHIZO super-graph · Synthetic search beats biologic at every density | epr_k

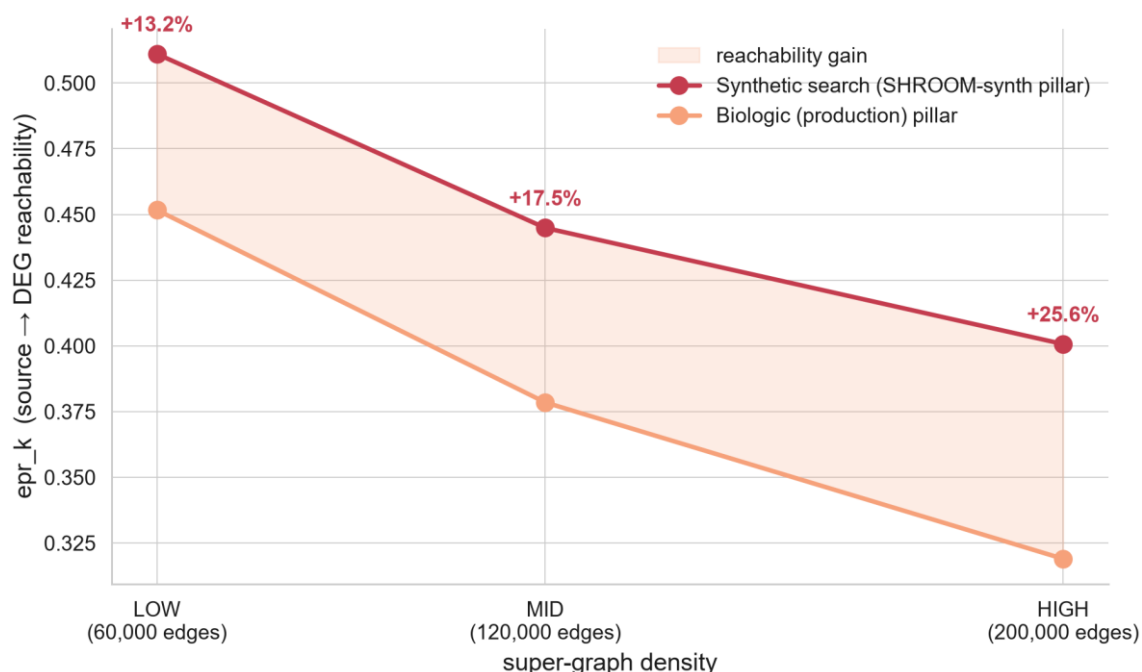


Figure 4.12 | Application-specific optimization improves reachability. Source-to-target reachability of a graph optimized for downstream signal flow against the production graph, across three edge densities. The application-tuned graph reaches more of each perturbation's true targets at every density, by 13 percent at low density, 18 percent at medium, and 26 percent at high, and the gap widens as the graph grows denser.

4.8.2 — Use case: a downstream perturbation-prediction model (SPECTRA)

The most direct consumer of a FUNGI graph is a downstream perturbation-prediction model such as SPECTRA, which reads the graph as the wiring it propagates a perturbation's effect along⁴³. What such a model needs from a graph is not only biological realism but good signal flow, because its prediction for a perturbed gene is only as good as the graph's ability to carry that perturbation to the genes it actually changes. A graph can be biologically valid and still route that signal poorly, bottlenecking it through a few edges so that distant targets are never reached, the over-squashing failure that section 4.5.7 introduced⁷⁸. FUNGI's secondary objective can target signal flow directly, preferring among biologically valid graphs the one whose structure lets the model's message passing reach a perturbation's true targets, so the same graph is at once a faithful regulatory map and an efficient substrate for the model that rides on it.

4.8.3 — Use case: the topology-aware arbiter (RHIZO)

The downstream model this thesis builds makes the secondary objective concrete, because its idea of a good graph can be written down as explicit structural quantities. RHIZO, the arbiter model developed in Chapter 5, predicts a perturbation's response by propagating it from the perturbed source across the graph, so the graph properties that help it are the ones that govern that propagation, a low effective resistance from a source to the genes it regulates, short and reachable paths from perturbed sources to their targets, and a directional coherence that keeps signal flowing the way regulation does^{78,79}. These are exactly the kinds of quantities the desirability secondary combines, so FUNGI can be pointed at a RHIZO-shaped objective and will select, from among the biologically valid graphs, the one that best serves it. The topology-aware

pruner and the topology-aware model then optimize for the same structure, which is what lets one be tuned to the other.

4.8.4 — Use case: an industry-standard model (GEARS)

The same mechanism extends to models FUNGI did not help design. GEARS is an established perturbation-prediction model that runs a graph neural network over a fixed external knowledge graph, the same generic gene-ontology graph for every dataset regardless of the cells it is applied to ¹¹. A graph curated for both biological validity and the way GEARS propagates signal is a drop-in replacement for that fixed input, one that is specific to the dataset at hand rather than generic and tuned to the model that will consume it rather than assembled independently of it. This is the practical payoff of the dual objective. A single FUNGI graph can be handed to a downstream model as an input that is at once biologically faithful and shaped for the model's own architecture, without the two goals being pursued in separate graphs.

4.8.5 — Why the DASH kernel enables this

None of this would be possible with a fixed pruning rule, and the reason FUNGI can carry a second objective at all is the structure of the DASH kernel. The kernel is a parameterized family of scoring functions rather than a single formula, with the searchable exponents and terms of section 4.5.9 setting how strongly each piece of evidence weighs on an edge, and the search already ranges over that family to find the biological shape. The secondary objective rides on the same machinery. It re-weights the same kernel terms to favor downstream-friendly structure, drawing from the same space of prunings the primary search explores, so optimizing for the application is not a separate procedure bolted onto FUNGI but the same search reading a second scoreboard once the first is satisfied. Because the biological objective is a set of bands with slack inside them, that second scoreboard has graphs to choose between, and because the kernel is expressive enough to move a graph within those bands, the choice is real. The DASH kernel is what turns a biologically valid pruning from a single answer into a family of answers that FUNGI can then optimize over, and that is what lets one graph be biologically optimal and model-optimal at the same time.

4.9 — Baseline Pruning Methods

FUNGI is compared against the two structure-blind pruners the downstream evaluation carries through end to end, top-K by weight and the k-nearest-neighbors graph. Both are co-expression baselines, both reduce the dense parent by a single local rule, and neither targets the global shape of a regulatory network. What makes the pair a fair test rather than a pair of straw men is that between them they bracket the two ways a structure-blind pruner goes wrong. Top-weight over-concentrates the graph, letting a few genes monopolize the edges, and k-nearest-neighbors over-flattens it, forcing every gene to look the same. FUNGI's degree targets are meant to sit between those two failures, so measuring against both is what shows whether it actually does.

4.9.1 — Top-K by weight

The most natural baseline keeps the highest-weighted edges and discards the rest, either globally or under a per-gene budget, the global weight threshold that network-backbone methods take as their point of comparison ⁸⁹. It takes the scores the dense generator already produced and trusts them directly, which makes it the zero-effort pruning and a genuinely strong reference, because those raw weights do carry real regulatory signal and any method worth using has to beat them. Its weakness is that it trusts them completely. A high co-expression weight is not the same as a regulatory relationship, as sections 3.2 and 3.3 set out, so keeping edges by weight also keeps the co-expression pairs that share an upstream driver or sit in the same complex, which correlation alone cannot tell apart from a true regulatory edge ¹⁴, and it lets whichever genes happen to carry the highest-scored edges accumulate as many as their weights allow. Nothing caps a gene's out-degree, nothing enforces a hub hierarchy, and

nothing reads the direction of what survives. The result is a graph whose shape is a by-product of the weight distribution rather than a target, and top-weight pruning is the primary reference FUNGI is measured against in the downstream evaluation.

4.9.2 — K-nearest-neighbors graph

The k-nearest-neighbors pruner keeps each gene's top k edges, capping every gene's out-degree at the same value k. It is in one respect an improvement on top-weight, because the per-gene cap guarantees coverage, giving every gene its k best edges so that no gene is orphaned and no hub can monopolize the budget, which is why it is a standard reference in single-cell GRN benchmarking ⁶. That same cap is its fatal flaw as a regulatory graph. If every gene has exactly k out-edges, the out-degree distribution is a spike at k and its inequality is zero, the exact opposite of the scale-free hub hierarchy of section 4.3.1, so a master regulator that should govern hundreds of targets is held to the same k as a terminal effector and the graph cannot represent a hub at all. Where top-weight lets the strong genes take everything, k-nearest-neighbors gives every gene the same share, and both are wrong for the same underlying reason, that a single local rule cannot produce the unequal, hub-dominated degree distribution a regulatory network actually has. That is the gap FUNGI's degree and inequality targets are built to close, and the comparison against both baselines in Chapter 6 is where that claim is tested.

Table 4.2 brackets the two structure-blind baselines by the single quantity each optimizes and the way each fails.

Table 4.2 | The two structure-blind baselines. Each reduces the dense parent by one local rule and misses the global shape of a regulatory network in an opposite direction, which is why FUNGI's degree targets are meant to sit between them.

Baseline	What it keeps	Failure mode
Top-K by weight	The highest-weighted edges, no degree cap	Over-concentrates; a few genes monopolize the edges
K-nearest-neighbors	Each gene's top k edges, one uniform cap	Over-flattens; out-degree Gini is zero, no hub hierarchy

4.10 — Datasets and Experimental Setup

FUNGI is evaluated on the two Perturb-seq datasets the pipeline runs end to end, each a CRISPR-interference screen read out by single-cell RNA sequencing and prepared by SPORE before the generators of Chapter 3 turn it into the dense parent FUNGI prunes ⁸. Their construction belongs to those chapters; what matters here is their scale and their splits.

The primary substrate is the RPE1 genome-scale Perturb-seq screen of Replogle and colleagues ²¹. FUNGI runs on its feature-selected 5,000-gene essential panel, across 1,514 perturbations partitioned by gene into 1,058 training, 152 validation, and 304 test perturbations under a fixed seed, so that no perturbed gene appears in more than one split. The second substrate is the H1 human embryonic stem cell screen from the Virtual Cell Challenge, a smaller CRISPR-interference Perturb-seq dataset on a 5,024-gene panel with 148 perturbations split into 108 training, 20 validation, and 20 test ⁵⁵. RPE1 carries the main evaluation, and H1-hESC is the second cell line on which the pipeline's behavior is checked for whether it holds beyond a single dataset.

The setup follows the leakage firewall the whole pipeline is built around. The dense parent, and therefore every graph FUNGI prunes from it, is built from the training perturbations alone, so the held-out validation and test perturbations are seen by neither the inference, the pruning, nor the bounds the diagnostics calibrate. Within that firewall FUNGI's pruned graph is compared against the top-weight and k-nearest-neighbors baselines of section 4.9, each reduced to the same edge budget, and every resulting graph is judged not on its own topology but on how well

the downstream model reads it to predict held-out perturbation responses, the evaluation Chapter 6 carries out.

4.11 — From Dense Parent to Curated Graph

FUNGI reframes pruning as an optimization toward the shape of a real regulatory network. It treats that shape as an explicit objective, a set of topology targets spanning the scale-free degree hierarchy, the inequality of regulatory output and input, the hub fraction, the modular and clustered structure, the degree assortativity, and the directed reciprocity, and it collapses how well a whole graph matches them into a single utopian loss whose reachable floor is bounded by what the dense parent can support. The DASH kernel turns that whole-graph objective into a per-edge decision, scoring each candidate edge by combining the inferred regulatory weight with biological priors and topology terms, and the optimization pipeline searches the kernel's hyperparameters, a coarse Sobol sweep followed by focused refinement, for the graph that best matches the targets. Because the objective is a set of bands rather than a single point, the same machinery can then optimize a secondary, application-specific objective among the graphs that already satisfy the biology, so one curated graph can be biologically valid and tuned to the model that will consume it at once.

None of this fixes the graph's worth in place, and that is the note the chapter ends on. FUNGI only removes edges, so the curated graph is bounded above by the dense parent it was pruned from and its value is decided below by the model that reads it. Its own topology numbers, however close to the targets, are a means rather than the verdict. The edge list FUNGI outputs is the wiring the downstream arbiter of Chapter 5 propagates a perturbation across, and whether the curation earns its place, against the structure-blind baselines and across both cell lines, is the question the integrated evaluation of Chapter 6 answers.

CHAPTER 5

Predictive Models

MOREL & RHIZO

Turning the curated graph into a prediction. Two complementary instruments, one a linear propagator on a CPU and one a neural network on a GPU, that test whether a topology-aware graph predicts held-out perturbation responses.

5.1 — Why the pipeline ends in a predictive model

Every stage of MYCELIUM produces a graph, and yet a graph is never the deliverable. SPORE cleans the data, SHROOM and HYPHAE infer the dense parent, and FUNGI prunes it into a curated regulatory map, but the reason to build that map is to predict how a cell responds when one of its genes is silenced. The pipeline therefore ends in a predictive model, and this chapter builds the two that MYCELIUM uses. They are complementary by design, one a small linear propagator that runs on a CPU and one a neural message-passing network that runs on a GPU, and the sections below introduce each in turn.

5.1.1 — A graph's worth is a model's held-out prediction

A curated graph cannot be judged on its own topology score, because that score does not say whether the graph helps predict anything. A dense graph can reproduce a curated interaction database and still be the wrong input for a model, and a pruned graph can lose a little edge-level precision and still be the better substrate once a model propagates signal across it. Neither outcome is visible from the graph in isolation. The only test that settles it is a downstream one, whether a fixed model, handed the graph, predicts held-out perturbation responses better than when it is handed another graph. This is a demanding bar, and the field's own experience sets the expectation low. A recent benchmark found that current deep-learning perturbation predictors do not yet beat deliberately simple linear baselines⁹⁰, so the goal here is not a state-of-the-art predictor. It is a sensitive, honest instrument, a model whose prediction changes when and only when the graph it reads changes. Both models in this chapter predict the same object, the genome-wide response to silencing a chosen gene, and both sit in the graph-consuming family of perturbation predictors¹¹ rather than the family that maps expression to expression with no explicit network²³.

5.1.2 — Two instruments, one claim: MOREL and RHIZO

MYCELIUM tests its central claim with two models rather than one. MOREL is a directed linear propagator, a small model that runs on a CPU and reads the graph through sparse linear algebra. RHIZO is a nonlinear message-passing graph neural network that runs on a GPU. They are the CPU yin and the GPU yang of the same idea, and they are deliberately built on opposite technical foundations so that a shared conclusion cannot be an artifact of one architecture. What they hold in common is the property that makes either one a clean instrument. Neither imports any external feature, no ontology, no pretrained embedding, no per-gene identity prior, so the graph is the only thing that varies when one graph is swapped for another. A result that holds on a linear CPU propagator and again on a neural GPU network is far harder to dismiss than a result from a single model, which is why the pipeline is validated on both.

Figure 5.1 shows the two instruments side by side, one linear and one neural, reading the same graph.

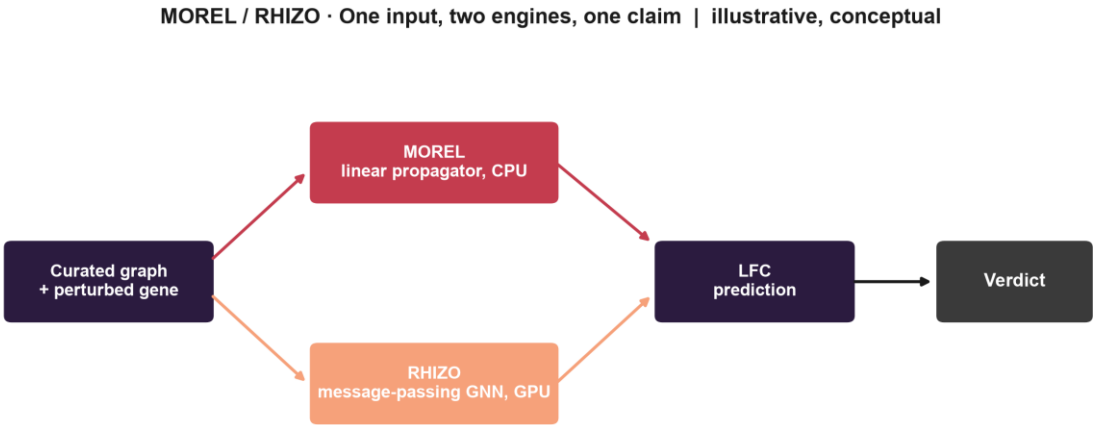


Figure 5.1 | The two instruments. A single curated graph and a perturbed gene feed both models. MOREL propagates the perturbation with linear algebra on a CPU, RHIZO with a message-passing network on a GPU, and both return a predicted expression response. Confirming a result on two independent instruments is stronger than on one.

Table 5.1 sets the two instruments side by side.

Table 5.1 | The two predictive models. MOREL and RHIZO compared across the axes that make them a complementary pair, one a linear propagator on a CPU and one a neural network on a GPU, both reading the graph with no external features.

Axis	MOREL	RHIZO
Engine	Directed weighted linear propagation	FAGCN message-passing network
Hardware	CPU	GPU
External features	None	None
Readout	Per-gene ridge regression	Neural readout
Capacity	Tens of thousands of coefficients	Small network (node dimension 16 to 64)
Role	Load-bearing linear instrument	Load-bearing neural instrument

5.2 — MOREL: Mechanistic Ontology-free Regulatory Estimation by Linear-propagation

The linear instrument, the yin to RHIZO’s neural yang. MOREL propagates a perturbation across the graph with sparse linear algebra and reads it out with a small per-gene model, light enough to train on a CPU in minutes.

MOREL is the linear instrument, built to be the smallest model in which the only path from a perturbation to a response runs through the graph. It is a directed weighted linear propagator, and its deliberate simplicity is the point rather than a limitation.

5.2.1 — What MOREL is: a directed linear-propagation instrument

MOREL propagates a perturbation across the directed, weighted graph and reads out each gene’s response with a tiny per-gene linear model, with no node features anywhere in the pipeline. It follows the lineage of CellOracle, which infers a regulatory network and then simulates a perturbation by propagating its effect across that network in silico ⁹¹, but it strips the

idea down to a measuring device. There are no embeddings, no per-gene latent vectors, and no cell-level features. The whole model is sparse matrix multiplication followed by a small ridge regression, so it trains on a CPU in minutes per graph and never touches the GPU. MOREL is not built to top a leaderboard. It is built to be a sensitive and load-bearing instrument, one whose only source of graph-specific signal is the graph itself.

5.2.2 — The load-bearing design: the only path runs through the edges

The absence of node features is what makes MOREL a clean test of a graph. Because a gene carries no features of its own, the only way a perturbation of gene p can change the predicted expression of gene t is through a directed path of edges from p to t . Swapping the graph therefore swaps the entire mechanism by which a prediction is formed, which makes a graph comparison a direct causal test rather than a correlational one. The instrument property is exact at the boundary. Given an empty graph, no edge carries the perturbation anywhere, and the model predicts zero specific signal for every gene by construction. This design is a direct response to how the previous downstream model failed. A model seeded with rich pretrained embeddings could predict a response from gene identity alone, so a structure-blind random graph matched a curated one at inference and the graph stopped being the variable under test^{43,90}. MOREL removes that bypass by removing the features that enabled it.

5.3 — MOREL: how it forms a prediction

MOREL turns a graph and a perturbed gene into a predicted response in three moves, a localized injection, a directed propagation, and a per-gene readout, each kept as simple as the instrument role allows.

5.3.1 — Perturbation injection and directed propagation

A perturbation enters the model as a localized shift at the perturbed gene. The initial signal $h^{(0)}$ is zero everywhere except at the perturbed gene p , where it is set to δ_p , the knockdown shift, defaulting to $-\mu_{\text{ctrl}}[p]$ so that CRISPR interference drives the silenced gene toward zero²¹. The signal is then propagated across the graph with a direction-preserving normalization. Each edge weight is column-normalized so that a regulator distributes its shift across its targets, and the graph is never symmetrized, because symmetrizing would erase the edge direction that the whole experiment is built to test. The propagation runs for K hops, $h^{(k)} = \tilde{A} h^{(k-1)}$, with the spectral radius of $\tilde{A}$ held at or below one so a multi-hop signal cannot explode. The hop count is capped low, around three, because past a few hops a dense graph over-smooths toward its degree-stationary vector, at which point the propagator has become a structure-blind smoother that no longer distinguishes one graph from another.

Figure 5.2 shows a perturbation spreading from its source over the directed graph, with unreached genes held at zero.

MOREL, illustrative graph · Propagation only reaches what the edges connect | seed=42, conceptual, not real data

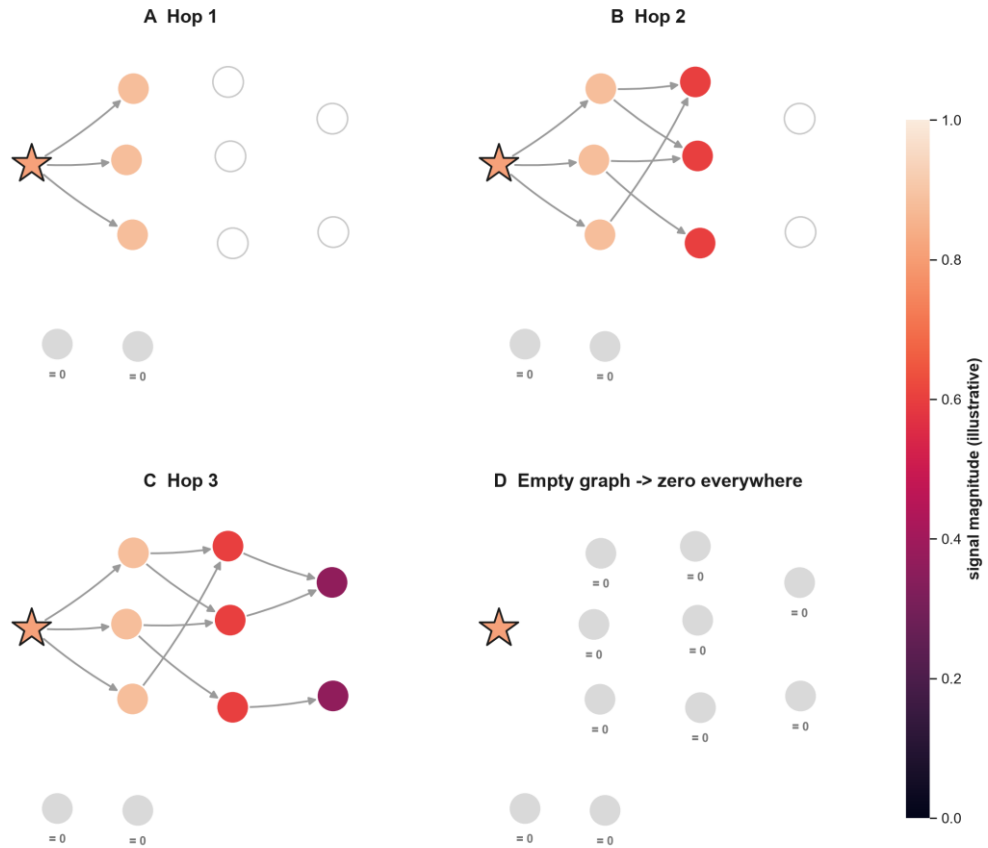


Figure 5.2 | MOREL propagation and the load-bearing property. A perturbation injected at a source gene spreads across the directed graph one hop at a time, and a gene is predicted only if a directed path reaches it. A gene with no path from the source stays at zero, and an empty graph predicts zero everywhere, which is what makes the graph the only source of signal. Illustrative.

5.3.2 — Per-gene ridge readout on the propagated features

The readout is a separate linear model for each gene, fit to that gene's own propagated signal. For a perturbation of gene p , the features of gene g are its values at each hop, $[h_g^{(1)}, \dots, h_g^{(K)}]$, and MOREL fits an independent ridge regression per gene in closed form, parallelized across CPU cores. Keeping the readout per-gene is what preserves the load-bearing property. Each gene's prediction is a function of its own propagated features alone, which are exactly zero when no path reaches it, so a gene the graph does not connect to the perturbation cannot borrow a prediction from any other gene. The model stays small, a few tens of thousands of coefficients in total, which is part of why it trains in minutes and why its capacity cannot substitute for the graph.

Figure 5.3 gives the model in code, the propagation loop and the per-gene ridge.

```
dwlp.py :: propagate() + DWLP.fit()
def propagate(At, pert_gene_idx, delta, K):
    H = np.zeros((N, P)) # H^(0): zero except the perturbed gene
    H[rows, cols] = delta[valid]
    for k in range(K):
        H = At @ H # direction-preserving, out-normalized adjacency
        feats[k] = H
    return feats # (K, N, P)

# ... per-gene ridge, no intercept: W_g = (X_g^T X_g + alpha I)^-1 X_g^T y_g
Gram = np.einsum("pgk,pgl->gkl", X, X) # (G, K, K), one tiny system per gene
Gram += self.alpha * np.eye(self.K)[None, :, :]
Xty = np.einsum("pgk,pg->gk", X, y)
self.W = np.linalg.solve(Gram, Xty[:, :, None])[ :, :, 0]
```

Figure 5.3 | MOREL in a few lines. The K-hop propagation over the direction-preserving normalized adjacency, followed by the independent per-gene ridge readout. The whole model is sparse matrix multiplication and a small regression.

5.3.3 — Training on the specific residual

MOREL is trained to predict the part of a response that is specific to the perturbed regulator, not the part shared across every perturbation. Silencing an essential gene drives a cell into a shared arrest program that looks much the same whichever gene was hit, and that shared response dominates the raw expression change while carrying no regulator-specific information³⁶. MOREL removes it before training. The target for perturbation p is the specific residual $s_p = \Delta_p - \bar{\mu}$, the perturbation's expression change with the training-set mean response subtracted, and the shared response $\bar{\mu}$ is computed on the training perturbations alone so that no held-out information leaks into it. Training on the specific residual has a clean consequence that makes the graph comparison honest. A model that predicts the shared mean and nothing else scores zero by construction, so any score above zero is credit for recovering regulator-specific structure, which is the only thing a graph can help with.

5.4 — RHIZO: Regulatory Hop Inference, Zero Ontology

The neural instrument, the yang to MOREL's linear yin. RHIZO reasons over the curated graph with a message-passing network, carrying a perturbation from its source to the genes it reaches.

"I believe that mycelium is the neurological network of nature."

Paul Stamets, Mycelium Running

Every stage of MYCELIUM to this point produces a graph, and RHIZO is the stage that finally uses one. SPORE cleans the perturbation data, SHROOM and HYPHAE infer the dense parent, and FUNGI prunes it into a curated regulatory map, but a graph is a means rather than an end. RHIZO is the downstream arbiter, the model that reads a curated gene regulatory network and predicts how a cell responds when one of its genes is silenced. It is the last stage of the pipeline and the instrument the integrated evaluation runs on, because a prediction is the thing a regulatory map is ultimately for. The name carries the design. RHIZO performs regulatory hop inference, propagating a perturbation across the graph one regulatory hop at a time, and it imports zero external ontology, which is the point the rest of the chapter builds to.

5.4.1 — The arbiter: from a curated graph to a perturbation prediction

RHIZO takes a graph and the identity of a perturbed gene and returns the predicted genome-wide response, a vector of log-fold-changes with one entry per gene. This is the central computational problem set out at the start of this thesis, the map from a regulatory network and a perturbation to the expression shift it causes, and RHIZO is the model that solves it over an explicit graph. Two families of perturbation predictor exist in the field. One learns the response directly from expression data with no explicit network, inferring the effect of a perturbation from the patterns it has seen across many measured perturbations²³. The other propagates the

perturbation over an explicit gene graph with a graph neural network, so the prediction for a gene is built from the genes upstream of it ¹¹. RHIZO is the second kind, and it is built so the graph it reasons over is the curated MYCELIUM graph rather than a fixed external one. Its input side is a graph and a perturbed gene, its output side is a differential-expression vector, and it is deliberately lightweight, a design choice developed in the architecture below.

5.5 — RHIZO: Zero Ontology

RHIZO reads only two things, the graph it is given and the training-perturbation responses it is fit on, and nothing else. That refusal to import anything from outside the data is the conceptual heart of the model and the load-bearing half of its name, the zero ontology.

5.5.1 — What "zero ontology" means, and what it excludes

Zero ontology means RHIZO carries no external knowledge about what genes are or how they relate, and the exclusions are concrete. It uses no gene-ontology or knowledge graph, unlike GEARS, which runs its graph neural network over a fixed gene-ontology graph that is the same for every dataset regardless of the cells it is applied to ¹¹. It uses no pretrained embeddings, unlike SPECTRA, which seeds each gene node with a scGPT expression embedding, an identity prior learned on outside data ⁴³. It uses no external identity priors of any kind, no curated transcription-factor motifs, no ontology annotations, no per-gene features keyed to which gene a node happens to be. Its inputs are the graph topology and the training-perturbation responses, and its edge features are computed purely from the graph. The rule is enforced in the code rather than left as an aspiration. Every edge feature RHIZO reads is a deterministic function of the graph it is given, computed identically for every graph, so nothing tied to gene identity or an outside database can enter the model.

5.5.2 — Why graph-agnosticism makes RHIZO the clean instrument

Because RHIZO imports no external prior, any difference in its predictions is attributable to the graph it was handed rather than to a knowledge base it also carries, and that is exactly the property the pipeline needs from its final judge. A model loaded with external ontology confounds the graph-quality test. When such a model predicts well, the win cannot be separated into the part that came from the curated graph and the part that came from the pretrained knowledge, so the graph and the ontology are measured together and neither is measured cleanly. RHIZO's blindness to outside data removes that ambiguity. When it predicts a held-out response better on a FUNGI-curated graph than on a structure-blind prune of the same dense parent, the graph is the only thing that changed, so the difference is the graph. Zero ontology is what lets RHIZO isolate the quality of the substrate and the pruning, which is the entire job the arbiter has in the thesis.

5.5.3 — Alignment with the MYCELIUM philosophy

Zero ontology is not a local choice for the model, it is the pipeline's governing principle extended to its final stage. SPORE derives its cleaning thresholds from the data, SHROOM and HYPHAE infer the graph from the expression matrix rather than importing a network, and FUNGI reads its topology targets from the dense parent rather than assuming what a regulatory network should look like. Each stage answers a question from the data in front of it instead of from an outside reference. RHIZO carries that same principle into the downstream model, importing no external ontology anywhere in the pipeline, so the final verdict rests on the data and the curated graph alone. The field offers the opposite trade. Methods that lean on a fixed knowledge graph ¹¹ or on pretrained embeddings ⁴³ buy a boost in accuracy with an external prior, at the cost of no longer being able to say whether a result is about the graph or about the prior. RHIZO refuses that trade on purpose, so that when the integrated evaluation reports a number, the number is a statement about the graph MYCELIUM built.

5.6 — RHIZO: Architecture, a lightweight message-passing model

RHIZO turns a graph into a prediction by passing information along the graph's edges, which makes the graph the medium of the computation rather than a side input to it. The model is a small message-passing graph neural network, lighter than the DASH kernel that precedes it, and the rest of this section gives its propagation basis, how a perturbation is injected and carried across the graph, the features it reads off each edge, and the objective it is trained on.

5.6.1 — Message passing over the regulatory graph (the FAGCN basis)

RHIZO propagates signal with a frequency-adaptive graph convolution, the FAGCN scheme, which separates the low-frequency and high-frequency components of the signal on the graph and learns how much of each to pass at every step⁷⁷. The choice is dictated by the biology of the graph. A regulatory network is heterophilic, meaning a regulator and its target often sit in different expression programs and carry different profiles, so a plain graph convolution that simply averages each node with its neighbors blurs a regulator into the genes it controls and erases the contrast that carries the regulatory signal²⁴. FAGCN keeps the high-frequency contrast that a regulatory graph depends on, which a homophily-assuming convolution discards⁷⁷. RHIZO propagates this over K hops, with K in the range of roughly three to twelve, so a perturbation reaches genes several regulatory steps downstream of its source. It combines the states from every hop with a jumping-knowledge readout and a teleport term that re-injects the source signal, which stops the deepest hops from driving every gene toward the same value, the over-smoothing failure that degrades deep graph networks⁴⁰. The model's capacity is kept small, with a node dimension on the order of sixteen to sixty-four, because the pipeline needs to train one arbiter per graph across many graphs and a small model keeps each of those comparisons cheap.

5.6.2 — Propagating a perturbation from its source, and the $\psi(0)=0$ invariant

RHIZO produces a prediction by injecting the perturbation at the perturbed gene and propagating it across the graph's directed edges to the genes those edges reach⁷⁷. This gives the model one invariant that governs everything downstream of it. A gene that message passing never reaches from the perturbed seed predicts exactly zero, which we write $\psi(0) = 0$. The behavior is correct rather than a defect. If no path of directed edges carries the perturbation from its source to a given gene, the model has been given no signal about that gene and returns no change for it, and the property is built into the architecture so that the readout and every optional component preserve it. The invariant has a consequence that the evaluation returns to. A causal graph carries outgoing edges only from genes that were actually perturbed, so for a gene it has no outgoing edges from, propagation reaches nothing and the prediction is zero across the whole genome. This is the structural reason a purely causal graph is mute on a held-out gene it never saw perturbed, a coverage boundary developed later in the chapter.

Figure 5.4 shows the same propagation for RHIZO, with an unreachable gene held at exactly zero by the $\psi(0)=0$ invariant.

RHIZO · An unreachable gene predicts exactly zero | illustrative graph, seed=42

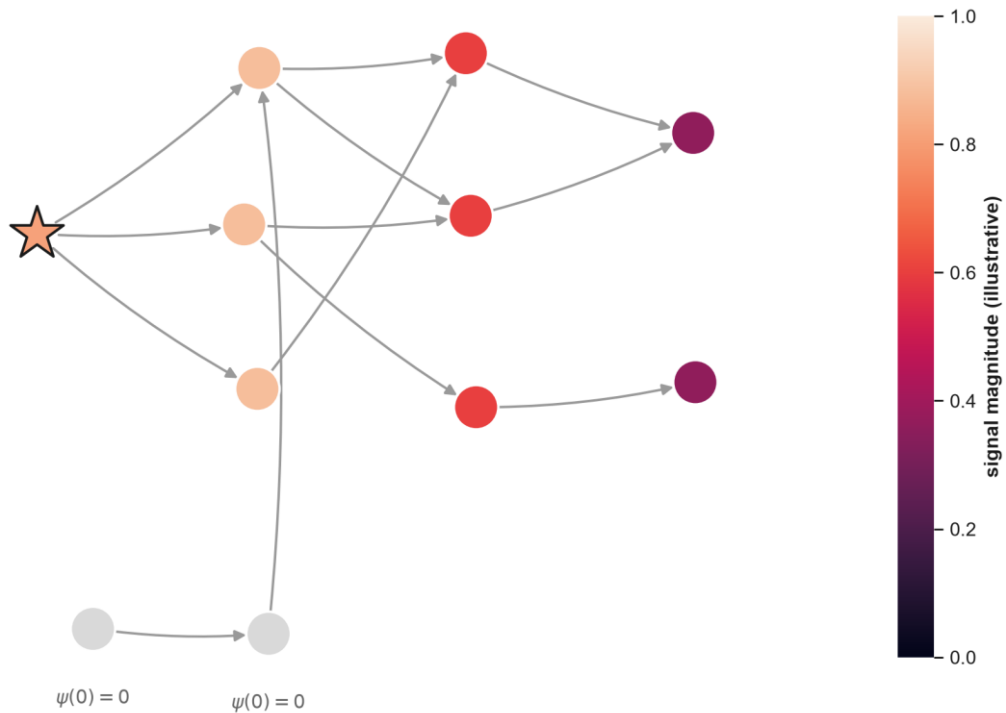


Figure 5.4 | RHIZO message passing and the $\psi(0)=0$ invariant. Drawn on the same graph as the linear case, a perturbation propagates from its source through the frequency-adaptive network, and a gene the message passing never reaches predicts exactly zero. The inset shows the low-frequency and high-frequency split a heterophilic regulatory graph needs. Illustrative.

Figure 5.5 shows the invariant in code, the readout that holds an unreachable gene at zero.

```
model_v3.py :: DNPv3.forward (RHIZOFinal backbone)
# seed injection: x0 is zero everywhere except the perturbed node
x0 = torch.zeros(N, B, d, device=dev, dtype=w.dtype)
valid = seed_nodes >= 0
bidx = torch.arange(B, device=dev)[valid]
x0[seed_nodes[valid], bidx, :] = deltas[valid].unsqueeze(-1)
x = x0

# ... K hops of message passing produce f, which is exactly 0 for any node no path reaches ...

# multiplicative, zero-preserving readout: 0 * anything = 0, so psi(0) = 0 survives every optional term
if self.use_ovsq and hop_w is not None:
    f = f * hop_w.view(-1, 1, 1)          # over-squash weighting
if self.use_film and c_g is not None:
    gamma = 1.0 + torch.tanh(self.film(c_g))
    f = gamma.unsqueeze(1) * f             # FiLM, multiplicative only
out = self.read_out(F.gelu(self.read_in(f))) # bias-free => psi(0) = 0
```

Figure 5.5 | The $\psi(0)=0$ -preserving readout. The perturbation seed injection and the multiplicative readout that keeps a gene the message passing never reaches at exactly zero.

5.6.3 — Graph-derived edge features

Each directed edge in the graph carries a small vector of features, and RHIZO computes every one of them from the graph itself. The load-bearing features are an edge's reciprocity, whether the reverse edge from target back to source is also present, and its community role, whether it bridges two communities or sits inside one. These are degree-blind, so an edge does not score

highly merely because its endpoints are hubs with many connections, and they are read off the graph's own wiring with no reference to which genes the two nodes are. This is the zero-ontology principle carried down into the architecture. The model's view of an edge is a function of where that edge sits in the graph and nothing else. The same features are computed identically for every graph RHIZO is asked to read, so when one graph is swapped for another the comparison stays fair, because no graph can carry a feature that another structurally cannot.

5.6.4 — Training objective and the lightweight design

RHIZO is fit on the training perturbations alone, with the held-out validation and test perturbations never seen, the same leakage firewall the rest of the pipeline is built around. Its loss combines a correlation term, the primary objective, which rewards recovering the shape of the response across genes, with a small mean-squared-error term at a tenth of the weight, which anchors the predicted magnitude, and training stops early once the validation score stops improving. The prediction target is the response measured relative to the control mean, so the model learns how far a perturbed cell moves from the unperturbed baseline rather than its absolute state. The whole design is kept small and fast by intent. Because the pipeline's question is which graph is the better input, RHIZO has to be trained once for every graph across many substrates and seeds, and a heavy, high-capacity model would make that comparison unaffordable and would also risk letting the model's own capacity, rather than the graph, account for a result. A light arbiter that trains quickly is what keeps the graph the variable under test.

5.7 — RHIZO: Capabilities, the switchable components

The core message-passing engine of the previous section carries a small set of optional components, each of which can be switched on or off without changing anything else, and each was included because it plausibly helps a model read a curated regulatory graph. A component that plausibly helps is not yet a component that earns its place, so the final experiment of this work treated the arbiter's configuration as the object of study rather than a fixed setting: it doubled the model's capacity, and it removed or added each optional component one at a time, and it scored every variant on regulator-specificity, the de-confounded measure that credits a model only for the response that is specific to the perturbed regulator. All of these tests were run on the fused Super substrate, so the graph was held fixed and only the model changed. The study settles which components are load-bearing and fixes the production configuration; Table 5.2 summarizes what was varied and what each variation showed, and the subsections that follow take the switchable components one at a time.

The reading that survives this study is a plain one. Only the topology-conditioned readout is load-bearing, the over-squash-aware readout is safe to keep but does no measurable work at the depth RHIZO runs, the directed edge features help on a purely causal graph but not on the fused substrate, and doubling the model's capacity does not reward the curated graph at all. The arbiter that goes forward is therefore the small one described in the previous section, with a node dimension of sixty-four, a hidden width of sixty-four, and eight hops, with the topology-conditioned readout and the over-squash-aware readout enabled and the directed edge-feature profile left off. A light model is not a compromise the results forced on us; it is the setting the results endorse.

5.7.1 — Topology-conditioned readout (FiLM)

The topology-conditioned readout lets the model shape each gene's predicted response according to which perturbation is being applied, rather than reading the propagated signal the same way for every perturbation. It is built on feature-wise linear modulation, a general mechanism that conditions a network's computation by scaling and shifting its features with a signal derived from the context ⁹², carried into the graph setting as a modulation of the messages a node reads from its neighbors ⁹³. In RHIZO the conditioning signal is the identity of the

perturbed gene, so the readout is told which regulator was knocked down and can weight the propagated hop-states accordingly. The configuration study confirms this component is load-bearing. Removing it lowers both of the configurations that actually read graph structure by about 0.03 on regulator-specificity, a drop that is significant and that appears wherever the model is being asked to extract a graph-specific answer, which is the signature of a component that is doing real work rather than decorating the architecture. The readout is therefore kept on in the production model.

5.7.2 — Over-squash-aware readout

Over-squashing is a known failure of message-passing networks in which signal travelling a long path is forced through a graph bottleneck and compressed into a fixed-size vector, so that information from distant nodes arrives degraded and a model cannot fit a prediction that depends on long-range structure⁹⁴. The severity of the effect grows with depth and with the topology of the bottlenecks the signal must cross⁷⁸, which makes it a real concern for a dense regulatory graph in principle. RHIZO carries an over-squash-aware readout that is designed to preserve this long-range signal, and the configuration study put it to the test by removing it. The result is the one place in the whole study where a change moves nothing: with the component removed the model's regulator-specificity is statistically indistinguishable from the production configuration, the single variation with no significant effect. The explanation is that RHIZO runs shallow, at eight hops, and over-squashing bites only when a model is deep enough for the bottleneck to compound, so at this depth the phenomenon simply does not arise. The component is retained in the production model because it is harmless and cheap insurance for deeper graphs, but it is reported honestly as insurance rather than as a lever that improved the result.

5.7.3 — Directed regulatory edge features (what a co-expression graph lacks)

The directed edge features encode the one property that distinguishes a regulatory edge from a mere association, its orientation from a source regulator to the target it controls, together with the directed-motif roles that orientation makes possible, such as an edge's participation in a feed-forward loop¹⁸. This is exactly the signal a co-expression graph structurally cannot carry, because a correlation is symmetric and has no direction, so directed edge features are a capability a causal graph has and a co-expression graph does not, read on top of the directed propagation the architecture already performs⁷⁷. The configuration study gives this component a genuinely two-sided verdict, and reporting both sides is what keeps the claim credible. On a purely causal graph at a smaller scale the directed features earned a real and significant gain of roughly 0.045 on regulator-specificity, which is the result one would expect if directionality is informative. On the fused Super substrate, however, turning them on lowers the graph-reading configurations by about 0.02, so the clean profile with the directed features switched off is the correct default there. The reconciliation is that the fused substrate already carries co-expression edges that supply breadth the causal core lacks, and once that breadth is present the extra directional features add noise faster than signal, whereas on a graph that is causal end to end the same features pay their way. Directionality is a real asset of a causal graph; it is simply an asset the fused substrate no longer needs to lean on.

The capacity result closes the section by answering the one objection the rest of the chapter leaves open. The parity between pruning methods established for the downstream models could, in principle, have been an artifact of an arbiter too small to read a curated graph's hub structure, so that a larger model would finally reward FUNGI's topology with better prediction. The configuration study tested exactly this by doubling the arbiter, and the data moves the other way: the doubled model lowers the curated-graph configuration by about 0.06 while raising the naive top-weight prune by about 0.06, so extra capacity does not help the model extract more from a topology-balanced graph, it helps the model exploit the rawer, higher-confidence direct edges that a greedy prune preserves. A bigger model rewards the greedy pruner, not the curated one. This is why RHIZO is held deliberately small, and it is why the parity result of the next

chapter is a property of the graphs rather than a limit of the model. The full ranking of pruning methods across substrates and regimes belongs to the integrated evaluation; the point that belongs here is narrower and about RHIZO itself, that its light design is a tested choice which more capacity was given every opportunity to overturn and did not.

Table 5.2 sets out the configuration study behind RHIZO's final settings.

Table 5.2 | The RHIZO configuration study. Each variation changes exactly one component of the arbiter, holding the fused super-graph fixed, and is scored on regulator-specificity relative to the production configuration. Only the topology-conditioned readout is load-bearing; the model is kept deliberately small.

Component varied	Change tested	What the study found
Topology-conditioned readout (FiLM)	Removed	Load-bearing. Both graph-reading configurations fall by about 0.03, so the conditioning is kept on.
Over-squash-aware readout	Removed	Not load-bearing at eight hops. The one change with no significant effect (about 0.00); kept as cheap insurance for deeper graphs.
Directed regulatory edge features	Enabled	Harmful on the fused super-graph (about 0.02 lower), so the clean default is correct here; the same features helped by about 0.045 on a purely causal graph.
Model capacity	Doubled	No help to the curated graph. It falls by about 0.06 while the greedy top-weight prune rises by about 0.06.

Effects are per-perturbation shifts in regulator-specificity and are statistically significant unless noted.

5.8 — Evaluation regimes: in-distribution and zero-shot

MOREL and RHIZO are each scored under two regimes, and the distinction between them governs how every result in the next chapter has to be read. One regime measures how well a model reads a graph for a gene the graph was built to describe, and the other measures whether a model generalizes to a gene the graph has never seen perturbed.

5.8.1 — rung1: in-distribution graph quality

The in-distribution regime scores a model on a training perturbation, splitting that perturbation's cells into two halves and asking the model, fit on one half, to predict the other. The queried gene is one the graph was built from, so it has outgoing edges and the propagation has somewhere to go. This regime measures how faithfully a graph encodes a response it was allowed to learn, which is a real and useful measure of graph quality, but it is not a test of generalization. The graph already contains the answer, so a strong in-distribution score must never be reported as held-out prediction. Keeping that caveat attached to every in-distribution number is what stops the evaluation from overstating what a graph can do.

5.8.2 — zeroshot: the held-out coverage boundary

The zero-shot regime is the true generalization test. It scores a model on a held-out perturbation whose gene is disjoint from every training perturbation, so only the gene's identity crosses the split and its cells and responses are never seen during training. This regime exposes a hard structural boundary of a purely causal graph. Because a causal graph carries outgoing edges only from genes that were actually perturbed, a held-out gene has no outgoing edges, propagation from it reaches nothing, and the prediction is zero across the genome. RHIZO's $\psi(0) = 0$

invariant makes this exact, and MOREL's empty-path behavior makes the same point in the linear setting. This is not a failure of either model. It is the coverage boundary that motivates fusing a causal graph with a co-expression graph, which supplies outgoing edges for the genes a perturbation-derived graph is blind to, and it is why the zero-shot regime is the decisive test of whether a curated graph generalizes.

Figure 5.6 contrasts the two regimes and the coverage boundary between a causal and a co-expression graph.

MOREL / RHIZO, illustrative graphs · In-distribution vs zero-shot, and the coverage boundary that motivates fusion | seed=42, conceptual, not real data

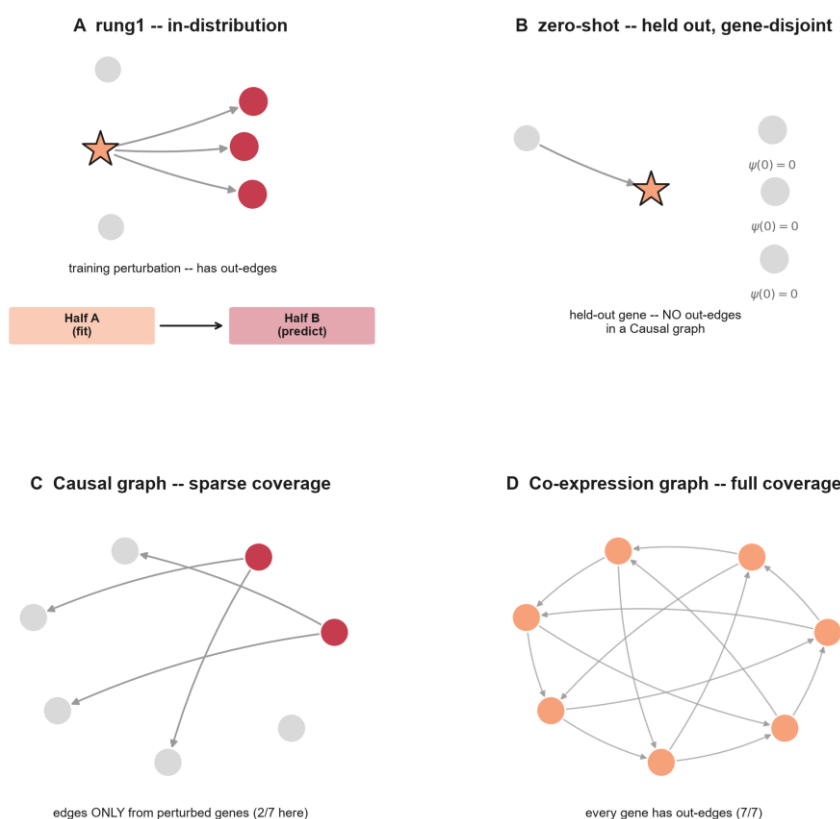


Figure 5.6 | The two evaluation regimes. In the in-distribution regime the queried gene was part of the training graph and has outgoing edges, so the model predicts, though the graph already encodes the answer. In the zero-shot regime the queried gene is held out and gene-disjoint, and a causal graph has no outgoing edges from it, so its prediction is zero. A causal graph covers only perturbed genes while a co-expression graph covers all, the coverage boundary that motivates fusing them. Illustrative.

5.8.3 — What the models are scored on

Both models are scored on metrics that remove the shared response before comparing prediction to truth, so that a model is credited only for the regulator-specific signal and the shared-mean baseline scores zero by construction ³⁶. The scoreboard combines this de-confounded correlation with the interventional, perturbation-response measures the causal-inference benchmarks use ⁹. The full metric machinery, its controls, and its significance testing belong to the integrated evaluation and are developed there rather than here.

5.9 — From predictive models to verdict

MOREL and RHIZO are two instruments built for one purpose, to turn a curated graph into a held-out prediction so that the quality of the graph can be measured rather than asserted. They are deliberately unlike, one a linear propagator on a CPU and one a neural network on a GPU,

and they are deliberately alike in the one respect that matters, that neither imports anything external, so a difference in their predictions is a difference in the graph. That is what makes either one a clean test and both together a strong one. The instruments are built; the question they exist to answer is whether a FUNGI-curated graph lets them predict held-out responses better than the structure-blind baselines, across substrates, pruning methods, coverage regimes, and cell lines. The integrated evaluation of the next chapter puts them to that test.

CHAPTER 6

Anastomosis

Results & Discussion

In a mycelium, anastomosis is the fusion of two separate hyphal branches into one continuous network. Here the branches of the whole pipeline fuse together, as two cell lines, three substrates, two independent predictors, and every experiment of the project converge on a single verdict.

Separate threads, grown apart, reach across and join into one body.

6.0 — Overview: From a Constructed Substrate to a Downstream Verdict

By the end of Chapter 5 the reader knows what we built: a pipeline that takes raw Perturb-seq counts, aggregates them into a leakage-safe substrate (SPORE), infers a causal gene-regulatory network from the perturbation responses (SHROOM) and a co-expression network from the same panel (HYPHAE), fuses the two into the Super graph, prunes each to a working topology (FUNGI), and reads the resulting graph with a directed-FAGCN arbiter (RHIZO) to predict how the transcriptome responds to a perturbation. The question this chapter answers is the only one that matters for a thesis: so what. Does the substrate we hand the predictor, its biological composition and its pruned topology, actually change what it can predict, and is the fused Super graph, pruned by FUNGI, the right answer.

The verdict, previewed here so the construction story that follows has a destination, is yes on the part that counts. At matched edge count and in-distribution, a causal-containing graph encodes regulator-specific signal that pure co-expression categorically lacks, the fused Super substrate is the best-composed of the graphs we tested, and the result replicates across edge density, across two very different cell lines, and on a second evaluator that shares none of the arbiter's machinery. FUNGI's contribution is more specific than "it predicts better," and this chapter is careful to state it as what the data actually supports: FUNGI produces a biologically legible graph, a real hub hierarchy in literature-faithful bands, at no cost to prediction, and the advantage it and the Super graph carry is a property of the wiring, not of degree, coverage, or fine edge weights. Where the advantage ends, at held-out perturbations, and along the coverage axis, is characterized rather than hidden, because a de-confounded evaluation that catches its own confounds is what makes the positive claims credible.

The chapter is deliberately built rather than dumped. It first sets up the two datasets that give every result two points of view (section 6.1) and the evaluation lens that makes any result interpretable (section 6.2); it then constructs and validates the substrate, culminating in the Super graph (section 6.3); lands the headline result (section 6.4); proves the headline is real regulatory structure (section 6.5); gives FUNGI its due on the axis it actually wins (section 6.6); shows the result generalizes (section 6.7); attributes the signal to the graph (section 6.8); bounds the claims honestly (section 6.9); closes the pruning question with a capacity and architecture study (section 6.10); and consolidates the case (section 6.11).

6.1 — The Two Testbeds: Dense versus Sparse Perturbation Coverage

Every result in this chapter is read through two contrasting datasets, chosen so that a method which works on both is demonstrably robust rather than tuned to one screen.

The first is the genome-scale RPE1 Perturb-seq screen of Replogle et al. ²¹, a near-saturating CRISPRi perturbation of an immortalized retinal pigment epithelial line, our **dense-coverage** viewpoint. After the substrate-recovery steps described in Chapter 2, the RPE1 causal graph draws out-edges from **866** perturbed regulators, a source-gene coverage of **17.3 %** of the 5,000-gene panel. The second is the h1-hESC dataset released with the Virtual Cell Challenge ⁵⁵, a human embryonic stem-cell screen with an order of magnitude fewer training perturbations, our **sparse-coverage** viewpoint, with only **88** perturbed regulators as out-edge sources (**1.8 %** of the panel). **Figure 6.1** renders the contrast directly: each panel is the 5,000-gene panel drawn as a track, with every gene that is a perturbation source marked; RPE1's track is densely ticked, h1-hESC's is sparse, a roughly tenfold difference in the coverage that a causal graph has to work with.

The two graphs are built on the same 5,000 highly-variable-gene panel, differing only in a per-line protected-gene list, so that any cross-line difference in the results is attributable to biology

and coverage rather than to a change in panel size. This is the single design choice that makes the two-dataset comparison interpretable: dense RPE1 is the regime where a causal graph has the most out-edges to act on, sparse h1-hESC is the stress test, and the strongest results in this chapter are the ones that hold on both.

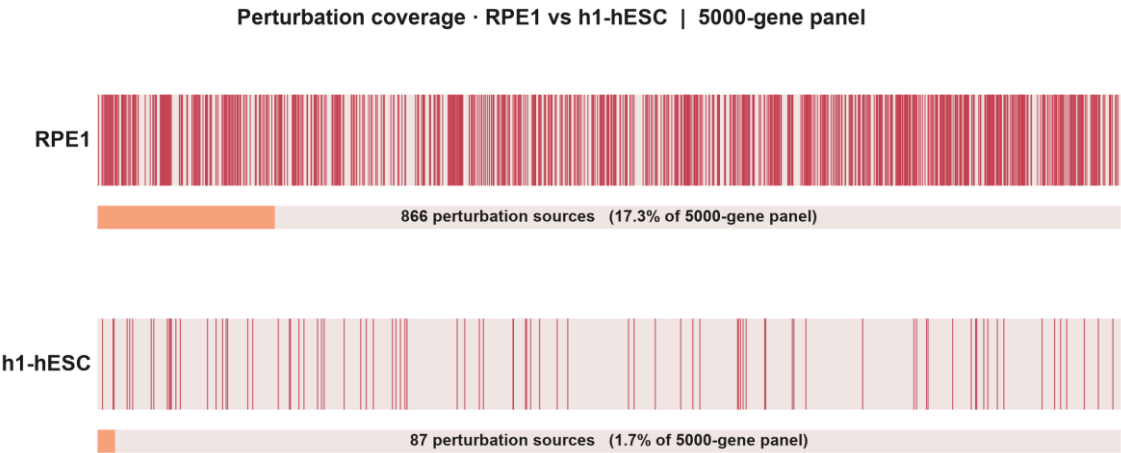


Figure 6.1 | Perturbation coverage in the two datasets. Each cell line's 5,000-gene panel is drawn as a track, with every gene that is a perturbation source marked by a tick. RPE1 is densely covered, with 866 source genes (17.3 percent of the panel), while h1-hESC is sparse, with 88 sources (1.8 percent), a roughly tenfold contrast in the coverage a causal graph has to work with.

Table 6.1 sets the two testbeds side by side.

Table 6.1 | The two datasets. Both graphs use the same 5,000-gene panel, so any cross-line difference is biology and coverage rather than a panel artifact.

Property	RPE1	h1-hESC
Role	Dense-coverage viewpoint	Sparse-coverage viewpoint
Perturbation sources (out-edge)	866	88
Out-edge source coverage of panel	17.3%	1.8%
GRN panel	5,000 highly-variable genes	5,000 highly-variable genes

Coverage is the fraction of the 5,000-gene panel that can emit a causal out-edge. RPE1 is from Replogle et al. (2022); h1-hESC is from the Virtual Cell Challenge.

6.2 — The Evaluation Lens: Why Regulator-Specificity Is the Decision Axis

Before any result, two things need to be fixed: which metrics decide, and why one of them is not enough on its own. This section introduces the metric vocabulary the rest of the chapter uses, then makes the empirical case for the de-confounded decision axis.

6.2.1 — The metrics, defined

A perturbation predictor is scored by comparing a predicted log-fold-change response vector $\hat{\delta}_p$ against the measured response δ_p for each held-out perturbation p , over the panel genes. Several complementary quantities are used throughout this chapter, and because most are introduced here for the first time, each is defined before it appears in a figure.

Regulator-specificity (`pearson_systema``), the decision axis. Every perturbation in a screen elicits a large shared response, a systematic axis, dominated in these data by cell-cycle arrest, that any predictor captures simply by predicting "a strong perturbation happened." A raw correlation therefore rewards a method for getting that shared axis right even when it has no

idea which *specific* genes a given regulator controls. Regulator-specificity removes this confound by projecting the shared mean-perturbation response $\bar{\delta}$ out of both vectors before correlating:

$$\text{pearson_systema}(p) = \text{corr}(\hat{\delta}_p - \bar{\delta}, \delta_p - \bar{\delta}).$$

Higher is better, and the sign is diagnostic: a positive value means the graph resolves perturbation-specific regulatory structure, a value at or below zero means it does not. This is the axis on which every headline comparison in the chapter is decided.

Residual-Specific Correlation (RSC). The arbiter's own working metric, and the metric of the bypass-free evaluator in section 6.6, is a closely related quantity, the Pearson correlation of predicted and true responses over the train-defined signal genes S , excluding the perturbed gene, Fisher-averaged across perturbations:

$$RSC = \text{operatorname{Fisher-mean}}_p \text{corr}(\hat{\delta}_p[S], \delta_p[S]).$$

RSC is a raw correlation: it captures any of the response, including the shared systematic axis, so it rises whenever a graph can act on more genes. It is reported as a companion, and, as section 6.9 shows, the divergence between RSC and regulator-specificity is itself one of the chapter's results.

Direction agreement ('cosine_delta'). The cosine similarity between the predicted and true response vectors, $\cos(\hat{\delta}_p, \delta_p)$; like RSC it is dominated by the systematic axis and is read with regulator-specificity, never alone.

****Differentially-expressed-gene recovery (DEG-AUPRC and F_1).** *A predictor should also identify which* genes respond, framed as retrieval of the true top-responding gene set. For a rank threshold k , precision and recall of the predicted responders against the true set give an F_1 score, $F_1@k = 2PR/(P + R)$; sweeping k and integrating gives the area under the precision-recall curve (DEG-AUPRC). Higher is better; both are the field-standard "did you find the right responding genes" complement to the correlation axes⁹⁵.*

Prediction error (RMSE / weighted MSE). The pseudobulk error between prediction and truth, $\text{RMSE} = \sqrt{1/n \sum_g (\hat{\delta}_g - \delta_g)^2}$; the weighted variant concentrates the error on the signal genes so that agreement on the many unresponsive genes does not dominate. Lower is better, and it is carried as a sanity companion, not a decision axis.

Graph-intrinsic specificity ('spec_prec', 'stat_prec', and the contamination gap). section 6.3 evaluates the graphs *directly*, before any prediction, by asking whether a graph's own top- K edges connect regulators to genes that genuinely respond to perturbing them. Holding out test cells, 'stat_prec' measures the fraction of a graph's top- K regulator→target edges whose target shifts significantly when the regulator is knocked down relative to control; 'spec_prec' measures the same against the *global perturbed* centroid, so that only regulator-specific residual counts. Their difference,

$$\text{sysvar_gap} = \text{stat_prec} - \text{spec_prec},$$

is the share of a graph's apparent precision that is actually the shared systematic response, a contamination measure, higher meaning more contaminated.

Topology metrics (out-degree Gini, reciprocity, assortativity). section 6.6 measures the *shape* of a graph. The out-degree Gini coefficient, $G \in [0,1]$, is the inequality of the out-degree distribution: $G = 0$ is a perfectly uniform out-degree (every gene regulates the same number of targets, no hubs), and higher values indicate a hub hierarchy where a few master regulators dominate, as real regulatory networks do. Reciprocity is the fraction of edges whose reverse also exists (regulatory networks are directional, so low is realistic), and out-in degree assortativity is the tendency of high-out-degree hubs to point at low-in-degree targets (mildly negative in real networks). These three are the axes on which a graph is "biologically legible."

6.2.2 — Why de-confounding is not optional: the CHITIN evidence

The obvious objection to a de-confounded metric is that one could instead *remove* the systematic variation from the data by preprocessing, and then score on a raw metric. We tested this exhaustively under the CHITIN program (Chapter 3), and it does not work. **Figure 6.2.3** places every correction arm by how much systematic variation it removes against how much causal precision it retains: the quadrant that both cleans and preserves is empty. Blind subtraction methods clean less than 5 %; a latent-variable method makes contamination worse; and the one arm that genuinely cleans a meaningful fraction, regressing out the cell-cycle-arrest program, does so at a real cost to causal precision (–11 % specificity), because the systematic variation it removes *is* genuine regulatory biology, not a removable artifact. The conclusion is the one that governs the whole chapter: you cannot preprocess the confound away without removing signal, so you must measure around it. Regulator-specificity is that measurement.

CHITIN correction · Clean vs preserve | K = 5k

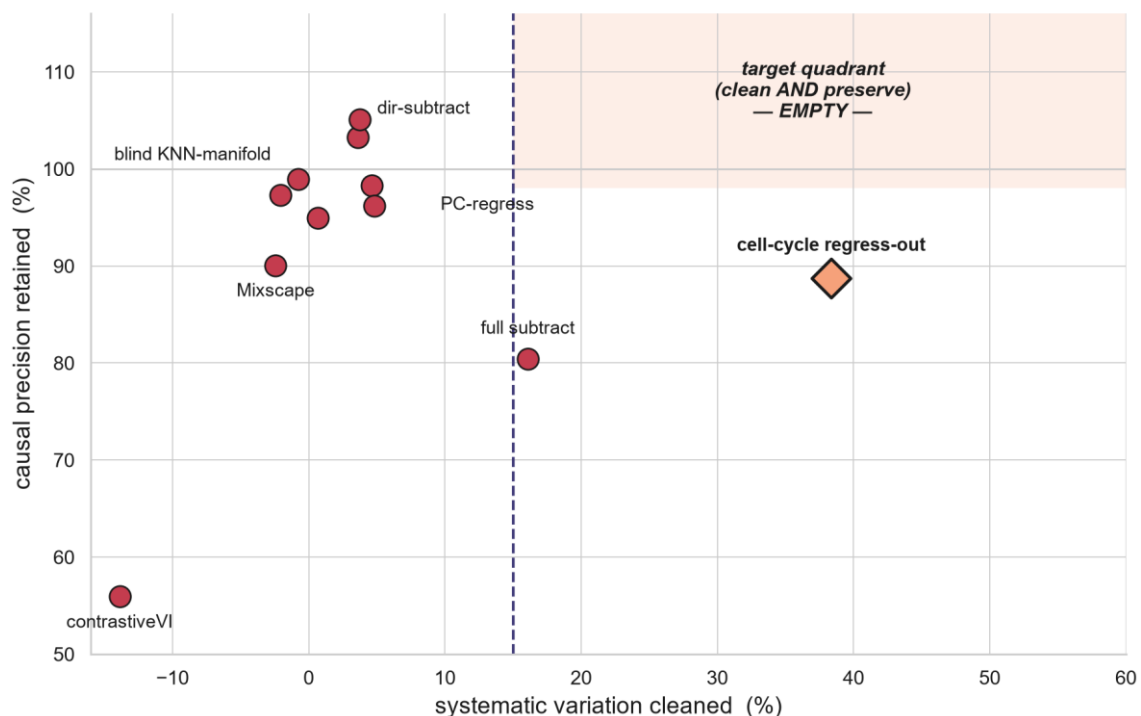


Figure 6.2.3 | The clean-versus-preserve wall for systematic-variation correction. Every correction arm is placed by how much systematic cell-cycle-arrest variation it removes against how much causal precision it retains, relative to the no-correction reference. The quadrant that both cleans and preserves is empty. Blind subtraction removes almost nothing, a latent-variable method makes contamination worse, and the only arm that genuinely cleans does so by removing real regulatory biology at a cost to precision. This is why the systematic response is de-confounded at evaluation rather than removed in preprocessing.

6.3 — Building the Substrate: From Single Cells to the Super Graph

Before the downstream test, this section establishes that each substrate is a deliberately engineered, validated object, and that the graphs are already good on an intrinsic, model-free evaluation, culminating in the Super graph, the artifact the rest of the chapter evaluates.

6.3.1 — The generators, and the fusion

SHROOM infers the **causal** substrate by self-training a classifier on the perturbation responses⁴⁵; because a gene's out-edges can only be learned by observing the transcriptome's response to perturbing it, the causal graph draws out-edges only from the perturbed regulators, and this is

the origin of both its high specificity and its coverage ceiling. HYPHAE infers the **co-expression** substrate from the same panel by tree-based regression^{14,38}; it draws out-edges from all panel genes, giving broad coverage, but on its own it carries no directed regulatory specificity, as section 6.4 shows, pure co-expression is negative on the decision axis. The two are complementary in exactly the way that motivates fusing them: the causal core supplies specificity, the co-expression scaffold supplies coverage. **Figure 6.3.4 (diagram)** shows the composition schematically, a handful of high-out-degree causal hubs unioned with the broad co-expression mesh, Borda-fused into the **Super** graph that carries both. The Super graph is not an incidental ensemble; it is the architecture designed to resolve the specificity-versus-coverage tension, and it was selected against alternative construction variants before adoption (Chapter 5).

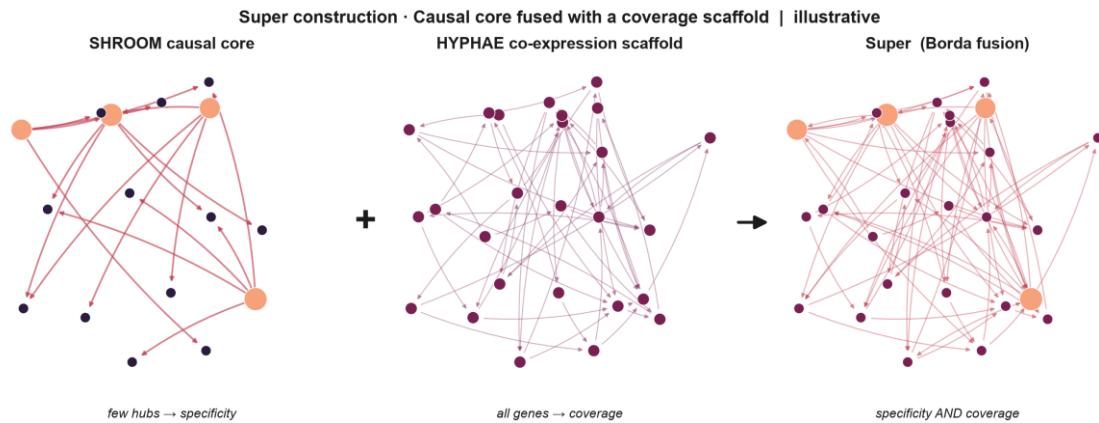


Figure 6.3.4 (diagram) | The Super graph as a fusion of causal core and co-expression scaffold. A handful of high-out-degree causal hubs, which supply regulator-specificity, are unioned with the broad co-expression mesh, which supplies coverage, and the two are rank-fused into the Super graph that carries both. The subgraph shown is illustrative.

6.3.2 — The graphs are good intrinsically, on both datasets

The substrate ranking can be seen before any perturbation prediction is run, by evaluating the graphs' own top- K edges directly. **Figure 6.3.5(a)** sweeps the edge budget K from one thousand to two hundred thousand and plots regulator-specific precision for the three substrates on both cell lines. The result reproduces the downstream headline intrinsically: causal and Super graphs carry regulator-specificity that co-expression categorically lacks, a factor of roughly 7 to 34 times at every K , on both datasets, while co-expression sits near the zero floor ('spec_prec' of order 0.02–0.04 throughout). At $K = 5,000$ on RPE1 the values are 0.586 for the causal core, 0.321 for Super, and 0.036 for co-expression; on h1-hESC the separation is even starker (0.861, 0.494, 0.025). The intrinsic ranking is 'Causal $\geq$ Super $\gg$ Co-exp': the *pure* causal core is the most regulator-specific graph, Super second because it deliberately dilutes that core with co-expression coverage.

That Super ranks second here but wins downstream (section 6.4) is not a contradiction, it is the composition principle working exactly as designed. Intrinsic specificity is maximized by the pure causal core, but the downstream task rewards specificity *and* the coverage the co-expression scaffold contributes, which only the Super graph carries. Intrinsic and downstream evaluations agree on the load-bearing fact, causal composition beats co-expression by roughly an order of magnitude, and differ on Causal-versus-Super precisely along the axis the fusion was built to exploit. **Figure 6.3.5(b)** makes the same point through contamination: the Super graph keeps its systematic-variation gap bounded below the pure causal core across the whole edge budget, staying out of the contaminated band, while co-expression sits at a gap of near zero for the trivial reason that it has no specific signal to contaminate. A methodological note carried from this analysis: the causal-database enrichment metric commonly used to score GRNs does *not* separate these substrates and in fact inverts on co-expression, because a co-expression graph's

densest edges overlap curated databases by construction; regulator-specificity is the correct intrinsic readout, and enrichment must not be read as substrate quality (a confound documented in Chapter 4).

Figure 6.3.4 (plot) consolidates the construction argument across every seed, both datasets, and all three densities on the two axes that define it, out-edge coverage and regulator-specificity. The causal cores sit low-coverage and high-specificity, the co-expression scaffolds high-coverage and low-specificity, and the Super fusions occupy the high-coverage, high-specificity corner. The corner-binding holds as everything is pooled: fusion buys both axes at once.

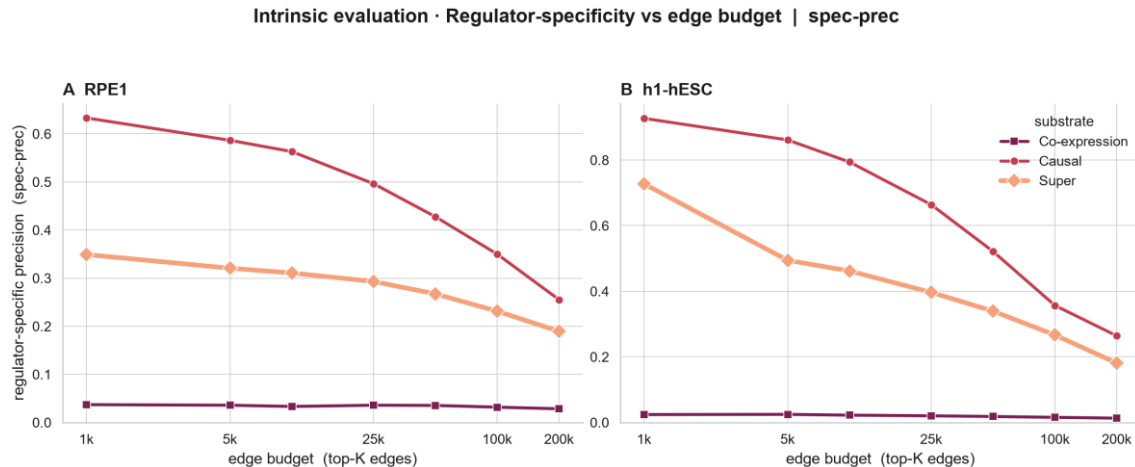


Figure 6.3.5(a) | Intrinsic regulator-specificity against edge budget. Regulator-specific precision of the three substrates as the top-K edge budget grows, on both cell lines. The causal and Super graphs carry regulator-specificity that co-expression categorically lacks at every budget and on both datasets, while co-expression sits near the zero floor.

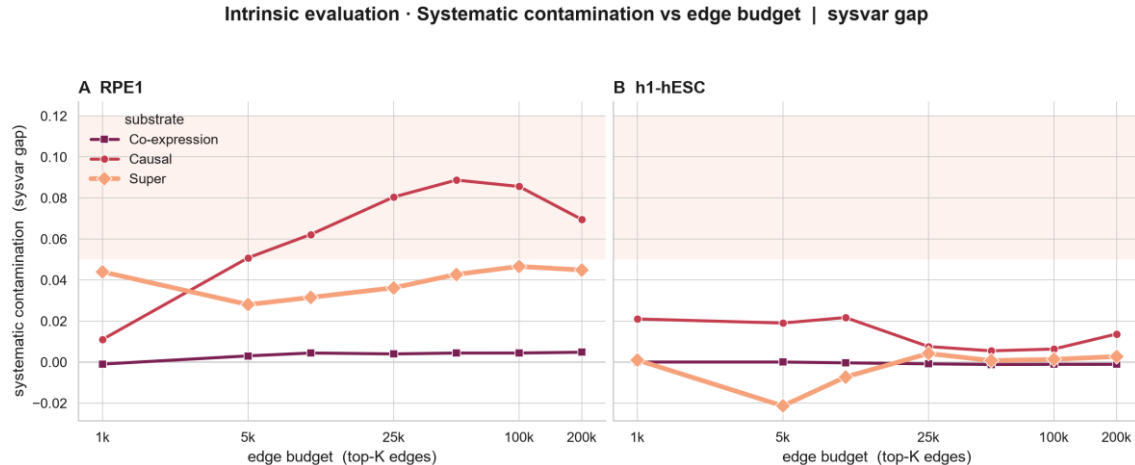


Figure 6.3.5(b) | Systematic contamination against edge budget. The share of a graph's apparent precision that is actually the shared systematic response, plotted against the edge budget. The Super graph keeps its contamination bounded across the whole budget, below the pure causal core, while co-expression sits near zero because it has no specific signal to contaminate.

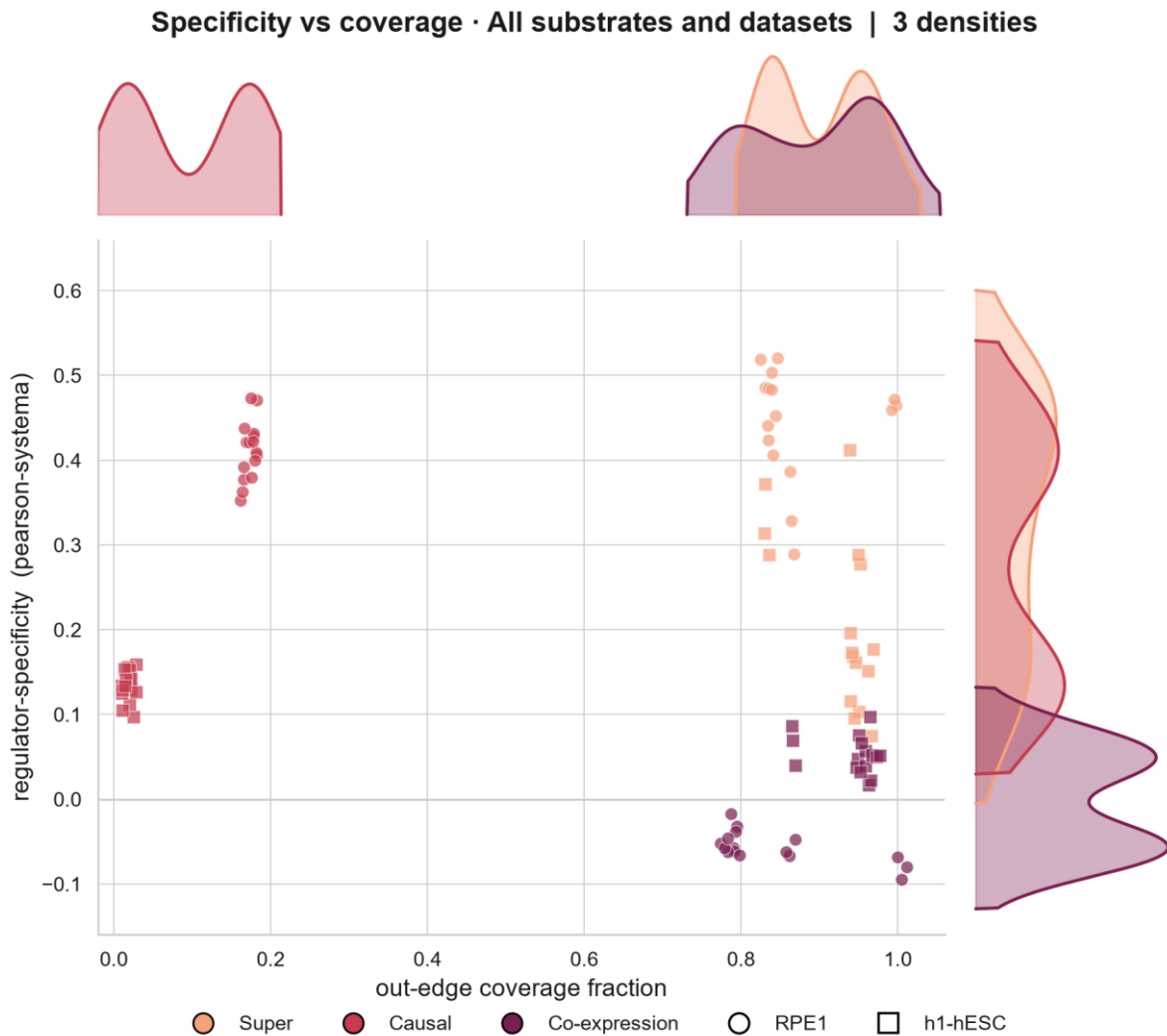


Figure 6.3.4 (plot) | The specificity-and-coverage plane, pooled across all data. Every seed of all substrates, both datasets, and all three edge densities placed on out-edge coverage against regulator-specificity. The causal cores stay low-coverage and high-specificity, the co-expression scaffolds high-coverage and low-specificity, and the Super fusions occupy the high-coverage, high-specificity corner. Fusion buys both axes at once.

6.4 — The Crown Jewel: Substrate Biology Decides Perturbation-Specificity

This is the headline result of the thesis. At strictly matched edge count and in-distribution, a causal-containing graph encodes regulator-specific signal that pure co-expression does not, on both datasets, and the fused Super substrate is the best-composed of the graphs tested.

Figure 6.4.1(a) is the primary evidence: the full per-perturbation distribution of regulator-specificity for all nine arms, the three pruners (FUNGI, Top-Weight, KNN) crossed with the three substrates, faceted by substrate, with RPE1 above ($n = 878$ perturbations) and h1-hESC below ($n = 88$). The separation is categorical and it is on the de-confounded axis built specifically to strip the co-expression precision confound. On RPE1, every causal and Super arm sits well positive, with mean regulator-specificity between $+0.42$ and $+0.54$, while every pure co-expression arm has its mass straddling or below zero (-0.05 to -0.09). The weakest causal arm beats the strongest co-expression arm by roughly 0.47 in absolute terms. The pattern is not carried by a single metric: across the full suite the causal and Super arms simultaneously post the lowest prediction error and the highest DEG-recovery, and co-expression sits at or below the trivial baseline on every axis (Table 6.4.1a). It reproduces an earlier pilot almost number-for-

number, a cross-campaign reproduction that certifies the result as signal rather than run-variance.

Figure 6.4.1(b) shows the same result as a heatmap over the three-by-three arm grid, and the at-a-glance reading is the whole argument: the co-expression column is cold on both datasets, the causal and Super columns are warm. Substrate biology, not the choice of pruner, decides. Among the arms, the fused Super graph is the best-rounded, the two highest direction-agreement values in the cohort belong to the Super arms, and FUNGI Super is top-tier on every axis while also carrying the topology argument of section 6.6 that none of the others can. **Figure 6.4.1(c)** confirms the substrate separation is stable across edge density: plotting every arm and seed at all three densities on the two decision axes at once, the three substrates separate cleanly and stay separated, Super in the top-right, co-expression in the bottom-left.

One caveat governs every sentence in this section, and it is stated once here: this is the *in-distribution* regime, where the queried gene has out-edges in the graph, so the result is a claim about what causal GRN structure can *represent*, not about generalization to unseen perturbations. Generalization is the subject of section 6.9, and it is an honest boundary rather than a hidden one.

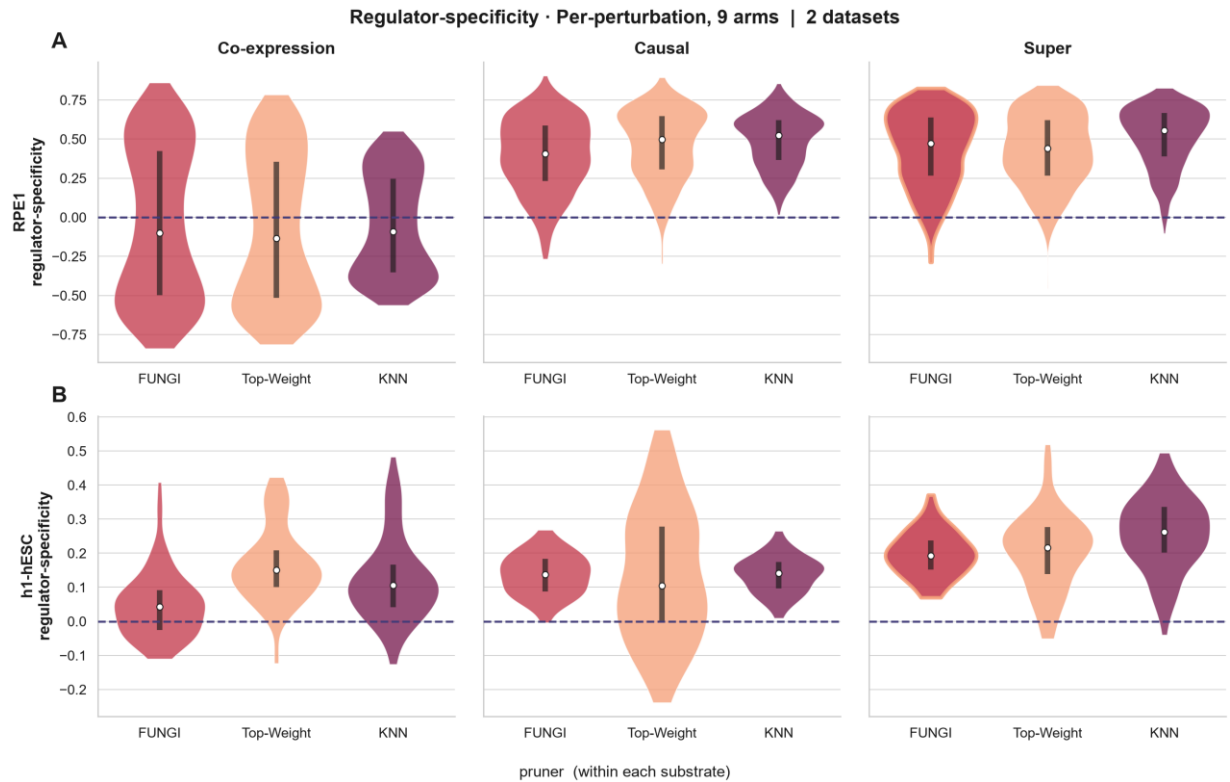


Figure 6.4.1(a) | Per-perturbation sign separation on the decision axis. The full per-perturbation distribution of regulator-specificity for all nine arms, the three pruners crossed with the three substrates, faceted by substrate, with RPE1 above (878 perturbations) and h1-hESC below (88). On RPE1 every causal and Super arm sits well positive while every co-expression arm straddles or falls below zero, and the pattern replicates, compressed, on the sparse dataset.

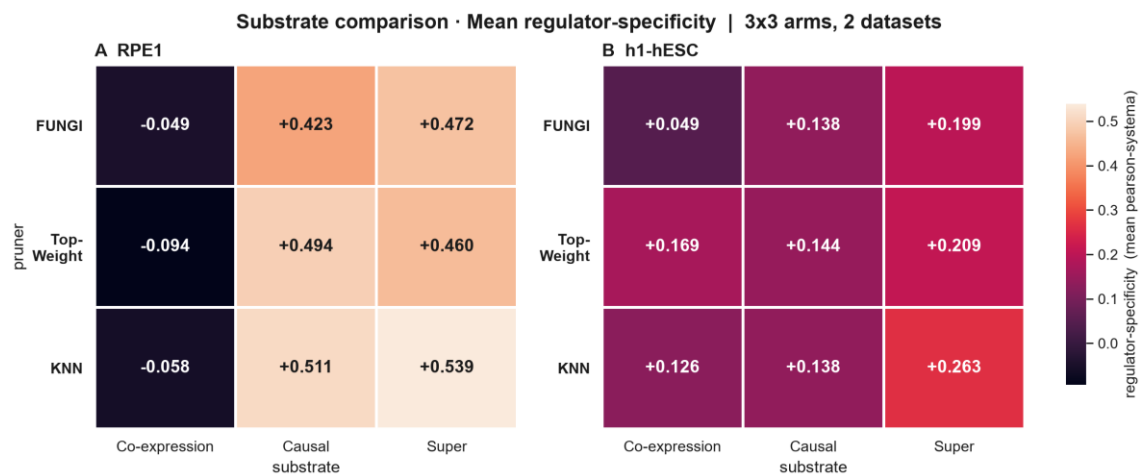


Figure 6.4.1(b) | The three-by-three arm grid on the decision axis. Mean regulator-specificity across the three pruners and three substrates on both datasets. The co-expression column reads cold and the causal and Super columns read warm, so substrate biology, not the choice of pruner, decides.

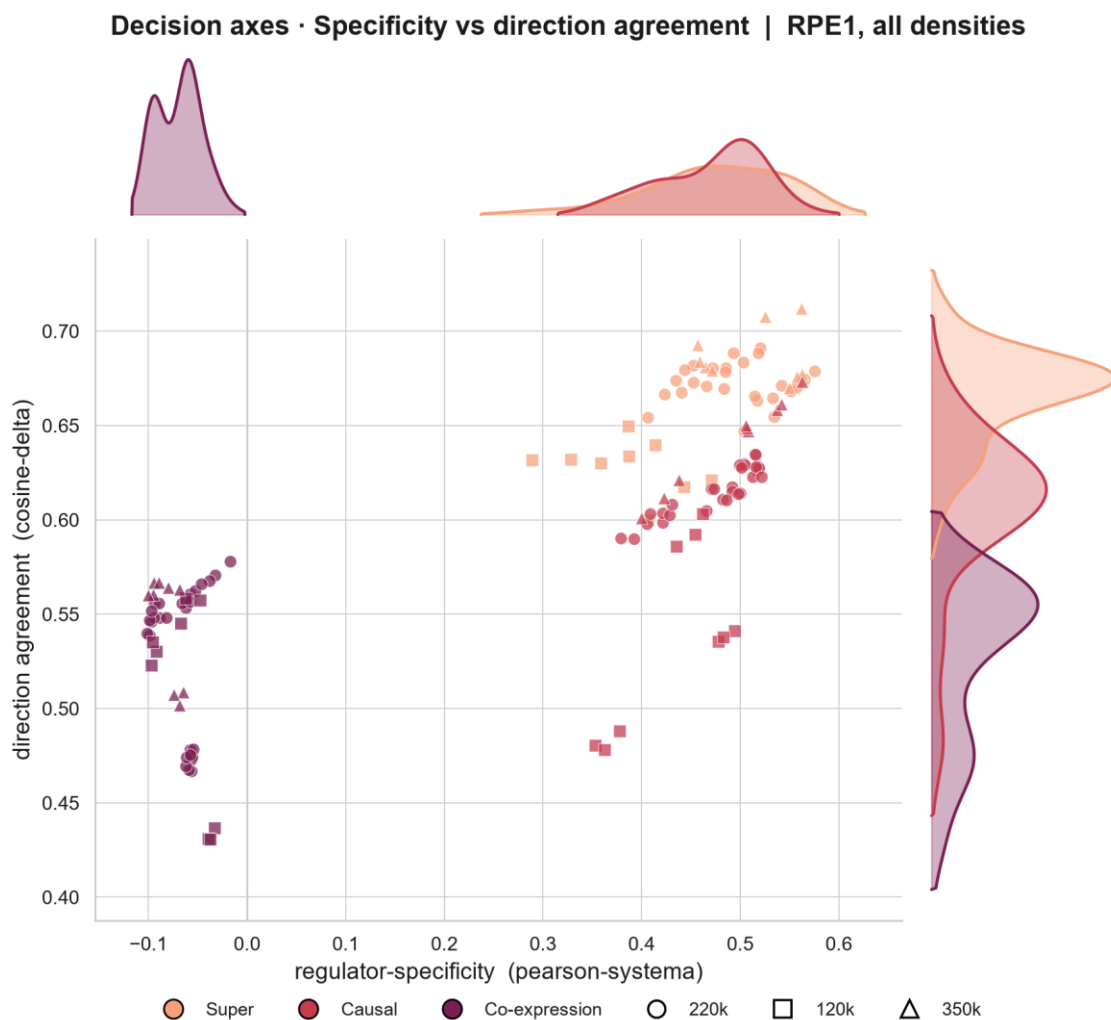


Figure 6.4.1(c) | Substrate separation across edge density. All nine arms, every seed, at all three densities, plotted on regulator-specificity against direction agreement. The three substrates separate cleanly and stay separated, with Super in the top-right corner and co-expression in the bottom-left.

Table 6.2 | Full metric suite on RPE1, in-distribution, sorted by the decision axis. Every causal and Super arm is positive on regulator-specificity while every co-expression arm is negative.

Arm	Reg-specificity	DEG-AUPRC	Cosine	RSC	RMSE
KNN Super	+0.539	0.516	0.666	0.710	0.079
KNN Causal	+0.511	0.501	0.629	0.696	0.082
Top-Weight Causal	+0.494	0.450	0.615	0.626	0.081
FUNGI Super	+0.472	0.514	0.675	0.686	0.080
Top-Weight Super	+0.460	0.509	0.679	0.682	0.079
FUNGI Causal	+0.423	0.470	0.603	0.634	0.088
Linear baseline	+0.198	0.445	0.593	0.587	0.086
FUNGI Co-expression	-0.049	0.442	0.563	0.434	0.108
KNN Co-expression	-0.058	0.430	0.473	0.517	0.112
Top-Weight Co-expression	-0.094	0.437	0.548	0.573	0.112

RMSE is lower-is-better; all other columns are higher-is-better. The perturbation-mean baseline is omitted, as its regulator-specificity is a structural minus one rather than a score. Ten seeds.

Table 6.3 | Full metric suite on h1-hESC, in-distribution. The direction and ordering replicate at the sparse dataset's roughly tenfold lower coverage, with the magnitude compressed.

Arm	Reg-specificity	DEG-AUPRC	Cosine	RSC
KNN Super	+0.263	0.308	0.472	0.364
Top-Weight Super	+0.209	0.305	0.474	0.328
Linear baseline	+0.204	0.287	0.484	0.319
FUNGI Super	+0.199	0.306	0.505	0.330
Top-Weight Co-expression	+0.169	0.294	0.472	0.332
Top-Weight Causal	+0.144	0.164	0.414	0.170
FUNGI Causal	+0.138	0.210	0.444	0.217
KNN Causal	+0.138	0.222	0.440	0.222
FUNGI Co-expression	+0.049	0.272	0.356	0.247

Ten seeds.

6.5 — Why the Super Win Is Real: The Mechanism Controls

A skeptic's first move against section 6.4 is that the advantage might be an artifact of degree, coverage, or edge weights rather than genuine regulatory structure. The mechanism controls close that door directly, by surgically destroying one property of the graph at a time and measuring the collapse.

Figure 6.5 shows the intact causal and Super graphs against three controls, on both datasets, with the co-expression floor drawn as a threshold. A degree-preserving **scramble**, which keeps each gene's out-degree but randomizes which genes it points to, collapses regulator-specificity by 60 to 75 %, so the degree sequence alone carries almost none of the signal; it is the specific choice of which regulator controls which target that matters. **Reversing** every edge direction

collapses it further, to the co-expression floor or below (on RPE1, Super under direction-reversal falls slightly below zero), which is exactly what a directed-causal reading predicts and a co-expression reading cannot explain. **Binarizing** the edge weights, by contrast, is tolerated, the intact advantage is retained to within a few percent, and on h1-hESC the weight-stripped Super graph is statistically indistinguishable from the intact one. The reader uses the *topology*, not the fine edge weights. The controls behave identically on the sparse dataset, so the mechanism is cross-line robust. Every contrast is significant at $q \approx 0$ over the roughly 878 per-perturbation values, so the verdicts are read on effect size rather than on p-values, and the tiny weight-strip differences on RPE1 are statistically distinguishable but biologically negligible. The advantage is the specific directed regulatory wiring, carried by topology, which is precisely the property the next section shows FUNGI renders faithfully.

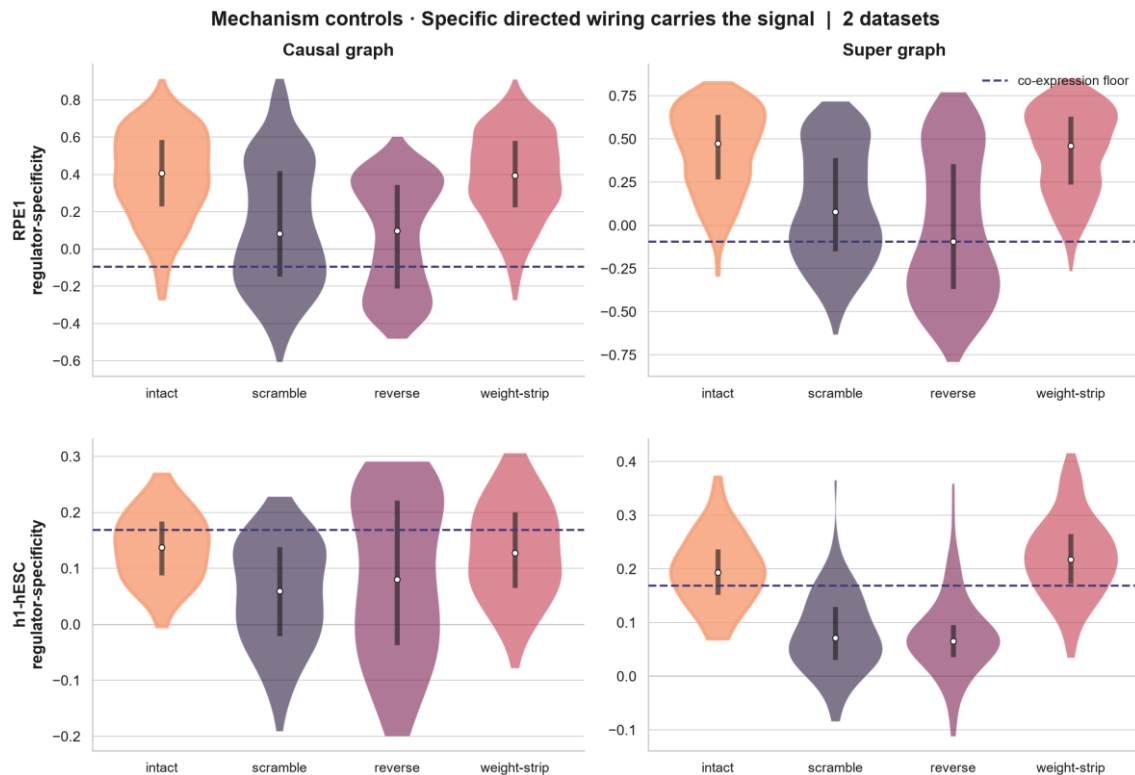


Figure 6.5 | The advantage is specific directed wiring. Per-perturbation regulator-specificity of the intact causal and Super graphs against a degree-preserving scramble, an edge reversal, and a weight-strip control, on both datasets, with the co-expression floor drawn as a threshold. Scramble and reversal collapse the advantage toward or below the floor, while stripping the edge weights is tolerated, so the signal is carried by the directed topology rather than by degree or fine weights.

Table 6.4 | Mechanism controls. The advantage collapses toward the co-expression floor when the directed wiring is scrambled or reversed, and is tolerated when the edge weights are stripped.

Graph	Intact	Scramble	Reverse	Weight-strip
RPE1, FUNGI Causal	+0.394	+0.124	+0.063	+0.386
RPE1, FUNGI Super	+0.438	+0.111	-0.005	+0.424
h1-hESC, FUNGI Causal	+0.137	+0.053	+0.080	+0.131
h1-hESC, FUNGI Super	+0.197	+0.080	+0.076	+0.222

Values are per-perturbation mean regulator-specificity. The co-expression floor is minus 0.094 on RPE1 and plus 0.169 on h1-hESC. Scramble and reverse collapse to the floor; weight-strip is tolerated within a few percent.

6.6 — FUNGI's Distinct Contribution: A Legible Graph at Predictive Parity

Section 6.5 establishes that the advantage lives in the directed topology. It follows that the pruner which renders topology faithfully is the valuable one, and this is where FUNGI earns its place, on the axis it actually wins, with the pruning question stated honestly first.

6.6.1 — The pruning wash, stated plainly

On raw prediction, the choice of pruner is a wash. At matched edges, FUNGI ties the naive pruners rather than beating them: within the Super substrate on RPE1, FUNGI, Top-Weight, and KNN score 0.472, 0.460, and 0.539 on regulator-specificity, so KNN in fact leads on this axis and FUNGI sits between (Table 6.6.1). This is stated up front because it is the honest baseline, and because it is the setup for the actual FUNGI result, not because it is a defeat. As section 6.10 shows, the wash is not an artifact of an under-powered model; it survives a doubled-capacity arbiter and three architecture changes. The predictive parity of the three pruners is a real, robust property, and it is the correct frame: FUNGI matches the naive pruners on prediction while being the only one of the three that produces a biologically legible graph.

Table 6.5 | The pruning wash. At matched edge count the three pruners tie on prediction, so the choice of pruner does not decide the axis.

Substrate	Metric	FUNGI	KNN	Top-Weight
Super	Reg-specificity	0.472	0.539	0.460
Super	Cosine	0.675	0.666	0.679
Super	DEG-AUPRC	0.514	0.516	0.509
Causal	Reg-specificity	0.423	0.511	0.494
Co-expression	Reg-specificity	-0.049	-0.058	-0.094

Raw prediction is a wash across pruners. FUNGI's distinct contribution is topological legibility, not accuracy.

6.6.2 — Legibility at parity, and a longer hub hierarchy

Figure 6.6.2(a) plots the three Super pruners on prediction against hub concentration (out-degree Gini), seed by seed. FUNGI and Top-Weight sit at a literature-faithful hub concentration (Gini $\approx$ 0.72 and 0.70) while KNN's graph is markedly flatter (0.57); at matched prediction, FUNGI holds a hub hierarchy that KNN does not. **Figure 6.6.2(b)** makes this concrete through the out-degree distribution: the FUNGI Super graph decays as a smooth long-tailed hub hierarchy out to a maximum out-degree of 1,132, while the matched-size KNN graph caps with a sharp cliff at 287, a markedly shorter hub reach. It is worth being precise about this comparison rather than overclaiming it. An idealized structural KNN pruner with a uniform per-gene out-edge cap has an out-degree Gini of exactly zero and literally cannot represent a hub hierarchy (the baseline discussed in Chapter 4); the crown-jewel KNN Super graph here is not that degenerate object, its Gini is 0.57, not 0, so the honest contrast is a real but milder one: FUNGI reaches further and concentrates more of its output on genuine hubs, without the flat cliff that a degree-capped pruner imposes. The clean, unequivocal FUNGI topology win is the next figure.

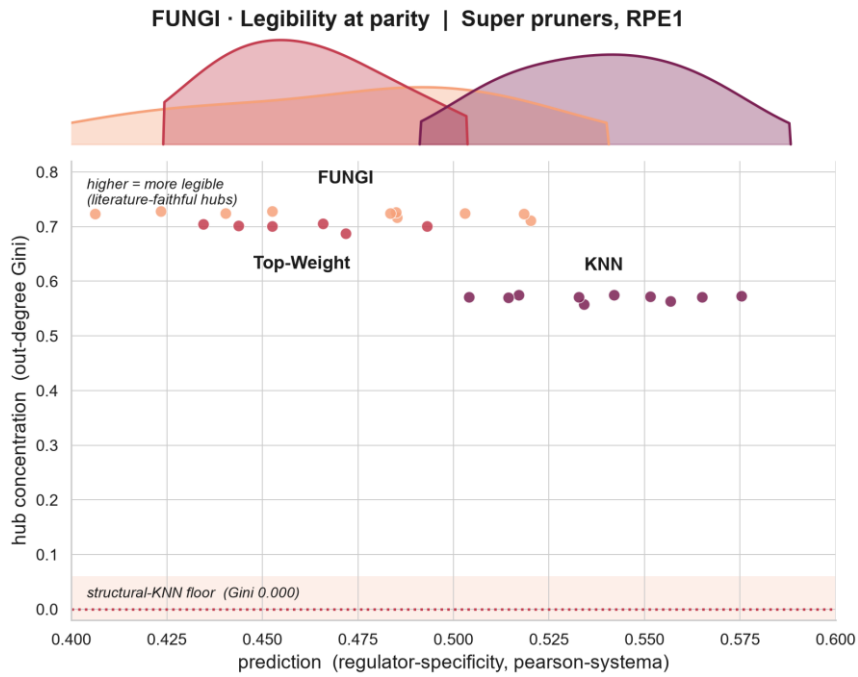


Figure 6.6.2(a) | Legibility at predictive parity. The three Super pruners' seeds on prediction against hub concentration. FUNGI and Top-Weight sit at a literature-faithful hub concentration while KNN is markedly flatter, so at matched prediction FUNGI holds a hub hierarchy that KNN does not.

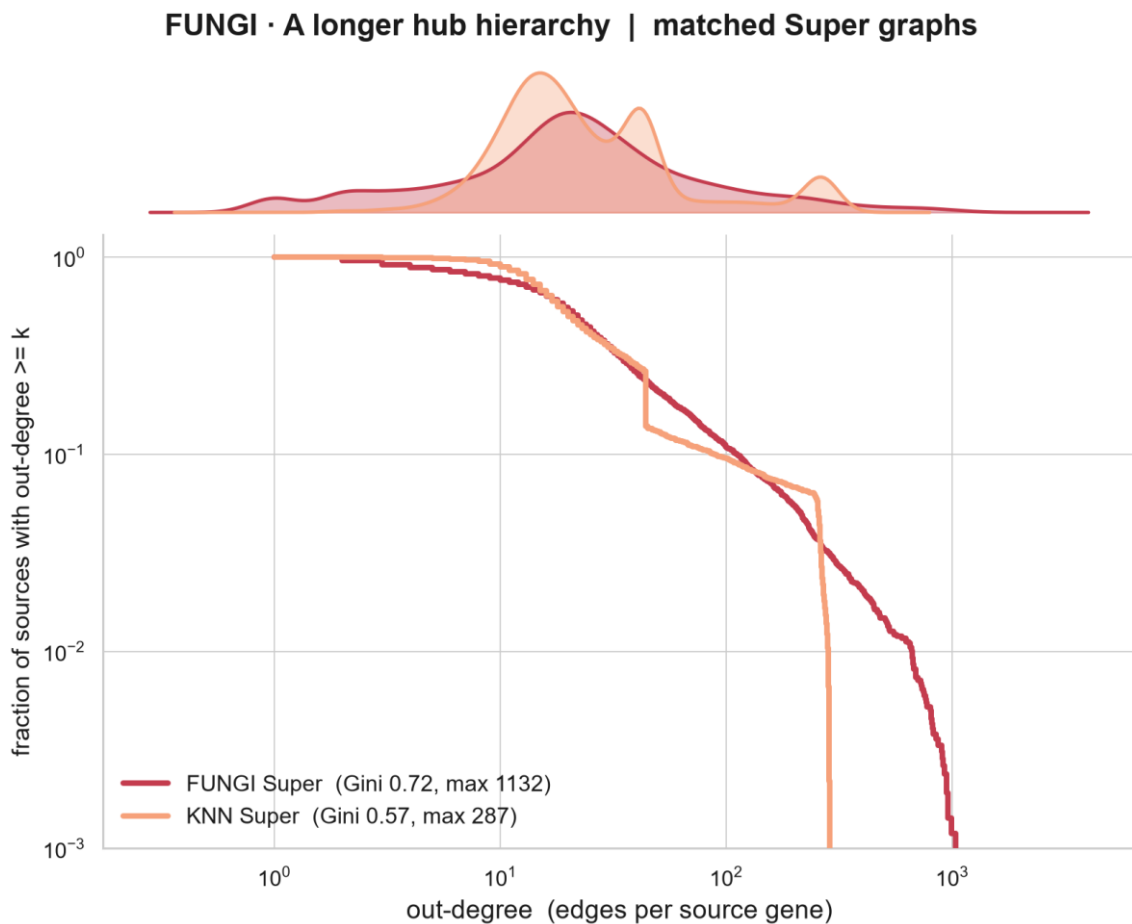


Figure 6.6.2(b) | A longer hub hierarchy. The out-degree distribution of the matched-size FUNGI Super and KNN Super graphs. FUNGI decays as a smooth long-tailed hub hierarchy out to a maximum out-degree of 1,132, while KNN caps with a sharp cliff at 287, a markedly shorter hub reach.

6.6.3 — FUNGI is literature-faithful on both datasets, and beats every structure-blind graph

The strongest form of FUNGI's topology claim is not "it has a longer tail than KNN" but "it is the only pruner whose topology is consistently regulatory-network-like across every density and both datasets." **Figure 6.6.5** shows this as a head-to-head landscape: every Super graph, three pruners crossed with both datasets and all three densities, eighteen graphs in total, placed in the plane of two literature topology axes, reciprocity (regulatory networks are directional, so low is realistic) against hub concentration (out-degree Gini, where a hub hierarchy sits in a moderate band), with the literature-faithful zone shaded. FUNGI's graphs cluster inside the zone across every density and both cell lines. Top-Weight drifts right, into over-reciprocity, its edges retain too many bidirectional co-expression pairs. KNN drops low, too flat on RPE1. This is a comparative win, not a FUNGI-alone showcase: FUNGI is the single pruner that lands in the regulatory-network-like region regardless of dataset or edge budget, while each naive pruner fails it on a different axis.

That legibility is not merely cosmetic; it corresponds to a graph whose edges carry real signal. **Figure 6.6.3** shows the result from MOREL, the bypass-free evaluator introduced in Chapter 5, a propagator that carries no node features, so the graph is the only channel that can produce a prediction. In that setting FUNGI's real wiring beats a degree-preserving shuffle of itself by roughly fourfold on RSC (+0.085 versus +0.021) and beats every structure-blind baseline, a random graph, a co-expression KNN, a spanning tree, and the empty graph, while the direction- and identity-destroying controls collapse to zero. Two independent instruments, the learned arbiter downstream and the bypass-free propagator intrinsically, therefore agree that FUNGI's topology is load-bearing. FUNGI's contribution is stated exactly: a graph as predictive as the alternatives, and the only one that is also biologically legible.

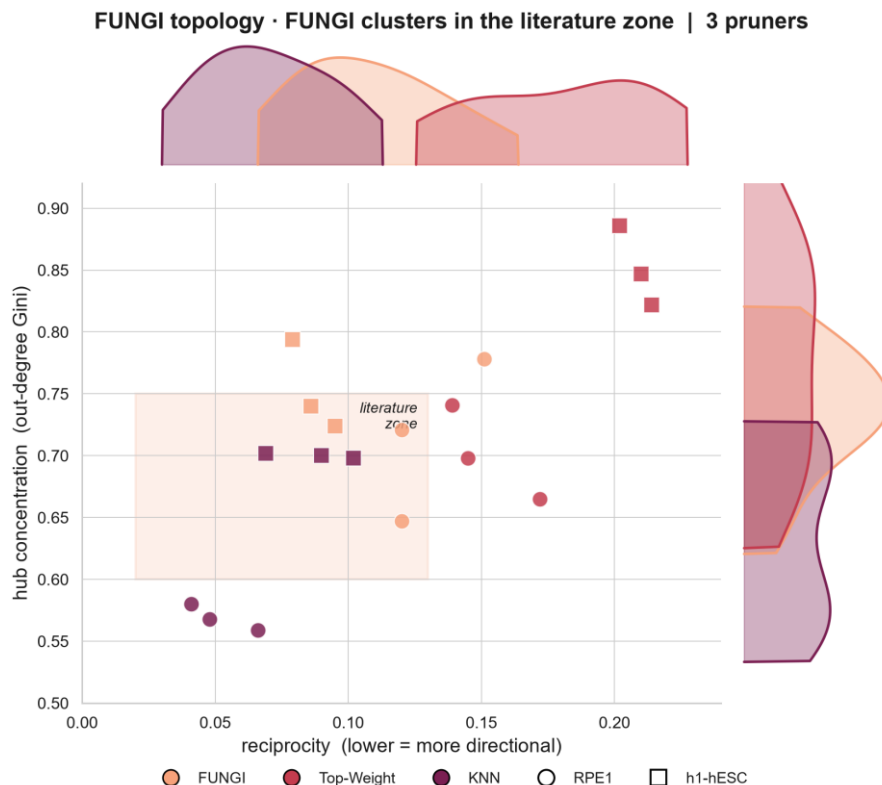


Figure 6.6.5 | The three pruners in the literature-topology plane. Every Super graph, the three pruners crossed with both datasets and all three densities, placed by reciprocity against hub concentration, with the literature-faithful zone shaded. FUNGI's graphs cluster inside the zone across every density and both cell lines, while Top-Weight drifts into over-reciprocity and KNN drops too flat, so FUNGI is the only pruner that is regulatory-network-like regardless of dataset or budget.

MOREL · FUNGI beats structure-blind graphs | RPE1

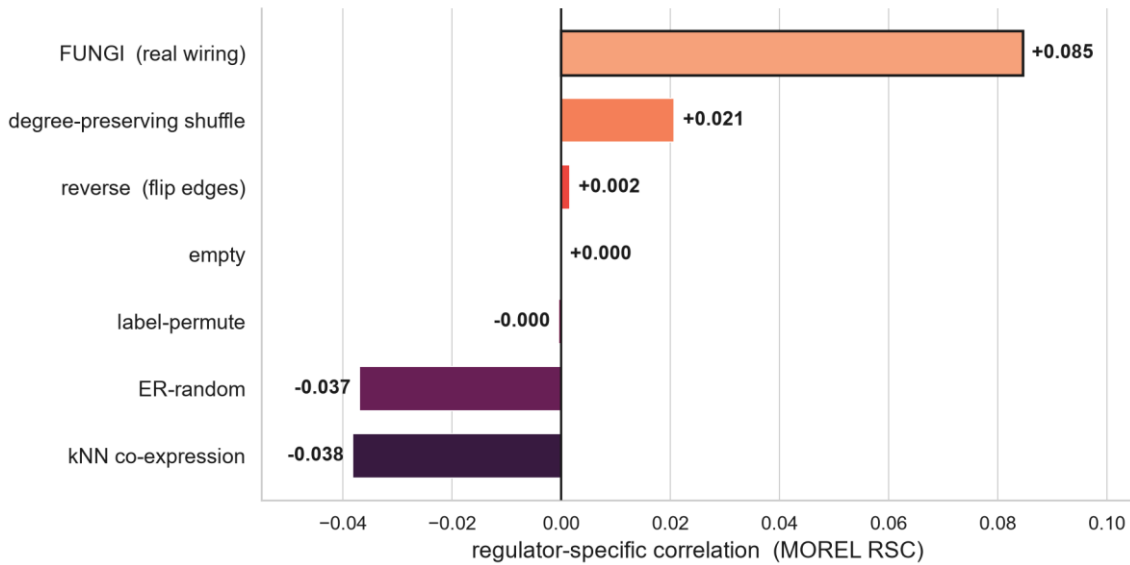


Figure 6.6.3 | FUNGI beats every structure-blind graph on the bypass-free evaluator. In MOREL, where the graph is the only channel that can produce a prediction, FUNGI's real wiring beats a degree-preserving shuffle of itself roughly fourfold and beats every structure-blind baseline, while the direction- and identity-destroying controls collapse to zero. The topology is load-bearing.

6.7 — The Result Generalizes: Density, Cell Line, and an Independent Instrument

A single-condition result is a curiosity; the substrate advantage is not one. It holds across edge density, replicates on the second cell line, and reproduces on the bypass-free evaluator.

Figure 6.7.1 sweeps the edge budget and plots the Super-minus-co-expression specificity gap at each density. On dense RPE1 the advantage grows monotonically with density, from +0.45 at the sparsest budget to +0.59 at the densest, richer graphs win by more. On sparse h1-hESC the advantage is largest at the sparsest budget and dips slightly negative at the densest extreme, which is shown rather than hidden and is consistent with the coverage story of section 6.9: the two datasets have opposite density optima because they have opposite coverage. The cross-line replication is the robustness headline of the chapter, the Super arms top the arm ranking on both datasets, and FUNGI Super posts the highest direction-agreement on both, with the magnitude compressed on the roughly tenfold-lower-coverage h1-hESC exactly as the coverage mechanism predicts. And on MOREL, the node-feature-free instrument, the substrate ranking holds, causal and Super beat co-expression, with the Super substrate carrying the highest absolute signal, reproducing the headline on an evaluator that shares none of the arbiter's architecture (section 6.6.3). A result that survives a change of density, a change of cell line, and a change of the entire prediction model is a property of the substrate, not of any one condition.

Density sweep · The Super advantage vs edge density | 2 datasets

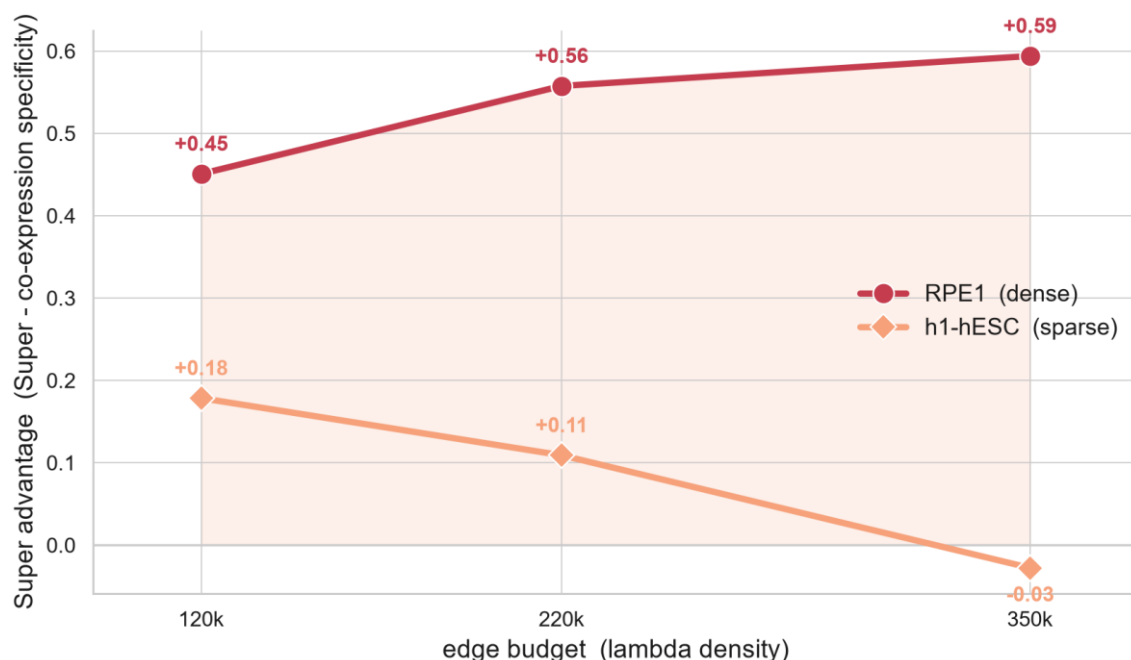


Figure 6.7.1 | The substrate advantage against edge budget. The Super-minus-co-expression specificity gap at each edge density. On dense RPE1 the advantage grows monotonically with density, while on sparse h1-hESC it is largest at the sparsest budget and dips slightly negative at the densest extreme, an opposite density optimum that tracks the opposite coverage of the two datasets.

6.8 — Attribution: The Graph Carries the Signal, and RHIZO Reads It

Having shown the substrate advantage is real and general, the last question is where the signal lives: in the graph we built, or in the model that reads it.

Figure 6.8.1 decomposes each substrate's in-distribution performance into a graph-free linear floor, a zero-parameter propagation reader, and the learned RHIZO reader, on both datasets. The load-bearing result is on RPE1: the FUNGI-Super graph read by RHIZO clears the strong linear baseline that the perturbation-prediction field has repeatedly failed to beat ⁹⁰, by +0.28 for the causal substrate and +0.29 for Super, while co-expression read by the same model collapses *below* that floor. Beating the linear baseline in-distribution is a stronger statement than "causal beats co-expression," and it is the substrate that makes it possible: the specific regulatory structure supplies the capacity, and the learned reader converts it. The honest wrinkle is on h1-hESC, and it is shown in the same figure: on the sparse dataset, zero-parameter propagation on the graph already reaches the specificity ceiling and the learned reader adds nothing in-distribution, the graph's structure is doing the work, and the heavier model has little sparse structure left to extract. The reading across both panels is that the graph is the product; RHIZO is the reader that, given enough coverage, turns it into a floor-beating prediction. That the arbiter itself is a capable reader, that its directed-topology design reads causal graphs better than a topology-blind reader, and that its architecture is the right one, is established by the capacity and ablation study in section 6.10.

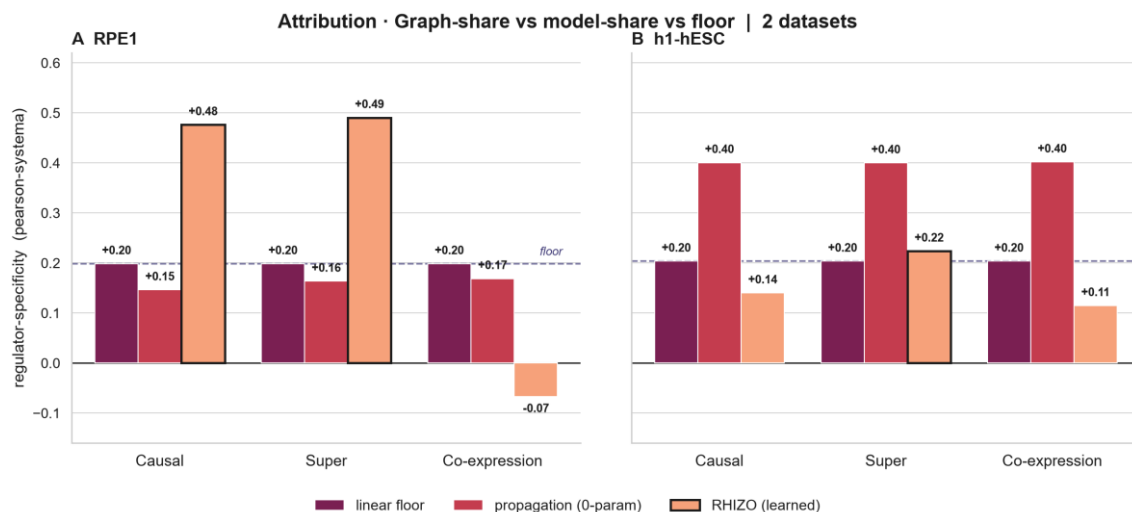


Figure 6.8.1 | Attribution across a linear floor, a propagation reader, and the learned reader. For each substrate, regulator-specificity of a graph-free linear floor, a zero-parameter propagation reader, and the learned RHIZO reader, on both datasets. On RPE1 the causal and Super graphs read by RHIZO clear the linear floor the field has repeatedly failed to beat, while co-expression collapses below it. On h1-hESC zero-parameter propagation already reaches the ceiling and the learned reader adds nothing.

Table 6.6 | Graph reader versus model reader, against the linear floor. On RPE1 the learned reader on a causal or Super graph clears the linear floor the field cannot beat; on h1-hESC the graph alone already reaches the ceiling.

Dataset	Substrate	Linear floor	Propagation	RHIZO	RHIZO minus floor
RPE1	Causal	+0.198	+0.146	+0.476	+0.278
RPE1	Super	+0.198	+0.163	+0.491	+0.293
RPE1	Co-expression	+0.198	+0.169	-0.067	-0.265
h1-hESC	Causal	+0.204	+0.401	+0.138	-0.066
h1-hESC	Super	+0.204	+0.400	+0.224	+0.020

Values are regulator-specificity, in-distribution. Propagation is a zero-parameter reader; RHIZO is the learned reader.

6.9 — Honest Boundaries: Where the Advantage Ends

The advantage has two boundaries, and characterizing them is what makes sections 6.4–6.8 credible. A de-confounded evaluation that catches its own confounds is a rigor asset, not a liability.

The first boundary is zero-shot generalization. At a held-out, gene-disjoint perturbation the causal graph has no out-edges from the queried gene, it is structurally mute, predicting a constant, which the degenerate-row rule correctly scores as undefined rather than as a fabricated zero (Chapter 5). The graphs that do score at zero-shot converge to the linear baseline: no method here, ours included, beats a simple linear model on unseen perturbations, which is the field-wide result of Ahlmann-Eltze et al. reproduced in our pipeline⁹⁰. This is a boundary, not a failure, it places the work exactly where the field's best models sit, and the very fact that nothing beats the linear baseline at zero-shot is what certifies the in-distribution win as real signal rather than leakage. The causal graft that would give held-out genes out-edges, and so give the causal channel something to say at zero-shot, is the natural next step and is left to future work.

The second boundary is the coverage axis, and it is a genuine and instructive divergence. **Figure 6.9.2** takes the RPE1 causal graph rebuilt from 25, 50, 75, and 100 % of the training perturbations and plots two metrics against coverage: raw arbiter correlation (RSC) climbs

steeply and monotonically ($\rho = +1.0$), while de-confounded regulator-specificity stays flat, already near its ceiling at 25 % coverage, with a slope that is not statistically distinguishable from zero. The two lines cross. The reading is exact and it is not the one we hypothesized: coverage buys *breadth* of raw prediction, the graph can say something about more genes, but it does not buy sharper regulator-specificity, which is an intrinsic, coverage-robust property of the causal wiring, present in full even at minimal coverage. This is a textbook demonstration of why the de-confounded axis matters: an analyst reading only the raw correlation would have declared a triumphant coverage law that is, on the specificity axis, a systematic-response confound. The cross-line gap between RPE1 and h1-hESC is likewise not explained by coverage alone, the sparse dataset sits well off the dense dataset's specificity-versus-coverage curve, so what governs that gap beyond coverage is a real open question for future work.

Coverage ladder · Breadth rises, specificity stays flat | RPE1

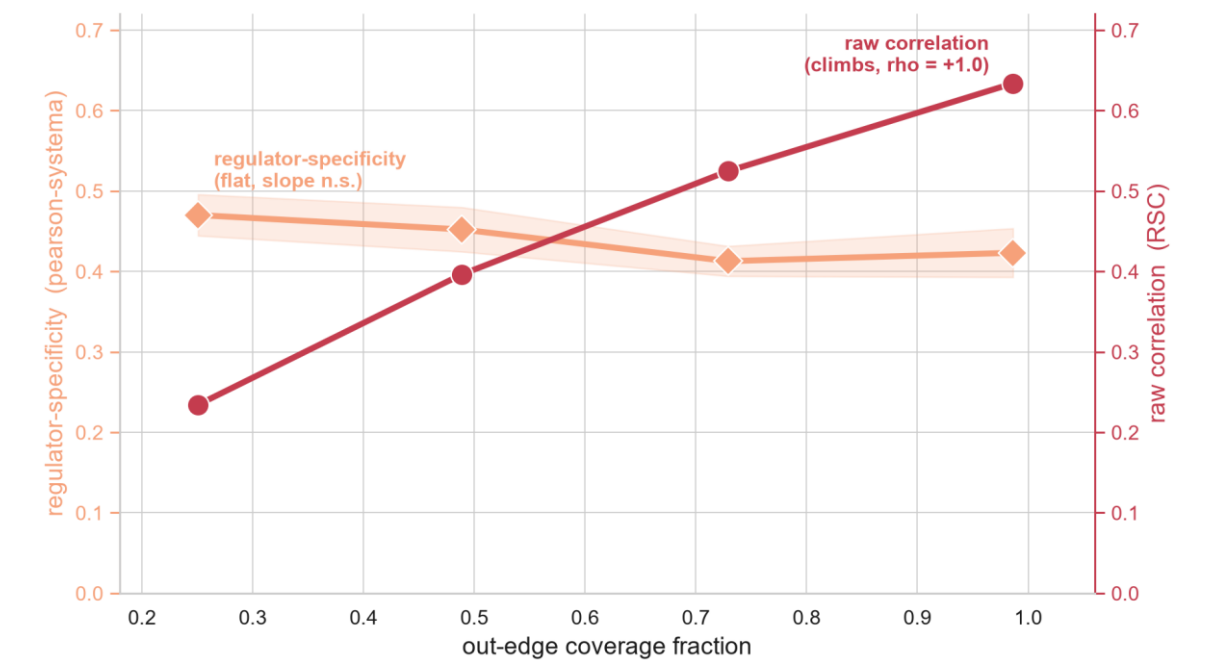


Figure 6.9.2 | Coverage buys breadth, not specificity. Over the RPE1 coverage ladder, raw arbiter correlation climbs steeply and monotonically while de-confounded regulator-specificity stays flat, already near its ceiling at 25 percent coverage. The two lines cross. Coverage lets the graph say something about more genes, but it does not sharpen regulator-specificity, which is an intrinsic property of the causal wiring.

Table 6.7 | The coverage divergence. Raw correlation climbs with coverage while regulator-specificity stays flat, near its ceiling from 25 percent coverage.

Coverage fraction	Reg-specificity	RSC (raw)
0.25	+0.470	0.234
0.49	+0.452	0.396
0.73	+0.413	0.525
0.99	+0.423	0.634
Trend	slope -0.073 (n.s.)	slope +0.541, $\rho = +1.0$

On RPE1. Coverage buys breadth of raw prediction but not sharper regulator-specificity, which is near-ceiling even at minimal coverage.

6.10 — RHIZO Capacity and Architecture: The Pruner Question

Section 6.6.1 stated that the pruner axis is a wash, and left open the possibility that the wash was an artifact of an under-powered arbiter, that a larger or more topology-aware model would finally reward FUNGI's curated topology with better prediction. It does not. This section closes that question with a capacity probe and three architecture ablations, all on the Super arms.

Doubling the arbiter's capacity does not convert the wash into a FUNGI win; it moves the result the other way. Under the larger model, FUNGI Super *drops* by 0.058 to last place among the three pruners, while the naive Top-Weight pruner *rises* by 0.060 to first, both at $q \approx 10^{-4}$ (Table 6.10). The mechanism is the opposite of the one the capacity hypothesis proposed: more model capacity does not help the reader extract more from FUNGI's topology-balanced graph, it helps the reader exploit the rawer, higher-confidence direct edges that a greedy top-weight prune preserves and that FUNGI deliberately spends to hit its topology targets. Across the capacity probe and all three architecture ablations, KNN or Top-Weight is greater than or equal to FUNGI on the decision axis without exception, and Top-Weight gains from every model change while FUNGI gains from none. FUNGI does not win the pruner axis under any capacity or architecture we tested.

The ablations also characterize the production arbiter, which feeds the model showcase of section 6.8. Feature-wise conditioning (FiLM) is load-bearing, removing it costs both graph-reading arms⁹². Over-squash mitigation is not, removing it is statistically indistinguishable from the anchor, the only ablation that preserves the pruner ranking, indicating that the graph is not deep enough for over-squashing to bite^{78,94}. Directed-topology awareness is mildly harmful on the Super substrate, so the clean default is correct here. The anchor configuration is confirmed as the right production choice.

The scope of this result must be stated precisely, because it is easy to overread. The capacity and ablation study ran only the three Super arms, it varies the *pruner* within the Super substrate, holding the substrate fixed, so it tests the pruner axis and says nothing directly about the substrate ranking, which is owned by section 6.4 and corroborated intrinsically in section 6.3.2. And the cap-big drop is stated plainly rather than buried: at the production capacity FUNGI is at parity with the naive pruners, and scaling the model up does not rescue it and slightly widens the gap, exactly what one expects if FUNGI's value is legibility rather than predictive accuracy. Far from denting the FUNGI framing, this is the result that hardens it: "interpretability at predictive parity" is not a hedge we chose, it is the conclusion the data forces after we gave the model every chance to prefer FUNGI and it did not.

Table 6.8 | Arbiter capacity and architecture study, on the Super arms. Doubling the model does not convert the pruning wash into a FUNGI win, and the ablations fix the production configuration.

Change to the arbiter	FUNGI Super	Top-Weight Super	KNN Super	Reading
Doubled capacity	-0.058	+0.060	-0.024	FUNGI to last, Top-Weight to first
Feature-wise conditioning removed	-0.032	+0.010	-0.033	Conditioning is load-bearing
Over-squash reader removed	+0.012	+0.004	-0.000	Not load-bearing at this depth
Directed edge features on	-0.019	+0.040	-0.020	Harmful on the Super substrate

Values are the change in regulator-specificity relative to the production arbiter. Across every change, KNN or Top-Weight is at least FUNGI, and Top-Weight gains from each change while FUNGI gains from none.

6.11 — Discussion Synthesis: The Two-Pillar Case, Consolidated

Read as a whole, the chapter makes a two-pillar case, and it is worth stating each pillar in the exact, defensible form the evidence supports.

The **Super graph** wins on the axis that decides, regulator-specificity, in-distribution, and it wins robustly: at every edge density, on two cell lines that differ tenfold in perturbation coverage, and on a second evaluator that shares none of the arbiter's machinery (sections 6.4, 6.7). It is also the most deliberately constructed of the substrates, engineered to resolve the specificity-versus-coverage tension, and its advantage is corroborated intrinsically at the graph level before any model reads it (section 6.3.2). The mechanism is understood: the advantage requires the specific directed regulatory wiring and is carried by topology, not by degree, coverage, or fine weights (section 6.5), and the graph, not the model, is what carries the specificity, to the point of beating the linear baseline the field cannot beat, in-distribution on the dense dataset (section 6.8). The composition principle behind the Super graph, a causal core for specificity, a co-expression scaffold for coverage, is the chapter's most transferable contribution: it is a statement about how to *build* a substrate, not a property of one dataset.

FUNGI is the pruner that renders that topology faithfully, and its contribution is legibility at predictive parity. On raw prediction it ties the naive pruners, and the study of section 6.10 shows that parity is a real, capacity-invariant property rather than an under-powered-model artifact, which means the honest, correct claim for FUNGI is not "it predicts better" but "it produces a biologically legible graph, uniquely in every literature band on both datasets, at no cost to prediction" (section 6.6). That legibility is not cosmetic: the same graph is load-bearing on the bypass-free evaluator, where FUNGI's real wiring beats every structure-blind alternative. For a gene-regulatory-network inference thesis, a graph that predicts as well as a black box and is the only one a biologist can read is the objective.

The boundaries define the frame and the future work. The advantage is in-distribution: at zero-shot every method converges to the linear baseline, which is the field-standard result and the certification that the in-distribution win is real; and along the coverage axis, coverage buys breadth rather than specificity, a divergence that vindicates the de-confounded evaluation. The causal graft for zero-shot coverage, and the question of what governs the cross-line gap beyond coverage, are the two threads the concluding chapter takes forward. The de-confounded evaluation that made every one of these results measurable, and that caught its own confounds along the way, is itself a methodological contribution the thesis carries out the door.

Table 6.9 | The two-pillar case, consolidated.

Pillar	Wins on	Evidence	Honest boundary
The Super graph	The decision axis, in-distribution, on both datasets	Crown jewel, density sweep, and the bypass-free evaluator	Zero-shot tie; compressed magnitude on the sparse dataset
FUNGI	Topological legibility and mechanism at predictive parity	Mechanism controls, the legibility landscape, and the structure-blind test	Raw-prediction pruning is a wash

Super wins prediction, reproduced three ways; FUNGI wins legibility and mechanism; the boundaries are stated.

CHAPTER 7

Conclusion

The threads tied off. A recap of what MYCELIUM built, why the Super graph and FUNGI are its answer to the best regulatory map the data allows, and the one step that remains.

The mycelium, fused into one body, sends up its final form.

7.1 — What MYCELIUM Delivers

MYCELIUM is a complete and reproducible route from raw single-cell perturbation counts to a curated regulatory map, and to a downstream test that the map, rather than the model reading it, is what carries the result. The pipeline runs end to end, from the leakage-safe substrate that SPORE prepares, through the SHROOM and HYPHAE dense-graph generators, the FUNGI pruner, and the MOREL and RHIZO predictors. Every stage is also a standalone tool. A user who needs only a clean co-expression graph can run HYPHAE alone, and a user who has any dense graph and wants a biologically faithful topology can point FUNGI at it. The pipeline is the recommended path, and the parts are independently useful, which is the property that lets the work reach past the single question it was built to answer.

7.2 — The Recipe for the Best GRN

The central result is a concrete recipe for the best regulatory map that interventional data allows. Fuse a specific causal core, learned by SHROOM from the perturbation responses, with a broad co-expression scaffold from HYPHAE, into the Super graph, and prune it to a literature-faithful topology with FUNGI. The Super graph is the headline artifact because it carries the causal core's regulator-specificity and the scaffold's whole-panel coverage in one object, and it wins on the de-confounded decision axis, in-distribution, on two cell lines that differ tenfold in coverage and on two evaluators that share none of each other's machinery.

FUNGI is the pruner that makes that graph legible. On raw prediction the choice of pruner is a wash, and that parity survives a doubled-capacity model and a full architecture sweep, so FUNGI's contribution is stated for exactly what it is. It produces a graph that predicts as well as an opaque one and is the only pruner that lands inside the literature topology bands on both datasets, which is the graph a biologist can actually read. Framing network quality as measurable topological faithfulness, and resolving the specificity-and-coverage tension by fusion, are the two contributions the thesis carries out the door, and both are statements about how to build a substrate rather than properties of one dataset.

7.3 — Boundaries and the Next Step

The boundaries are stated plainly, because a de-confounded evaluation that catches its own confounds is what makes the positive claims credible. At held-out perturbations a purely causal graph has no out-edges to offer, and every method converges to the linear baseline, which is the current field-wide result rather than a shortcoming of this pipeline. Added coverage buys breadth of raw prediction but not sharper regulator-specificity, a divergence that confirms why the de-confounded axis was necessary in the first place.

The natural next step is a causal graft that gives held-out genes out-edges, so the causal advantage that is decisive in-distribution can be carried into true zero-shot generalization. The whole pipeline was built and run on a single consumer workstation, so that step, and the wider use of MYCELIUM to build the best regulatory map a dataset allows, is within reach of anyone with the data and an ordinary machine. The graph is the product, the Super graph is the best one the data yields, and FUNGI is what makes it both faithful and legible. That is the thesis.

References

1. Karlebach, G. & Shamir, R. Modelling and analysis of gene regulatory networks. *Nat. Rev. Mol. Cell Biol.* **9**, 770–780 (2008).
2. Lee, T. I. & Young, R. A. Transcriptional regulation and its misregulation in disease. *Cell* **152**, 1237–1251 (2013).
3. Lambert, S. A. et al. The human transcription factors. *Cell* **172**, 650–665 (2018).
4. Barabási, A. L. & Oltvai, Z. N. Network biology: understanding the cell's functional organization. *Nat. Rev. Genet.* **5**, 101–113 (2004).
5. Boyer, L. A. et al. Core transcriptional regulatory circuitry in human embryonic stem cells. *Cell* **122**, 947–956 (2005).
6. Pratapa, A., Jalihal, A. P., Law, J. N., Bharadwaj, A. & Murali, T. M. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nat. Methods* **17**, 147–154 (2020).
7. Salzberg, S. L. Open questions: how many genes do we have? *BMC Biol.* **16**, 94 (2018).
8. Dixit, A. et al. Perturb-seq: dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell* **167**, 1853–1866 (2016).
9. Chevalley, M., Roohani, Y., Mehrjou, A., Leskovec, J. & Schwab, P. CausalBench: a large-scale benchmark for network inference from single-cell perturbation data. Preprint at arXiv:2210.17283 (2023).
10. Barabási, A.-L., Gulbahce, N. & Loscalzo, J. Network medicine: a network-based approach to human disease. *Nat. Rev. Genet.* **12**, 56–68 (2011).
11. Roohani, Y., Huang, K. & Leskovec, J. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nat. Biotechnol.* **42**, 927–935 (2023).
12. Hindorff, L. A. et al. Potential etiologic and functional implications of genome-wide association loci for human diseases and traits. *Proc. Natl. Acad. Sci. USA* **106**, 9362–9367 (2009).
13. Takahashi, K. & Yamanaka, S. Induction of pluripotent stem cells from mouse embryonic and adult fibroblast cultures by defined factors. *Cell* **126**, 663–676 (2006).
14. Huynh-Thu, V. A., Irrthum, A., Wehenkel, L. & Geurts, P. Inferring regulatory networks from expression data using tree-based methods. *PLoS One* **5**, e12776 (2010).
15. Newman, M. E. J. Assortative mixing in networks. *Phys. Rev. Lett.* **89**, 208701 (2002).
16. Hartwell, L. H., Hopfield, J. J., Leibler, S. & Murray, A. W. From molecular to modular cell biology. *Nature* **402**, C47–C52 (1999).
17. Segal, E. et al. Module networks: identifying regulatory modules and their condition-specific regulators from gene expression data. *Nat. Genet.* **34**, 166–176 (2003).
18. Milo, R. et al. Network motifs: simple building blocks of complex networks. *Science* **298**, 824–827 (2002).
19. Mangan, S. & Alon, U. Structure and function of the feed-forward loop network motif. *Proc. Natl. Acad. Sci. USA* **100**, 11980–11985 (2003).
20. Norman, T. M. et al. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science* **365**, 786–793 (2019).
21. Replogle, J. M. et al. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell* **185**, 2559–2575 (2022).
22. Love, M. I., Huber, W. & Anders, S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biol.* **15**, 550 (2014).
23. Lotfollahi, M., Wolf, F. A. & Theis, F. J. scGen predicts single-cell perturbation responses. *Nat. Methods* **16**, 715–721 (2019).
24. Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. Preprint at arXiv:1609.02907 (2016); Int. Conf. on Learning Representations (2017).
25. Qi, L. S. et al. Repurposing CRISPR as an RNA-guided platform for sequence-specific control of gene expression. *Cell* **152**, 1173–1183 (2013).
26. Gilbert, L. A. et al. Genome-scale CRISPR-mediated control of gene repression and activation. *Cell* **159**, 647–661 (2014).
27. Ran, F. A. et al. Genome engineering using the CRISPR-Cas9 system. *Nat. Protoc.* **8**, 2281–2308 (2013).
28. Przybyla, L. & Gilbert, L. A. A new era in functional genomics screens. *Nat. Rev. Genet.* **23**, 89–103 (2022).

29. Doench, J. G. Am I ready for CRISPR? A user's guide to genetic screens. *Nat. Rev. Genet.* **19**, 67–80 (2018).
30. Adamson, B. et al. A multiplexed single-cell CRISPR screening platform enables systematic dissection of the unfolded protein response. *Cell* **167**, 1867–1882 (2016).
31. Replogle, J. M. et al. Combinatorial single-cell CRISPR screens by direct guide RNA capture and targeted sequencing. *Nat. Biotechnol.* **38**, 954–961 (2020).
32. Islam, S. et al. Quantitative single-cell RNA-seq with unique molecular identifiers. *Nat. Methods* **11**, 163–166 (2014).
33. The ENCODE Project Consortium. An integrated encyclopedia of DNA elements in the human genome. *Nature* **489**, 57–74 (2012).
34. Zheng, G. X. Y. et al. Massively parallel digital transcriptional profiling of single cells. *Nat. Commun.* **8**, 14049 (2017).
35. Luecken, M. D. & Theis, F. J. Current best practices in single-cell RNA-seq analysis: a tutorial. *Mol. Syst. Biol.* **15**, e8746 (2019).
36. Viñas Torné, R. et al. Systema: a framework for evaluating genetic perturbation response prediction beyond systematic variation. *Nat. Biotechnol.* (advance online publication, 2025).
37. Young, M. D. & Behjati, S. SoupX removes ambient RNA contamination from droplet-based single-cell RNA sequencing data. *GigaScience* **9**, giaa151 (2020).
38. Moerman, T. et al. GRNBoost2 and Arboreto: efficient and scalable inference of gene regulatory networks. *Bioinformatics* **35**, 2159–2161 (2019).
39. Aibar, S. et al. SCENIC: single-cell regulatory network inference and clustering. *Nat. Methods* **14**, 1083–1086 (2017).
40. Chen, D., Lin, Y., Li, W., Li, P., Zhou, J. & Sun, X. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. *Proc. AAAI Conf. Artif. Intell.* **34**, 3438–3445 (2020).
41. Hossain, I., Fischer, J., Burkholz, R. & Quackenbush, J. Pruning neural network models for gene regulatory dynamics using data and domain knowledge. Preprint at arXiv:2403.04805 (2024).
42. Wang, G. et al. Modeling and predicting single-cell multi-gene perturbation responses with scLAMBDA. Preprint at bioRxiv <https://doi.org/10.1101/2024.12.04.626878> (2024).
43. Calabrò, M. SPECTRA. GitHub, <https://github.com/MCalabroCode/SPECTRA> (2026).
44. Baran, Y. et al. MetaCell: analysis of single-cell RNA-seq data using K-nn graph partitions. *Genome Biol.* **20**, 206 (2019).
45. Song, X., Deng, K., Chen, M. & Guan, Y. PSGRN: gene regulatory network inference from single-cell perturbational data through self-training with synthetic gold standards. *Sci. Adv.* **12**, eaeb3376 (2026).
46. Kaufman, S., Rosset, S., Perlich, C. & Stitelman, O. Leakage in data mining: formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data* **6**, 15 (2012).
47. Kapoor, S. & Narayanan, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* **4**, 100804 (2023).
48. Sandve, G. K., Nekrutenko, A., Taylor, J. & Hovig, E. Ten simple rules for reproducible computational research. *PLoS Comput. Biol.* **9**, e1003285 (2013).
49. Amezquita, R. A. et al. Orchestrating single-cell analysis with Bioconductor. *Nat. Methods* **17**, 137–145 (2020).
50. Wolf, F. A., Angerer, P. & Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol.* **19**, 15 (2018).
51. Wolock, S. L., Lopez, R. & Klein, A. M. Scrublet: computational identification of cell doublets in single-cell transcriptomic data. *Cell Syst.* **8**, 281–291 (2019).
52. Stuart, T. et al. Comprehensive integration of single-cell data. *Cell* **177**, 1888–1902 (2019).
53. Korsunsky, I. et al. Fast, sensitive and accurate integration of single-cell data with Harmony. *Nat. Methods* **16**, 1289–1296 (2019).
54. Sculley, D. Web-scale k-means clustering. In *Proc. 19th Int. Conf. World Wide Web (WWW '10)* 1177–1178 (ACM, 2010).
55. Roohani, Y. et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell* **188**, 3370–3374 (2025).
56. Marbach, D. et al. Wisdom of crowds for robust gene network inference. *Nat. Methods* **9**, 796–804 (2012).

57. Ruysinck, J. et al. NIMEFI: gene regulatory network inference using multiple ensemble feature importance algorithms. *PLoS One* **9**, e92709 (2014).
58. Dwork, C., Kumar, R., Naor, M. & Sivakumar, D. Rank aggregation methods for the web. In *Proc. 10th Int. Conf. on World Wide Web (WWW '01)* 613–622 (2001).
59. Szklarczyk, D. et al. The STRING database in 2023: protein-protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Res.* **51**, D638–D646 (2023).
60. Garlaschelli, D. & Loffredo, M. I. Patterns of link reciprocity in directed networks. *Phys. Rev. Lett.* **93**, 268701 (2004).
61. Erdős, P. & Rényi, A. On random graphs I. *Publ. Math. Debrecen* **6**, 290–297 (1959).
62. Madan Babu, M., Luscombe, N. M., Aravind, L., Gerstein, M. & Teichmann, S. A. Structure and evolution of transcriptional regulatory networks. *Curr. Opin. Struct. Biol.* **14**, 283–291 (2004).
63. Guelzim, N., Bottani, S., Bourguin, P. & Képès, F. Topological and causal structure of the yeast transcriptional regulatory network. *Nat. Genet.* **31**, 60–63 (2002).
64. Basso, K. et al. Reverse engineering of regulatory networks in human B cells. *Nat. Genet.* **37**, 382–390 (2005).
65. Clauset, A., Shalizi, C. R. & Newman, M. E. J. Power-law distributions in empirical data. *SIAM Rev.* **51**, 661–703 (2009).
66. Gini, C. *Variabilità e Mutabilità* (Tipografia di Paolo Cuppini, Bologna, 1912).
67. Dixon, P. M., Weiner, J., Mitchell-Olds, T. & Woodley, R. Bootstrapping the Gini coefficient of inequality. *Ecology* **68**, 1548–1551 (1987).
68. Amaral, L. A. N., Scala, A., Barthélémy, M. & Stanley, H. E. Classes of small-world networks. *Proc. Natl. Acad. Sci. USA* **97**, 11149–11152 (2000).
69. Luscombe, N. M. et al. Genomic analysis of regulatory network dynamics reveals large topological changes. *Nature* **431**, 308–312 (2004).
70. Newman, M. E. J. & Girvan, M. Finding and evaluating community structure in networks. *Phys. Rev. E* **69**, 026113 (2004).
71. Newman, M. E. J. Modularity and community structure in networks. *Proc. Natl. Acad. Sci. USA* **103**, 8577–8582 (2006).
72. Blondel, V. D., Guillaume, J.-L., Lambiotte, R. & Lefebvre, E. Fast unfolding of communities in large networks. *J. Stat. Mech.* P10008 (2008).
73. Alon, U. Network motifs: theory and experimental approaches. *Nat. Rev. Genet.* **8**, 450–461 (2007).
74. Csárdi, G. & Nepusz, T. The igraph software package for complex network research. *InterJournal Complex Syst.* **1695**, 1–9 (2006).
75. Watts, D. J. & Strogatz, S. H. Collective dynamics of 'small-world' networks. *Nature* **393**, 440–442 (1998).
76. Gavish, M. & Donoho, D. L. The optimal hard threshold for singular values is $4/\sqrt{3}$. *IEEE Trans. Inf. Theory* **60**, 5040–5053 (2014).
77. Bo, D., Wang, X., Shi, C. & Shen, H. Beyond low-frequency information in graph convolutional networks. *Proc. AAAI Conf. Artif. Intell.* **35**, 3950–3957 (2021).
78. Di Giovanni, F., Giusti, L., Barbero, F., Luise, G., Liò, P. & Bronstein, M. M. On over-squashing in message passing neural networks: the impact of width, depth, and topology. *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, PMLR **202**, 7865–7885 (2023).
79. Spielman, D. A. & Srivastava, N. Graph sparsification by effective resistances. *SIAM J. Comput.* **40**, 1913–1926 (2011).
80. Achlioptas, D. Database-friendly random projections: Johnson–Lindenstrauss with binary coins. *J. Comput. Syst. Sci.* **66**, 671–687 (2003).
81. Traag, V. A., Waltman, L. & van Eck, N. J. From Louvain to Leiden: guaranteeing well-connected communities. *Sci. Rep.* **9**, 5233 (2019).
82. Page, L., Brin, S., Motwani, R. & Winograd, T. The PageRank citation ranking: bringing order to the web. Stanford InfoLab Tech. Rep. SIDL-WP-1999-0120 (1999).
83. Sobol', I. M. Distribution of points in a cube and approximate evaluation of integrals. *USSR Comput. Math. Math. Phys.* **7**, 86–112 (1967).
84. Meinshausen, N. & Bühlmann, P. Stability selection. *J. R. Stat. Soc. B* **72**, 417–473 (2010).

85. Ester, M., Kriegel, H.-P., Sander, J. & Xu, X. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)* 226–231 (1996).
86. Bergstra, J., Bardenet, R., Bengio, Y. & Kégl, B. Algorithms for hyper-parameter optimization. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **24**, 2546–2554 (2011).
87. Breiman, L. Random forests. *Mach. Learn.* **45**, 5–32 (2001).
88. Derringer, G. & Suich, R. Simultaneous optimization of several response variables. *J. Qual. Technol.* **12**, 214–219 (1980).
89. Serrano, M. Á., Boguñá, M. & Vespignani, A. Extracting the multiscale backbone of complex weighted networks. *Proc. Natl. Acad. Sci. USA* **106**, 6483–6488 (2009).
90. Ahlmann-Eltze, C., Huber, W. & Anders, S. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nat. Methods* **22**, 1657–1661 (2025).
91. Kamimoto, K. et al. Dissecting cell identity via network inf
92. Perez, E., Strub, F., de Vries, H., Dumoulin, V. & Courville, A. FiLM: visual reasoning with a general conditioning layer. *Proc. AAAI Conf. Artif. Intell.* **32**, 3942–3951 (2018).
93. Brockschmidt, M. GNN-FiLM: graph neural networks with feature-wise linear modulation. *Proc. 37th Int. Conf. on Machine Learning (PMLR)* **119** 1144–1152 (2020).
94. Alon, U. & Yahav, E. On the bottleneck of graph neural networks and its practical implications. *Int. Conf. on Learning Representations* (2021).
95. Wu, Y. et al. PerturBench: benchmarking machine learning models for cellular perturbation analysis. Preprint at arXiv (2024).